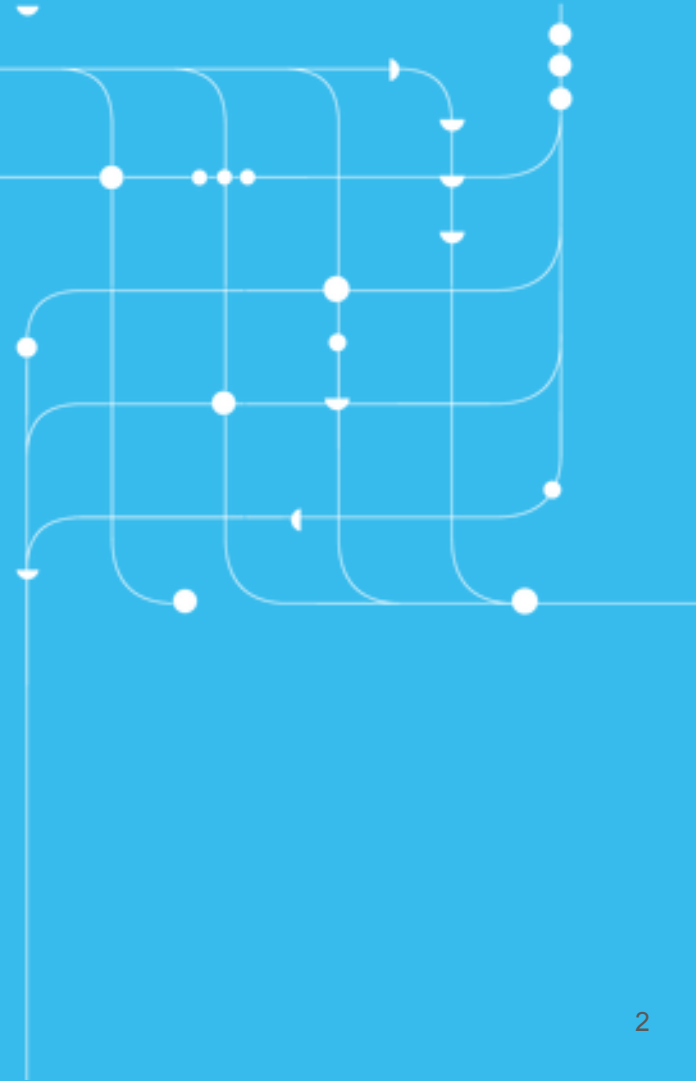


Seguridad Informática y Ciberseguridad



Pensemos:
**¿Qué es
la
seguridad?**



¿Qué es la Seguridad Informática?

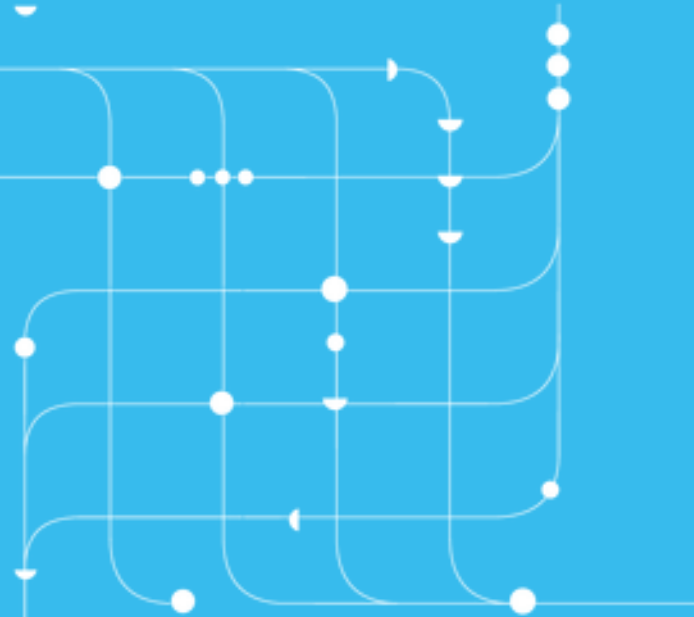
Característica o cualidad que implica:

“Un sistema este libre de peligro, daño o riesgo”.

“La seguridad es una utopía, es una característica inalcanzable o una tendencia”

Tiene que ver con la Fiabilidad

Fiabilidad: Probabilidad de que un sistema se comporte como tal (que actúe de acuerdo a lo esperado)



La Seguridad Informática persigue la confiabilidad y para esto:

Confidencialidad: Los objetos son accesibles a elementos autorizados.

Integridad: Modificados, autorizados y controlados.

Disponibilidad: Los objetos están utilizables para ser usados

Autenticidad: Que el autor sea identificable.

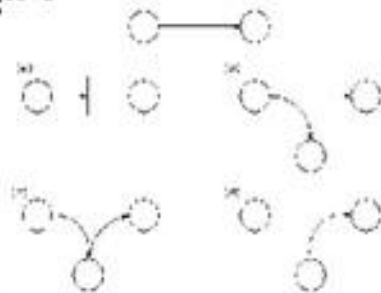
Auditabilidad: Registrable para su control



Tipos de Ataques...

¿Que queremos Proteger?

Los tipos de Ataques:



- a) **Interrupción:** Un objeto del sistema se pierda, inutilizable o indisponible.
- b) **Interceptación:** Acceso no autorizada a un objeto del sistema.
- c) **Modificación:** Interceptar y modificar.
- d) **Fabricación:** Objeto original fabricado.



¿Qué queremos proteger?

Fundamentalmente:

Hardware, Software, Datos



¿De qué nos queremos Proteger?



Personas Amenazas Lógicas Catástrofes



¿Cómo podemos protegernos?

¿Con qué mecanismos de seguridad?



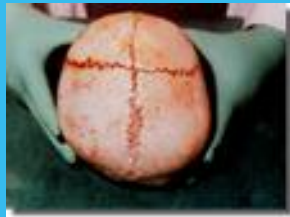
Prevención.



Detección.

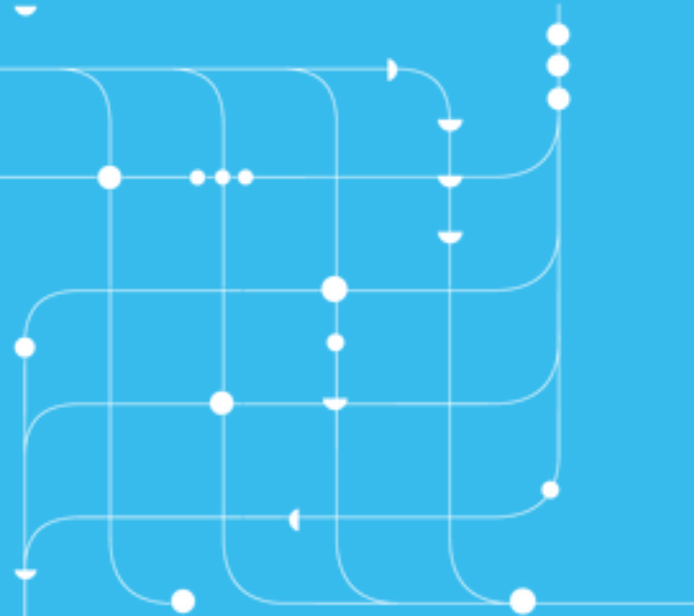


Recuperación.



Análisis forense.

“¡Mejor prevenir que curar!”



Hay que tener un Plan de Seguridad

Gestionar: Políticas de Seguridad de la Información

1- Identificación de los Activos de Información.

2- Identificación de las Amenazas y vulnerabilidades.

Análisis de riesgo (sobre 1 y 2).

Administración del Riesgo.

- **Comité de Seguridad de la Información.**
- **Responsable de Seguridad Informática.**
- **Dueño de la Información.**



Comité de Seguridad de la Información

El problema de la Seguridad de la Información no es un problema del área de Sistemas o de Informática. Es un problema de toda la Organización.

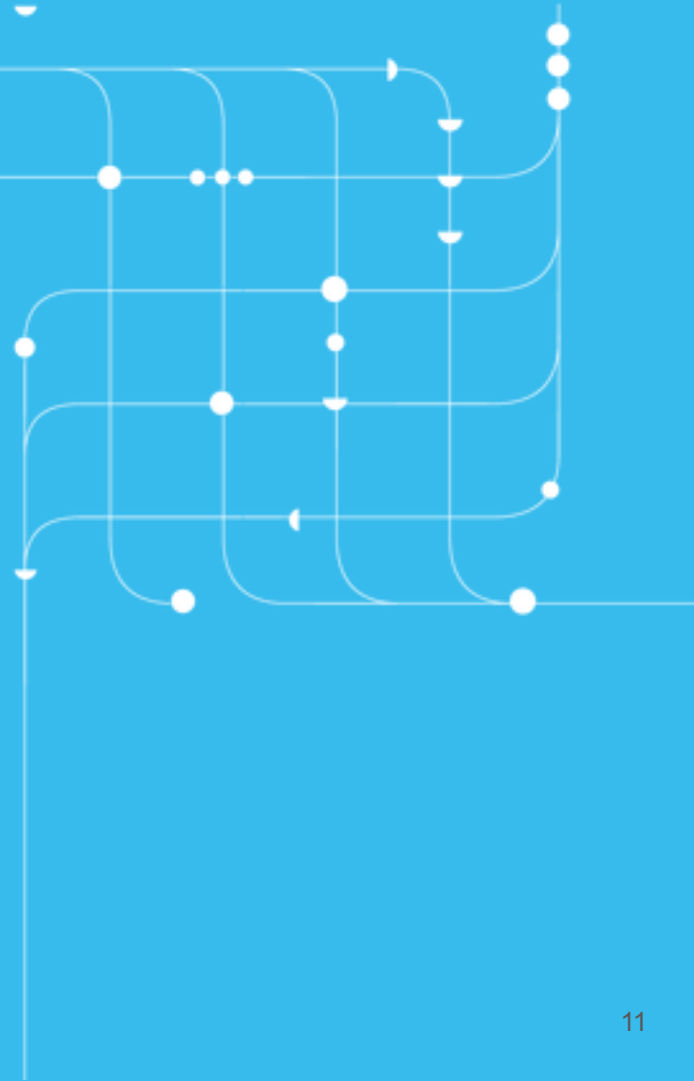
"Podemos pensar que la seguridad de la organización es una cadena donde, cada individuo es un eslabón y su fuerza se basa en la fortaleza del eslabón más débil".

**Ing. Pablo Lazaro experto
en Seguridad Informática**

Es fundamental la toma de conciencia de los miembros y su constante capacitación.



¿Qué es la Ciberseguridad?



Algunas definiciones

¿Qué es la CIBERSEGURIDAD?

“Protección de **activos de información** para mitigar los riesgos en el Ciberespacio ... “

ISACA 2018

¿Qué es el CIBERESPACIO?

“Es el espacio virtual determinado por la conexión entre personas y dispositivos a través de redes ”

Estrategia Nacional de Ciberseguridad



Algunas reflexiones

"..En estos escenarios de conflicto, por primera vez en la historia de la humanidad, es el ciudadano quien se convierte en la primera línea de defensa..."

General de División Carlos Gómez de Medina
Mando Conjunto de Ciberdefensa / España



Otra vez... ¿qué protegemos?



Algunos consejos:

**Actualizar el software y el sistema operativo.
Utilizar software antivirus.**

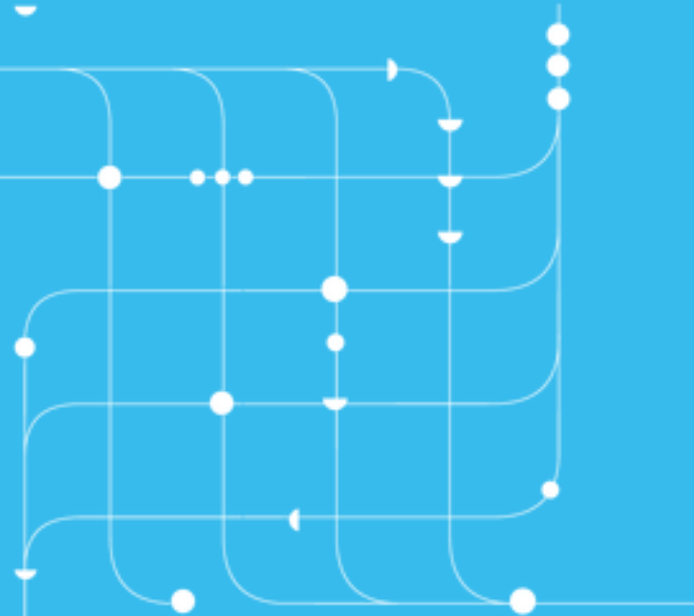
Utilizar contraseñas seguras.

No abrir archivos adjuntos de correos electrónicos de remitentes desconocidos.

No hacer clic en los vínculos de los correos electrónicos de remitentes desconocidos.

No abrir sitios web desconocidos.

Evitar el uso de redes Wi-Fi no seguras en lugares públicos.



Phishing y otras yerbas...

Malware:
(Virus, troyanos, spyware, Ransomware, Adware, Botnets)

Spear (Hacia un objetivo específico)

SmiShing (Mensajes de texto o Whatsapp)

Vishing (voice Fishing, llamadas fraudulentas)

BEC (Business Email Compromise, Dirigidos)
SIM swapping (Duplicando el SIM)
Ataque tipo "Man-in-the-middle". (ej. Wi-Fi)



Ingeniería social

El acto de manipular a las personas para que realicen acciones o divulguen información confidencial...



NIST SP 800-63-3

**Atajos cognitivos y
la psicología de la
disuasión**

R. Cialdini - "Influence: The Psychology of Persuasion"
2006

Ingeniería social - La psicología de la disuasión

1. *Reduzca la velocidad* e investigue los hechos
2. No publique sus datos personales o fotos
3. Elimine directamente cualquier solicitud de información financiera o contraseñas
4. Rechazar solicitudes de ayuda u ofertas de ayuda
5. No deje que un enlace controle donde vaya a parar cuando nevega por Internet.
6. No revele datos confidenciales (por ejemplo, contraseñas)
7. No evite las políticas y procedimientos de seguridad que le informen.
8. Reporte cualquier actividad sospechosa

Herramientas para Plan de Seguridad

Framework de Estrategias
Espacio de trabajo: conjunto estandarizado de
conceptos, prácticas y criterios.



ISO 27.001



CSF - Deloitte



NIST



Legislación en la Argentina

Leyes relacionadas a la ciberseguridad:

Ley 26.388 de Delito informático

Ley 25.326 de Protección de Datos Personales

Decreto Reglamentario N° 1558/2001

Ley 25.506 de Firma Digital

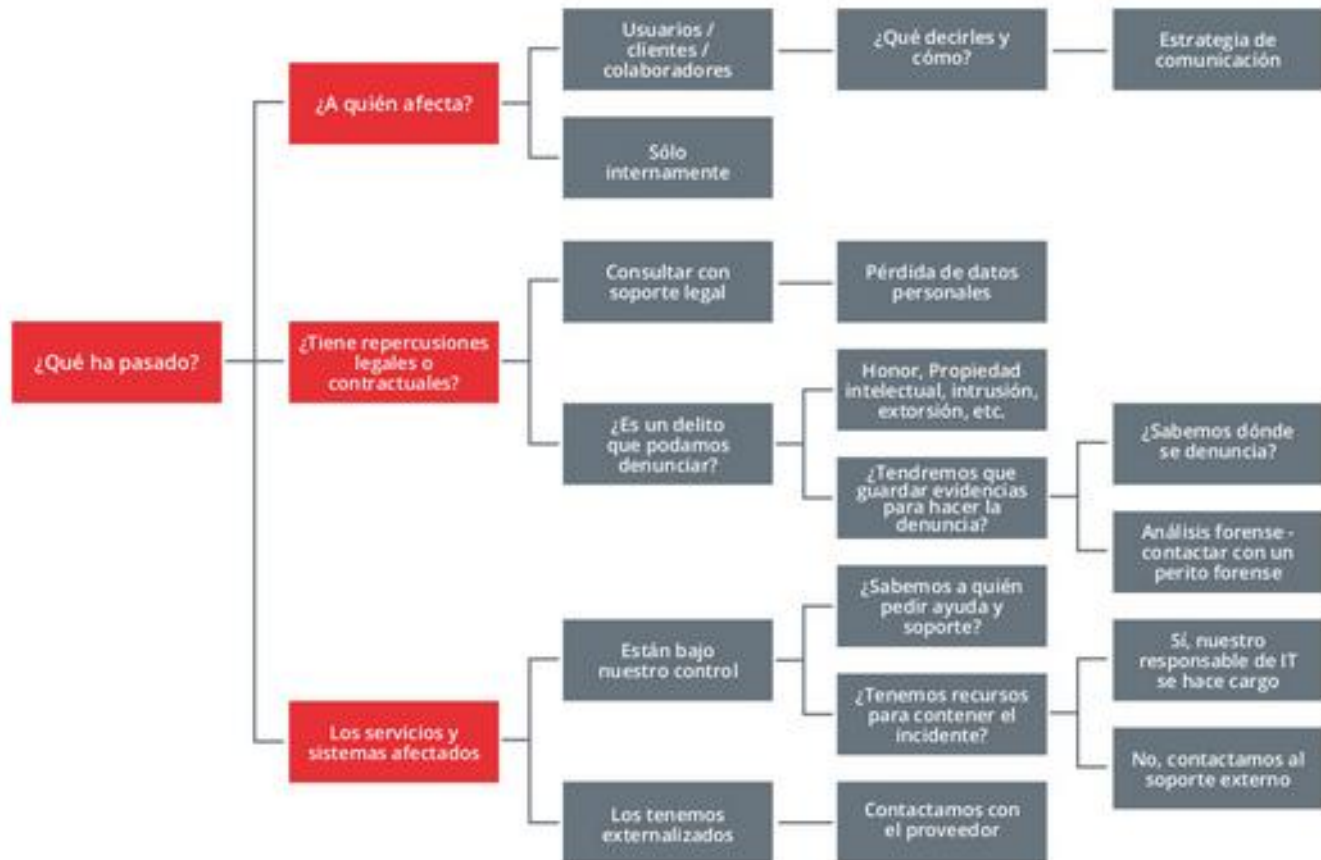
Decreto Reglamentario N° 2628/2002

Ley 26.904 de Grooming

<https://www.argentina.gob.ar/jefatura/innovacion-publica/direccion-nacional-ciberseguridad/normativa>

¡A trabajar!

¿Qué hacer? Flowchart



¡Gracias!

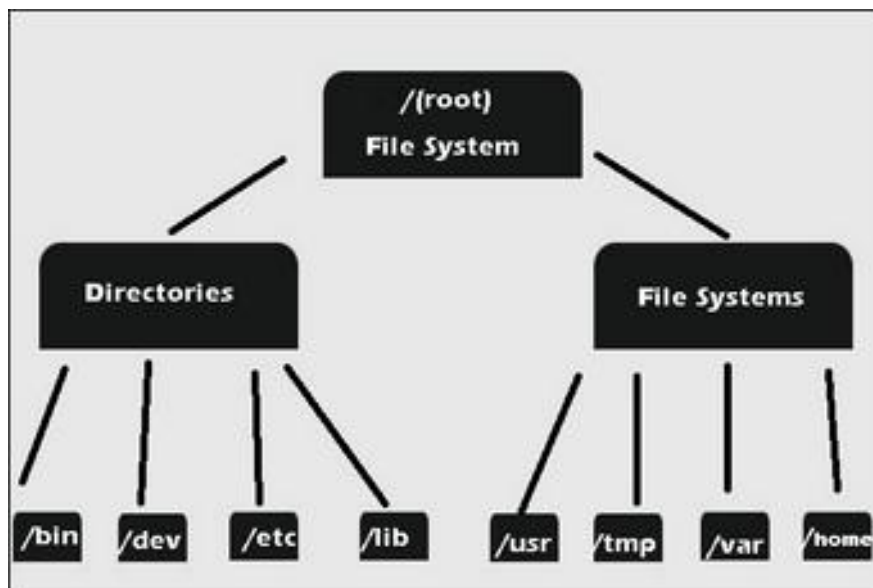
¿Preguntas?

carlos@fi.mdp.edu.ar
valeria@fi.mdp.edu.ar
hhinojal@fi.mdp.edu.ar





SISTEMA DE ARCHIVOS



- Introducción de las funciones del Sistema de Archivo
- Tipos de Sistema de archivo
- Los mas populares
- Aspectos para evaluar un F. S.
- Diseño de un F. S.
 - Concepto de Bloque
 - Archivos – Directorios
 - Asignación de Espacio
 - Control de Espacio Libre
- Casos de Estudio
 - FAT
 - Unix File-Systems
 - Inodos
 - Manejo de inodos
 - NTFS
 - MFT
- Abstracción del Sistema de archivo



Introducción al Sistema de Archivo

Es el que indica como se **organiza la información** dentro de los **medios físicos**

Entre sus funciones están:

- El guardado y recuperación de la información;
- Administración del espacio libre,
- El guardado de los atributos de seguridad del sistema operativo (En los filesystem nuevos).

Estos programas se encargan de:

- Identificar y localizar archivos
- Seguridad
- Asignación de espacio
- Coordinación de transferencia (sistema multiusuario)



Introducción al Sistema de Archivo

Sistema de archivos: Subsistema del S.O. encargado de la gestión de la memoria secundaria (concretamente del almacenamiento de la información en dispositivos de memoria secundaria).

El medio sobre el que se almacenan los archivos se divide en bloques de longitud fija, siendo el sistema de archivos el encargado de asignar un número adecuado de bloques a cada archivo.

Funciones del sistemas de archivos:

- Crear y borrar archivos
- Permitir el acceso a los archivos para que sean leídos o escritos
- Automatizar la gestión de la memoria secundaria
- Permitir referenciar un archivo por su nombre simbólico
- Proteger los archivos frente a fallos del sistema
- Permitir el uso compartido de archivos a usuarios autorizados

BLOQUES:

SECTOR 512 Bytes.

El Bloque puede tomar varios sectores:

ej. 1k, 2 k o 4 k



Tipos de Sistema de archivo

Pueden ser clasificados por:

- medio
- red
- propósito especial.

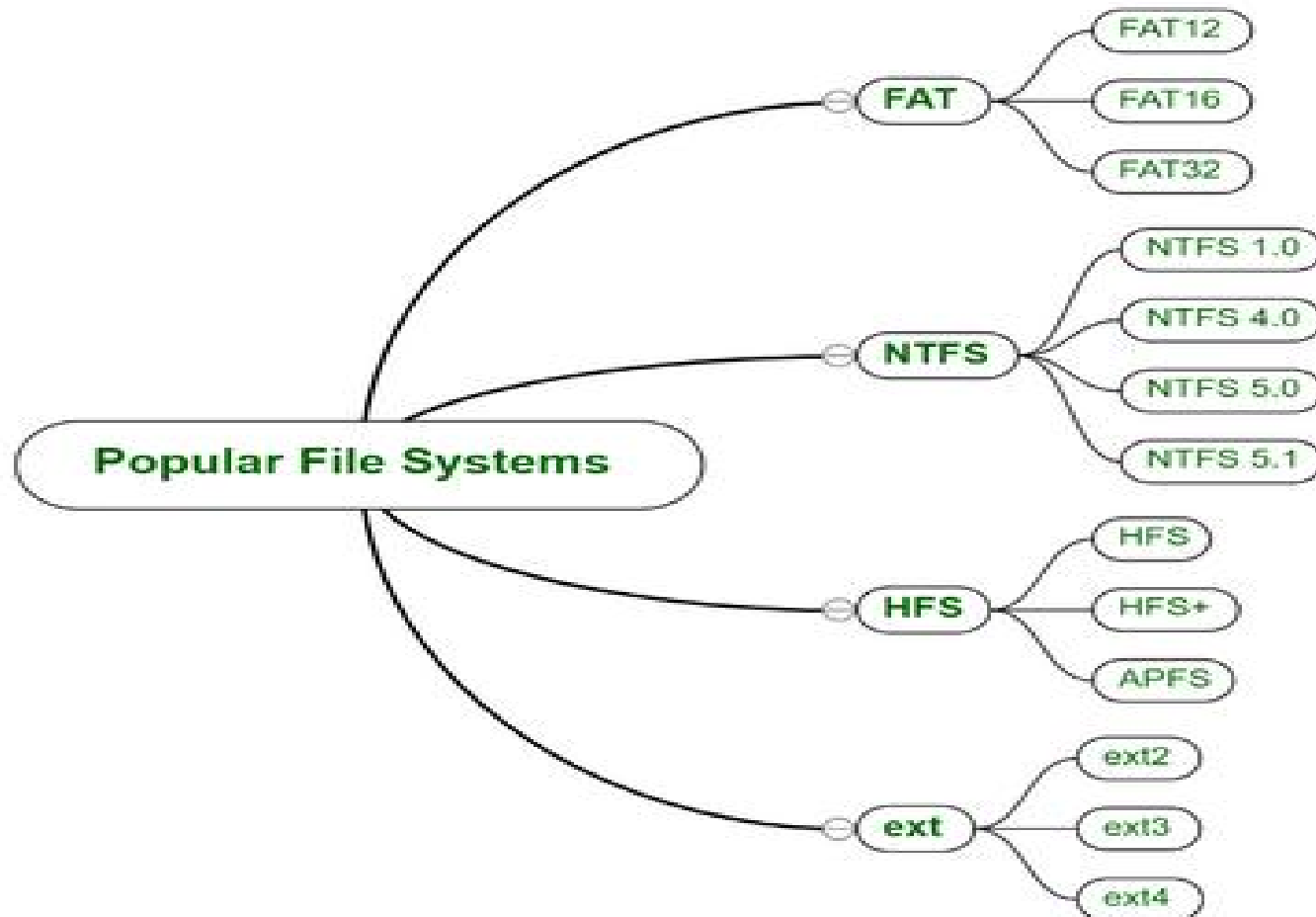
- **Discos de plato magnetico o Flash** (acceso aleatorio): FAT, NTFS, HFS+, UFS, EXT2/3/4, XFS, BTRFS, Veritas File system, ZFS, ReiserFS
- **Discos Opticos:** ISO9660, UDF
- **Flash file System** (Especialmente embebidos): JFFS, UBIFS, LogFS, F2FS, SquashFS

- **Network File System:** NFS, AFS (Sistema de comparticion de apple), SMB (Sistema de comparticion de microsoft), FTP, Webdav, SSHFS.
- **Sistemas de archivos en Cluster:**
 - **Shared-disk / storage area network:** GFS2 (Red hat), GPFS (IBM), CSV (Microsoft), OCFS (Oracle)
 - **Distributed file systems:** GFS (Google), HDFS (Apache), Ceph (Red Hat), DFS (Microsoft), GlusterFS (Red Hat)

- **Especiales:**
 - **procfs** --> Mapeo de procesos y estructuras en linux
 - **configfs y sysfs** --> Muestra como archivos los parametros de consulta y configuración de un Linux
 - **devfs** --> los archivos en esta fs representa a dispositivos. Y permite utilizar las primitivas de cualquier fs para acceder al dispositivo.
 - **Superpuestos** --> overlayFS, UnionFS, aufs



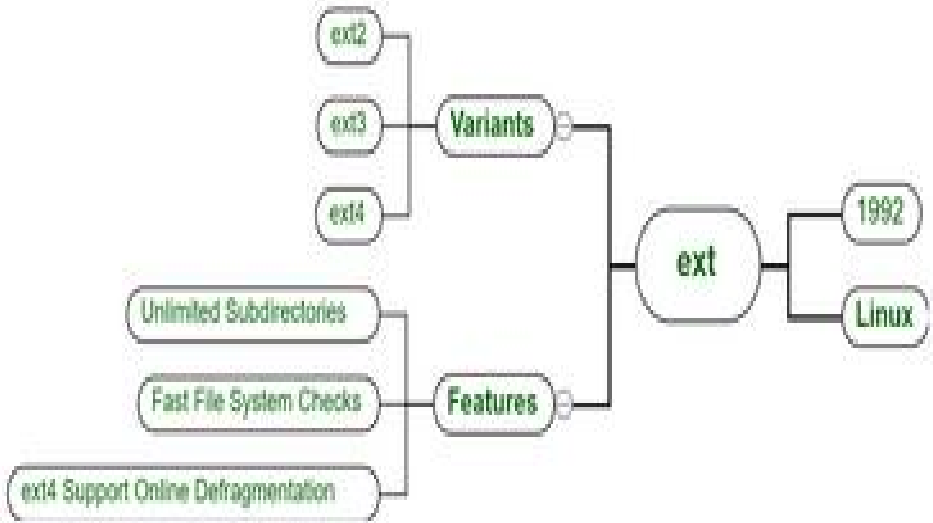
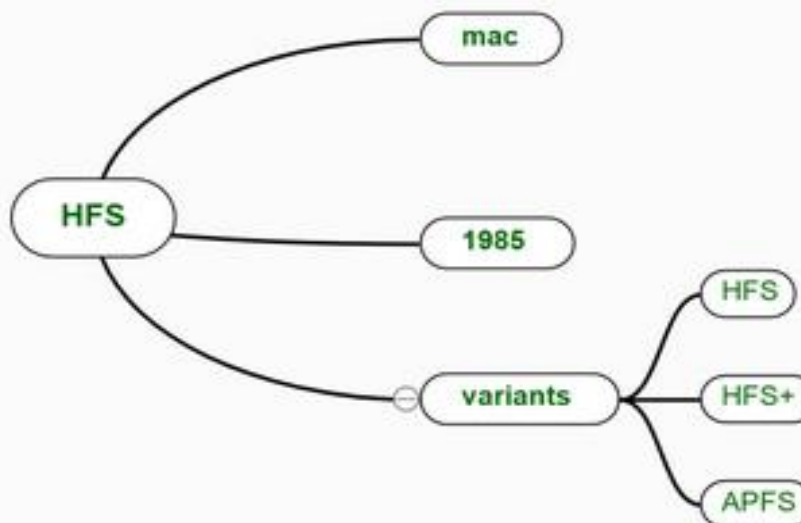
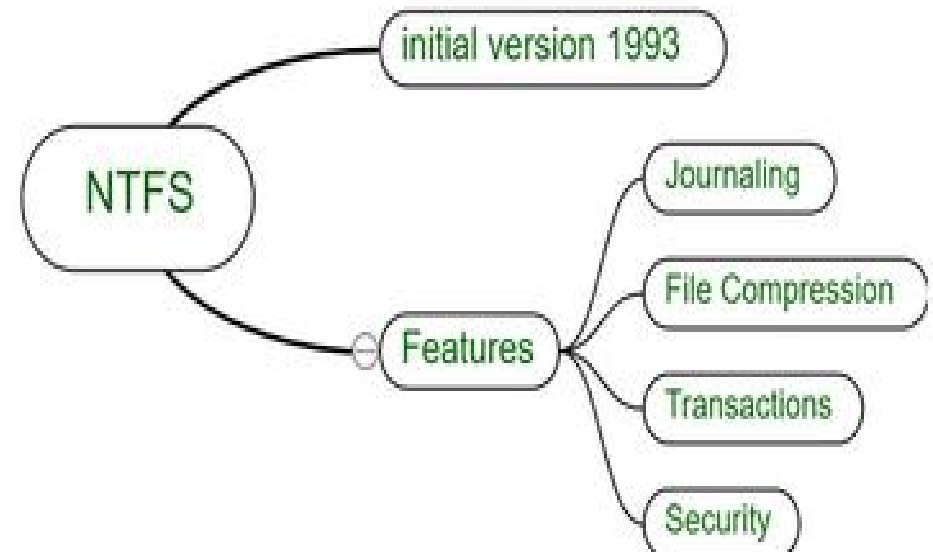
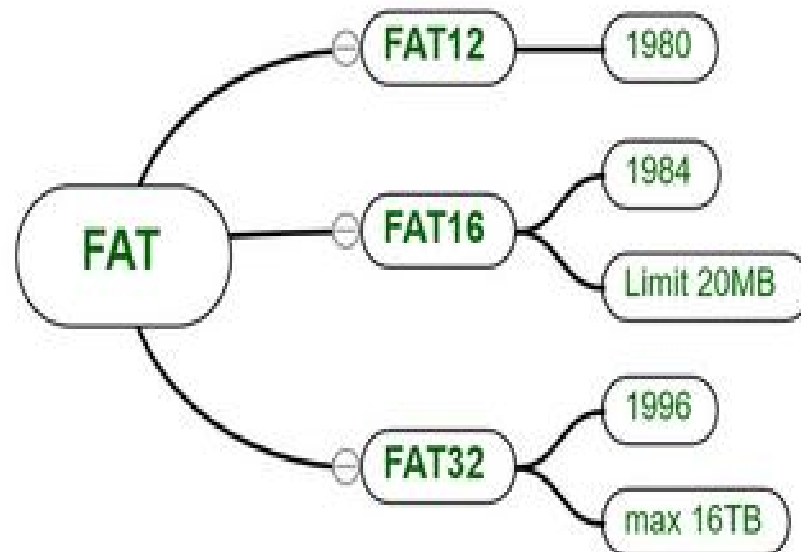
Los mas populares



- FAT (File Allocation Table).
- NTFS (New Technology File System).
- HFS (Hierarchical File System).
- EXT (Extended File System).



Los mas populares





Aspectos a evaluar para un F.S.

- **Manejo de espacio**
 - Manejo de nombres de los objetos (directorio y archivos)
 - La estructura de directorios utilizados
 - Metadata que se le puede asociar a los objetos (directorio y archivos)
- **Utilidades:**
 - Creación, Chequeo, Defrag, Tunning, Resize, Undelete, etc.
- **Mecanismos de permisos y restricción.**
 - ACL
 - Capacidades
- **Integridad de la información**
- **Limitiaciones impuestas por diseño.**
- **Extras**
 - Encriptado a nivel archivo o directorio
 - Compresión a nivel archivo o directorio
 - Desduplicacion



Diseño de un FS

EL CONCEPTO DE BLOQUE

Un bloque es la unidad mínima de asignación de espacio en un soporte específico.

Siempre que se pida un cierto espacio en un soporte de almacenamiento, el sistema de archivo proporciona una cantidad que, por lo regular, es mayor que la que se pide. Problema **Fragmentación Interna**.

Ejemplo de Archivo lógico Vs Físico:

```
dd if=/dev/urandom of=./uno.txt bs=512 count=1
dd if=/dev/urandom of=./dos.txt bs=512 count=8
dd if=/dev/urandom of=./tres.txt bs=512 count=9
ls -l
du -h *
```

Ejemplo de direccionamiento:

Lógico Vs Físico

```
blockdev --getss /dev/sdb
blockdev --getpbsz /dev/sdb
```

Manipulación de Archivos:

Entrada/salida lógica	Relación con Usuarios y Aplicaciones
Supervisor de E/S básico	Comienza y termina E/Salidas
Sistema de archivo básico (E/S física)	Bloques de datos, desconoce estructura, de datos.
Controladores	(device driver) directo sobre el Hardw.



Diseño de un FS

ATRIBUTOS DE LOS ARCHIVOS:

Nombre del archivo: Univoco dentro de un directorio.

Tipo de archivo: Formato (texto, binario, Enlace simbólico, etc)

Datos de la creación o modificación: fecha, hora, etc.

Tamaño del archivo: Especificado en bytes, bloques, etc.

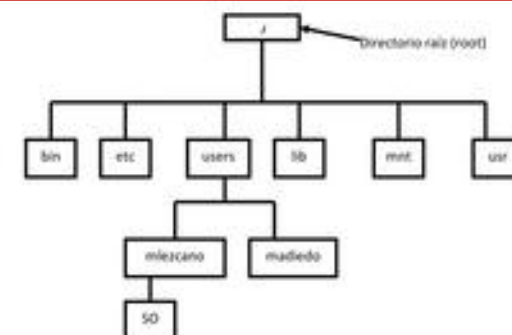
Atributos de protección. Permisos.

Ejemplo comandos:

dir en windows
ls en linux

ESTRUCTURA DE DIRECTORIO:

- Todo sistema de archivo posee, como parte de su organización, una estructura de datos denominada tabla de directorio.
- Todos los medios de almacenamiento poseen esa estructura de datos, que se construye cuando se le da formato al soporte.
- La forma que tenga el directorio depende del SO.



Una Estructura Posible (Solo didáctica)

NOMBRE	TIPO	DIRECCIÓN DE INICIO	FECHA	HORA	TAMAÑO	PROPIETARIO



Diseño de un FS

EL CONCEPTO DE BLOQUE

Un bloque es la unidad mínima de asignación de espacio en un soporte específico.

Siempre que se pida un cierto espacio en un soporte de almacenamiento, el sistema de archivo proporciona una cantidad que, por lo regular, es mayor que la que se pide. Problema **Fragmentación Interna**.

ASIGNACIÓN DE ESPACIO:

Para pensar...

Cuando se cree un nuevo archivo, ¿se le asignará todo el espacio que necesita?

¿Qué tamaño debe tener un bloque?

¿Que pasa si los Bloques son muy pequeños?

¿Que pasa si los Bloques son muy Grandes?

La idea es tener un tamaño optimo de Bloque para no provocar tanta fragmentación interna y para no tener que usar demasiados recursos para controlarlos.

No es tan fácil definir el tamaño “ideal” de un bloque.





Diseño de un FS

TIPOS DE ASIGNACIÓN DE ESPACIO:

- **Contigua:** Asigna un conjunto de bloques contiguos cuando se crea al archivo.
- **Enlazada:** Cada bloque tiene un puntero al próximo bloque y pueden estar en cualquier parte de la unidad de almacenamiento.
- **Indexada:** Cada archivo usa uno de los bloques para almacenar las localizaciones de los bloques que pertenecen al archivo.

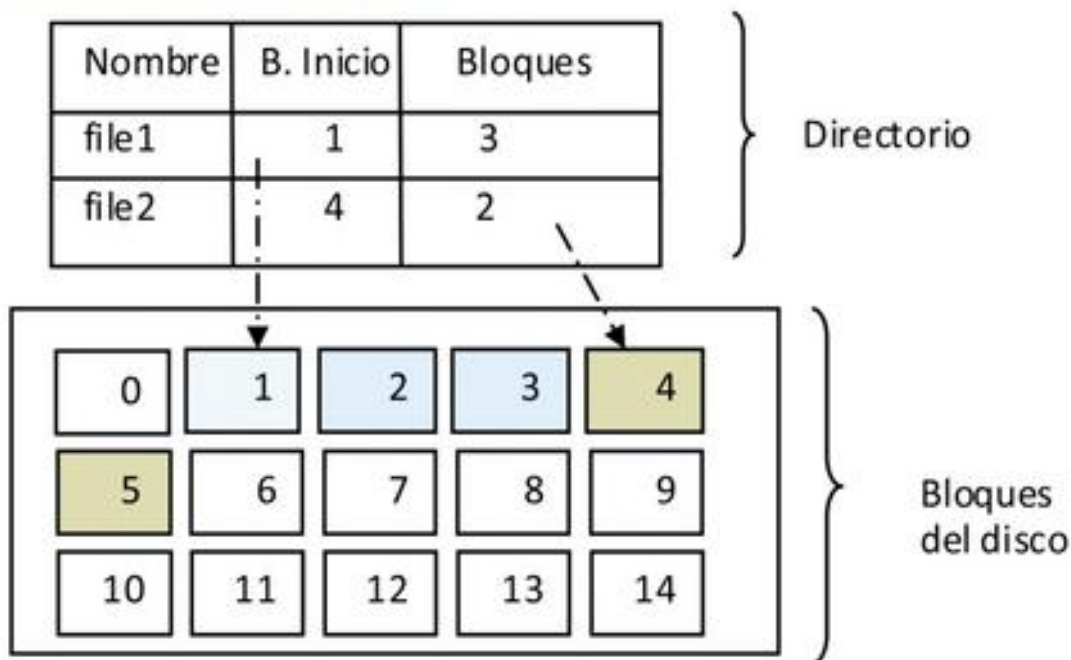
CADA UNA TIENE:

- **Ventajas y desventajas.**
- **Estrategias:** Cada Estructura tiene asociada sus estrategias de manipulación.
- **Problema:** Como pensar un F. S. multi-proposito o bien un F. S. especializado.



Diseño de un FS

ASIGNACIÓN CONTINUA:



Desventajas:
El crecimiento.
Fragmentación Externa.

DES FRAGMENTAR:
Cual es el problema de la des fragmentación?

Ventajas

Acceso directo a los datos.

Rápido: Se efectúan menos movimientos mecánicos del cabezal de lectura/escritura.

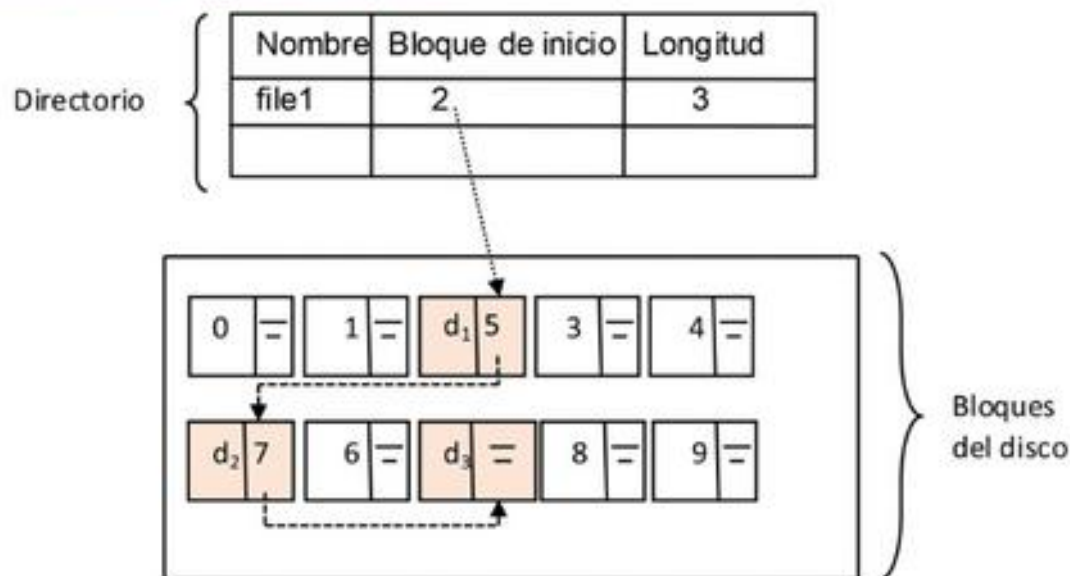
Fácil Recuperación de datos Borrados.





Diseño de un FS

ASIGNACIÓN ENLAZADA:



Problema:

¿ Que tipo de acceso utiliza?
¿ Mejora o empeora la situación?
Comparando con la anterior....

Ventajas

Crecimiento mientras existan bloques libres.
Elimina el problema de la fragmentación externa.

Desventajas:

No Tiene acceso directo; para acceder al bloque n hay que recorrer los $n-1$ bloques que le preceden.
Lentitud: Por dispersión de los bloques.
Gasta espacio adicional: Cada bloque contiene un campo dedicado al puntero.



Diseño de un FS

ASIGNACIÓN INDEXADA:

Nombre	B. Inicio
file1	4

Directorio

Bloque 4 ampliado

0	1	2	3	4
5	6	7	8	9
10	11	12	13	14

5
9
13
Nulo
Nulo
Nulo



Ventajas

No provoca fragmentación externa.

Permite el acceso aleatorio.

Los archivos pueden crecer mientras haya espacio y forma de hacer referencia a sus bloques.

Desventajas

Archivos están dispersos.

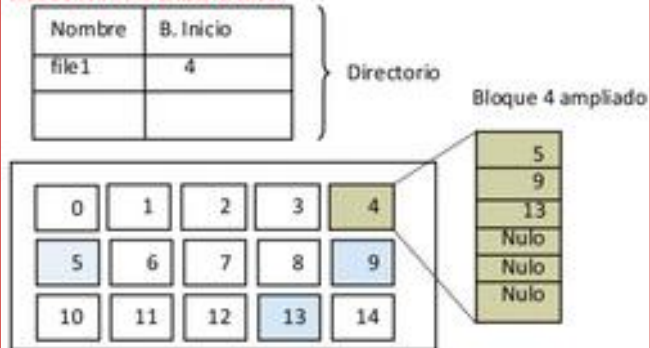
Acceso mas lento que la Continua.

LA INDEXADA ES LA MÁS UTILIZADA ACTUALMENTE



Diseño de un FS

ASIGNACIÓN INDEXADA:



Algunas Reflexiones:

- Los Archivos del Directorio apuntan a un Nodo de Direcciones.
- El Primer Nodo de Direcciones contiene todas las direcciones de bloque del Archivo.
- Que hacer Si el Archivo requiere mas bloques que la cantidad que tiene el Nodo de direcciones?



Algunas Aplicaciones de este modelo:

- La solución que ofrece el SO Unix se basa en esta idea.
- La Solución de Windows, en sus FAT, se basa en Generar una gran tabla de índices para todos los archivos del F. S.
 - FAT tiene una lista enlazada que es necesario recorrer para localizar los bloques de datos.
 - **FAT NO ES UNA LISTA ENLAZADA**

La solución del tipo de asignación depende del problema



Diseño de un FS

CONTROL DE ESPACIO LIBRE:

Para controlar el espacio libre, se pueden usar varios tipos de estructuras de datos.

- Utilizar la Misma estructura. (Caso de la FAT). Poco eficiente.
- Control a partir de una Tabla.
- Utilizando un mapa de Bits.

CONTROL A PARTIR DE UNA TABLA:

DIRECCIÓN DE INICIO	TAMAÑO
10	23
80	12

Tener cuidado con la eliminación de Archivos y revisar antes de actualizar la tabla de Espacio libre.

CONTROL UTILIZANDO MAPA DE BITS:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
1	1	1	0	1	0	0	0	1	1	0	0	0	0	0	1	1	1	1	0

La longitud del mapa es igual a la cantidad de Bloques del FS.
Las estrategias cambian de acuerdo a si es Contigua, Enlazada, o Indexada.

DES FRAGMENTAR:

Cual es el problema de la des fragmentación?





Caso de Estudio F. S.

File Allocation Table (FAT)

Tabla de Directorios:

Nombre	Extensión	Atributos	Reservado	Hora	Fecha	Cluster	Longitud
carta	doc					12	
pepe	txt					23	

* atributos (oculto, de lectura solamente, sistema y archivo)

Tabla de Datos

	0	1	2	3	4	5	6	7	8	9
0	0	0	eof	8	0	0	0	0	2	0
1	0	0	3	0	0	0	0	0	0	0
2	0	0	0	30	0	0	0	0	0	0
3	31	38	0	0	0	0	0	0	50	0
4	0	0	0	0	0	0	0	0	0	0
5	eof	0	0	0	0	0	0	0	0	0
6	0	0	0	0	0	0	0	0	0	0
7	0	0	0	0	0	0	0	0	0	0
8	0	0	0	0	0	0	0	0	0	0
9	0	0	0	0	0	0	0	0	0	0



Doble Propósito:

- Localizar los bloques de datos de cada Archivo.
- Controlar el espacio libre.

Los 2 primeros Clúster no se utilizan (el 1ro es MBR)

El S.O. realiza los cálculos para ubicar físicamente el Bloque en el disco.



Caso de Estudio F. S.

El sistema de archivos FAT se compone de cuatro secciones:

- **El sector de arranque**
 - Siempre es el primer sector de la partición (volumen) e incluye información básica, punteros a las demás secciones, y la dirección de la rutina de arranque del sistema operativo.
 - Contiene además la geometría del disco. (para el cálculo del SO).
- **La región FAT**
 - Contiene dos copias de la tabla de asignación de archivos (por motivos de seguridad).
 - Estos son mapas de la partición, indicando qué clusters están ocupados por los archivos.
- **La región del directorio raíz.**
 - Es el índice principal de carpetas y archivos.
- **La región de datos.**
 - Es el lugar donde se almacena el contenido de archivos y carpetas.
 - Por tanto, ocupa casi toda la partición.
 - El tamaño de cualquier archivo o carpeta puede ser ampliado siempre que queden suficientes clúster libres.
 - Cada clúster está enlazado con el siguiente mediante un puntero.
 - Si un determinado clúster no se ocupa por completo, su espacio remanente se desperdicia.

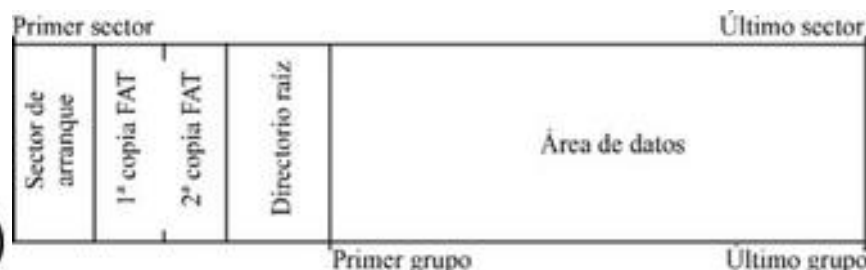




Caso de Estudio F. S.

FAT - Estructuras

- Boot (sector de arranque)
- Región FAT (dos copias)
- Directorio raiz (Índice principal de carpetas y directorios)
- Datos



00004000	f0 ff ff 0f	ff ff ff 0f	f8 ff ff 0f	ff ff ff 0f	...	...	...	...
00004010	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	...	...	...	...
00004020	09 00 00 00	0a 00 00 00	0b 00 00 00	0c 00 00 00	...	...	...	...
00004030	0d 00 00 00	0e 00 00 00	0f 00 00 00	10 00 00 00	...	...	...	...
00004040	11 00 00 00	12 00 00 00	13 00 00 00	ff ff ff 0f	...	...	...	...
00004050	15 00 00 00	16 00 00 00	17 00 00 00	18 00 00 00	...	...	...	...
00004060	19 00 00 00	1a 00 00 00	1b 00 00 00	1c 00 00 00	...	...	...	...
00004070	1d 00 00 00	1e 00 00 00	1f 00 00 00	20 00 00 00	...	...	...	...
00004080	21 00 00 00	ff 00 ff ff	0f 00 00 00	00 00 00 00	!	...	...	...
00004090	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	...	...	...	...

FAT ID	Indicación de fin de cadena
Clusters vacíos	El directorio raiz cabe en un sólo cluster
Primer cadena (un solo cluster: 0003)	Segunda cadena (12 clusters: 0008 a 0013)
Tercera cadena (16 clusters: 0014 a 0021)	



Caso de Estudio F. S.

La FAT

- Cada entrada, campo o celda, contiene información sobre un clúster:
 - La dirección del siguiente clúster en la cadena.
 - Si es pertinente, la indicación de "fin de archivo" (EOF).
 - Un carácter especial para indicar que el clúster es defectuoso.
 - Un carácter especial para indicar que el clúster está reservado (es decir, ocupado por un archivo).
 - El número cero para indicar que el clúster está libre.
- El tamaño de estas entradas también depende de la variante FAT
 - FAT12 de 12 bytes.
 - FAT16 de 16 bytes.
 - FAT32 de 32 bytes.
- Una entrada representa un bloque o cluster del volumen.



Se Mantienen 2 FAT ya que por seguridad Resguarda una copia.



Caso de Estudio F. S.

FAT Directorio Raíz

Este índice es un tipo especial de archivo que almacena las sub carpetas y archivos que componen cada carpeta.

Cada entrada del directorio contiene:

- **Nombre del archivo o carpeta (máximo 8 caracteres)**
- **Extensión (máximo 3 caracteres)**
- **Atributos (archivo, carpeta, oculto, del sistema, o volumen)**
- **Fecha**
- **Hora**
- **Dirección del primer cluster donde están los datos.**
- **Tamaño que ocupa.**

El directorio raíz ocupa una posición concreta en el sistema de archivos, pero los índices de otras carpetas ocupan la zona de datos como cualquier otro archivo. Los nombres largos se almacenan ocupando varias entradas en el índice para el mismo archivo o carpeta.

En realidad los directorios son un tipo especial de fichero, que en lugar de datos de usuario, contienen metadatos (punteros a otros ficheros).





Caso de Estudio F. S.



Sistema de Archivos	Tamaño Mínimo de Partición	Tamaño Máximo de Partición	Tamaño de Cluster Común	Aplicación Típica
FAT12	720 KB	16 MB	0.5 KB (512 bytes)	Disquetes (hasta 1.44 MB)
FAT16	32 MB	2 GB	2 KB a 32 KB	Primeros discos duros (DOS)
FAT32	512 MB	2 TB	4 KB a 32 KB	Discos duros y USB modernos

Tamaño de la FAT según el sistema de archivos

Sistema de Archivos	Bits por Entrada (Cluster)	Máximo de Clusters	Tamaño Máximo de la FAT
FAT12	12 bits (1.5 bytes)	4,096	Hasta 6 KB aproximadamente
FAT16	16 bits (2 bytes)	65,536	Hasta 128 KB aproximadamente
FAT32	32 bits (4 bytes)	~268,435,456	Hasta 1 GB aproximadamente

Cálculo del Tamaño de la FAT

Para calcular el tamaño de la FAT, usamos esta fórmula aproximada:

$$\text{Tamaño de la FAT} = \text{Número de clusters} \times \text{Tamaño de cada entrada}$$



Caso de Estudio F. S.

Fórmula General para el Tamaño de la Partición en FAT16

Tamaño total de partición = Espacio de datos + $2 \times$ Tamaño de la FAT + Otros sectores

Determina el tamaño del cluster: En FAT16, el tamaño del cluster varía según el tamaño de la partición. Aquí hay una guía común:

Tamaño de Partición	Tamaño del Cluster
Hasta 32 MB	512 bytes
32 MB - 128 MB	2 KB
128 MB - 256 MB	4 KB
256 MB - 512 MB	8 KB
512 MB - 1 GB	16 KB
1 GB - 2 GB	32 KB



Calcula el número de clusters:

$$\text{Número de clusters} = \frac{\text{Espacio de datos}}{\text{Tamaño del cluster}}$$

Calcula el tamaño de la FAT:

- En FAT16, cada entrada de la FAT ocupa **2 bytes** (16 bits).

$$\text{Tamaño de la FAT} = \text{Número de clusters} \times 2 \text{ bytes}$$



Caso de Estudio F. S.

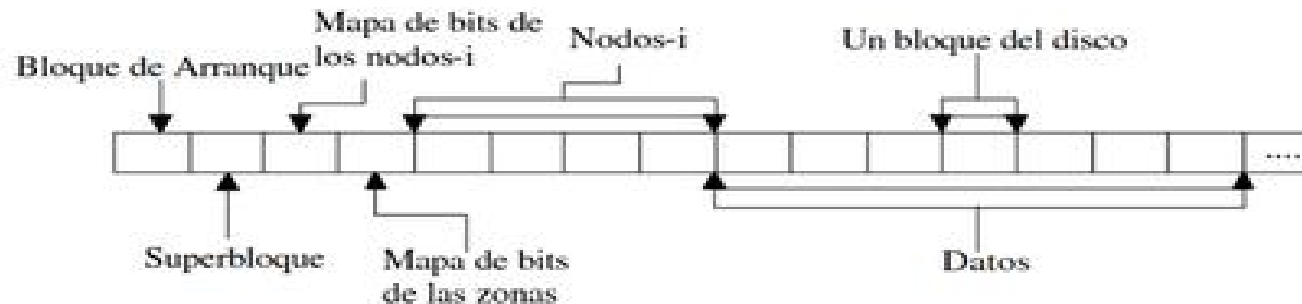


	FAT12	FAT16	FAT32
Desarrollador	Microsoft		
Nombre completo	Tabla de Asignación de Archivos		
	(versión de 12 bits)	(versión de 16 bits)	(versión de 32 bits)
Introducido	FAT12: 1980-08 (SCP 86-DOS 0.42)	FAT16: 1984-08 (PC DOS 3.0), FAT16B: 1987-11 (Compaq MS-DOS 3.31)	FAT32: 1996-08 (Windows 95 OSR2)
Identificador de partición	0x01 (MBR)	0x04, 0x06, 0x0E (MBR)	0x0B, 0x0C (MBR) EBD0A0A2-B9E5-4433-87C0-68B6B72699C7 (GPT)
Estructuras	FAT12	FAT16	FAT32
Contenido de carpeta	Tabla		
Ubicación de archivo	Lista enlazada		
Bloques defectuosos	Lista enlazada		
Limites	FAT12	FAT16	FAT32
Tamaño máximo de archivo	32 MiB	2 GiB (4 GiB - 1)	4 GiB - 1
Número máximo de archivos	4.068 for 8 KiB clusters	65.460 for 32 KiB clusters	268.173.300 for 32 KiB clusters
Longitud máxima del nombre de archivo	8.3 (11) o 255 caracteres cuando se usan LFNs (Long File Names)		
Tamaño máximo del volumen	32 MiB	2 GiB	2 TiB



Caso de Estudio F. S.

Esquema de un Sistema de Archivos Basado en Minix



Bloque de arranque

Cada sistema de archivo comienza con un bloque de arranque, el cual ocupa siempre un sólo bloque.

Superbloque

Ocupa un solo bloque. Contiene información relativa al tamaño de las distintas partes del sistema de archivos.

Mapa de bits de los nodos-i

Se utiliza para llevar el control de los nodos-i libres y ocupados. Por cada nodo-i existe un bit en dicha tabla que indica su estado. 0,1.

Mapa de bits de zonas

Se utiliza para llevar el control de las zonas de datos. Su funcionamiento es igual del mapa de bits de los nodos-i.

Nodos-i

Son estructuras en las que se encuentra información de cada fichero físico existente en el sistema de archivos, principalmente la relativa a la situación de las zonas de datos de dicho fichero.

Datos

Es el conjunto de bloques que se utiliza para el almacenamiento de los ficheros.

Basado en Minix

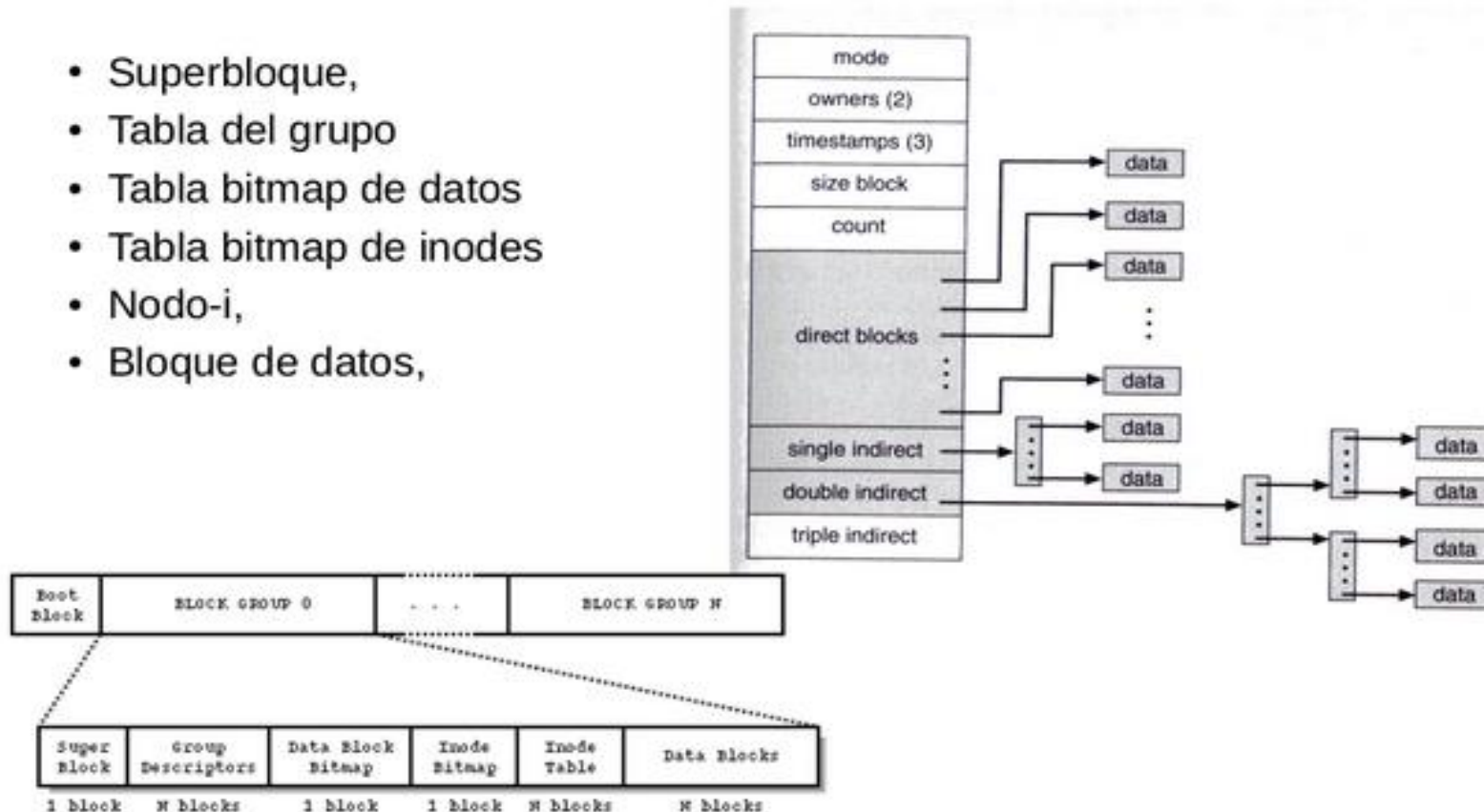


Caso de Estudio F. S.



Ext2/3/4

- Superbloque,
- Tabla del grupo
- Tabla bitmap de datos
- Tabla bitmap de inodes
- Nodo-i,
- Bloque de datos,



El superbloque tiene información del sistema de archivos en conjunto, como su tamaño (la información precisa aquí depende del sistema de archivos).



Caso de Estudio F. S.

Características Ext4

- Permite archivos de 1(EiB) y hasta 16(TiB)
- Extends
- Compatibilidad con Ext3/2
- Pre alojamiento de espacio
- Alojamiento demorado
- Sin limitaciones de directorios
- Mejoramiento en el timestamp
- Encriptación transparente
- Journal Checksum

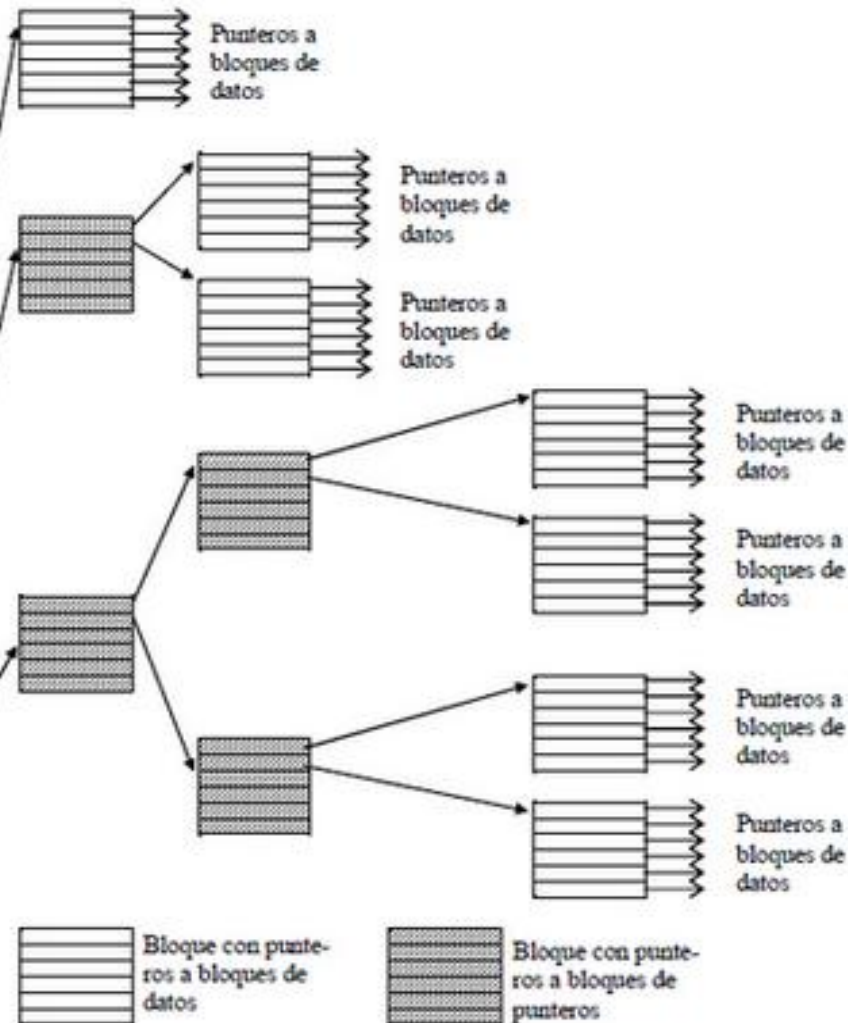




Caso de Estudio F. S.



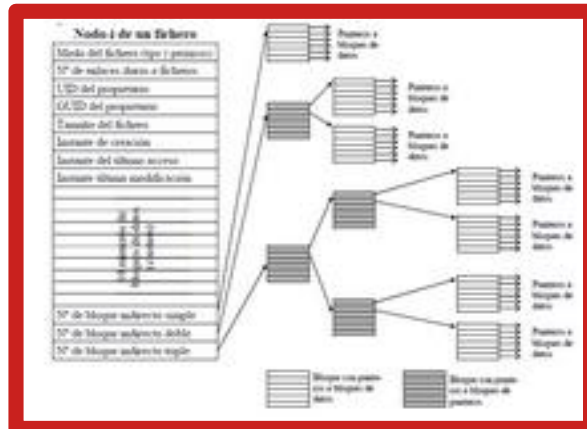
Nodo-i de un fichero	
Modo del fichero (tipo y permisos)	
Nº de enlaces duros a ficheros	
UID del propietario	
GUID del propietario	
Tamaño del fichero	
Instante de creación	
Instante del último acceso	
Instante última modificación	
10 números de bloques de datos (clusters)	
Nº de bloque indirecto simple	
Nº de bloque indirecto doble	
Nº de bloque indirecto triple	



Donde está el Nombre del Archivo?



Caso de Estudio F. S.



Inode: Cada archivo está representado por una estructura llamada un inode. Cada inode contiene la descripción del fichero, tipo de archivo, los derechos de acceso, marcas de tiempo, el tamaño y los punteros a los bloques de datos.

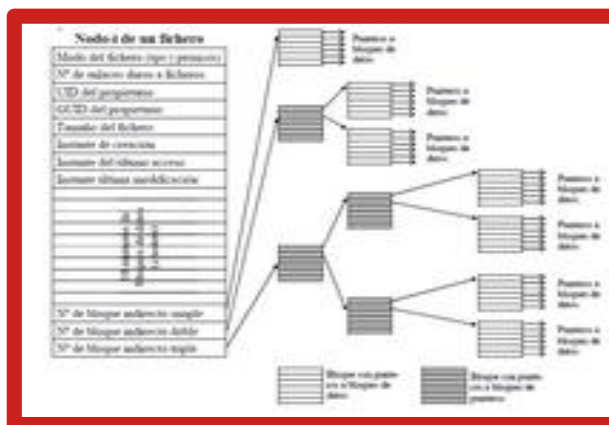
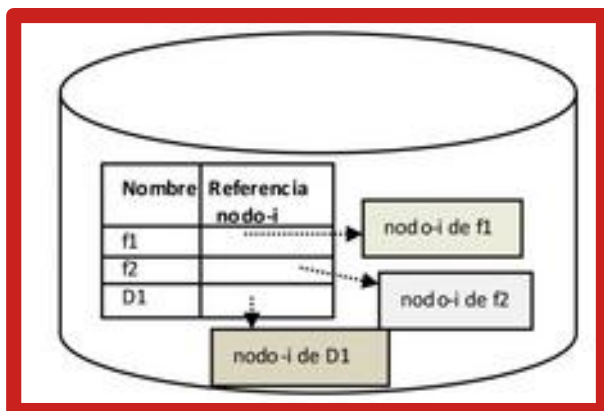
Un nodo-i tiene toda la información de un archivo, **salvo su nombre**. El nombre se almacena en el directorio, junto con el número de nodo-i. Una entrada de directorio consiste en un nombre de archivo y el número de nodo-i que representa al archivo.

El nodo-i contiene los números de varios bloques de datos, que se utilizan para almacenar los datos en el archivo. Sólo hay espacio para unos pocos números de bloques de datos en el nodo-i; en cualquier caso, si se necesitan más, más espacio para punteros a los bloques de datos son colocados de forma dinámica.

Estos bloques colocados dinámicamente son bloques indirectos; el nombre indica que para encontrar el bloque de datos, primero hay que encontrar su número en un bloque indirecto.



Caso de Estudio F. S.

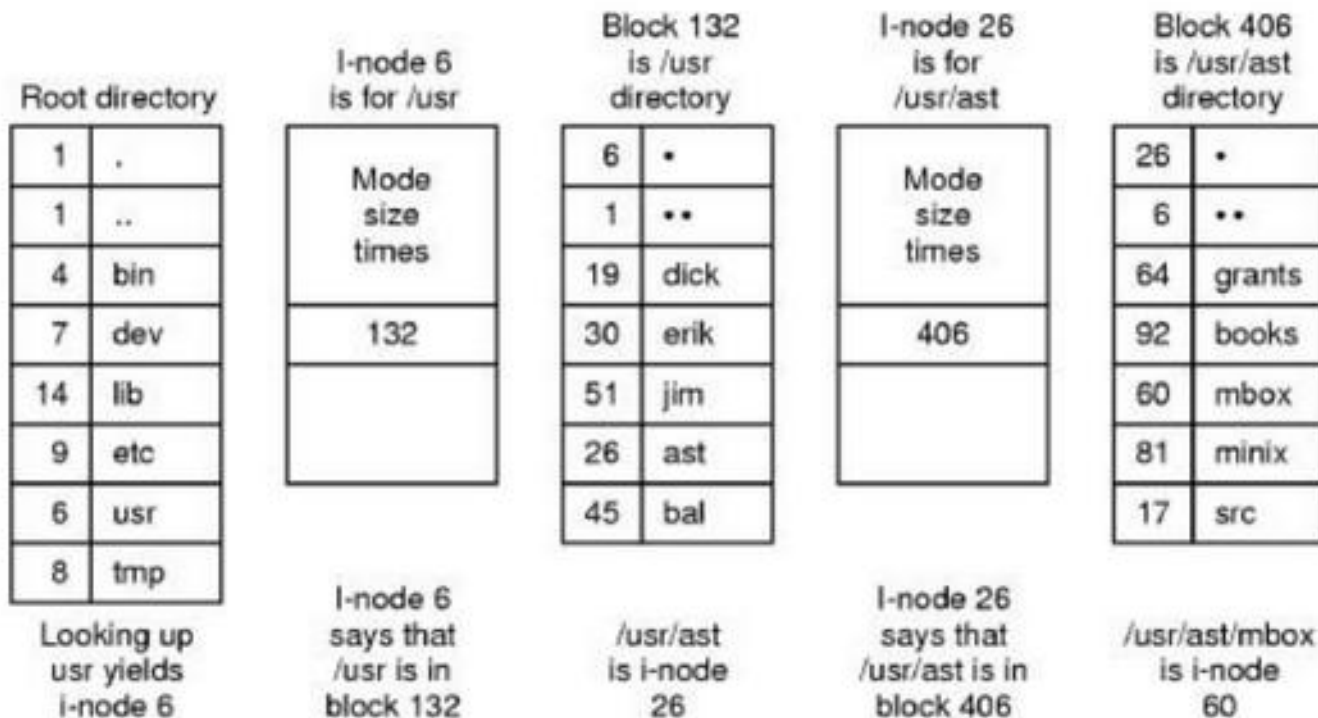


- **Directorios:** Los directorios se estructuran en un árbol jerárquico. Cada directorio puede contener archivos o directorios. Los directorios son un tipo especial de archivos.
- **Enlaces:** Sistemas de archivos Unix implementar el concepto de enlace. Varios nombres se pueden asociar con un inodo. El inodo contiene un campo que contiene el número asociado con el archivo.
- **Archivos especiales de dispositivo:** En los sistemas operativos tipo Unix, los dispositivos pueden acceder a través de ficheros especiales. Un archivo especial de dispositivo no utiliza ningún espacio en el sistema de ficheros. Es sólo un punto de acceso al controlador de dispositivo. Existen dos tipos de archivos especiales: carácter y bloquear archivos especiales. La primera permite operaciones I / O en modo de caracteres mientras que el segundo requiere datos a escribir en modo bloque a través de las funciones de caché del búfer. Cuando una solicitud de E / S se realiza en un archivo especial, se envía a un (pseudo) controlador de dispositivo. Un archivo especial hace referencia a un número importante, que identifica el tipo de dispositivo, y un número menor de edad, que identifica la unidad.



Caso de Estudio F. S.

apertura del archivo /usr/ast/mbox





Caso de Estudio F. S.

Estructura de Ext4

[https://ext4.wiki.kernel.org/index.php/Ext4 Disk Layout](https://ext4.wiki.kernel.org/index.php/Ext4_Disk_Layout)

Enlaces:

- <http://web.mit.edu/tytso/www/linux/ext2intro.html>
- <http://www.nongnu.org/ext2-doc/ext2.html#DEFINITIONS>
- <http://upload.wikimedia.org/wikipedia/commons/a/a2/Ext2-inode.gif>
- <http://cs.smith.edu/~nhowe/262/oldlabs/img/ext2.png>
- <https://www.cs.cmu.edu/~mihaib/fs/fs.html>





Caso de Estudio F. S.



Del ejercicio 5 representese mediante un esquema esa misma situación del disco pero suponiendo que en vez de una FAT se dispone de un sistema de asignación tipo UNIX. Supóngase una estructura de inodo con 3punteros directos, 1 índice simple indirecto, 1 índice doble indirecto y 1 índice triple indirecto.

Dir Simplificada	FAT		Asignado a
A.txt → 6	0	X	
B.txt → 8	1	EOF	C.txt
C.txt → 12	2	BAD	
	3	EOF	
	4	15	A.txt
	5	FREE	
	6	4	A.txt
	7	13	B.txt
	8	7	B.txt
	9	EOF	A.txt
	10	14	C.txt
	11	FREE	
	12	16	C.txt
	13	EOF	B.txt
	14	1	C.txt
	15	9	A.txt
	16	10	C.txt



Caso de Estudio F. S.



Tabla de inodos

Inode0 (root directory)

Tipo	Dir
P. Dir 1	Bdatos 0
P. Dir 2	Null
P. Dir 3	Null
P. Ind. Simple	Null
P. Ind. Doble	Null
P. Ind. Triple	Null

Inode1 (root directory)

Tipo	Dir
P. Dir 1	Bdatos 1
P. Dir 2	Bdatos 2
P. Dir 3	Bdatos 3
P. Ind. Simple	Bdatos 4
P. Ind. Doble	Null
P. Ind. Triple	Null

Tabla de bloques de datos

Bloque Datos0

P. a Tabla inodo	Nombre Archivo
Inode1	A.txt
Inode2	B.txt
Inode3	C.txt

Bloque Datos1

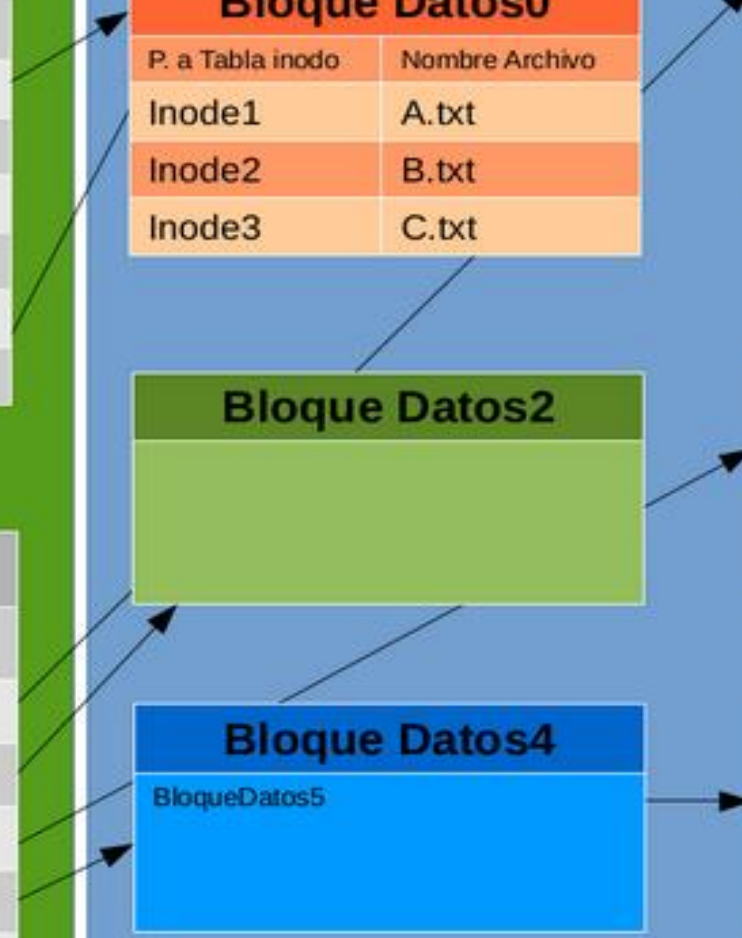
Bloque Datos2

Bloque Datos3

Bloque Datos4

BloqueDatos5

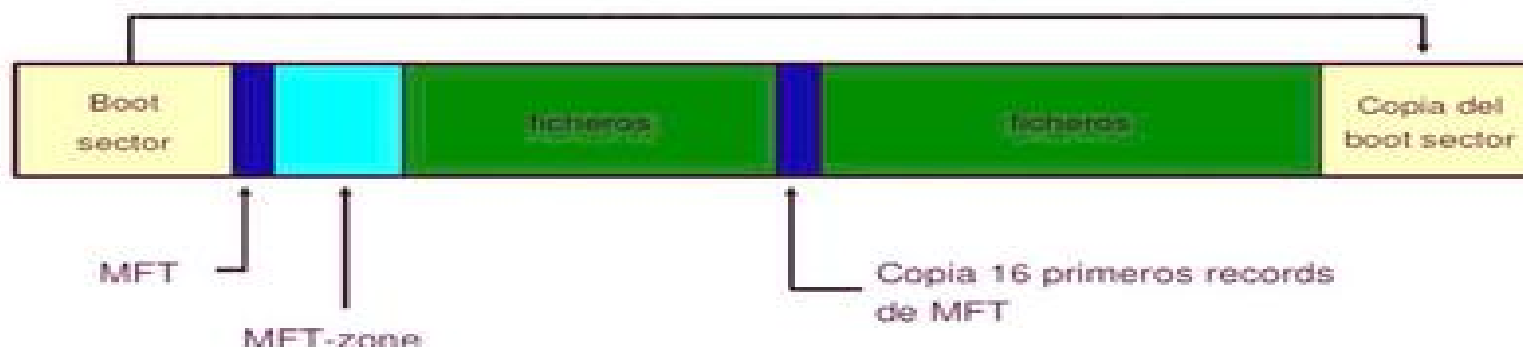
Bloque Datos5





Caso de Estudio F. S.

Estructura de un volumen NTFS:



En **NTFS** todo son archivos, y existen dos categorías, "Metadatos" (Archivos que contienen información sobre el volumen) y "Normal" (Archivos normales que contienen sus datos)

La principal estructura de datos administrativos es la tabla **MFT** (Master File Table) que permite registrar los atributos del archivo y punteros a los bloques, de los archivos del volumen. Cumple una función similar a la tabla de FAT, pero es mucho más completa y sofisticada.



Caso de Estudio F. S.

Los archivos de Metadatos más importantes son los siguientes:

1. **\$MFT.** - Tabla maestra de archivos - Un índice de todos los archivos .
2. **\$MftMirr.** - Una copia de seguridad de los 4 primeros registros de la MFT .
3. **\$Logfile.** - Archivo de registro de transacciones.
4. **\$Volumen.** - Número de serie, el tiempo de creación,...
5. **\$AttrDef.** - Definiciones de atributos.
6. **\$(Punto).** - Directorio raíz del disco.
7. **\$Bitmap.** - Contiene un mapa de clusters de volumen (en uso vs libre).
8. **\$Boot.** - Registro de arranque del volumen.
9. **\$BadClus.** - Listas clústeres defectuosos en el volumen .
10. **\$I30.** - Índice de elementos borrados.
11. **\$.Secure.** - Lista de control de acceso.
12. **\$.Upcase.** - Usado para convertir todos los caracteres en mayúscula a caracteres unicode minúscula.
13. **\$.Extend.** - Listado de cuotas o identificadores de objetivos



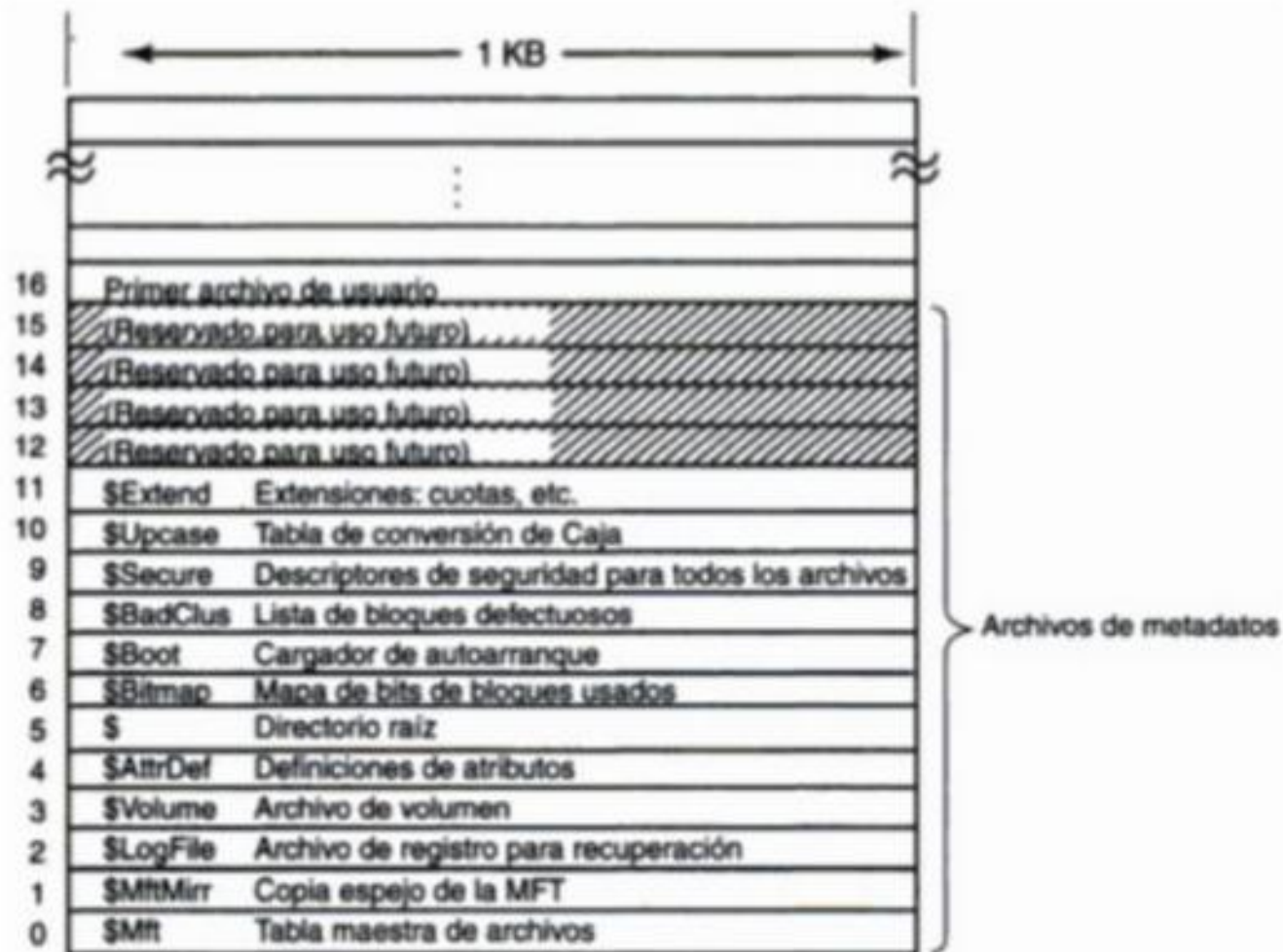
MFT (Metadata File Table)

- Habrá una fila o record (entrada en la tabla) por cada archivo que exista en el sistema de archivos.
- Ahora bien, el concepto de archivo es bastante amplio: toda entidad almacenada individualmente es un archivo. La propia MFT es considerada un archivo y tiene su propia entrada en la MFT.
- Las primeras 16 entradas de la MFT están reservadas para almacenar archivos del sistema, es decir, información sobre el propio sistema de archivos. El resto de entradas se corresponden con los archivos y carpetas de datos.



Caso de Estudio F. S.

Detalles de la tabla MFT:





Caso de Estudio F. S.

NTFS

System File	File Name	MFT Record	Purpose of the File
Master file table	\$Mft	0	Contiene en el registro de archivo todos los archivos y directorios en el volumen NTFS. Si supera el registro se utiliza otro.
MTF Mirro	\$MtfMirr	1	Duplica el registro MFT para mejor los problemas de falla.
Log file	\$LogFile	2	Contiene información para recuperacion. Mantiene consistencia de los metadatos despues de una falla.
Volume	\$Volume	3	Contine información sobre el volumen, como el nombre o la versión
Attribute definitions	\$AttrDef	4	Lista de nombres, numeros y descriptores.
Root file name index	.	5	La carpeta raiz
Cluster bitmap	\$Bitmap	6	Representa en bits los cluster usados y no usados.
Boot sector	\$Boot	7	Incluye el BPB y el codigo de carga adicional si el volumen es booteable.
Bad cluster file	\$BadClus	8	Contine la lista de clusters dañados.
Security file	\$Secure	9	Contiene los descriptores unicos de seguridad para todos los archivos del volumen.
Uppcase table	\$Uppcase	10	Convierte caracteres para machear los caracteres UNICODE upcase.
NTFS extension file	\$Extend	11	Información adicional, como cuotas.
		12-15	Reservado para futuros usos.





Caso de Estudio F. S.

Características NTFS

- Cuotas
- Maximo Tamaño: 256TiB con cluster 64KiB. Tamaño maximo del archivo: 16EiB a 1KiB o 16TiB a 64KiB.
- Date resolution 100ns.
- Jornaling
- Hard Links
- Alternative Data Stream: Permite mas de un archivo con el mismo nombre. Formato filename:streamname. Comandos del PowerShell Add-Content, Clear-Content, Get-Content, Get-Item, Out-String, Remove-Item, Set-Content.
- Compresión
- Sparse File. Archivos con espacios libres.
- Volumen Shadow Copy (o snapshot). Realiza una copia de todo archivo o directorio modificado.
- Encryption
- Puntos de Montado
- Deduplicación.



Estructura NTFS

[https://technet.microsoft.com/en-us/library/cc781134\(WS.10\).aspx](https://technet.microsoft.com/en-us/library/cc781134(WS.10).aspx)

Enlaces Sugeridos:

<http://ftp.kolibrios.org/users/Asper/docs/NTFS/ntfsdoc.html>

<https://technet.microsoft.com/en-us/library/cc781134%28v=ws.10%29.aspx>

<http://www.tuxera.com/community/ntfs-3g-manual/>



Caso de Estudio F. S.

VIRTUAL FILE SYSTEMS

Es una capa virtual que sirve como capa de abstracción del sistema de archivo real. Esta capa es la que utiliza el usuario para acceder al sistema de archivo real. Esta capa permite controlar y/o manejar de la misma manera un sistema de archivo local como un sistema de archivo de red.

El primer mecanismo de sistema de archivo virtual en un sistema Unix fue introducido en el SunOS de 1985. Esto permitía acceder a su sistema de archivo local llamado UFS como acceder a sistemas remotos NFS en forma transparente.

Podría decirse que el servicio más importante que la capa VFS ofrece una caché de datos I/O uniforme. Linux mantiene cuatro cachés de datos de E/S: page cache, i-node cache, buffer cache y directory cache. El cache de páginas (page cache) combina datos de la memoria y archivos virtuales. El buffer cache es una interfaz con el dispositivo de bloques y cachea los meta-datos de discos de bloques. El cache de i-nodo mantiene el acceso reciente a los i-nodos de archivo. El cache de directorio (d-cache) mantiene en la memoria de un árbol que representa una parte de la estructura de directorios del sistema de archivos.

La estructura de VFS de linux se parece a la estructura de ext2.



Caso de Estudio F. S.

VFS Superblock

Todo sistema de archivo montado esta representado por el VFS superblock. El VFS contiene:

- Device/Dispositivo: Este es el identificador de dispositivo. por ejemplo /dev/sda1
- Puntero a inodos/Inode pointers: Apunta al primer inodo del sistema de archivo. El puntero de inodo cubierto representa el inodo del directorio donde esta montado dentro del sistema. Si es el root file system no tiene puntero cubierto.covered pointer,
- Tamaño de Bloque/Blocksize: es el tamaño de bloque en el sistema de archivo. ej 4096 bytes.
- Operaciones de superbloque/Superblock operations: Apunta a grupo de rutinas de superbloque. Entre otras cosas estas rutinas son las que permite la lectura y escritura de inodos y superbloques.
- Tipo de sistema de archivo/File System type: Es el puntero a los datos de estructura del tipo de sistema de archivo.
- Especifico del sistema de archivo/File System specific: Es el puntero a la información necesario para este sistema de archivo.



Caso de Estudio F. S.

El VFS inodo

Como el sistema de archivo Ext2, todo archivo, directorio y otros es representado por algún VFS inodo. FS inode.

La información de cada inodo VFS es construida desde la información extraída del sistema de archivo mediante rutinas específicas. Cada VFS inodo existe solo en memoria del sistema y es mantenida dentro del cache de inodo. VFS inodo contiene los siguientes datos:

- Dispositivo/device: Es el identificador de dispositivo donde se encuentra el inodo real.ode represents,
- Numero de Inodo/inode number: Es un numero unico de inodo dentro del sistema de archivo. Es una combinación entre el dispositivo y el el numero de inodo.
- Modo/mode: Como el EXT2 este campo describe los permisos al inodo.
- ID de usuario/user ids: Describe los ids de usuario
- Tiempos/times: De creación, modificación, y de escritura.
- Tamaño de Bloque/block size: Tamaño de bloques.
- Operaciones de inodo/inode operations: Es un puntero a la direcciones de los procedimientos de inodo. Por ejemplo la opeacion de truncar el archivo que representa el inodo.
- Cuenta/count: Es el numero de componetes de sistema que estan usando este inodo. Cuando la suma llega a cero puede ser descartado o reusado.
- lock: Este campo es usado cuando el inodo es lockeado cuando es usado por el sistema.
- Sucio/dirty: Indica si el inodo VFS ha sido escrito y el sistema de archivo por debajo necesita ser modificado.

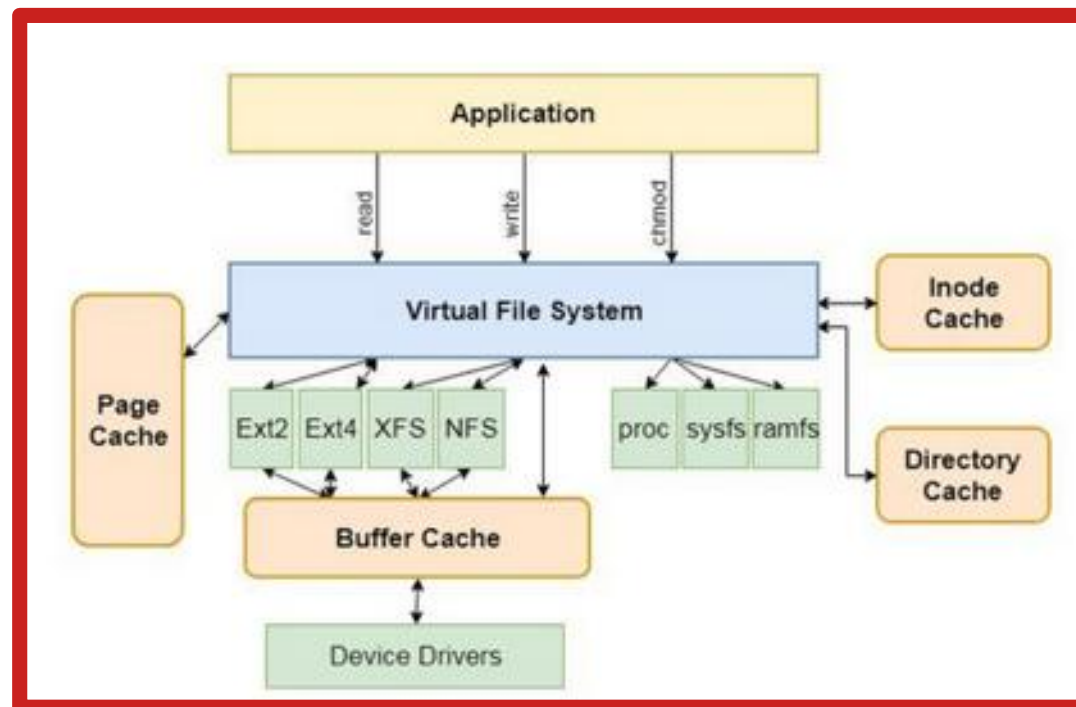


Caso de Estudio F. S.

Registro de sistema de archivos

Todo en linux todo nuevo sistema de archivo debe ser registrado. Los sistemas de archivos pueden ser creados como modulos y cargados bajo demanda. Cuando son cargados el kernel registra el sistema de archivo.

Uno puede ver si el archivo esta registrado haciendo un cat al archivo `/proc/filesystems`.





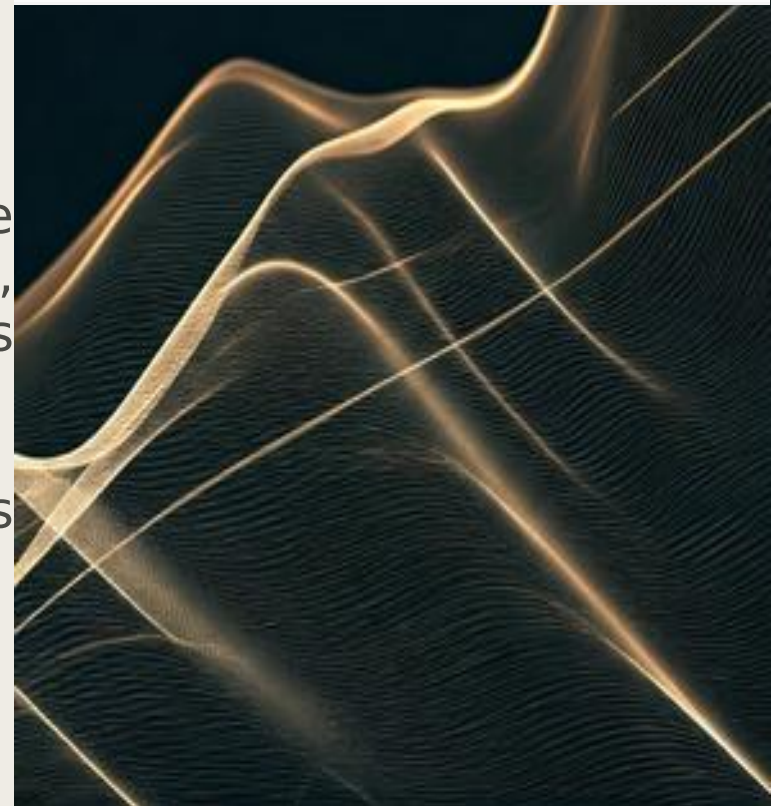
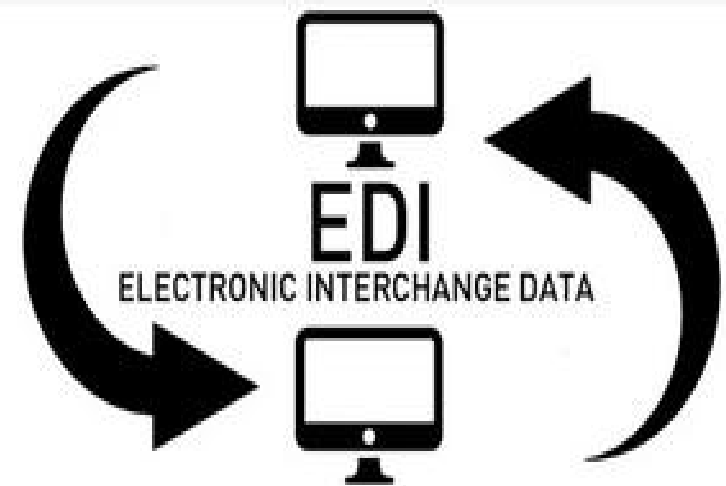
FORMATOS DE INTERCAMBIO DE DATOS

Introducción al Intercambio Electrónico de Datos (EDI)

El Intercambio Electrónico de Datos (EDI) es un sistema de intercambio de información comercial estructurada entre organizaciones.

EDI permite la transferencia electrónica de documentos comerciales, como facturas, pedidos y albaranes, entre sistemas informáticos.

El intercambio se puede realizar en distintos Formatos.





FORMATOS DE INTERCAMBIO DE DATOS



EDI es un formato estándar para intercambiar información entre dos organizaciones de forma electrónica en lugar de utilizar documentos en papel

UNB+UNOA+777+222+2702 :17+
999999'UNH+99999+ORDERS:1'BG
M+105+0001+270296'NAD+ST+++
mayor5'NAD+SE+++cerrada2'CUX
+ESP:OC'UNS+D/LIN+1+++333:EN+
555+44444'DTM+002+280296'UNS
+S'UNT+10+99999'UNZ+1+99999'

Todas las transacciones de EDI se definen mediante estándares de mensajes de EDI.

En general, hay dos tipos básicos de transmisión de EDI:

- Conexiones de punto a punto o directas: dos computadoras o sistemas se conectan sin intermediarios a través de Internet, generalmente con protocolos seguros.
- Red de valor añadido (VAN): una red de terceros gestiona la transmisión de datos, generalmente con un paradigma de buzón de correo.

En transacciones de **EDI**, la información se traslada directamente desde una aplicación informática en una organización a una aplicación de sistema en otra. Los **estándares de EDI** definen la ubicación y el orden de la información en un formato de documento.



FORMATOS DE INTERCAMBIO DE DATOS



Estandares EDI: (Documentos EDI estándar)

Los estándares definen las reglas a tener en cuenta para la emisión e interpretación de una transacción concreta.

Diferente tipos de estándares EDI:

Los desarrollados específicamente para industrias concretas.
Los extendidos en más sectores o regiones.

Los más utilizados son:

Estándar EDI - UN/EDIFACT

United Nations/Electronic Data Interchange for Administration, Commerce and Transport, desarrollado por Naciones Unidas durante la década de los 80 gracias al trabajo del grupo Working Party 4.

Se trata de un lenguaje EDI multi país y multi industria, muy usado en Europa especialmente en el sector Retail, primera industria en adoptarlo, si bien con el paso de los años, su uso se ha generalizado a otros ámbitos como la sanidad, logística y transporte o construcción.



FORMATOS DE INTERCAMBIO DE DATOS



Los más utilizados son:

Estándar EDI - ANSI ASC X12

American National Standards Institute (ANSI) creó a finales de los 70 el Accredited Standards Committee (ASC)

Un estándar de mensajería uniforme para el intercambio de documentos electrónicos.

ANSI X12 inicialmente como un estándar a emplear por empresas norteamericanas de diferentes sectores.

Más información sobre ANSI X12

Estándar EDI - ODETTE

Organization for Data Exchange by Tele Transmission in Europe (ODETTE)

Representa los intereses de la industria automovilística en Europa

Aplica un protocolo propio de comunicación: OFTP (Odette File Transfer Protocol)



FORMATOS DE INTERCAMBIO DE DATOS



Los más utilizados son:

Estándar EDI - UBL (Universal Business Language)

Basado en XML, diseñado inicialmente para el sector financiero. La Unión Europea, en el marco de su estrategia 2020, ha designado el UBL como el estándar de referencia para la contratación pública como base para definir los mensajes que deben intercambiar con sus proveedores.

Por supuesto que existen otros Estándares Internacionales y Estándares que Gestionan los Gobiernos de cada País.

También Existen Estándares dentro de una misma Organización.

Ejemplo: Relación de los Sistemas del SIU con los Sistemas de la Universidades.

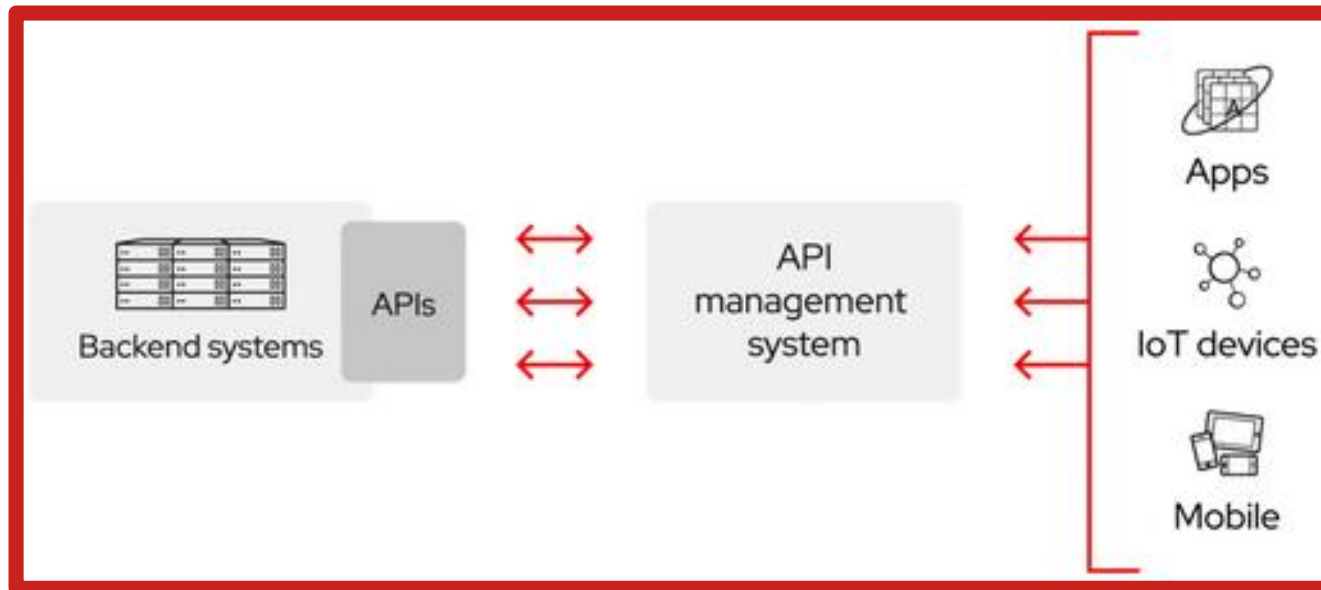


FORMATOS DE INTERCAMBIO DE DATOS

API

API son las siglas de **Application Programming Interface**, en español Interfaz de Programación de Aplicaciones.

Se trata de un conjunto de reglas que va a definir cómo se comunican dos aplicaciones entre sí.





FORMATOS DE INTERCAMBIO DE DATOS

API

API son las siglas de **Aplication Programming Interface**, en español Interfaz de Programación de Aplicaciones.

Se trata de un conjunto de reglas que va a definir cómo se comunican dos aplicaciones entre sí.

Los tres tipos básicos de APIs

APIs toma tres formas básicas: local, como web y como programa. Aquí hay un vistazo a cada tipo

Local APIs

La API original, creada para proporcionar servicios de sistema operativo o middleware a los programas de aplicación

Web APIs

Diseñado para representar recursos ampliamente utilizados como páginas HTML y se accede mediante un protocolo HTML simple. A menudo llamadas APIs REST vs APIs RESTFUL

Programa APIs

Basado en la tecnología RCP que hace que un componente de programa remoto parezca ser local al resto del software



FORMATOS DE INTERCAMBIO DE DATOS

Actualmente las API web normalmente usan **HTTP/s** para solicitar mensajes y proporcionar una definición de la estructura de los mensajes de respuesta.

Generalmente, las respuestas toman la forma de un archivo **XML** o **JSON** Formatos preferidos por lo fácil de manejar para otras aplicaciones.

Se basan en Servicios contemplados en **SOA**.

La arquitectura orientada a los servicios (SOA) es un tipo de diseño de software que permite reutilizar sus elementos gracias a las interfaces de servicios que se comunican a través de una red con un lenguaje común.

Protocolos de API remotas (SOAP)

Protocolo de Acceso a Objetos Simples (SOAP):

Usan **XML** para el formato de sus mensajes.

Reciben solicitudes a través de **HTTP** o **SMTP**.

SOAP, facilita que la comunicación entre aplicaciones que funcionan en entornos distintos o están escritas en diferentes lenguajes compartan información.

Transferencia de Estado Representacional (REST):

Las API web que funcionan con las limitaciones de arquitectura **REST** se llaman **API de RESTful**.

Usan **JSON** para el formato de sus mensajes.

No hay ningún estándar oficial para las API web de **RESTful**.



FORMATOS DE INTERCAMBIO DE DATOS



Permite: XML

HTTP, FTP, POP3, TCP, etc



Permite: XML, JSON, Binarios (imágenes, documentos), Text, etc.



FORMATOS DE INTERCAMBIO DE DATOS

EDI frente a API

Mientras que los EDIs tienen grandes medidas de fiabilidad y seguridad, las **API** son prácticamente ilimitadas cuando se trata de los tipos de sistemas con los que se puede interconectar.

WHAT DO SHIPPERS WANT?	EDI	API
Increased Communication Speed Between Data Systems	✓	✓
Cost Effective	✓	✓
Promotes Growth	✓	✓
Computer-to-Computer Integration	✓	
Cloud-to-Computer Integration		✓
Individual Transactions		✓
Batch Transactions	✓	



FORMATOS DE INTERCAMBIO DE DATOS

Volvamos a
los Datos...



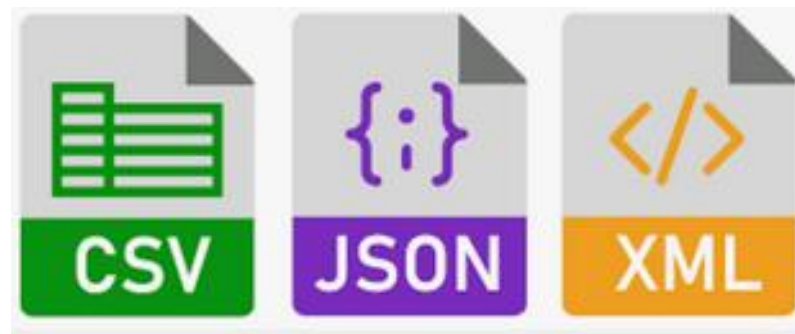
- 1) XML
- 2) JSON
- 3) YAML
- 4) CSV
- 5) Etc



Estos formatos estan pensados para intercambiar datos entre sistemas de dististintos lenguajes. Otra forma de compartir datos, pero no entre sistemas, son con documentos.



FORMATOS DE INTERCAMBIO DE DATOS



Formatos de Intercambio de Datos

XML

Extensible Markup Language (XML) es un formato de texto estandarizado para representar datos estructurados.

XML utiliza etiquetas para definir la estructura y el contenido de los datos, lo que lo hace legible por humanos y máquinas.

JSON

JavaScript Object Notation (JSON) es un formato ligero para el intercambio de datos, legible por humanos y máquinas. JSON utiliza una sintaxis basada en objetos y matrices para representar datos, lo que lo hace fácil de procesar.

CSV

Comma-Separated Values (CSV) es un formato de texto simple para datos tabulares.

CSV utiliza comas para separar valores en filas y columnas, lo que lo hace adecuado para datos estructurados.



CSV

Las siglas CSV vienen del inglés "Comma Separated Values" y significan **valores separados por comas**.

Un archivo CSV es cualquier archivo de texto en el cual los caracteres están separados por comas, haciendo una especie de **tabla en filas y columnas**. Donde las columnas quedan definidas por cada punto y coma (;).

Nota: Esto es adaptable a sus necesidades. (puedo seleccionar otro carácter especial de limitación.

Los archivos CSV sirven para manejar una gran cantidad de datos en formato tabla.



Limitaciones:

los archivos CSV tan solo admiten datos, es decir, no podremos dar formato a los mismos.

La capa de presentación la debe resolver el programa receptor.



CSV

ID,Nombre,Edad,Curso,Correo

1,Juan Perez,20,A,juan.perez@example.com

2,Ana Gomez,22,B,ana.gomez@example.com

Limitaciones:

los archivos CSV tan solo admiten datos, es decir, no podremos dar formato a los mismos.

La capa de presentación la debe resolver el programa receptor.



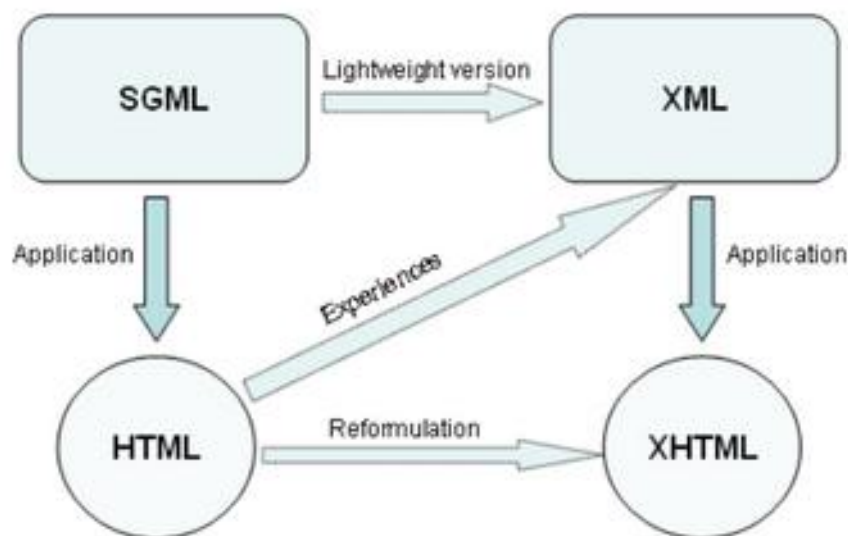
FORMATOS DE INTERCAMBIO DE DATOS

SGML:

SGML es un acrónimo de Standard Generalized Markup Language o Lenguaje de Señalización General Normalizado.

Esta es una norma ISO derivada de una anterior (**GML de IBM**).

SGML permite que la estructura de un documento pueda ser definida en base a la relación lógica de sus partes.



HTML (*Hyper Text Markup Language*)
Lenguaje de marcas de texto.
utilizado normalmente en la **www**.

XHTML Junta la capacidad de formato de HTML
Consolida con la formalidad del XML (y sus reglas).
Permite ser Validado por herramientas Estandar.



XML



```
1 <persona>
2   <nombres>Elsa</nombres>
3   <apellidos>Zambrano</apellidos>
4   <fecha-de-nacimiento>
5     <día>18</día>
6     <mes>6</mes>
7     <año>1996</año>
8   </fecha-de-nacimiento>
9   <ciudad>Pamplona</ciudad>
10</persona>
```

SGML es un acrónimo de Standard Generalized Markup Language o Lenguaje de Señalización General Normalizado. Esta es una norma ISO derivada de una anterior (GML de IBM). **SGML** permite que la estructura de un documento pueda ser definida en base a la relación lógica de sus partes.

Que es el XML?

XML es un subconjunto de **SGML** (Estándar Generalised Mark-up Language), simplificado y adaptado a Internet.

XML es un meta-lenguaje que nos permite definir lenguajes de marcado adecuados a usos determinados.



¿Para que sirve XML?

Representar información estructurada en la web (todos documentos), de modo que esta información pueda ser almacenada, transmitida, procesada, visualizada e impresa, por muy diversos tipos de aplicaciones y dispositivos.

XML No es una versión mejorada de **HTML**
HTML es una aplicación de **SGML** por lo tanto de
XML No es un lenguaje para hacer paginas WEB

Problema del HTML

- Define mas la presentación que su contenido
- No es fácilmente procesables por las maquinas
- Su estructura es caótica
- Su interpretación es ambigua dependiendo S.W utilizado
- Solo vale para paginas WEB

Ventajas del XML

- Fácilmente procesable
- Separa radicalmente el contenido y el formato de presentación
- Diseñado para cualquier lenguaje y alfabeto. (encoding)



Características

- XML es un subconjunto de SGML que incorpora sus tres características más importantes:
 - Extensibilidad
 - Estructura
 - Validación
- Basado en texto.
- Orientado a los contenidos no presentación.
- Las etiquetas se definen para crear los documentos, no tienen un significado preestablecido.
- No es sustituto de HTML.
- No existe un visor genérico de XML.

Aplicaciones de XML

- Publicar e intercambiar contenidos de bases de datos.
- Formatos de mensaje para comunicación entre aplicaciones (B2B)
- Descripción de metacontenidos.

B2B: "business to business" = Negocio a negocio.
B2C: "business to consumer" = Negocio a consumidor



Documento XML

- Conjunto de datos con sus respectivas etiquetas de marcado XML.
- Se almacena como texto en archivo con extensión .xml.
- Un documento XML puede incluir cualquier flujo de datos basado en texto:
 - un artículo de una revista
 - un resumen de cotizaciones de bolsa
 - un conjunto de registros de una base de datos,
 - etc..

Estructura de un documento XML

- Un documento XML está formado por **datos de caracteres** y **marcado**, el marcado lo forman las etiquetas.
- Dentro del documento existen dos partes: el **Prologo** y el **Cuerpo**

Prologo:

```
<?xml version="1.0" encoding="ISO-8859-1" standalone="no" ?>
```

```
<!DOCTYPE persona SYSTEM "persona.dtd">
```

Cuerpo:

```
<persona>
```

```
    <nombre> Franco </nombre>
```

```
    <apellido> Kühn </apellido>
```

```
</persona>
```




Estructura Básica

La estructura esta compuesta por:

- Elementos
- Atributos
- Contenido del elemento





Componentes de un documento XML

- En un documento XML existen los siguientes componentes:
 - **Elementos:** Pieza lógica del marcado, se representa con una cadena de texto(dato) encerrada entre etiquetas. Pueden existir elementos vacíos (
). Los elementos pueden contener atributos.
 - **Instrucciones:** Ordenes especiales para ser utilizadas por la aplicación que procesa
<?xml-stylesheet type="text/css" href="estilo.css">
 - **Las instrucciones XML.** Comienzan por <? Y terminan por ?>.
 - **Comentarios:** Información que no forma parte del documento. Comienzan por:
<!-- y terminan por -->.
 - **Declaraciones de tipo:** Especifican información acerca del documento:
<!DOCTYPE persona SYSTEM "persona.dtd">
 - **Secciones CDATA:** Se trata de un conjunto de caracteres que no deben ser interpretados por el procesador:
<![CDATA[Aquí se puede meter cualquier carácter, como <, &, >, ... Sin que sean interpretados como marcación]]>



Árbol de Análisis

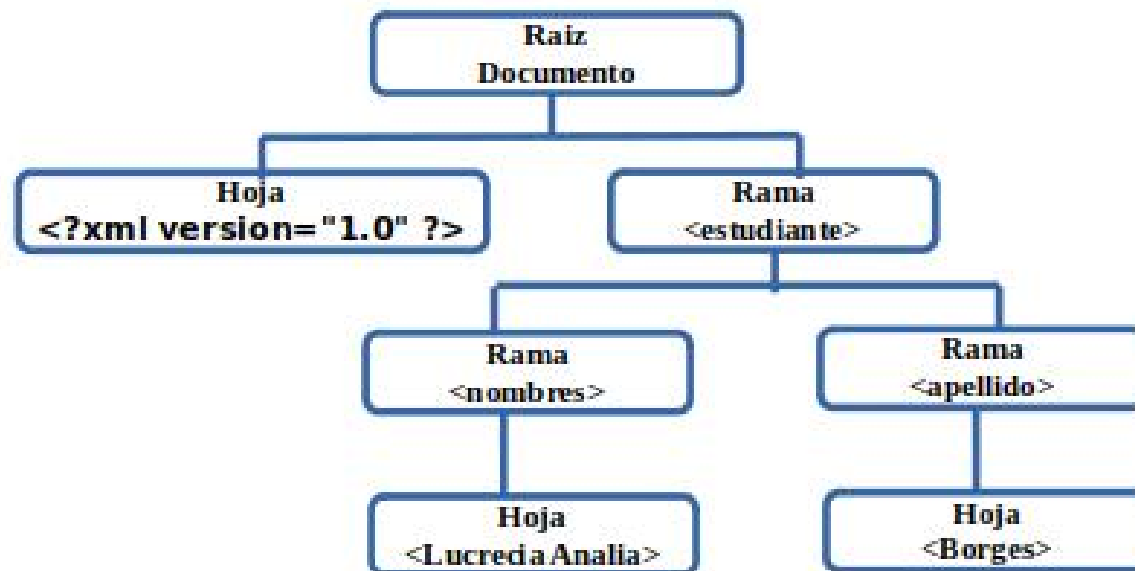
- Estructura que muestra los objetos que forman el documento y las relaciones entre ellos.
- Los componentes de un documento se les llama **objetos** (elementos, comentarios y cadenas de texto). El propio documento es un objeto.
- A cada objeto del árbol se le denomina **nodo**.
- El nodo principal que contiene a los demás se le llama nodo **raíz**.
- Cuando un nodo contiene a otro se le denomina **rama**.
- Los nodos finales, que no contienen otros nodos, se llaman **hojas**.

```
<?xml version="1.0" ?>
```

```
<estudiantes><persona>
```

```
<nombres> Lucrecia Analia </nombre>
```

```
<apellido> Borges </apellido>
```





Aclaración

- A la hora de diseñar se nos plantean dudas entre que escoger atributo o elemento. Hay que tener en cuenta lo siguiente:
 - **los atributos no pueden contener subelementos ni subatributos.**
 - No se organizan en ninguna jerarquía por lo que la representación es mucho más reducida que los elementos
 - La utilización de los atributos será una mera modificación de los elementos a que se aplican la información que deben de contener debe de ser de poca entidad sencilla y sin estructura.
- Aun así muchas veces llegamos a la misma conclusión utilizando atributos y elemento

Definición y Validación de XML (Reglas de Construcción)

- Un documento XML puede:
 - **Estar bien formado:** Cumple con la **sintaxis de XML**.
- O puede:
 - **Ser válido:** Además de estar bien formado, el documento cumple unas determinadas reglas y normas.
- Un analizador que comprueba la validez de un documento se dice que es un **analizador de validación**.
- Para establecer las reglas de construcción de documentos XML se utilizan **las DTD's** y los **Esquemas XML**.

Para definir y validar XML se puede utilizar las:

DTD

y/o

Esquemas XML



Definición de Tipo de Documento (DTD)

Define los elementos, atributos, entidades y notaciones que pueden utilizarse para construir un tipo de documentos, así como las reglas para su utilización.

- Mediante ellas se comprueba la validez de un documento.
- Tienen su origen en SGML.
- Posee una sintaxis especializada.

Ejemplo:

```
<!ELEMENT persona (nombre, apellidos, teléfono*)>  
<!ELEMENT nombre (#PCDATA)>  
<!ELEMENT apellidos (#PCDATA)>  
<!ELEMENT teléfono (#PCDATA)>
```

- La DTD establece la gramática de un vocabulario XML, el cual, determina la estructura de los documentos XML.
- Para que un documento se adhiera a una DTD, deberá incluir en su cabecera una declaración de tipo de documento:

```
<!DOCTYPE ElementRaiz SYSTEM dtd>
```

- ElementRaiz identifica el elemento de documento.
- Dtd representa la definición de la DTD.
- La declaración de tipo de documento, debe aparecer después de la declaración XML y antes del primer elemento de un documento.



La Declaración de Tipo de Documento (DTD-Document Type Data):

Al definir el lenguaje XML ya nos referimos a la Definición del Tipo de Documento (Document Type Definition DTD) que, en resumen, cumple las siguientes funciones:

- Una DTD especifica la clase de documento
 - Describe un formato de datos
 - Usa un formato común de datos entre aplicaciones
 - Verifica los datos al intercambiarlos
 - Verifica un mismo conjunto de datos
- Una DTD describe
 - Elementos: cuáles son las etiquetas permitidas y cuál es el contenido de cada etiqueta
 - Estructura: en qué orden van las etiquetas en el documento
 - Anidamiento: qué etiquetas van dentro de cuáles



Declaraciones de elemento

- Para declarar los elementos, que pueden ser utilizados por los documentos que se ajusten a esa **DTD**, se utiliza la expresión:

<!ELEMENT NombreElemento Contenido>

- *Contenido*, representa el contenido del elemento y puede ser:

- **Una lista de elementos secundarios.**

Cuando un elemento está formado por otros elementos, estos se declaran entre paréntesis separados por comas o por | si

se trata de elementos alternativos:

<!ELEMENT planeta (nombre+, tipo, (orbita|satélite+), composición*)>

- Cada subelemento puede llevar, además, los siguientes símbolos:
 - **Sin símbolo:** El elemento secundario debe aparecer una sola vez.
 - **Interrogación:** Puede aparecer una o ninguna vez.
 - **Asterisco (*):** Puede aparecer cualquier número de veces.
 - **Signo más (+):** Debe aparecer por lo menos una vez.

- **Datos de carácter.** Cuando un elemento solo va a contener datos de carácter analizados sintácticamente, se utiliza la declaración (#PCDATA):

<!ELEMENT nombre (#PCDATA)>

- Es posible declarar elementos mixtos, es decir, que contengan datos de carácter y elementos secundarios:

<!ELEMENT capitulo (#PCDATA|subcapitulo)*>

- **Elemento vacío.** Los elementos sin contenido se declaran:

<!ELEMENT saltolinea EMPTY>



Declaraciones de atributos

- Los atributos se utilizan para especificar información adicional del elemento.

```
<libro>  
  <capitulo numero="1">  
    <subcapitulo>  
      :
```

- Se declaran utilizando la siguiente sintaxis:

<!ATTLIST Elemento NomAtr Tipo Valpred>

- **NomAtr.** Es el nombre que se le da al atributo.
- **Tipo.** Es el tipo de atributo. Puede tomar uno de los siguientes valores:
 - **CDATA:** Datos de caracteres no analizados sintácticamente.
 - **Lista de valores:** Entre paréntesis y separados por |. Ejemplo: (Este | Oeste | Sur | Norte)
 - **ENTITY:** Entidad binaria externa.
 - **NOTATION:** Notación declarada en la DTD.
 - **ID:** Identificador único.
- **Valorpred.** Representa el valor predeterminado del atributo. Puede ser una de las siguientes posibilidades:
 - **#REQUIRED:** El atributo es obligatorio.
 - **#IMPLIED:** El atributo es opcional.
 - **#FIXED valor:** El atributo tiene ese valor fijo.
 - Un valor.



Declaraciones de atributos

Un elemento puede tener más de un atributo y pueden ser declarados todos en la misma línea:

```
<!ATTLIST curso
director CDATA #REQUIRED
horario (mañana | tarde | noche)
#IMPLIED
instalaciones (Exes | externas)
"Exes">

<!ELEMENT addressbook (contact)+>
<!ELEMENT contact (name, address+, city, state, zip, phone, email, web, company)>
<!ELEMENT name (#PCDATA)>
<!ELEMENT address (#PCDATA)>
<!ELEMENT city (#PCDATA)>
<!ELEMENT state (#PCDATA)>
<!ELEMENT zip (#PCDATA)>
<!ELEMENT phone (voice, fax?)>
<!ELEMENT voice (#PCDATA)>
<!ELEMENT fax (#PCDATA)>
<!ELEMENT email (#PCDATA)>
<!ELEMENT web (#PCDATA)>
<!ELEMENT company (#PCDATA)>
<!ELEMENT peliculas (pelicula)+>
<!ELEMENT pelicula(titulo, writer+, producer+, director+, actor*, comentarios?)>
<!ATTLIST pelicula
tipo (drama | comedy | adventure | sci-fi | mystery | horror | romance | documentary)
"drama"
clasificacion (G | PG | PG-13 | R | X) "PG"
review (1 | 2 | 3 | 4 | 5) "3"
año CDATA #IMPLIED>
<!ELEMENT titulo(#PCDATA)>
<!ELEMENT writer (#PCDATA)>
<!ELEMENT producer(#PCDATA)>
<!ELEMENT director (#PCDATA)>
<!ELEMENT actor (#PCDATA)>
```



Ejemplo de DTD

Recordemos que una DTD se puede guardar en un archivo de texto, como por ejemplo, en el archivo "list.dtd".:

```
<!ELEMENT List (Item)+>
<!ELEMENT Item (#PCDATA)>
<!ATTLIST Item
id CDATA IMPLIED
color CDATA IMPLIED>
<!ELEMENT Separator EMPTY>
```

Ejemplo de uso de DTD

Veamos ahora el documento .xml que hace referencia a esa DTD:

```
<?xml version="1.0" encoding="iso-8859-1"?>
<!DOCTYPE list SYSTEM "list.dtd">
<List>
  <Item>París</Item>
  <Item>Madrid</Item>
  <Separator />
  <Item color="rojo">Londres</Item>
</List>
```



Esquemas XML

- Misma finalidad que las **DTD**'s.
Describen los elementos y atributos que se pueden utilizar para construir documentos XML y las reglas de utilización.
- Permiten asociar tipos de datos con los elementos.
- Se crean utilizando la sintaxis XML.

Ejemplo:

```
<Schema xmlns="urn:schemas-microsoft-com:xml-data"
xmlns:dt="urn:schemas-microsoft-com:datatypes">
  <AttributeType name='id' dt:type='string' required='yes'/>
  <ElementType name='nombre' content='textOnly'/>
  <ElementType name='persona' content='mixed'>
    <attributetype type='id'/>
    <element type='nombre'/>
  </ElementType>
  <ElementType name='documento' content='eltOnly'>
    <element type='persona'/>
  </ElementType>
</Schema>
```



Fundamentos

- Surgen para dar respuesta a las limitaciones de las DTD's.
- Se basa en el vocabulario XML-Data que se utiliza para describir la estructura de los documentos.
- A la tecnología para la creación de esquemas se le conoce como XML Schema.
- IE 5.0 no realiza validación del documento a través del esquema.

Generalidades

- Los esquemas se definen en documentos XML.
- Se hace referencia a ellos a través de los espacios de nombres:

```
<películas xmlns="x-schema:PelículasEsquema.xml">
```

Espacios de nombres

- Un espacio de nombres es la dirección (URL o URN) que contiene la DTD o esquema, utilizado para validar los elementos del documento.
- Permiten evitar el conflicto de nombres dentro de un documento XML.

```
<películas  
  xmlns:película="http://www.aplicaciones.com/películas"  
  xmlns:música="http://www.aplicaciones.com/música"  
  <!-- Contenido del documento-->  
</películas>
```




Elementos de un Esquema

- **Schema.** Es el elemento raíz del documento, actúa como contenedor para el resto de los elementos. Contiene los atributos:
 - **name:** Nombre del esquema.
 - **xmlns:** Espacio de nombres del esquema. Hace referencia a la DTD donde se definen los elementos del esquema. Su valor se establece a:
um:schemas-microsoft-com:xml-data
- **ElementType.** Define los tipos de elementos que se utilizarán para la elaboración de documentos que sigan el esquema. Contiene los atributos:
 - **name:** Nombre del elemento.
 - **content:** Contenido del elemento:
 - **eltOnly:** Sólo puede contener elementos secundarios.
 - **textOnly:** Sólo texto.
 - **mixed:** Texto y elementos secundarios.
 - **empty:** Sin contenido.
 - **order:** Orden y frecuencia del grupo de elementos secundarios del elemento:
 - **One.** Sólo se permite una serie de elementos.
 - **Seq.** Los elementos deben producirse en la secuencia especificada.
 - **Many.** Los elementos pueden aparecer las veces que sea en cualquier orden.
- **dt:type.** Establece el tipo de contenido del elemento.



Elementos de un Esquema

- **Element.** Declara el modelo de contenido para un elemento. Dispone de los atributos:
 - **type.** Tipo de elemento. Es el nombre con el que ha sido declarado en ElementType.
 - **minOccurs.** Mínimo número de veces que el elemento puede aparecer. Su valor puede ser 0 o 1.
 - **maxOccurs.** Máximo número de veces que el elemento puede aparecer. Puede ser 1 o *.

```
<ElementType name="location" content="textOnly"/>
<ElementType name="comments" content="textOnly"/>
<ElementType name="session" content="eltOnly" order="seq">
  <element type="location" minOccurs="1" maxOccurs="1"/>
  <element type="comments" minOccurs="0" maxOccurs="1"/>
</ElementType>
```

- **AttributeType.** Se utiliza para definir los tipos de atributos que van a ser utilizados en los elementos. Los atributos son:
 - **name:** Nombre del atributo.
 - **dt:type:** Tipo de datos del atributo.
 - **dt:values:** Lista posible de valores de un atributo enumerado. Sólo se aplica cuando dt:type está establecido a enumeration.
 - **default:** Valor predeterminado.
 - **required:** Indica si el atributo es o no obligatorio. Su valor es **yes** o **no**.
- **attribute.** Se utiliza para declarar el modelo de contenido de un elemento. Sus atributos son:
 - **type.** Nombre del atributo
 - **default.** Valor predeterminado
 - **required.** Si es o no obligatorio



Tipos de datos de XML Schema

- Los tipos de datos de XML Schema están definidos en el espacio de nombres urn:schema-microsoft-com:datatypes.
- Entre los tipos de datos más importantes están:
 - **char**. Carácter.
 - **string**. Cadena de caracteres.
 - **boolean**. Booleano 0 o 1.
 - **int**. Número entero.
 - **float**. Número real.
 - **date**. Fecha.
 - **time**. Hora
 - **uri**. Identificador Uniforme de Recursos
 - **enumeration**. Tipo enumerado, sólo válido para atributos.
 - **ID**. Atributo de tipo identificador



Estándares relacionados con XML: xpath, xpointer, xlink, xslt, DOM.

- El XPath es un lenguaje de expresión que permite procesar valores conforme a los modelos de datos XML.
- XPointer (the XML Pointer language) permite armar hipervinculos que apuntan a partes específicas de fragmentos de documentos XML.
- XLink (the XML Linking language) define el metodo para crear links dentro de los documetos XML.
- XSLT soportar las transformaciones XSL. El XSLT permite las tranformaciones de documetos XML a otros formatos como XHTML.
- El dom xml define el estandar para acceder y manipular los documentos XML. El DOM presneta un documento XML com una estructura de arbol.
 - El DOM es separdo por la W3C en tres diferentes partes
 - CORE DOM: define el modelo estandar para cualquier documento estrucutrado
 - XML DOM: el modelo estandar para documentos XML.
 - HTML DOM: el modelo estadar para documentos HTML



Algunos sitios para leer

- <http://www.w3schools.com/xpath/>
- <http://es.wikipedia.org/wiki/XPath>
- <http://www.w3.org/TR/xpath20/>
- http://www.w3schools.com/xml/xml_xpointer.asp
- http://www.w3schools.com/xml/xml_xlink.asp
- <http://www.w3.org/TR/xlink11/>
- <http://www.w3schools.com/xsl/>
- <http://www.w3.org/TR/xslt20/>
- <http://www.w3schools.com/dom/>

Ejemplo sencillo

Ejemplo de Planilla de calculo

Ejemplo de Procesador de texto

Ejemplo de Bases de Datos mysql



Herramientas open-source para procesar XML.

butterflyXML (<http://sourceforge.net/projects/butterflyxml>) y conglomerate (<http://www.conglomerate.org/>), ambas con soporte para Docbook. El problema de ambas herramientas es que las últimas versiones estables son algo antiguas.

También está Quanta+, que es una herramienta de desarrollo de páginas web para KDE y forma parte del paquete kdewebdev. Es ampliable mediante plugins.

Otra opción libre es el editor XMLcopyeditor disponible tanto para GNU/Linux como Windows.

Herramientas open-source para procesar XML: xerces, xalan, fop, etc



```
<Estudiantes>
  <Estudiante>
    <id>1</id>
    <Nombre>Juan Perez</Nombre>
    <Edad>20</Edad>
    <Curso>A</Curso>
    <Correo>juan.perez@ejemplo.com</Correo>
  </Estudiante>
  <Estudiante>
    <id>2</id>
    <Nombre>Ana Gomez</Nombre>
    <Edad>22</Edad>
    <Curso>B</Curso>
    <Correo>ana.gomez@ejemplo.com</Correo>
  </Estudiante>
</Estudiantes>
```



JSON

JSON (JavaScript Object Notation)

Notación de Objetos de JavaScript es un formato ligero de intercambio de datos.

Alternativa para el **XML**. Es mas simple ya que esta construido por dos estructuras:

- Una colección de pares de nombre/valor. En varios lenguajes esto es conocido como un objeto, registro, estructura, diccionario, tabla hash, lista de claves o un arreglo asociativo.
- Una lista ordenada de valores. En la mayoría de los lenguajes, esto se implementa como arreglos, vectores, listas o secuencias.

Es por esto que es mas simple el parser. De hecho en JavaScript se puede utilizar el eval().

Definido por:

- ECMA-404 The JSON Data Interchange Standard.
- RFC - 7159



JSON

La sintaxis de intercambio de datos JSON

Un texto JSON es una secuencia de tokens formada a partir de puntos de código Unicode que se ajusta al valor JSON gramática.

El conjunto de tokens incluye seis tokens estructurales, cadenas, números y tres tokens de nombre literal.

[] { } : ,

Valores JSON

Un valor JSON puede ser un:

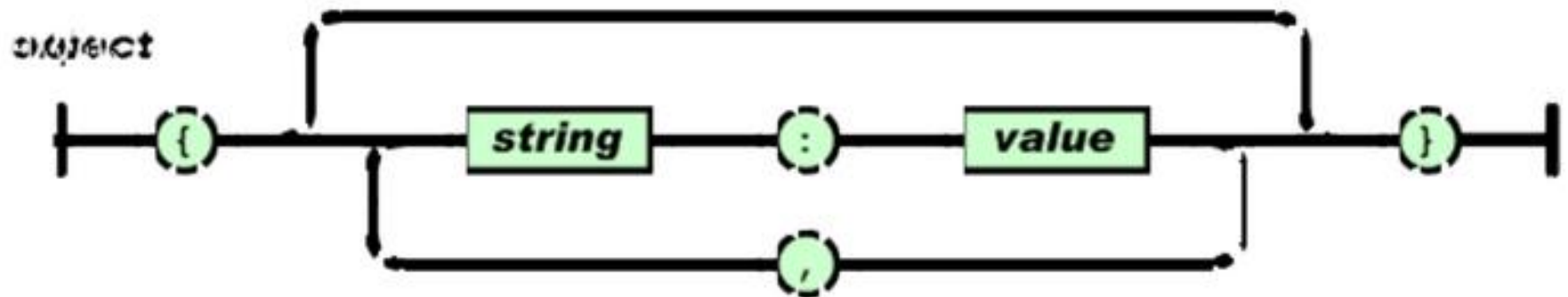
objeto
array
número
string
true
false
null

Objeto:

Es un conjunto desordenados de pares nombre/valor. Comienza con llave de apertura ({} y termina con llave de cierre (}) Cada nombre es seguido por dos puntos (:) y los pares son separados por comas (,).

Ejemplo:

```
{  
  "menu": {  "id": "file",  "value": "File",  },  
  "activado": 1  
}
```





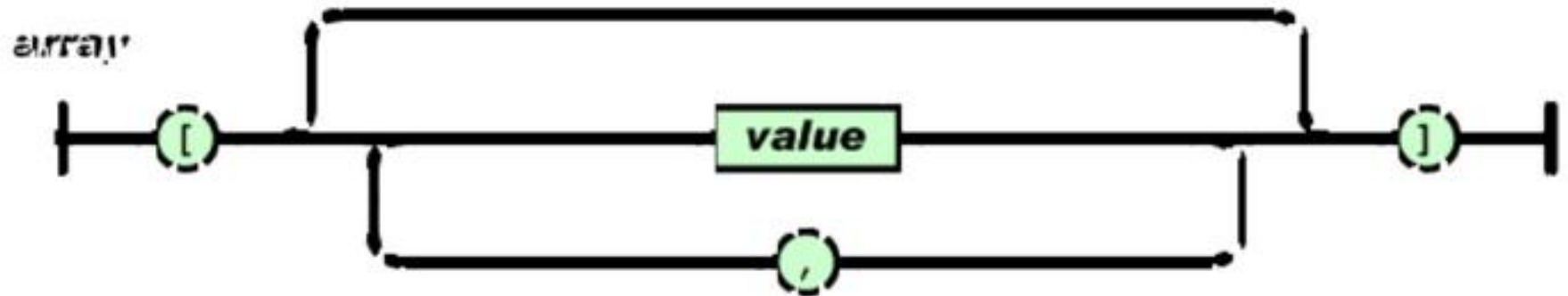
JSON

Arreglo:

Es una colección de valores. El arreglo empieza con corchete ([) y termina con el corchete de cierre (]). Los valores se separan por ,

Ejemplo:

```
[  
  "Valor1",  
  "Valor2"  
]
```



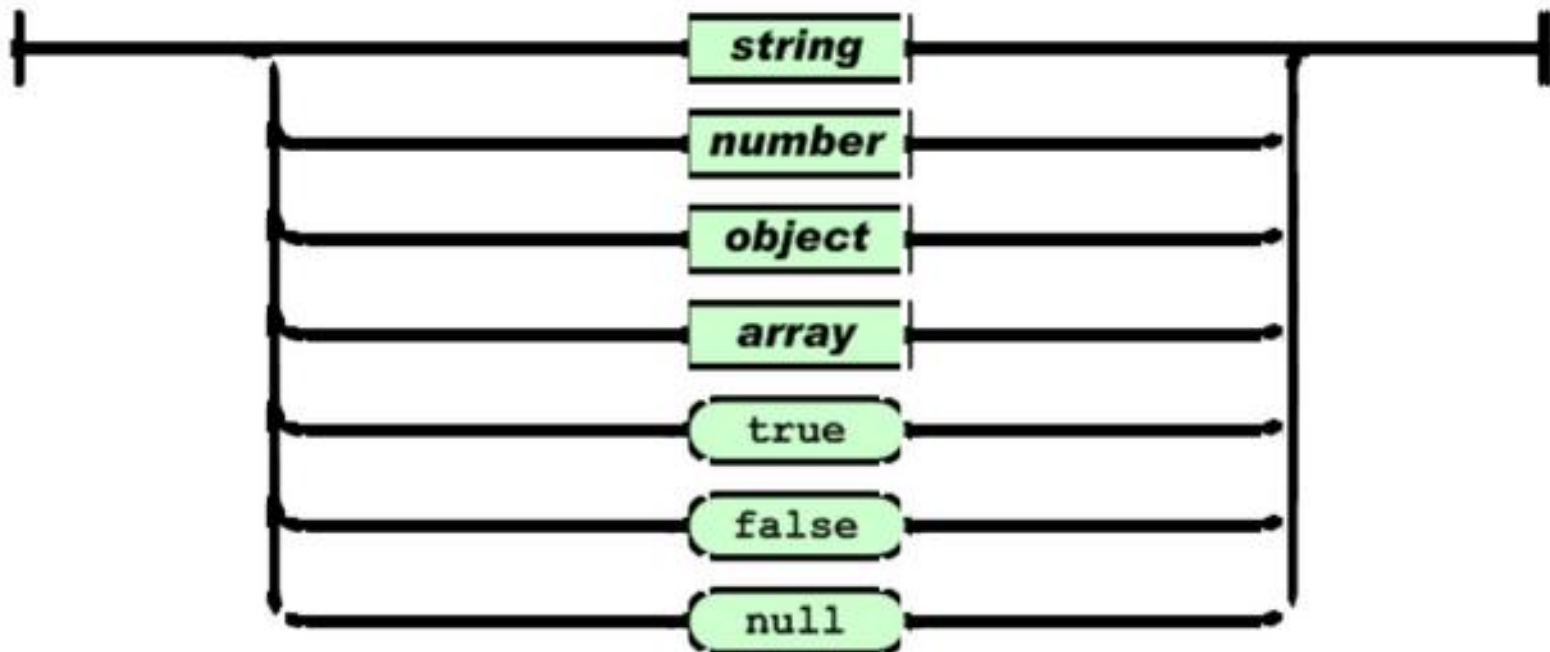


JSON

Valor:

Un valor puede ser una cadena de caracteres con comillas dobles, o un número, o true o false o null, o un objeto o un arreglo. Estas estructuras pueden anidarse.

value





JSON

```
{  
  "nombre": "Franco Kün",  
  "profesion": "Ing. Informático",  
  "edad": 35,  
  "lenguajes": ["PHP", "Javascript", "C++"],  
  "disponibilidadParaViajar": true,  
  "rangoProfesional": {  
    "aniosDeExperiencia": 5,  
    "nivel": "Senior"  
  }  
}
```

https://www.w3schools.com/js/js_json_intro.asp

Algo mas sobre JS JSON

JSON Example

```
{ "employees": [  
  { "firstName": "John", "lastName": "Doe" },  
  { "firstName": "Anna", "lastName": "Smith" },  
  { "firstName": "Peter", "lastName": "Jones" }  
]
```

XML Example

```
<employees>  
  <employee>  
    <firstName>John</firstName> <lastName>Doe</lastName>  
  </employee>  
  <employee>  
    <firstName>Anna</firstName> <lastName>Smith</lastName>  
  </employee>  
  <employee>  
    <firstName>Peter</firstName> <lastName>Jones</lastName>  
  </employee>  
</employees>
```



JSON

JSON.Parse(cadena)

Esta función permite convertir una cadena en formato JSON en una variable Javascript que contiene la representación del valor de esa cadena.

```
var objeto = JSON.parse('{ "nombre": "juan",  
"edad": 33, "casado": false }');
```

JSON.stringify(objeto)

Esta función Javascript convierte un objeto, un array u otro valor en su representación como cadena en formato JSON.

```
var coche = {  
  modelo: "Ford Focus",  
  anioFabricacion: "2020",  
  motorizacion: 'Gasolina'  
}  
var cadena = JSON.stringify(coche);
```

<https://desarrolloweb.com/home/json#track68>



Bibliografia:

- **json.org**
- **Json parser for C**
- **Json parser for C++**
- **JSON y BSON en MongoDB**
- **JSON en mysql**



JSON

```
{
  "Estudiantes": [
    {
      "id": 1,
      "Nombre": "Juan Perez",
      "Edad": 20,
      "Curso": "A",
      "Correo": "juan.perez@ejemplo.com"
    },
    {
      "id": 2,
      "Nombre": "Ana Gomez",
      "Edad": 22,
      "Curso": "B",
      "Correo": "ana.gomez@ejemplo.com"
    }
  ]
}
```




YAML

YAML es un acrónimo recursivo que significa **YAML Ain't Markup Language** (en castellano, '**YAML no es un lenguaje de marcado**').

=== Listas ===

```
<syntaxhighlight lang="yaml">
--- # Películas favoritas, formato de bloque
- BotijoAzul
- BotijoVerde
- Viridiana
- Psicosis
--- # Lista de la compra, formato en línea
[leche, pan, huevos]
[chorizo, morcilla, botijo, pollo]
</syntaxhighlight>
```

=== Vectores asociativos ===

```
<syntaxhighlight lang="yaml">
--- # Bloque
nombre: Pepe López
edad: 33
--- # En línea
{nombre: Pepe López, edad: 33}
</syntaxhighlight>
```

Programas compatibles con el archivo YAML

1. Adobe Dreamweaver.
2. gVim.
3. NotePad++ text editor.
4. Python.
5. Windows Notepad.



YAML

Estudiantes:

- id: 1

Nombre: Juan Perez

Edad: 20

Curso: A

Correo: `juan.perez@ejemplo.com`

- id: 2

Nombre: Ana Gomez

Edad: 22

Curso: B

Correo: `ana.gomez@ejemplo.com`

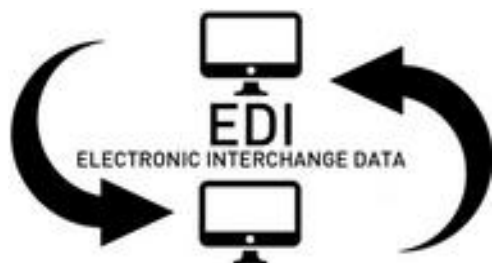


FORMATOS DE ARCHIVOS DE DATOS



Para que dos entidades puedan intercambiar información es necesario que tengan un acuerdo previo.

Las especificaciones de formatos son el acuerdo desde el punto de vista de los datos.





FORMATOS DE ARCHIVOS DE DATOS



Metadatos

Un metadato es información descriptiva acerca de otros datos.

El estándar ISO/IEC 11179 define el concepto de metadato. Como concepto, son previos a la informática.

https://es.wikipedia.org/wiki/ISO/IEC_11179



FORMATOS DE ARCHIVOS DE DATOS

Tipos de metadatos

Administrativos: Permiten gestionar los datos para controlar el acceso y el uso. Ejemplos: los bits `drwxrwxrwx` de cada archivo Unix, la extensión, que informa del tipo de documento (PDF, BMP, etc) o la fecha de recepción de un correo.

Estructurales: Describen las relaciones entre datos complejos y su estructura interna. Ejemplos: índices de bases de datos o archivos referenciados en una página web.

Descriptivos: Dan información pormenorizada de cada recurso de información. Permiten a los usuarios buscar y recuperar información. Ejemplo: la descripción de las etiquetas de un documento HTML.

**¿QUÉ DIABLOS SON
LOS METADATOS?**

```
<meta charset="utf-8" />
<base href="https://jorgesistemas.c
<meta name="keywords" content="slid
<meta name="author" content="Jorge
<meta name="robots" content="index,
<meta name="description" content="E
<meta name="generator" content="Joc
<title>Content Slider & SEBLOD
```



REPRESENTACIÓN Y CODIFICACIÓN DE DATOS

TEXTO PLANO

ASCII: (1963), EVOLUCIONÓ de los códigos usados en telegrafía. 7 bits datos, mas 1 bit de paridad. 128 símbolos, la versión actual utiliza 95 imprimibles.

CODEPAGES: (IBM) creó una serie de tablas de símbolos, distintas para cada país, conocidas como **CP-Número**, por ejemplo el famoso **CP-437**, mal llamado ASCII extendido.

ISO/IEC: 8859-x: En conjunto ISO y la IEC estándar Generaron el estandar de CODEPAGES, también para 8 bits. **ISO/IEC 8859-1** (español).

Windows-125x: El CODEPAGE **1252** se basó ANSI que se convirtió en la **ISO 8859-1**.





REPRESENTACIÓN Y CODIFICACIÓN DE DATOS

La ventaja de los codepages: Es que la mayor parte de los idiomas tienen cubierto su alfabeto con sus símbolos propios. Son compatibles hacia atrás. Los códigos imprimibles en el primitivo ASCII de 7 bits tienen la misma codificación numérica en los codepages de 8 bits.

El Problema: Falta de compatibilidad entre distintos codepages para los caracteres extra (del 128 al 255) Por otra parte, cualquier Ademas con 256 no puede cubrir la gran cantidad de ideogramas: **chinos, japoneses o coreanos.**

Primera solución: Double-Byte Character (DBCS) Esta solución (hoy Obsoleta) utiliza 8 bits para los comunes y el resto para los otros. Lo cual algunos símbolos tienen 8 bits y otros 16 bits. Generando el problema de **Ambigüedad de limites...**

Ademas en el lenguaje Chino tienen
Miles de símbolos para representar...

¿Y entonces que ????





REPRESENTACIÓN Y CODIFICACIÓN DE DATOS

Formas de codificar el Conjunto Universal de Caracteres ISO/IEC:10646

La ISO 10646 define diversas «formas de codificación» de caracteres para el Conjunto Universal de Caracteres (Universal Character Set).

- **UCS-4**, cuatro octetos para cada carácter.
- **UCS-2**, dos octetos para cada carácter.

<https://www.iso.org/standard/76835.html>

Unicode Transformation Format (UTF)

Las diferentes versiones de UTF son, hasta la fecha, el sistema más universal.

- **UTF-8** bytes descrito en "*UTF-8, a transformation format of ISO 10646*"
- **UTF-16** bytes descrito en "*https://es.wikipedia.org/wiki/UTF-16*"
- **UTF-32** bytes descrito en "*https://en.wikipedia.org/wiki/UTF-32*"



REPRESENTACIÓN Y CODIFICACIÓN DE DATOS

Unicode Transformation Format (UTF)

Las diferentes versiones de UTF son, hasta la fecha, el sistema más universal.

- *UTF-8 bytes descrito en "UTF-8, a transformation format of ISO 10646"*
- *UTF-16 bytes descrito en "<https://es.wikipedia.org/wiki/UTF-16>"*
- *UTF-32 bytes descrito en "<https://en.wikipedia.org/wiki/UTF-32>"*

UTF-8:

- 1 byte para los 128 caracteres que forman parte de **ASCII**.*
- 2 bytes para los 1920 caracteres de los principales idiomas.*
- 3 bytes para chino, japonés y coreano.*
- 4 bytes para los símbolos matemáticos y alfabetos clásicos.*

***UTF-16 y UTF-32** No son compatibles al **ASCII**.*

```
$char = "□";  
$utf8_string = utf8_encode($char);  
echo $utf8_string;
```



REPRESENTACIÓN Y CODIFICACIÓN DE DATOS

Unicode Transformation Format (UTF)

Las diferentes versiones de UTF son, hasta la fecha, el sistema más universal.

UTF-8: Usa 1 byte para los primeros 128 caracteres de ASCII y se expande hasta 4 bytes para caracteres más complejos de otros idiomas.

Ejemplos:

- El carácter "A" en UTF-8 es: 01000001 (1 byte, igual que en ASCII).*
- El símbolo "€" en UTF-8 es: 11100010 10000010 10101100 (3 bytes).*

UTF-16: Usa 2 o 4 bytes y es comúnmente utilizado en sistemas como Windows.

Ejemplos:

- El carácter "A" en **UTF-16**: 00000000 01000001 (2 bytes).*
- El símbolo "€" en **UTF-16**: 00100010 10101100 (2 bytes).*



FORMATOS DE ARCHIVOS DE DATOS

Volvamos a los datos...

Existen distintos tipos de Archivos de datos y pueden se categorizados de distintos puntos de vista.

Por ejemplo:



Ofimática



imagen



Sonido



Video



FORMATOS DE ARCHIVOS DE DATOS

Formatos en ofimática

*Estas aplicaciones además de los **CODEPAGES** requieren códigos especiales para fijar márgenes, tipos de letra, elementos flotantes, etc..*

- *Procesadores de texto.*
- *Hojas de cálculo.*
- *Programas para presentaciones.*
- *Bases de datos sencillas.*
- *Agendas.*



Nota: Los formatos de las aplicaciones de texto...

Nacieron considerando la página, asumiendo que debían ser imprimibles. Hoy hay distintas formas de presentación como el **ePub, mobi, djvu** y etc



FORMATOS DE ARCHIVOS DE DATOS

Formatos de imagen

Formatos sin compresión: *Existen decenas de formatos de imagen. Guardan la imagen original tal cual. Ventaja Tiempo de procesamiento. Y la desventaja es el Tamaño.*

Nombramos dos relevantes:

Formato crudo, en inglés raw: *Es la información tal cual sale del charge-coupled device (CCD), Es el formato que más memoria ocupa (36 a 48 bits/pixel).*

Bitmap Image File (BMP): *Cada imagen contiene una pequeña cabecera con metadatos y una matriz rectangular de pixels. Cada pixel se representa con entre 1 y 32 bits en función de si es monocromo o a color y de la profundidad de color (tiene varias versiones). Después del formato crudo es el que más ocupa.*





FORMATOS DE ARCHIVOS DE DATOS

Formatos de imagen

Formatos Vectoriales: Consisten en diagramas. Se guarda la forma de reproducir el dibujo, que consiste en elementos gráficos: puntos, segmentos, líneas curvas, texto. Se especifica atributos como su color, tipo de línea, terminaciones de los segmentos, fuente, etc.

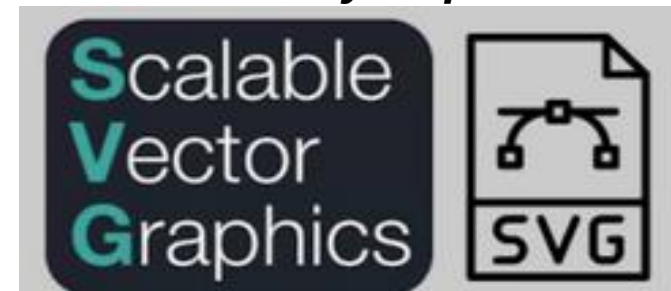
Ventajas: la posibilidad de escalar (agrandar o reducir) la imagen sin perder calidad y el mínimo tamaño que ocupan.

Nombramos dos relevantes:

Scalable Vector Graphics (SVG): Basado en XML y definido por el W3C. Popular en la web (Poco Peso)

Windows MetaFile (WMF): Es un conjunto de llamadas a funciones de dibujo.

Enhanced MetaFile (EMF): sucesor de WMF con más funciones y la posibilidad de ser usado como lenguaje gráfico para impresoras





FORMATOS DE ARCHIVOS DE DATOS

Formatos de imagen

Formatos Compresión Sin perdida: Tasa de compresión de estos algoritmos es modesta, y tienen el inconveniente del tiempo de proceso, sobre todo en la compresión. *(Se ampliará en la clase de Compresión).*

Nombramos dos relevantes:

Graphics Interchange Format (GIF): Ocupa 8 bits/pixel. Es antiguo pero aun muy usado. Con tasa de compresión entre 4:1 a 10:1. La compresión es sin pérdida para imágenes con número de colores ≤ 256 .

Portable Network Graphics (PNG): Permite más profundidad de color, hasta 48 bits, y monocromo de hasta 16 bits. Su tasa de compresión es entre un 10 % y un 30 % mejor que la de GIF.

Formatos Compresión Con perdida: El más popular es Joint Photographic Experts Group (**JPEG y JPEG 2000**), Su tasa de compresión ajustable, desde 10:1 a 1000:1; > tasa de compresión, < calidad de imager.





FORMATOS DE ARCHIVOS DE DATOS

Formatos de sonido



Formatos Sin Compresión:

Waveform Audio File Format (WAV): *Es el formato sin compresión. Ocupa mucho espacio y sólo es recomendable para edición.*

Formatos Compresión con pérdida:

MPEG-1 (o 2) Audio Layer III (MP3): *El ratio se puede ajustar y aproximadamente es de 1:10. Es sin duda el más popular, existen otros como : **WMA** de Microsoft y **OGG** usado por el software libre como Linux.*

Musical Instrument Digital Interface (MIDI): *Similar a los formatos vectoriales para gráficos. En lugar de codificar sonido contiene instrucciones para sintetizarlo por software. Ocupa mucho menos que MP3 pero no es posible comprimir en formato MIDI un sonido grabado desde un micrófono.*



FORMATOS DE ARCHIVOS DE DATOS

Formatos de video

codecs: (acrónimo de codificador/decodificador) Realizan la compresión y descompresión generando de corriente de datos (stream) en otros stream.

Audio Video Interleaved (AVI): Es el más usado en la actualidad. Admite varios codecs, como DivX, XviD, CinePack, Digital Video, Intel Indeo, etc. Su variedad de codecs le permite múltiples ratios y calidades de compresión.

Moving Pictures Expert Group (MPEG): Codecs, como MPEG-1, 2 y 4 para vídeo y MP3 para sonido.

MOV: (Apple). codec propio. La calidad es alta y el tamaño de los archivos bajo, es usado en páginas web. Admite streaming.

RM: Real Networks, tiene un Codec propio y admite streaming.

FLV: Es el formato de Adobe Flash. Los codecs son Sorenson Spark y On2



FORMATOS DE ARCHIVOS DE DATOS

Cuantos Formatos de Archivos existen??

Tipos de archivos....www.solvusoft.com

Formato cerrado: *Especificaciones no publicadas.*

Formato abierto: *Especificaciones publicadas y cualquiera puede hacer software que lea o escriba en él. Suelen ser legibles con un editor de texto.*

Formato propietario: *Es aquel que está protegido por derechos de autor o patentes. Sólo el propietario de los derechos puede hacer software que lea o escriba en ese formato.*

El problema de la dependencia!!!!

Es la justificación de un estándar!!!



Lenguaje Utilizado en muchas impresoras y también muy común como formato de transporte de archivos gráficos en talleres de impresión profesional. .

Formato : *Es completamente Abierto ya que consiste en un conjunto de insrucciones.*



```

10 def
11   /rank (T null) def
12
13   <N defNtr n1 tamaño de la página de
14
15   /pageSize (N) (T) def
16   <N defNtr n1 tamaño de la página de
17
18   /pageSize (N) (T) def
19   <N defNtr n1 tamaño de la página de
20
21   /pageSize (N) (T) def
22   <N defNtr n1 tamaño de la página de
23
24   /pageSize (N) (T) def
25   <N defNtr n1 tamaño de la página de
26
27   /pageSize (N) (T) def
28   <N defNtr n1 tamaño de la página de
29
30   /pageSize (N) (T) def
31   <N defNtr n1 tamaño de la página de
32
33   /pageSize (N) (T) def
34   <N defNtr n1 tamaño de la página de
35
36   /pageSize (N) (T) def
37   <N defNtr n1 tamaño de la página de
38
39   /pageSize (N) (T) def
40   <N defNtr n1 tamaño de la página de
41
42   /pageSize (N) (T) def
43   <N defNtr n1 tamaño de la página de
44
45   /pageSize (N) (T) def
46   <N defNtr n1 tamaño de la página de
47
48   /pageSize (N) (T) def
49   <N defNtr n1 tamaño de la página de
50
51   /pageSize (N) (T) def
52   <N defNtr n1 tamaño de la página de
53
54   /pageSize (N) (T) def
55   <N defNtr n1 tamaño de la página de
56
57   /pageSize (N) (T) def
58   <N defNtr n1 tamaño de la página de
59
60   /pageSize (N) (T) def
61   <N defNtr n1 tamaño de la página de
62
63   /pageSize (N) (T) def
64   <N defNtr n1 tamaño de la página de
65
66   /pageSize (N) (T) def
67   <N defNtr n1 tamaño de la página de
68
69   /pageSize (N) (T) def
70   <N defNtr n1 tamaño de la página de
71
72   /pageSize (N) (T) def
73   <N defNtr n1 tamaño de la página de
74
75   /pageSize (N) (T) def
76   <N defNtr n1 tamaño de la página de
77
78   /pageSize (N) (T) def
79   <N defNtr n1 tamaño de la página de
80
81   /pageSize (N) (T) def
82   <N defNtr n1 tamaño de la página de
83
84   /pageSize (N) (T) def
85   <N defNtr n1 tamaño de la página de
86
87   /pageSize (N) (T) def
88   <N defNtr n1 tamaño de la página de
89
90   /pageSize (N) (T) def
91   <N defNtr n1 tamaño de la página de
92
93   /pageSize (N) (T) def
94   <N defNtr n1 tamaño de la página de
95
96   /pageSize (N) (T) def
97   <N defNtr n1 tamaño de la página de
98
99   /pageSize (N) (T) def
100  <N defNtr n1 tamaño de la página de

```



FORMATOS DE ARCHIVOS DE DATOS

Analizando algunos Casos... PostScript(Page Description Language, PDL)

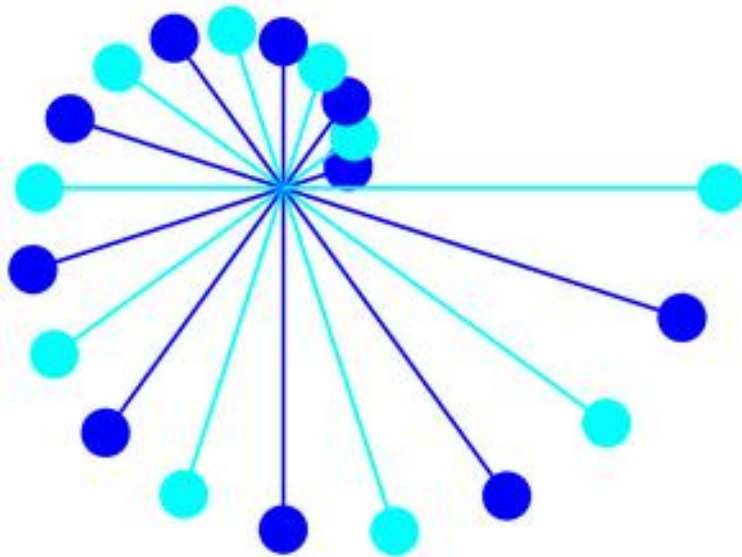


```
1 %!PS
2 /inch {72 mul} def
3
4 % Definir el tamaño de la página A4
5 <<
6   /PageSize [595.276 841.890]
7 >> setpagedevice
8
9 % Definir la fuente en negrita y el tamaño del texto
10 /Helvetica-Bold findfont 36 scalefont setfont
11
12 % Definir el color en verde
13 0 1 0 setrgbcolor
14
15 % Calcular el ancho del texto
16 /OrganizacionDeDatos (ORGANIZACION DE DATOS 2023) stringwidth pop def
17
18 % Calcular la ubicación central en la página
19 595.276 2 div OrganizacionDeDatos 2 div sub 841.890 2 div OrganizacionDeDatos 2 div sub translate
20
21 % Escribir "ORGANIZACION DE DATOS 2023" centrado en la página en negrita y verde
22 0 0 moveto
23 (ORGANIZACION DE DATOS 2023) show
24
25 % Restaurar el color a negro
26 0 0 0 setrgbcolor
27
28 % Finalizar el documento
29 showpage
30
```



FORMATOS DE ARCHIVOS DE DATOS

Analizando algunos Casos... PostScript(Page Description Language, PDL)



```
espiral.ps
2 % Ejemplo 5: Rayos en espiral a dos colores
3 /mm {72 mul 25.4 div} def % Conversion de mm a puntos
4 /W {210 mm} def % Ancho A4
5 /H {297 mm} def % Alto A4
6 << /PageSize [W H] >> setpagedevice
7 /ultimopunto {
8 % Define dos simbolos, "ultimox" y "ultimoy" iguales a las coordenadas
9 % del ultimo punto alcanzado
10 % Espera: -
11 % Devuelve: -
12 currentpoint exch
13 /ultimox exch def
14 /ultimoy exch def
15 } def
16 /linea {
17 % Linea generica a partir del punto y longitud dados,
18 % y registra el ultimo punto alcanzado
19 % Espera: longitud x y
20 % Devuelve: -
21 moveto % Mover a (x, y)
22 0 rlineto % Crear linea hacia la derecha
23 ultimopunto % Guardar ultimo punto
24 stroke % Trazar
25 } def
26 /N {20} def % Cantidad de rayos
27 W 2 div H 2 div translate % Origen de coordenadas al centro de la pagina
28 2 setlinewidth % Espesor de linea 2 pt
29 1 1 N {
30 360 N div rotate % Rotar espacio en (360/N) grados
31 dup % Preservar paso actual
32 2 mod 0 eq % Verificar paridad
33 {0 1 1} % Cian si es par
34 {0 0 1} % Azul si es impar
35 ifelse
36 setrgbcolor % Establecer color
37 dup % Preservar paso actual
38 4 mm mul 10 mm add % Obtener longitud como 4 mm * paso + 10 mm
39 0 0 linea % Linea a partir del origen
40 % Llenar circulo con centro en el ultimo punto alcanzado
41 % y radio 5 mm
42 ultimox ultimoy 5 mm 0 360 arc
43 fill
44 } for
45 showpage
```



FORMATOS DE ARCHIVOS DE DATOS

Analizando algunos Casos... PostScript(Page Description Language, PDL)

Ventajas:

- *Calidad de impresión*
- *Portabilidad.*
- *Escalabilidad*
- *Texto y gráficos*
- *Control preciso de la página*

```
1 %!PS
2 /Courier          % name the desired font
3 20 selectfont    % choose the size in points and establish
4                  % the font as the current one
5 72 500 moveto     % position the current point at
6                  % coordinates 72, 500 (the origin is at the
7                  % lower-left corner of the page)
8 (Hello world!) show % stroke the text in parentheses
9 showpage         % print all on the page
```

Desventajas:

- *Dificultad de lectura y escritura*
- *Tamaño del archivo*
- *Rendimiento limitado en dispositivos antiguos*
- *Incompatibilidad: No todos los dispositivos o visores.*
- *Dificultad para la edición*

Más Información: <https://www.adobe.com/products/postscript.html>

O también: <https://en.wikipedia.org/wiki/PostScript>



FORMATOS DE ARCHIVOS DE DATOS



Analizando algunos Casos...

PDF (siglas en inglés de Portable Document Format)

Desarrollado por la empresa Adobe Systems, Estándar abierto desde el 1 de julio de 2008 como ISO 32000-1.

Algunas Características:

- *Es multiplataforma.*
- *Combina: texto, vídeos, sonido, hipertexto como vínculos y marcadores, enlaces y miniaturas de páginas.*
- *Los archivos PDF no pierden el formato con el envío a otros usuarios.*
- *Es uno de los formatos más extendidos en Internet.*
- *Es una especificación abierta.*
- *Puede cifrarse e incluso firmarlo digitalmente.*
- *Un archivo PDF puede crearse desde varias aplicaciones.*
- *Puede generarse desde cualquier aplicación mediante la instalación de una impresora virtual*
- *Es el estándar ISO (ISO 19005-1:2008) para ficheros contenedores de documentos.*
- *Los ficheros PDF son independientes del dispositivo.*





FORMATOS DE ARCHIVOS DE DATOS



Analizando algunos Casos...

PDF (siglas en inglés de Portable Document Format)

Formato: *Es de tipo compuesto (imagen vectorial, mapa de bits y texto). La idea de contenedor de Archivos.*

Componentes de un archivo PDF

- **Objetos:** *Un PDF es una estructura de datos formada por objetos*
- **Estructura de Archivo:** *Determina como se almacenan, se accede, y se actualizan los Objetos*
- **Estructura de Documento:** *Especifica como se usan los tipos los objetos para representar los componentes de un documento PDF*
- **Flujos de contenido:** *Secuencia de instrucciones que describen la apariencia de una página u otra entidad gráfica.*



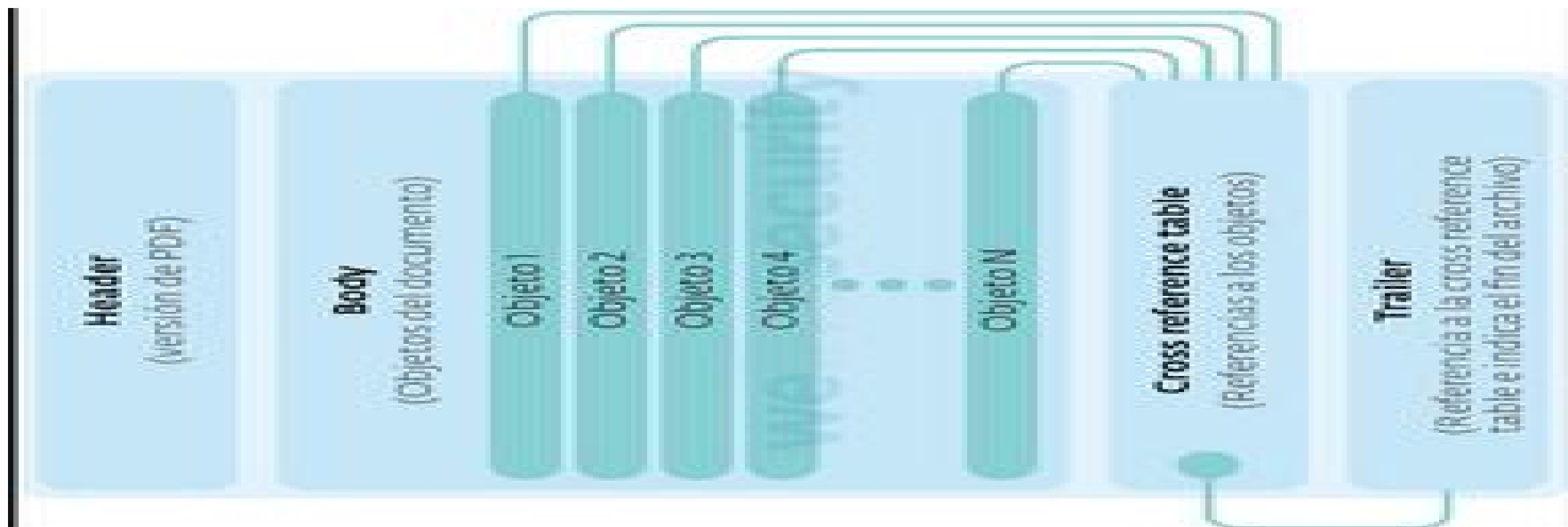


FORMATOS DE ARCHIVOS DE DATOS



Analizando algunos Casos...

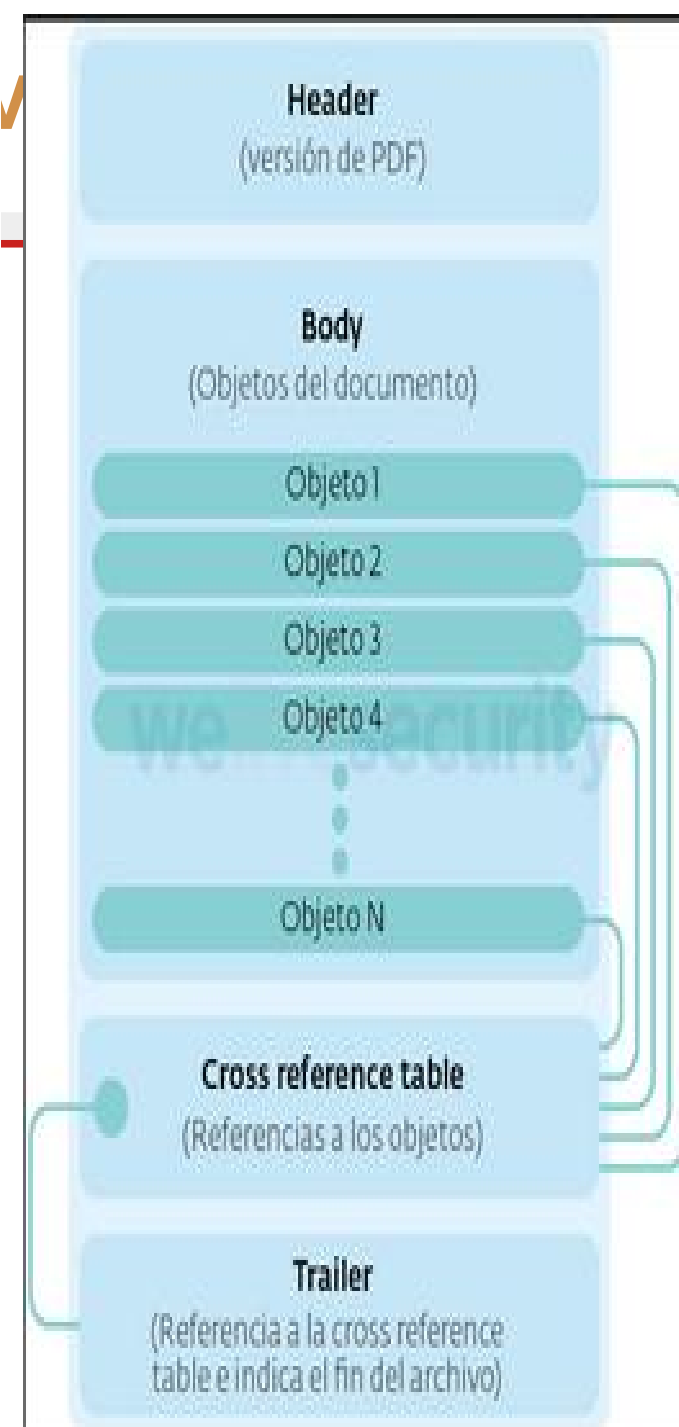
PDF (siglas en inglés de Portable Document Format)



Header	Body	xref	trailer
%PDF-1.7	1 0 obj<</Type /Catalog/...>> ... 11 0 obj<</Type /Font /Subtype ...>>	0 8 0000000000 65535 0000000009 00000 ...	<</Size 10/Root 1 0 R>> startxref 1888547 %%EOF
Cabecera	Cuerpo	Tabla de referencias cruzadas	Coda



%PDF-1.x		Cabecera Header	
<pre>1 0 obj << /Type /Catalog /Pages 2 0 R >> endobj</pre>		Root Object El objeto 1 es un catálogo (/Catalog) que apunta a la página en el objeto 2.	CUERPO
<pre>2 0 obj << /Type /Pages /Kids [3 0 R] /Count 1 >> endobj</pre>		El objeto 2 es un objeto de páginas (/Pages) que tiene una página como hijo.	
<pre>3 0 obj << /Type /Page /Parent 2 0 R /MediaBox [0 0 612 792] /Contents 4 0 R >> endobj</pre>		El objeto 3 es una página (/Page) que tiene un tamaño definido (MediaBox) y contenido en el objeto 4.	
<pre>4 0 obj << /Length 50 >> stream % Contenido del objeto (datos binarios) BT /F1 12 Tf 72 720 Td {¡Hola, mundo!} Tj ET endstream endobj</pre>		El objeto 4 es un objeto de contenido (/Contents) que tiene un flujo de datos binarios que representan el contenido de la página. En este caso, se muestra el texto "¡Hola, mundo!".	
<pre>xref 0 5 0000000000 65535 f 0000000017 00000 n 0000000096 00000 n 0000000174 00000 n 0000000327 00000 n</pre>	Se especifica la posición y el estado de cada objeto en el archivo PDF. La primera entrada, "0000000000 65535 f", indica que el objeto 0 está libre (f = free). Las siguientes entradas indican las posiciones y estados de los objetos restantes. ("n" significa "in use").	TABLA de REFERENCIA CRUZADA	COLA
<pre>trailer << /Size 5 /Root 1 0 R >> startxref 1234 %%EOF</pre>	Tamaño de la TRC (/Size) y objeto raíz (/Root). "startxref" especifica la posición del inicio de la sección xref. "1234" es la posición del inicio de la sección xref. "%%EOF" marca el final del archivo PDF.		





FORMATOS DE ARCHIVOS DE DATOS



Analizando algunos Casos...

PDF (siglas en inglés de Portable Document Format)

<code>%PDF-1.x</code>	HEADER
<code>1 0 obj << /Type /Catalog .. >> 20 0 obj << /page .. >></code>	BODY
<code>ref 0 5 00000000000 65535 f 00000000017 00000 n</code>	XREF
<code>trailer << /Size 5 /Root 1 0 R >> startxref 1234 %%EOF</code>	TRAILER



FORMATOS DE ARCHIVOS DE DATOS



Analizando algunos Casos...

PDF (siglas en inglés de Portable Document Format)

HEADER	BODY	XREF	TRAILER
--------	------	------	---------



CUANDO LO EDITAMOS PASA LO SIGUIENTE:

HEADER	BODY	XREF	TRAILER		BODY	XREF	TRAILER
--------	------	------	---------	--	------	------	---------



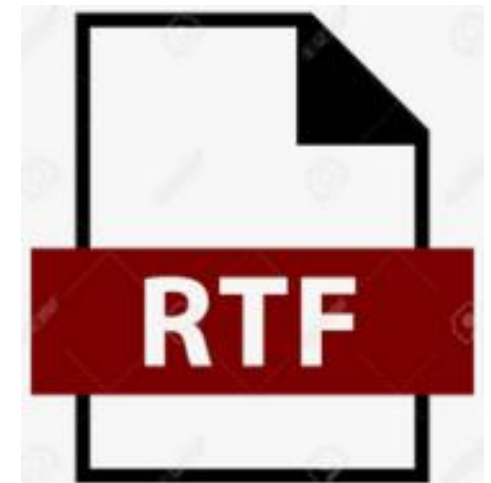
FORMATOS DE ARCHIVOS DE DATOS

Analizando algunos Casos... RTF (Rich Text Format)

Formato: *Texto enriquecido. desarrollado por Microsoft en 1987 para el intercambio de documentos multiplataforma. La mayoría de los procesadores de texto pueden leer y escribir documentos.*

Características:

- *Se puede leer en numerosos programas y plataformas.*
- *Es muy fácil de manejar*
- *No propaga virus.*
- *Problema Tamaño de los Archivos.*
- *No soporta todo tipo de imagen*





FORMATOS DE ARCHIVOS DE DATOS

Analizando algunos Casos... RTF (Rich Text Format)

Elementos Básicos: *Texto sin formato, Palabras de control, Símbolos de control y Grupos.*

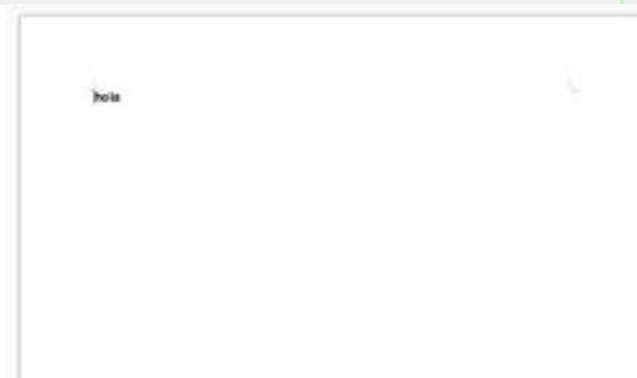
\PalabraControl<Delimitador>

```
{\rtf1\ansi\utf-8\deff0\deflang3082{\fonttbl{\f0\fnil\fcharset0 Arial;}}  
{\colortbl;\red0\green0\blue0;}  
\viewkind4\uc1\pard\cf1\b\fs24 hola\cf0\b0\par  
}
```

Link al Tutorial de RTF

link guia pocket RTF

link Wikipedia RTF





FORMATOS DE ARCHIVOS DE DATOS

Analizando algunos Casos... RTF (Rich Text Format)

```
{\rtf1\ansi\utf-8\deff0\deflang3082{\fonttbl{\f0\fnil\fcharset0 Arial;}}  
{\colortbl;\red255\green0\blue0;}  
\viewkind4\uc1\pard\cf1\b\fs24 Vamos la Academia\cf0\b0\par  
}
```



Encabezado RTF:

```
{\rtf1\ansi\utf-8\deff0\deflang3082{\fonttbl{\f0\fnil\fcharset0 Arial;}}
```

\rtf1: Indica que es un documento RTF versión 1.

\ansi: Utiliza la codificación de caracteres ANSI.

\utf-8: Define el uso de codificación UTF-8 para admitir caracteres especiales.

\deff0: Establece el idioma predeterminado en 0 (inglés).

\deflang3082: Establece el idioma principal en 3082 (español).



FORMATOS DE ARCHIVOS DE DATOS

Analizando algunos Casos... RTF (Rich Text Format)

```
{\rtf1\ansi\utf-8\deff0\deflang3082{\fonttbl{\f0\fnil\fcharset0 Arial;}}  
{\colortbl;\red255\green0\blue0;}  
\viewkind4\uc1\pard\cf1\b\fs24 Vamos la Academia\cf0\b0\par  
}
```

Tabla de fuentes:

```
{\fonttbl{\f0\fnil\fcharset0 Arial;}}
```

Tabla de colores:

```
{\colortbl;\red255\green0\blue0;}
```





FORMATOS DE ARCHIVOS DE DATOS

Analizando algunos Casos... RTF (Rich Text Format)

```
{\rtf1\ansi\utf-8\deff0\deflang3082{\fonttbl{\f0\fnil\fcharset0 Arial;}}  
{\colortbl;\red255\green0\blue0;}  
\viewkind4\uc1\pard\cf1\b\fs24 Vamos la Academia\cf0\b0\par  
}
```

Cuerpo del documento:

```
\viewkind4\uc1\pard\cf1\b\fs24 Vamos la Academia\cf0\b0\par
```

\viewkind4: Configura la vista en el documento.

\uc1: Utiliza la codificación UTF-8.

\pard: Reinicia el formato de párrafo.

\cf1: Cambia el color de la fuente al primer color de la tabla de colores (rojo).

\b: Activa la negrita.

\fs24: Establece el tamaño de fuente en 24 puntos.

Vamos la Academia: El texto que se mostrará.

\cf0: Restaura el color de la fuente al color predeterminado (negro).

\b0: Desactiva la negrita.

\par: Indica el final del párrafo.

\par → Fin de párrafo

\par\par\par → Fin de archivo



FORMATOS DE ARCHIVOS DE DATOS

Recordemos que para el intercambio de datos hoy utilizamos más:

- **JSON** (JavaScript Object Notation): Es un formato ligero y muy utilizado para el intercambio de datos en la web y aplicaciones.

Ejemplo:

```
{  
  "nombre": "Juan",  
  "edad": 30,  
  "ciudad": "Madrid"  
}
```

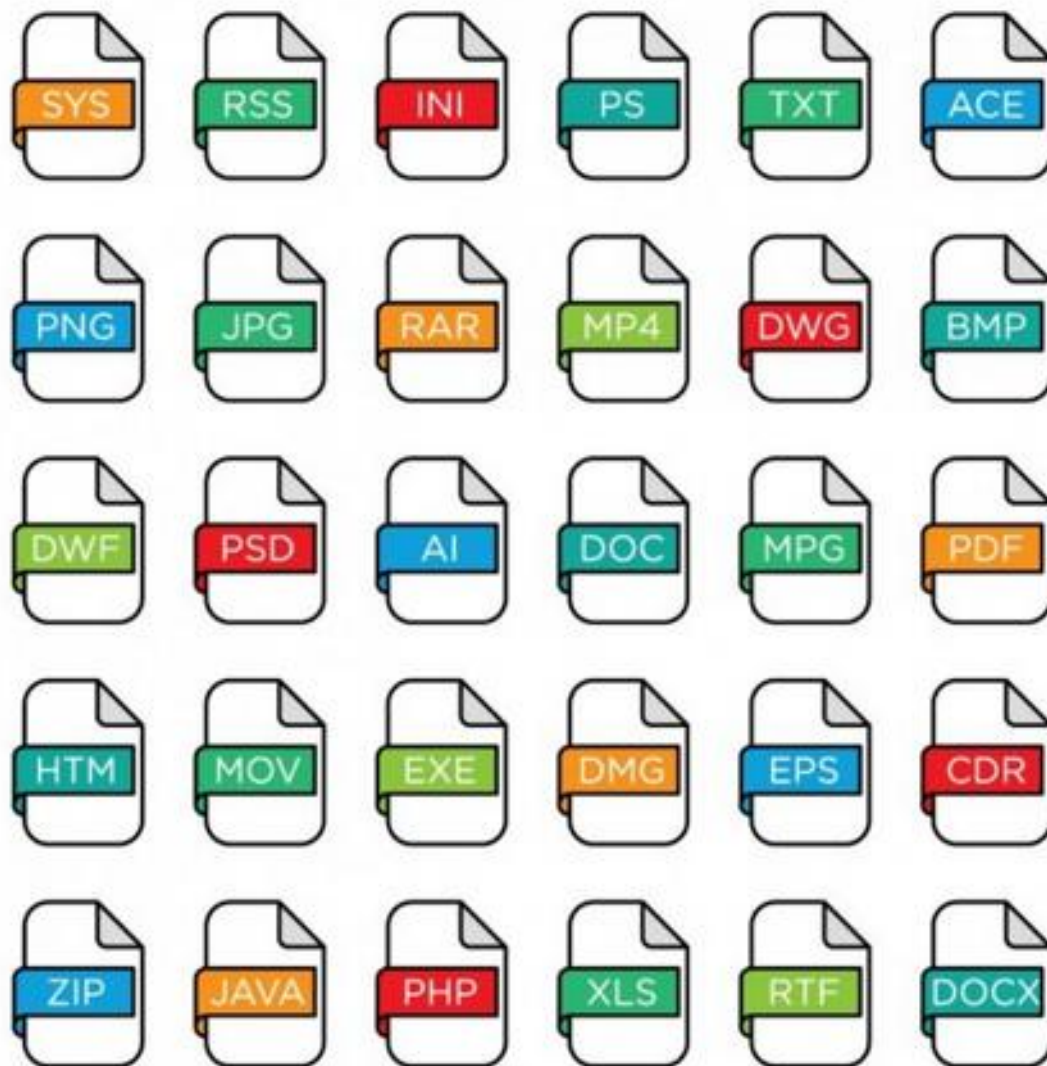
- **XML** (Extensible Markup Language): Un formato que organiza la información en etiquetas anidadas, utilizado en documentos y bases de datos.

Ejemplo:

```
<persona>  
  <nombre>Juan</nombre>  
  <edad>30</edad>  
  <ciudad>Madrid</ciudad>  
</persona>
```



FORMATOS DE ARCHIVOS DE DATOS





La comercialización



Software Libre

Open Source

Software con Dominio Público

Software Semi-libre

Freeware

Shareware

Software Propietario

Software Comercial (!= Propietario)

Adware (Publicidad)

Trial (te da un tiempito...)

Demo (Minusvalido... W95,200, w7,w8, etc...)

Crippleware (Versiones Lite)

Donationware (El autor quiere Donaciones)

Abandonware (Proyecto abandonado)





Las Licencias de software

Que es una Licencia de Software:

Es un **contrato** entre el **licenciante** (autor/titular de los derechos de explotación/distribuidor) y el **licenciatario** del programa informático (usuario consumidor /usuario profesional o empresa), para **utilizar** el **software** cumpliendo una serie de términos y condiciones establecidas dentro de sus cláusulas.



**No compramos el software!!!
Solo el permiso de Uso
con restricciones**



Las Licencias de software

Componentes de una Licencia de Software:

Personales: (*Licenciante, Garantía de titularidad y Licenciatario*).

Objetivos: (*Plazo y Precio*)

Tipos por Derecho de Autor:

Código abierto permisivas: (*BSD.MIT.Perl.PHP License v.3.0 ,etc.*)

Código abierto robustas: (**Herencia** a los cambios y a la obra generada).

Código abierto robustas fuertes: (*GPL V2.0,V3.0 , etc*)

Código abierto robustas débiles: (*GPL V2.1,Mozilla , etc*)

Código cerrado: (*Software propietario o Privativo*)

Dominio público (sin licencia) (*Vale todo!!*)

Destinatario: (*Distribuidores y Usuarios*)





SEGURIDAD INFORMATICA



¿Que es Seguridad?



Confiabilidad:

- **Confidencialidad**: Los Objetos son accesibles a elementos autorizados.
- **Integridad**: Modificados x Autorizados y controlados.
- **Disponibilidad**: Los Objetos están disponibles. Caso contrario--> Negación de Servicio.



SEGURIDAD INFORMATICA



Criptología

(del Griego Krypto y logos)
El estudio de lo oculto, lo escondido.



“Estudia problemas teoricos en la seguridad en el intercambio de mensajes”.

Dos Ramas: **Criptografia.**

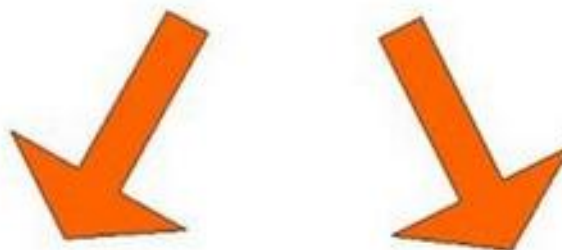


Criptoanalisis.





CRIPTOLOGÍA



CRIPTOGRAFÍA

Arte de Escribir
Mensajes Secretos

CRIPTOANÁLISIS

Arte de Descifrar
Mensajes Secretos



ELEMENTOS BÁSICOS DE LA CRIPTOGRAFÍA

EMISOR

MENSAJE

HOLA MUNDO

METODO
DE
CIFRADO



LLAVE

MENSAJE
CIFRADO

XSBRWIEIXA

EL MENSAJE CIFRADO NO
PUEDE SER ENTENDIDO POR
CUALQUIER INTERCEPTOR NO
AUTORIZADO



RECEPTOR

HOLA MUNDO

MENSAJE
DECIFRADO



METODO
DE
DESCIFRADO





SEGURIDAD INFORMATICA



CriptoSistemas



Definiremos un criptosistema como una quintupla (M, C, K, E, D) , donde:

- M representa el conjunto de todos los mensajes sin cifrar (lo que se denomina texto plano, o *plaintext*) que pueden ser enviados.
- C representa el conjunto de todos los posibles mensajes cifrados, o criptogramas.
- K representa el conjunto de claves que se pueden emplear en el criptosistema.
- E es el conjunto de *transformaciones de cifrado* o familia de funciones que se aplica a cada elemento de M para obtener un elemento de C . Existe una transformación diferente E_k para cada valor posible de la clave k .
- D es el conjunto de *transformaciones de descifrado*, análogo a E .

Todo criptosistema ha de cumplir la siguiente condición:

$$D_k(E_k(m)) = m$$



SEGURIDAD INFORMATICA

Criptografia

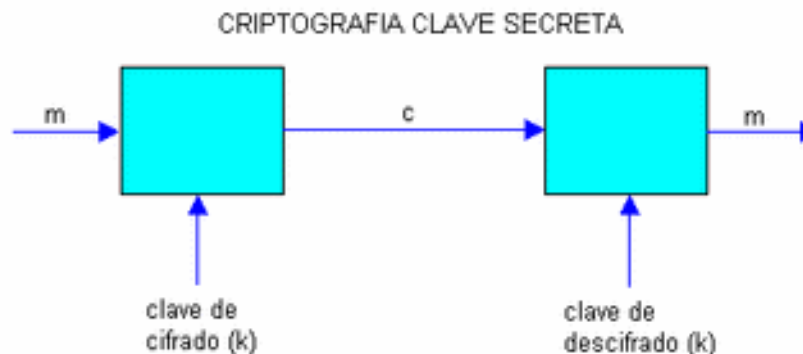


Criptosistema de clave Simetrica o Privada

Emplean la misma clave k tanto para cifrar como para descifrar

El problema es que para ser empleados en comunicaciones la clave k debe estar tanto en el emisor como en el receptor.

Como viaja la clave? Es segura?





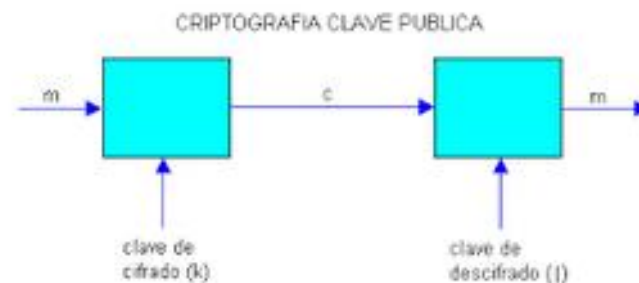
SEGURIDAD INFORMATICA

Criptografia



Criptosistemas Asimetricos o de clave Pública

Emplean una doble clave (k_p , k_P).
 k_p se conoce como clave privada
 k_P se conoce como clave pública.
 k_P sirve para la transformación E de cifrado.
 k_p para la transformación D de descifrado.



Estos criptosistemas deben cumplir además que el conocimiento de La clave pública k_P no permita calcular la clave privada k_p .

Pueden emplearse para establecer comunicaciones seguras por canales inseguros (SOLO viaja por el canal la **clave pública**, que sólo sirve para:

Cifrar.

Autenticar.



SEGURIDAD INFORMATICA



Otras formas de esconder

Esteganografía:

La idea es el empleo de canales subliminales.

Consiste en ocultar en el interior de una información, aparentemente inocua, otro tipo de información (cifrada o no).

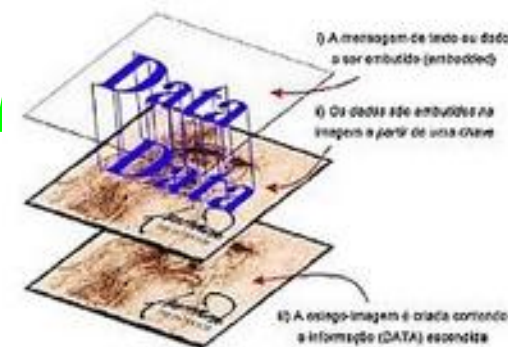
Muy utilizado ya permite burlar diferentes sistemas de control.

Un mensaje cifrado es detectable y al menos puede ser interrumpido. En cambio un mensaje camuflado es mas invisible.

El mensaje se envía como texto plano, pero entremezclado con cantidades ingentes de basura.

La técnica se la conoce como chaffing and winnowing, que vendría a traducirse como llenar de paja y separar el grano de la paja.

Creado por RIVEST en 1998 para como desafío a la política restrictiva del Gobierno de los EE.UU.





SEGURIDAD INFORMATICA

Criptologia



Criptoanálisis:

La idea es comprometer la seguridad de un criptosistema.

2 Formas: hacer descifrando un mensaje sin conocer la llave, o bien obteniendo a partir de uno o más criptogramas la clave que ha sido empleada en su codificación.

No es criptoanálisis el descubrimiento de un algoritmo secreto de cifrado.

Suponemos por el contrario que *los algoritmos siempre son conocidos.*



SEGURIDAD INFORMATICA

Criptologia



Criptoanálisis:

Algunas estimaciones probabilísticas de vocales y conectores
En castellano...

espacio	0.164
e	0.094
a	0.088
o	0.062
s	0.055
i	0.051
n	0.051
r	0.048
l	0.041
d	0.040
c	0.034
t	0.033
u	0.027
m	0.020
p	0.020
b	0.010
g	0.010
v	0.007
y	0.007
f	0.006
q	0.006
h	0.005
j	0.003
z	0.003
x	0.002
ñ	0.001
k	0.000
w	0.000

	frecuencia
de	0.0659
la	0.0351
a	0.0319
el	0.0270
en	0.0259
y	0.0258
que	0.0238
los	0.0157
del	0.0133
se	0.0111
las	0.0103
por	0.0098
con	0.0075
un	0.0073
su	0.0064
una	0.0063
no	0.0058
para	0.0057
al	0.0055
es	0.0053
lo	0.0042
pero	0.0017
ya	0.0012
hasta	0.0011



SEGURIDAD INFORMATICA

Criptologia



1. Escítala Espartana (siglo V a.C.):

- Método de transposición que usaba una vara alrededor de la cual se enrollaba un mensaje en una tira de pergamino. Para descifrar, se necesitaba una vara de idéntico grosor.

2. Cifrado Atbash (siglo VI a.C.):

- Cifrado de sustitución en el que cada letra es reemplazada por su opuesta en el alfabeto.

3. Cifrado César (siglo I a.C.):

- Atribuido a Julio César, este cifrado desplaza cada letra del mensaje un número fijo de posiciones en el alfabeto (ej. "A" a "D" con un desplazamiento de 3).

4. Cuadrado de Polibio (siglo II a.C.):

- Sistema de cifrado de sustitución en el que se organiza el alfabeto en una cuadrícula de coordenadas numéricas, donde cada letra se representa mediante una combinación de números.

5. Cifrado por Transposición (siglo II a.C. en adelante):

- Método que reorganiza el orden de las letras en el mensaje sin alterar las letras en sí. Ejemplos incluyen reordenamientos de columnas y otros patrones.



SEGURIDAD INFORMATICA

Criptologia



6. Cifrado de Alberti (siglo XV):

- Primer cifrado polialfabético desarrollado por Leon Battista Alberti, que usaba un disco para cambiar el alfabeto durante el cifrado.

7. Cifrado de Bellaso (siglo XVI):

- Inventado por Giovan Battista Bellaso, usaba una clave que se repetía para cifrar el mensaje, con desplazamientos variables en función de cada letra de la clave.

8. Cifrado de Vigenère (siglo XVI):

- Popularizado en Francia, es un cifrado polialfabético similar al de Bellaso que usa una clave repetida para cambiar el alfabeto de cada letra del mensaje.

9. Cifrado de Grille (siglo XVI):

- Cifrado por transposición que usaba una rejilla perforada para revelar el mensaje en ciertas posiciones. La rejilla se podía girar para cifrar mensajes más largos.

10. Cifrado de Playfair (1854):

- Creado por Charles Wheatstone y promovido por Lyon Playfair, este método cifra pares de letras en lugar de letras individuales, utilizando una cuadrícula de 5x5 basada en una palabra clave.



SEGURIDAD INFORMATICA

Criptologia



11. Máquina Jefferson o Cifrado Bazerics (1795):

- Usaba discos rotativos con diferentes alfabetos y era un precursor de los sistemas de cifrado mecánico moderno. Inventado por Thomas Jefferson y redescubierto en Francia.

12. Cifrado de Hill (1929):

- Un sistema polialfabético que utiliza matrices y álgebra lineal para cifrar bloques de texto. Fue creado por Lester S. Hill y es uno de los primeros cifrados en usar técnicas matemáticas complejas.

13. Cifrado de Vernam (One-Time Pad) (1917):

- Inventado por Gilbert S. Vernam, es un sistema simétrico que usa una clave aleatoria y de un solo uso de la misma longitud que el mensaje. Es teóricamente irrompible si se usa correctamente.

14. Máquina Enigma (década de 1920):

- Una máquina electromecánica desarrollada en Alemania que utilizaba rotores para cifrar mensajes. Fue ampliamente utilizada en la Segunda Guerra Mundial y es una de las primeras máquinas de cifrado complejas.



SEGURIDAD INFORMATICA

CriptoSistemas



El cuadrado de Polibio:

Inventado hacia 150 a. C. por el historiador Polibio, el cuadrado de Polibio fue utilizado principalmente por nihilistas rusos encerrados en las prisiones zaristas. Se trata de un algoritmo trivial, donde cada letra del alfabeto es reemplazada por las coordenadas de su posición en un cuadrado. Tomamos un cuadrado de Polibio con lugares cuadrados.

Es posible extenderlo a 36 para agregar cifras y signos de puntuación.

En este caso, pondremos la I y la J juntas para poder entrar en 25 lugares.

Carlos → 131142313443

	1	2	3	4	5
1	A	B	C	D	E
2	F	G	H	I, J	K
3	L	M	N	O	P
4	Q	R	S	T	U
5	V	W	X	Y	Z



SEGURIDAD INFORMATICA

CriptoSistema



Un metodo de los Confederados:

Es un tipo de sustitución polialfabética. Básicamente, se escoge una clave, que se va "sumando" al texto llano para dar lugar al texto cifrado. Eso de sumar significa que hacemos como si las letras fuesen números. Si hacemos $B=2$ y $C=3$, entonces $B+C=2+3=5=E$.

La Frase: HOY ES EL DIA

La clave:FE

H	O	Y		E	S		E	L		D	I	A
F	E	F		E	F		E	F		E	F	E
N	T	E		J	Y		J	P		I	Ñ	F

El criptograma:NTE JY JP IÑF



SEGURIDAD INFORMATICA

CriptoSistemas Criptografía Clasica



Cifrados Monoalfabéticos:

Sin desordenarlos símbolos dentro del mensaje, establecen una correspondencia única para todos ellos en todo el texto.

Es decir, si al símbolo A le corresponde el símbolo D, esta correspondencia se mantiene a lo largo de todo el mensaje.

El sistema Caesar: (Cesar, por Julio Cesar)
Alfabeto $A = \mathbb{Z}_m$ (enteros modulo m).

$$f(x) = ax + b, a \neq 0$$

$$1/f(x) = x - b).$$

Caesar utiliza $(1, b)$ es decir $a=1$.





SEGURIDAD INFORMATICA

Clasica

CESAR CON SALTO Y PALABRA:

Consiste en utilizar un salto y una palabra Clave.

A partir de la posición del salto escribir la palabra, sin repetir caracteres y completar



0	1	2	3	4	5	6	7	8	9	0	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26
A	B	C	D	E	F	G	H	I	J	K	L	M	N	Ñ	O	P	Q	R	S	T	U	V	W	X	Y	Z
					M	U	R	C	I	E	L	A	G	O												
T	V	W	X	Y	M	U	R	C	I	E	L	A	G	O	B	D	F	H	J	K	N	Ñ	P	Q	R	S



SEGURIDAD INFORMATICA

Clasica



Cifrados Polialfabéticos:

La sustitución aplicada a cada carácter varía en función de la posición que ocupe éste dentro del texto plano.

En realidad corresponde a la aplicación cíclica de n cifrados monoalfabéticos. Sin desordenarlos símbolos dentro del mensaje, establecen una

El sistema Vigénere: (Criptografo Frances):

Matriz de 26x 26. (a..z). desplazada de a 1 por fila.
clave= palabra de K letras ≥ 1 .

- 1) Una frase a encriptar y la clave.
- 2) Dividir la frase en grupos de k letras.
- 3) Repetir la clave por grupos de palabras de k letras.
- 4) Armar criptograma fila= frase, columna=clave.(letra

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
A	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
B	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A
C	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B
D	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C
E	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D
F	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E
G	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F
H	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G
I	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H
J	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I
K	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J
L	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K
M	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L
N	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M
O	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N
P	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
Q	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
R	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
S	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
T	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
U	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
V	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
W	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V
X	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
Y	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X
Z	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y



SEGURIDAD INFORMATICA

Criptografía



Cifrados de Transposición:

La idea es no sustituir unos símbolos por otros, sino que cambia su orden dentro del texto.

La Frase: "EL AMIGO DE MI AMIGO ES MI AMIGO"

La clave: Rotación 3,2,5,1,4

1	2	3	4	5											3	2	5	1	4
E	L		A	M												L	M	E	A
I	G	O		D											O	G	D	I	
E		M	I												M			E	I
A	M	I	G	O											I	M	O	A	G
	E	S		M											S	E	M		
I		A	M	I											A		I	I	M
G	O															O		G	

APLICAMOS ROTACIÓN {32514}

3	2	5	1	4
	L	M	E	A
O	G	D	I	
M			E	I
I	M	O	A	G
S	E	M		
A		I	I	M
	O		G	

APLICAMOS ROTACIÓN {32514}

El Criptograma : " LMEAOGDI M EIIMOAGSEM A IIM O G "

Los espacios valen!!!



SEGURIDAD INFORMATICA

Criptografía 2da Guerra



Máquinas de Rotores. La Máquina ENIGMA:

Se trataba de un instrumento, parecido a una máquina de escribir. Quien deseara codificar un mensaje sólo tenía que teclearlo y las letras correspondientes al mensaje cifrado se irían iluminando en un panel.

El destinatario copiaba dichas letras en su propia máquina y el mensaje original aparecía de nuevo.

La clave la constituían las posiciones iniciales de tres tambores o rotores





SEGURIDAD INFORMATICA

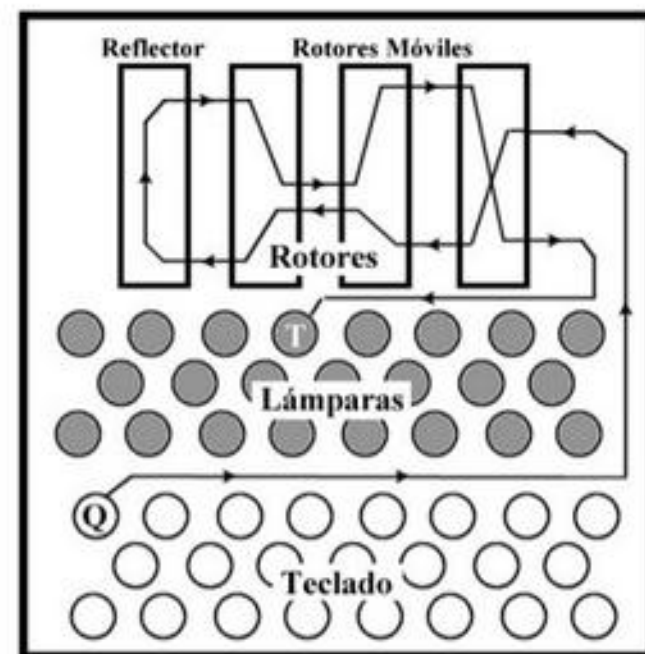


La Máquina ENIGMA:

Enigma era el nombre de una máquina que disponía de un mecanismo de cifrado rotatorio, que permitía usarla tanto para cifrar como para descifrar mensajes.

Durante la guerra: Le aplicaron varias mejoras, como incluir un pequeño sistema previo de permutación de letras, llamado Stecker. Incorporar rotores intercambiables

Se podían elegir y colocar en cualquier orden tres de entre cinco disponibles





SEGURIDAD INFORMATICA

Criptografía 2da Guerra



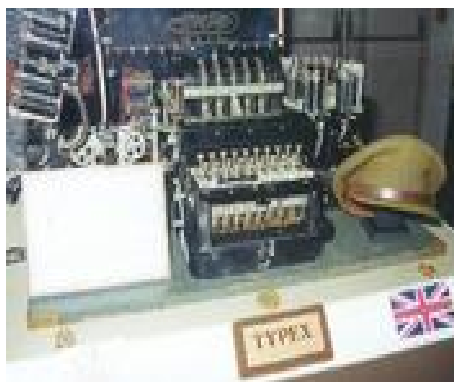
Otras Máquinas de Rotores:

Enigma fue la Estrella pero existieron otras maquinas de rotores en esa epoca **SIGABA** (USA) **TYPEX** (GB) ambas blanqueadas en el año 2000. Japon por su parte tuvo codigo RED y **PURPLE**.

SIGABA



TYPEX



PURPLE





SEGURIDAD INFORMATICA

Criptografía Moderna



Algoritmos Simétricos de Cifrado

La gran mayoría de los algoritmos de cifrado simétricos se apoyan en los conceptos de confusión y difusión inicialmente propuestos por Shannon.

Para conseguir algoritmos fuertes sin necesidad de almacenar tablas enormes es intercalar:

- **Connfusión** (sustituciones simples, con tablas pequeñas)
- **Difusión** (permutaciones).

En muchos casos el criptosistema no es más que un paso simple de sustitución-permutación repetido n veces, como ocurre con **DES**



SEGURIDAD INFORMATICA

Criptografía Moderna

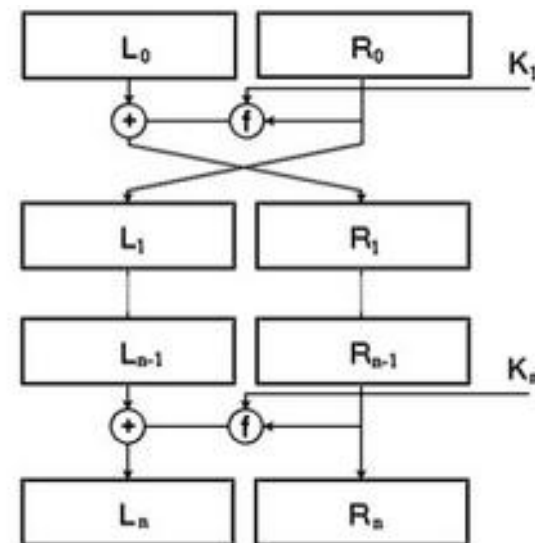


Redes de Feistel

Muchos de los cifrados de producto tienen en común que dividen un bloque de longitud n en dos mitades, **L** y **R**.

- 1) Seleccionar cadena, **N** de **64** o **128** bits dividir en **L** y **R**, de igual longitud ($N/2$)
- 2) Se toma una función, **F**, y una clave **K_i**
- 3) Se realizan una serie de operaciones con **F** y **K_i** y con **L** o **R** (solo uno de ellas)
La cadena obtenida intercambia por la cadena sin operar y se Hace nueva ronda

Utilizados por DES, Lucifer, FEAL, CAST, Blowfish,





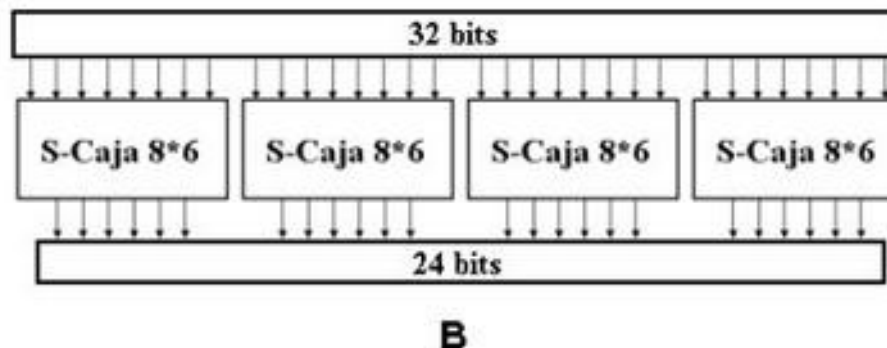
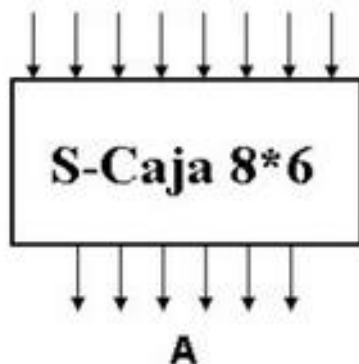
SEGURIDAD INFORMATICA

Criptografía Moderna



Cifrados con Estructura de Grupo

Como para poder construir buenos algoritmos de producto, intercambiamos sustituciones sencillas (confusión), con tablas pequeñas, y permutaciones (difusión). Estas tablas pequeñas de sustitución se denominan de forma genérica **S-Cajas**.



Se divide
por otro de n bits.

e sustituye



SEGURIDAD INFORMATICA

Criptografía Moderna

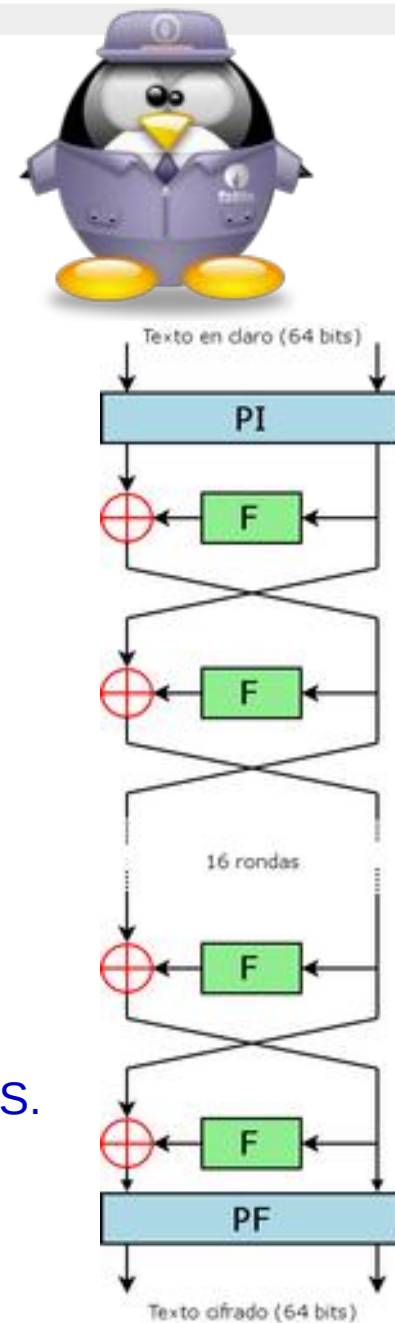
Criptosistema de clave secreta: (DES) Data Encryption Standard.

El original trabaja con 64 bit de cifrado. 56 de clave y 8 de paridad.

Divide la informacion en grupos de 64 bits.

Para cada grupo:

- aplica permutacion inicial fijo IP."desordena"
- divide en 2 grupos de 32
- para c/ subgrupo: Aplica 16 veces "vueltas", el algoritmo de cifrado compuesto de Permutaciones, substituciones y operacines
- Utilizado desde la decada del 70 Aprobado 1976.
- Reemplazado en el 2002 poe el AES Advanced Encryption Standard.
- Violado por fuerza bruta en 1998 Muchos usuarios de DES utilizan 3DES. (aplicar 3 veces DES)





SEGURIDAD INFORMATICA

Criptografía Moderna



Criptosistema de clave secreta: (IDEA) International Data Encryption Algorithm

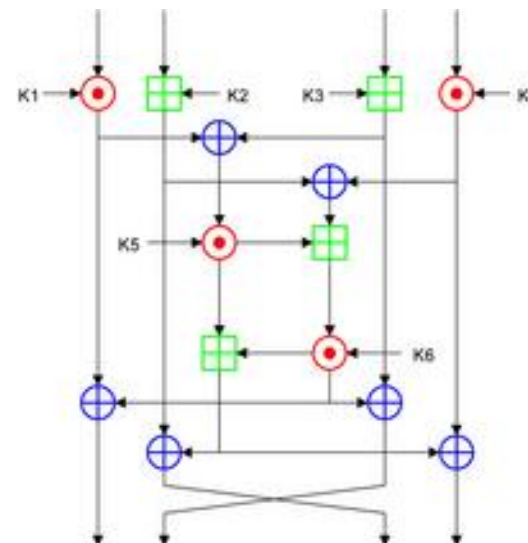
Trabaja con bloques de 64 bits de longitud y emplea una clave de 128 bits.

IDEA es un algoritmo bastante seguro, y hasta ahora se ha mostrado resistente a multitud de ataques, entre ellos el criptoanálisis diferencial.

Se basa en los conceptos de confusión y difusión, haciendo uso de las siguientes operaciones:

- XOR.
- Suma módulo 2^{16} .
- Producto módulo $2^{16} + 1$.

El algoritmo IDEA consta de ocho rondas.





SEGURIDAD INFORMATICA

Criptografía de Llave Pública

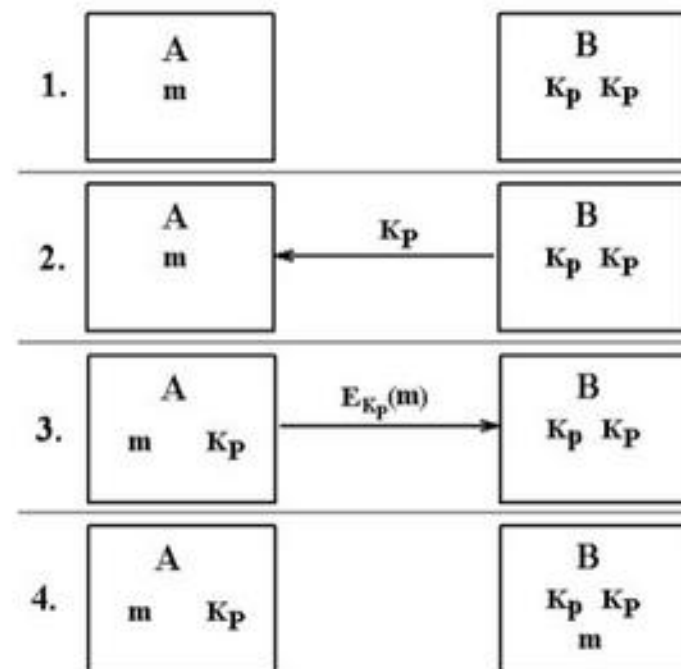


Algoritmos Asimétricos de Cifrado

Constan de dos claves diferentes en lugar de una, K_p y K_P , denominadas: clave privada y clave pública.

Una de ellas se emplea para codificar, mientras que la otra se usa para decodificar.

Dependiendo de la aplicación que le demos al algoritmo, la clave pública será la de cifrado o viceversa.





SEGURIDAD INFORMATICA

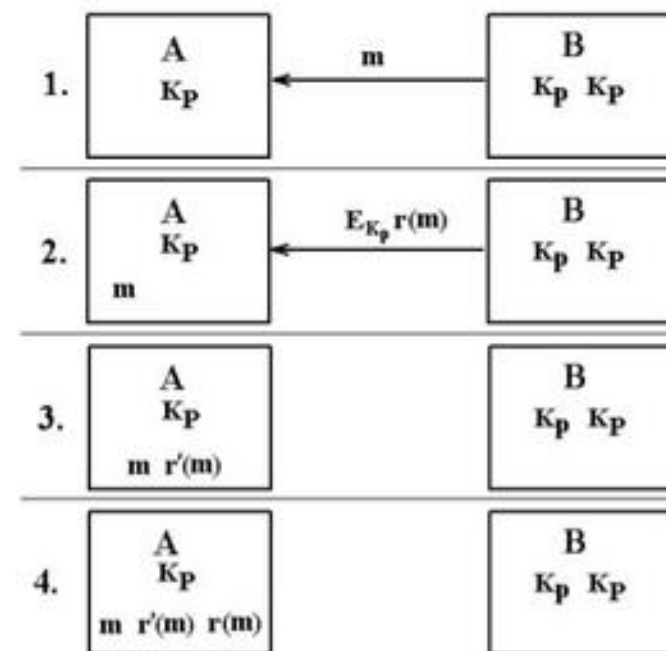
Criptografía de Llave Pública



Autenticación

La segunda aplicación de los algoritmos asimétricos es la autenticación de mensajes, con ayuda de funciones resumen

Las Fnciones de Resumen permiten obtener una firma a partir de un mensaje.





SEGURIDAD INFORMATICA

Criptografía de Llave Pública



El Algoritmo RSA

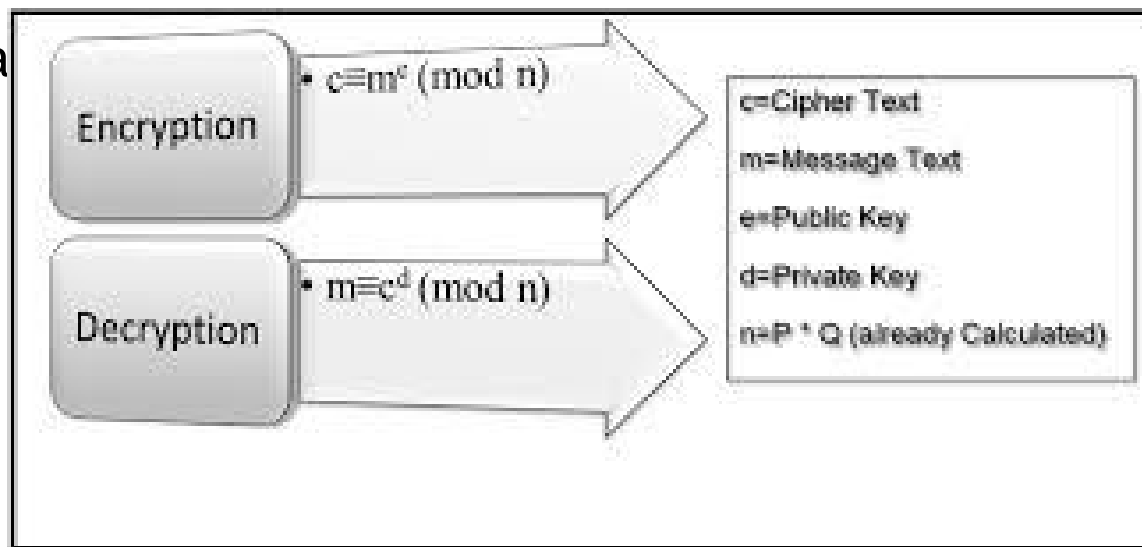
Su nombre proviene de sus inventores:

Ron Rivest, Adi Shamir y Leonard Adleman.

Desde su nacimiento nadie ha conseguido probar o rebatir su seguridad, pero se le tiene como uno de los algoritmos asimétricos más seguros.

RSA se basa en la dificultad para

Factorizar grandes números.





SEGURIDAD INFORMATICA

Criptografía de Llave Pública



Otros Algoritmos de Clave publica:

- **Codificación de ElGamal**
 - Cifrado basado en Logaritmo Discreto
- **Algoritmo de Rabin**
 - raíces cuadradas módulo un número compuesto
- **DSA (Digital Signature Algorithm)**
 - Firma digital



SEGURIDAD INFORMATICA

Criptografía de Llave Pública



Ejemplo de cifrado de mensaje: Ana envía un mensaje a David



- 1- Ana redacta un mensaje
- 2- Ana cifra el mensaje con la clave pública de David
- 3- Ana envía el mensaje cifrado a David a través de internet, ya sea por correo electrónico, mensajería instantánea o cualquier otro medio
- 4- David recibe el mensaje cifrado y lo descifra con su clave privada
- 5- David ya puede leer el mensaje original que le mandó Ana



SEGURIDAD INFORMATICA

Criptografía de Llave Pública



Ejemplo firma digital de clave asimétrica: David envía un mensaje a Ana



- 1- David redacta un mensaje
- 2- David firma digitalmente el mensaje con su clave privada
- 3- David envía el mensaje firmado digitalmente a Ana a través de internet, ya sea por correo electrónico, mensajería instantánea o cualquier otro medio
- 4- Ana recibe el mensaje firmado digitalmente y comprueba su autenticidad usando la clave pública de David
- 5- Ana ya puede leer el mensaje con total seguridad de que ha sido David el remitente



SEGURIDAD INFORMATICA

Criptografía de Llave Pública



La propagación de la confianza: (Puede darse como)

- **Infraestructura de clave pública o PKI.**
 - Entidades emisoras de certificados (Autoridades de certificación o CA del inglés Certification Authority)
- **Establecimiento de una web de confianza.**
 - No hay nodos aparte de los usuarios.
 - Los usuarios recogen claves públicas de otros usuarios y aseguran su autenticidad.
- **criptografía basada en identidad.**
 - PKG (acrónimo de Private Key Generator)
 - Hay un Generador que reparte las Claves Públicas y privada a quien corresponda.
- **Criptografía basada en certificados.**
 - En este modelo el usuario posee una clave privada y otra pública.
 - La clave pública la envía a una Autoridad de certificación que basándose en criptografía basada en identidad genera un certificado que asegura la validez de los datos.
- **criptografía sin certificados.**
 - KGC (acrónimo de Key Generator Center) es una clave parcial.
 - La clave privada completa se genera a partir de la clave privada parcial y un valor generado aleatoriamente por el usuario



SEGURIDAD INFORMATICA





ORGANIZACIÓN DE DATOS 2021

Conceptos de compresión de datos





Compresión

¿Por qué es importante conocer sobre compresión?

- Casi toda la información se maneja en forma digital (Redes Sociales, digitalización de documentos, audio, video).
- Cada vez se necesita más capacidad de almacenamiento y transmisión de información (cloud computing)
- Año a año se requiere un mejor uso de los recursos (tv-digital, telefonía IP, teleconferencias, redes en general). El contenido en alta definición o 4k necesita de altísimos anchos de banda y tasas de datos sin comprimir.



Compresión

¿Qué área de la ciencia aplica?

- La compresión como álgebra, análisis matemático, probabilidad y estadística y algoritmia aplicados.
- Aplica a Teoría de la Información.



Compresión

Teoría de la información

Según la teoría de la información de Shannon, definimos:

- **Fuente:** es todo aquello que emite el mensaje
- **Mensaje:** Es un conjunto de alfabetos que es transmitida por la fuente. En nuestro caso el alfabeto es 0 y 1.
- **Código:** Es el conjunto de alfabetos que representan el mensaje de acuerdo a reglas o convenciones.
- **Información:** Es el conjunto mínimo de alfabetos para que se requieran para representar un mensaje.



Compresión

Tipo de compresión

- Con perdida (lossy)
 - Imágenes
 - Audio
 - Video
- Sin perdida (lossless)



Compresión

Existen distintos tipo de fuentes de datos:

- Estructurada
- Desestructurada



Compresión

Entropía de una fuente

Ya hablamos del concepto de información, veremos ahora como se mide la misma . De acuerdo a la teoría de la información, el nivel de información de una fuente se puede medir según la entropía de la misma. Los estudios sobre la entropía son de suma importancia en la teoría de la información y se deben principalmente a Shannon, existen a su vez un gran numero de propiedades respecto de la entropía de variables aleatorias debidas a Kolmogorov.

Definimos a la probabilidad de ocurrencia(P_i) de un mensaje en una fuente como la cantidad de apariciones de dicho mensaje dividido el total de mensajes. L_i es la longitud del código utilizado para representar a dicho mensaje

$$H = \text{Sumatoria } (i=0 \rightarrow n) P_i * L_i$$

"H" se lo denomina "Entropía de la fuente". La entropía de la fuente determina el nivel de compresión que podemos obtener como máximo para un conjunto de datos.



Compresión

Código

Un código es decodificable sí y solo sí un código solo puede corresponder a un único mensaje.

Ejemplo:

a=0
b=01
C=10

Si el decodificador recibe el código: "0010" no puede distinguir si el mensaje original fue "aba" o "aac", ya que puede interpretarlo como 0 01 0 o como 0 0 10

En Cambio si los codigos se escribe asi:

a=0
b=10
c=11

El código "01011" solo puede corresponder a "abc"



Compresión

Compresión de Datos

La idea fundamental es muy simple, guardar la mayor cantidad de datos posibles en la menor cantidad de espacio posible y que dado el archivo comprimido pueda recuperarse el archivo original.

Teorema fundamental de la compresión de datos.

"No existe un algoritmo que sea capaz de comprimir cualquier conjunto de datos"

Técnicamente:

"Si un compresor comprime un conjunto de datos en un factor F entonces necesariamente expande a algún otro conjunto de datos en un factor MAYOR que F "

Compresión RLE

El método de compresión RLE (Run Length Encoding, a veces escrito RLC por Run Length Coding) es utilizado por muchos formatos de imagen (BMP, PCX, TIFF). Se basa en la repetición de elementos consecutivos.

El principio fundamental consiste en codificar un primer elemento al dar el número de repeticiones de un valor y después el valor que va a repetirse

En comunicaciones : ITU T-REC-T.45, MNP5

-En su forma más básica se basa en “contar” la repetición de determinado símbolos

Ejemplo :

AAAAAAAAAgh

Se comprime en

8Agh

Compresión RLE

En realidad, la compresión RLE está regida por reglas particulares que permiten que se ejecute la compresión cuando sea necesario y que se deje la cadena como está cuando la compresión genere pérdida. Las reglas son las siguientes:

Si se repiten tres o más elementos consecutivamente, se utiliza el método de compresión RLE.

De lo contrario, se inserta un carácter de control seguido del número de elementos de la cadena no comprimida.

Su principal ventaja radica en que RLE se puede usar aun cuando no se pueda analizar el texto o imagen completa a enviar

Compresión

Compresión estadística

Los compresores estadísticos conforman la columna vertebral de la compresión de datos. La idea básica es: El compresor "adivina" cual va a ser el input y escribe menos bits al output si adivinó correctamente. Notemos que el descompresor debe "adivinar" de la misma forma que el compresor para poder decodificar el archivo.

Los compresores estadísticos buscan representar a cada carácter con un número de bits proporcional a $-\log_2(P_i)$ (longitud ideal de acuerdo a la entropía). Siendo P_i probabilidad de aparición de caracteres

Compresión Estadística

Compresión estadística o basada en la entropía

El objetivo de la compresión estadística es en definitiva determinar cual deberá ser la longitud de los códigos correspondientes a los caracteres de un archivo, el valor de los códigos puede ser cualquiera siempre y cuando se respeten las longitudes, veremos a continuación una técnica que se utiliza frecuentemente para obtener las longitudes de estos códigos.

La compresión estadística se vale para poder comprimir una determinada fuente en el carácter estructurado de la misma, aquellas fuentes que son totalmente aleatorias, es decir de información pura, no pueden comprimirse.

El 90% de la información que se almacena en una computadora es de carácter netamente estructurado por lo que podremos comprimirla con facilidad. Cuanto mejor sea para archivos no aleatorios y peor para archivos random mejor será un algoritmo de compresión estadístico.

Compresión

Compresión estadística

Ejemplo:

Por ejemplo si solo tenemos cuatro caracteres a,b,c y d y el compresor sabe que la probabilidad de "a" es 0.5, "b" es 0.25, "c" es 0.25 y "d" es 0. Entonces intentaremos codificar de la forma:

- 1 bit para "a" ($-\log_2(.5)=1$)
- 2 bits para b y c ($-\log_2(.25)=2$)

Por ejemplo:

- a : 0
- b : 10
- c : 11

Como "d" tiene probabilidad de ocurrencia cero ni siquiera lo codificamos.

Si todos los caracteres ASCII son equiparables para un archivo entonces usaremos $\log_2(1/256) = 8$ bits para codificar cada uno, es decir que no podemos comprimir el archivo.

Compresión

Compresión estadística

Una forma de evitar el redondeo es ablocando símbolos, por ejemplo:

AA : 0

AB : 10

BA : 110

BB : 111

Con lo cual A es representado en alrededor de 0.7 bits y B en 1.4 bits (valores no enteros). El ablocar códigos para lograr valores enteros para los bloques que correspondan a valores no-enteros de los caracteres es, sin embargo, una tarea que puede llegar a resultar mucho mas costosa que el proceso de compresión en sí, por lo que su utilidad es únicamente teórica.



Compresión

Entropía de una fuente

Ya hablamos del concepto de información, veremos ahora como se mide la misma . De acuerdo a la teoría de la información, el nivel de información de una fuente se puede medir según la entropía de la misma. Los estudios sobre la entropía son de suma importancia en la teoría de la información y se deben principalmente a Shannon, existen a su vez un gran numero de propiedades respecto de la entropía de variables aleatorias debidas a Kolmogorov.

Definimos a la probabilidad de ocurrencia(P_i) de un mensaje en una fuente como la cantidad de apariciones de dicho mensaje dividido el total de mensajes. L_i es la longitud del código utilizado para representar a dicho mensaje

$$H = \text{Sumatoria } (i=0 \rightarrow n) P_i * L_i$$

"H" se lo denomina "Entropía de la fuente". La entropía de la fuente determina el nivel de compresión que podemos obtener como máximo para un conjunto de datos.

Compresión estadística

Huffman estático

Este algoritmo desarrollado por Huffman en 1952 es el mas popular y utilizado de los compresores estadísticos existiendo numerosas variantes, su popularidad se basa en que se demuestra que un árbol de Huffman es óptimo para la compresión de una cierta cantidad de caracteres ('cualquier' otro árbol que se use para comprimir genera un archivo igual o mayor al generado por Huffman).

Método :

Se le asigna a cada carácter que aparece en el archivo su frecuencia de aparición, caracteres con frecuencia cero se descartan pues no van a aparecer.

Se tienen pues n conjuntos de un caracter cada uno, con su respectiva frecuencia, a continuación mientras la cantidad de conjuntos sea mayor que uno se toman los dos conjuntos cuya sumatoria de frecuencias sea menor y se unen. Cuando queda un único conjunto, se ha formado un árbol binario (con el código Huffman).

Compresión

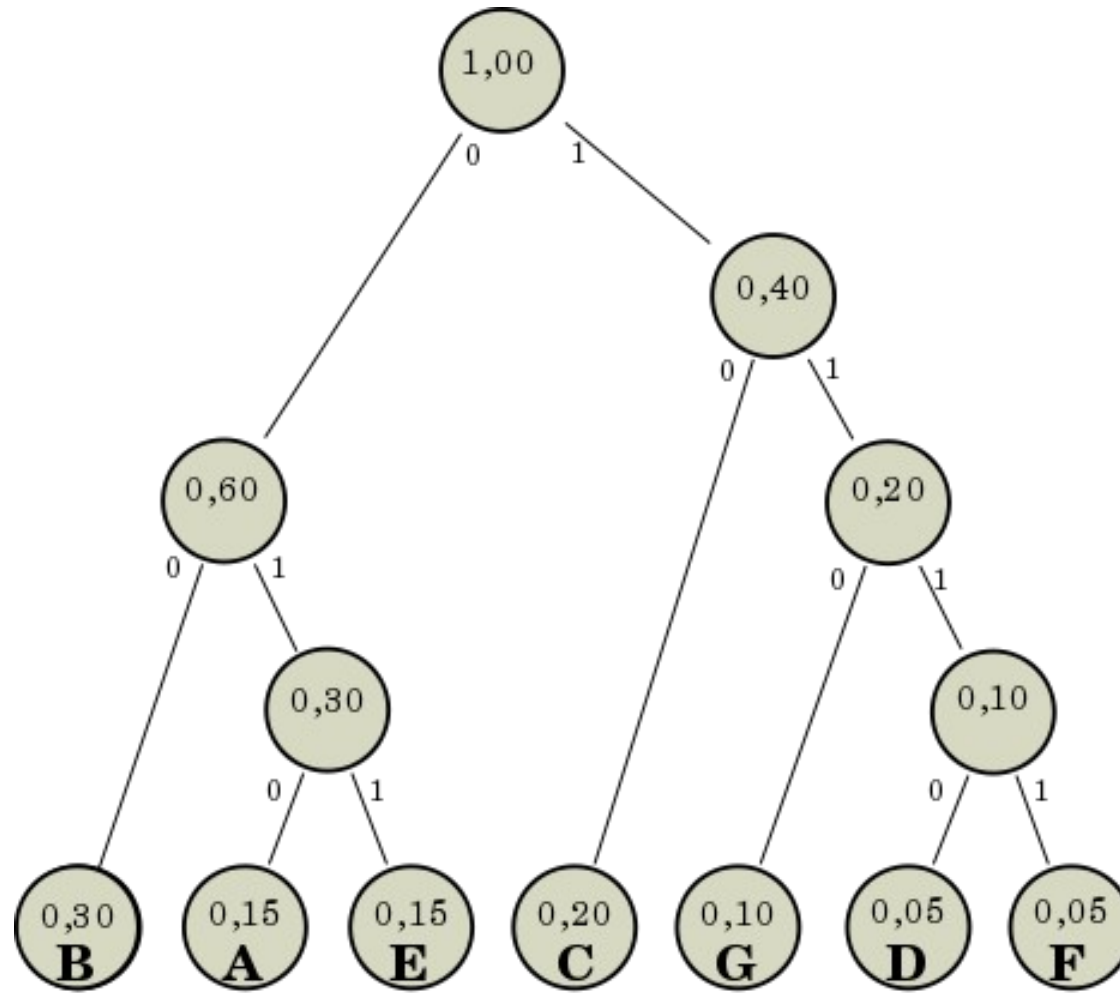
Huffman estático

Algoritmo

- 1.- Armar Histograma
- 2.- Agrupar de a dos caracteres siempre que su frecuencia de aparición sea menor a todos y se arma un nuevo símbolo, con la frecuencia de aparición de los dos anteriores.
- 3.- Repetir el punto dos hasta que quede un árbol binario donde el nodo principal tenga cadena con todos los caracteres.

El descompresor deberá primero recuperar el árbol y luego va descomprimiendo procesando bit por bit, comienza en la raíz del árbol y va doblando según encuentre ceros o unos, cuando llega a una hoja emite el carácter encontrado y vuelve a comenzar desde la raíz. El fin del archivo se indica al descompresor pre-concatenando al archivo comprimido la longitud del archivo original o bien inventando un carácter nuevo (código 256) que indique el fin de archivo.

Código de Huffman - Ejemplo



Compresión – Clasificación según su input

Básicamente cualquier algoritmo de compresión puede clasificarse en tres categorías según como se procesa el archivo de entrada.

- **Algoritmos estáticos:** Leen una vez el archivo para recolectar estadísticas y luego otra vez para comprimirlo.
- **Algoritmos semi-estáticos:** Levantan en memoria un bloque del archivo, lo procesan en forma estática y luego graban el bloque comprimido. Solo hacen una lectura del archivo original pero cada bloque es procesado dos veces (en memoria).
- **Algoritmos dinámicos:** Leen una vez el archivo de entrada y lo van comprimiendo a medida que lo leen.

Compresión – La necesidad de comprimir con pérdida - (loosy)

Tecnología de Red	Ancho de banda
Dial Up	56 kbps
ADSL lite	1.5 Mbps
T1	1.54 Mbps
ADSL 1	4 Mbps
Ethernet classic	10 Mbps
Wifi 802.11b	11 Mbps
ADSL 2+	24 Mbps
T3	44 Mbps
Wifi 802.11g	54 Mbps
Fast ethernet	100 Mbps
802.11n	150-300 Mbps
Gigabit ethernet	1 Gbps

imagenes

PNG ~ 2 – 4 kB

GIF ~ 6 – 800 kB

JPG ~ 9 – 10 MB

TIFF ~ 900 – 10 MB

BMP ~ 900 – 50 MB

Documentos

DOCX ~4 – 8 kB

PDF ~ 18 – 20 kB

ODT ~ 80 – 90 kB

Archivos multimedia

eBook ~ 1 – 5 MB

MP3 song ~ 3 – 15 MB

DVD Movie ~ 4 GB

HD Movie ~ 5 – 8 GB

Blu-Ray Movie ~ 20 – 25 GB

Sin comprimir

24-bit, 1080i @ 60 [fps](#): $24 \times 1920 \times 540 \times 60 = 1.49$ [Gbit/s](#)

24-bit, 1080p @ 60 [fps](#): $24 \times 1920 \times 1080 \times 60 = 2.98$ Gbit/s.



Compresión - JPEG

Las imágenes en formato crudo (RAW), almacenan los píxeles en formato RGB, por lo que su tamaño es proporcional a la resolución

Compresión de imágenes

El estándar JPEG (Grupo Unido de Expertos en Fotografía, del inglés Joint Photographic Experts Group) para comprimir imágenes fijas de tono continuo (es decir, fotografías) se desarrolló gracias a los expertos fotográficos que trabajan bajo los auspicios unidos de la ITU, ISO e IEC, otro organismo de estándares



Compresión – Archivos RAW

Los archivos RAW contienen la información necesaria para producir una imagen visible a partir de los datos del sensor de la cámara.

Utilidades

exifprobe

nconvert -fullinfo



Compresión - JPEG

El paso 1 es la preparación del bloque. Con fines de especificación, supongamos que la entrada JPEG es una imagen RGB de 640 x 480 con 24 bits/píxel. RGB no es el mejor modelo de color para utilizar en la compresión.

El ojo es mucho más sensible a la luminosidad o brillo de las señales de video que a la crominancia o color de las mismas. Por ende, calculamos primero la luminosidad Y y las dos crominancias, Cb y Cr, a partir de los componentes R, G y B. Las siguientes fórmulas se utilizan para valores de 8 bits que varían de 0 a 255:

$$Y = 165 + 0.26R + 0.50G + 0.09B$$

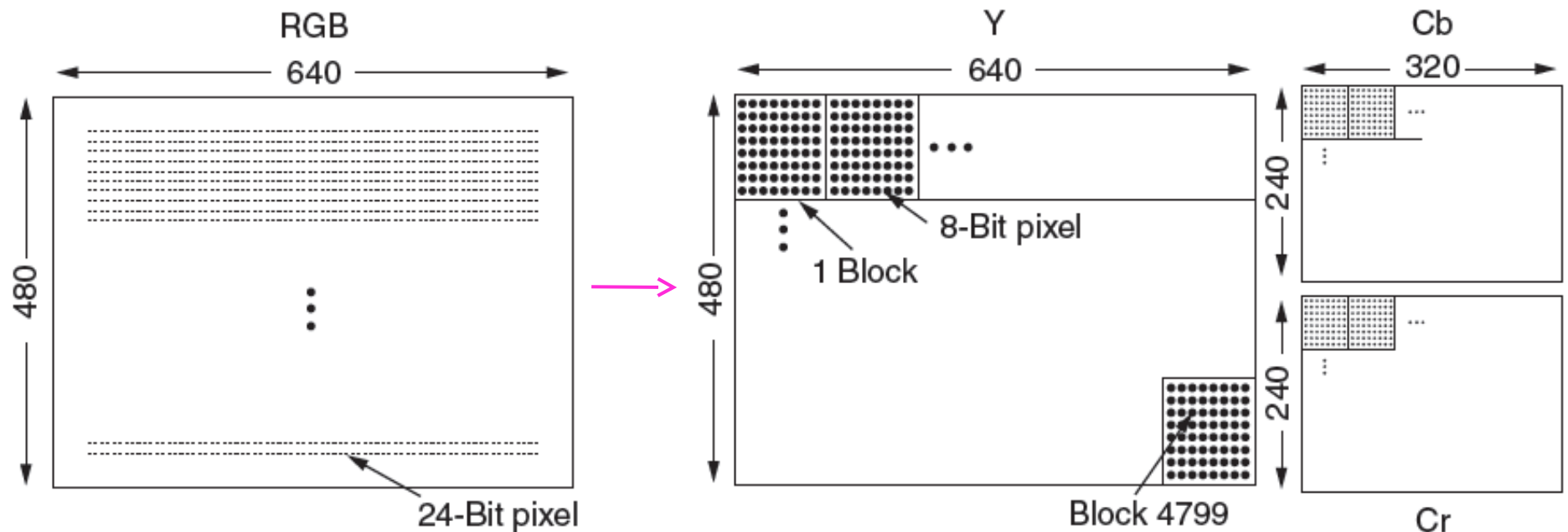
$$Cb = 128 + 0.15R + 0.29G - 0.44B$$

$$Cr = 128 + 0.44R + 0.37G + 0.07B$$

Compresión - JPEG

Los pixeles se mapean al espacio de luminancia/crominancia (YCbCr) y se sub-samplean

- El ojo es menos sensitivo a la crominancia



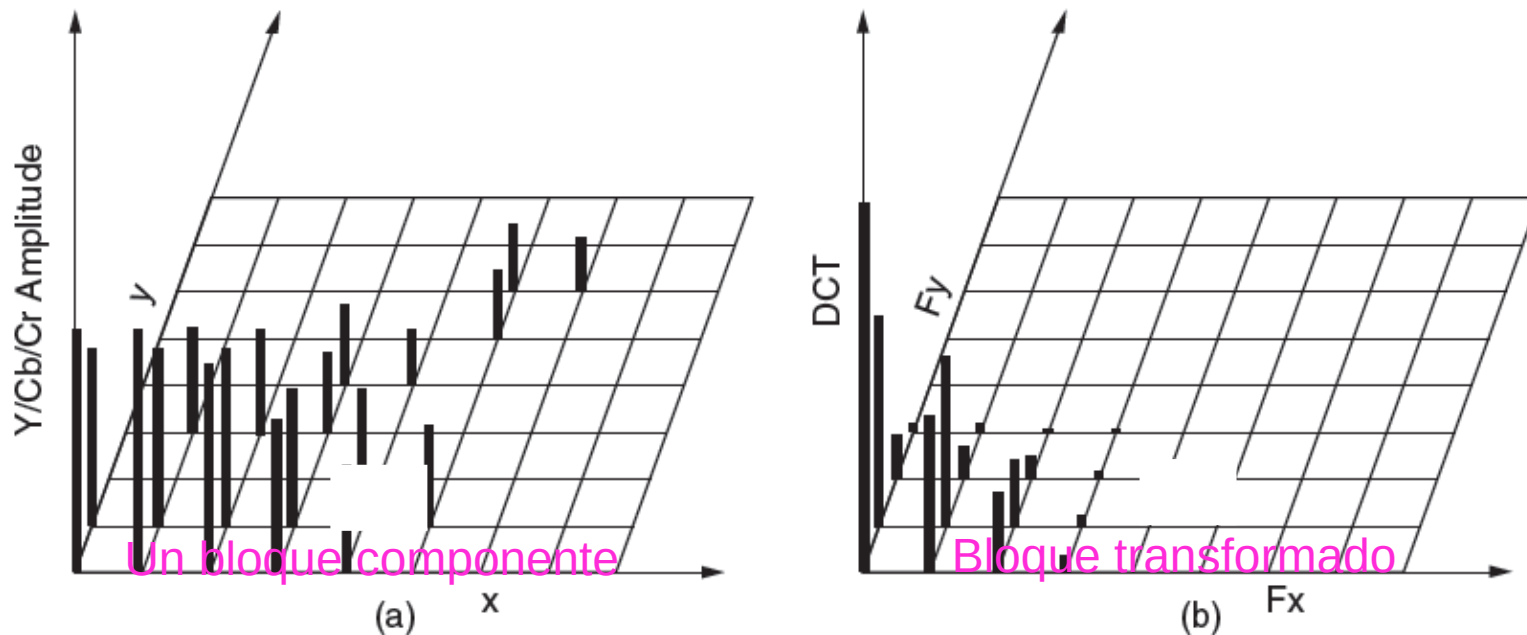
Entrada 24-bit RGB pixels

8-bit luminance
pixels

8-bit chrominances
every 4 pixels for

Compresión - JPEG

Paso 2: Cada bloque componente se transforma a frecuencias en el dominio espacial con DCT (Discrete Cosine Transformation), esto comprime por perder precisión(datos), pero captura los componentes clave



Compresión - JPEG

Paso 3: Los coeficientes DCT se cuantizan dividiendo por los valores en la tabla de cuantización; reduces bits in higher spatial frequencies

- El elemento superior izquierdo se diferencia sobre los bloques (paso 4)

DCT coefficients

150	80	40	14	4	2	1	0
92	75	36	10	6	1	0	0
52	38	26	8	7	4	0	0
12	8	6	4	2	1	0	0
4	3	2	0	0	0	0	0
2	2	1	1	0	0	0	0
1	1	0	0	0	0	0	0
0	0	0	0	0	0	0	0

Input

Quantization table

1	1	2	4	8	16	32	64
1	1	2	4	8	16	32	64
2	2	2	4	8	16	32	64
4	4	4	4	8	16	32	64
8	8	8	8	8	16	32	64
16	16	16	16	16	16	32	64
32	32	32	32	32	32	32	64
64	64	64	64	64	64	64	64

Thresholds

Quantized coefficients

150	80	20	4	1	0	0	0
92	75	18	3	1	0	0	0
26	19	13	2	1	0	0	0
3	2	2	1	0	0	0	0
1	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

Output

Compresión - JPEG

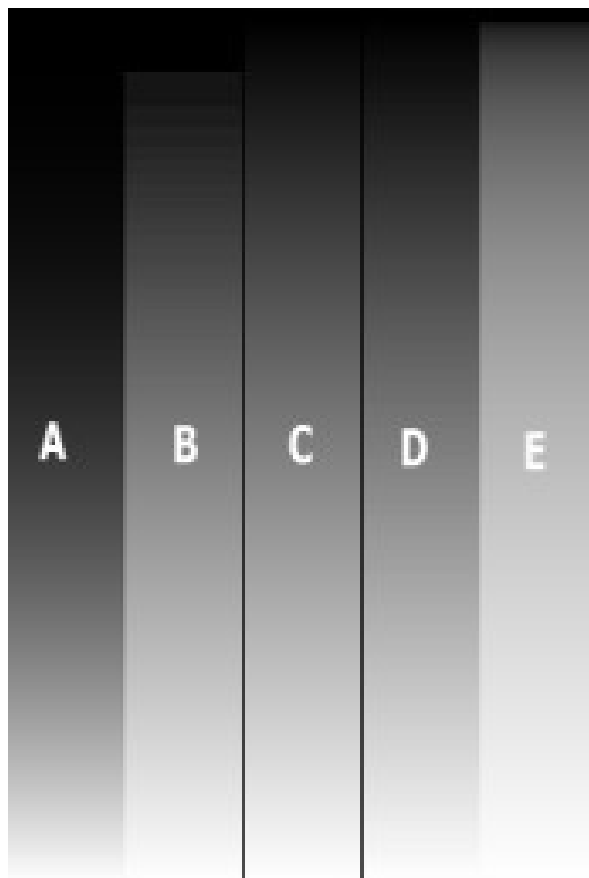
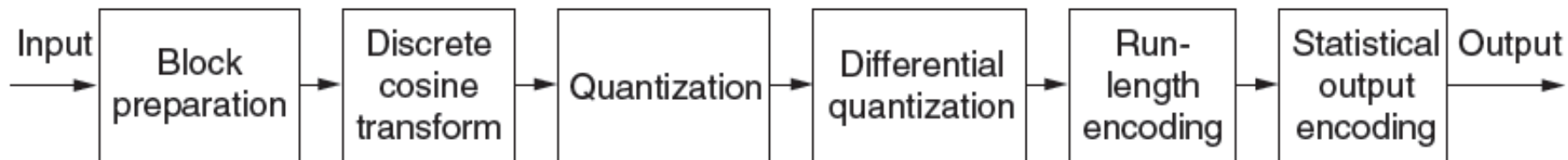
Paso 5: Se le aplica al bloque run-length codificada en zig-zag. Luego se codifica con Huffman (Paso 6)

150	80	20	4	1	0	0	0
92	75	18	3	1	0	0	0
26	19	13	2	1	0	0	0
3	2	2	1	0	0	0	0
1	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

Orden en que los coeficientes son enviados

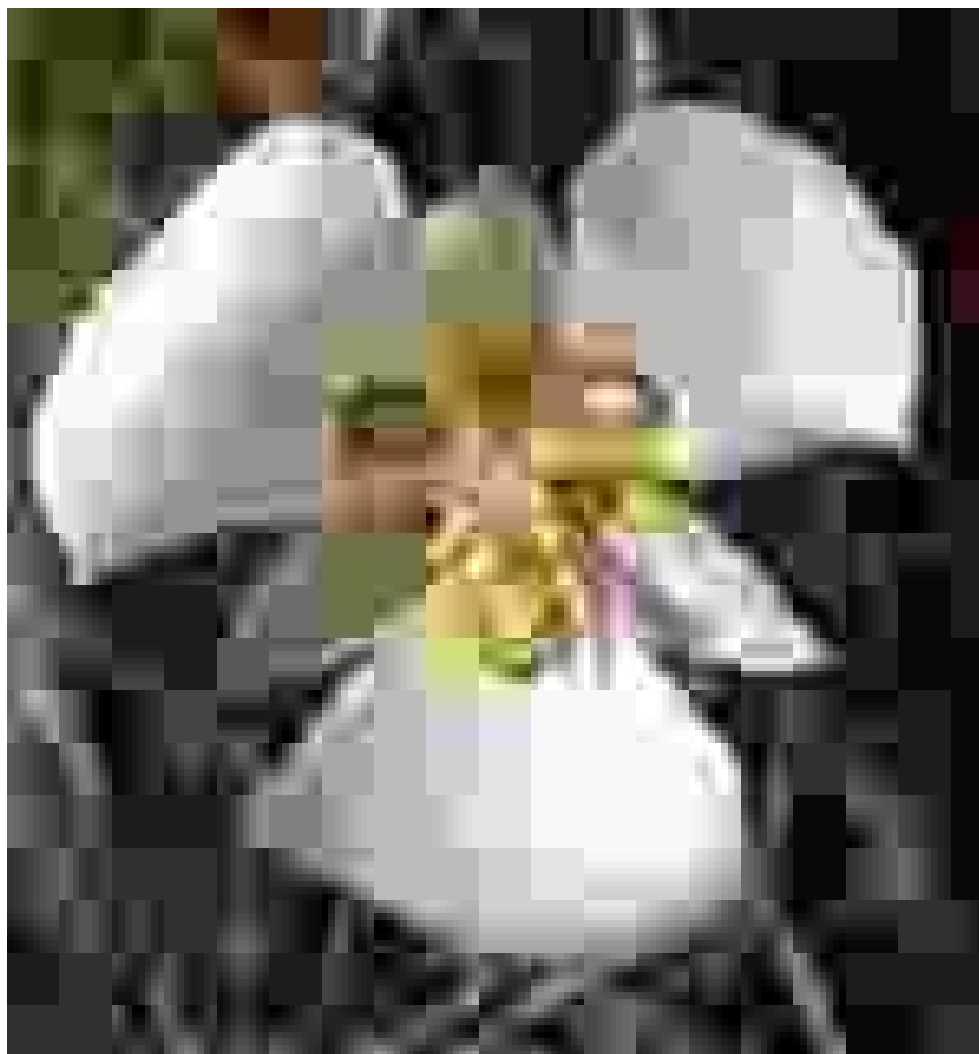


Compresión - JPEG





Compresión - JPEG

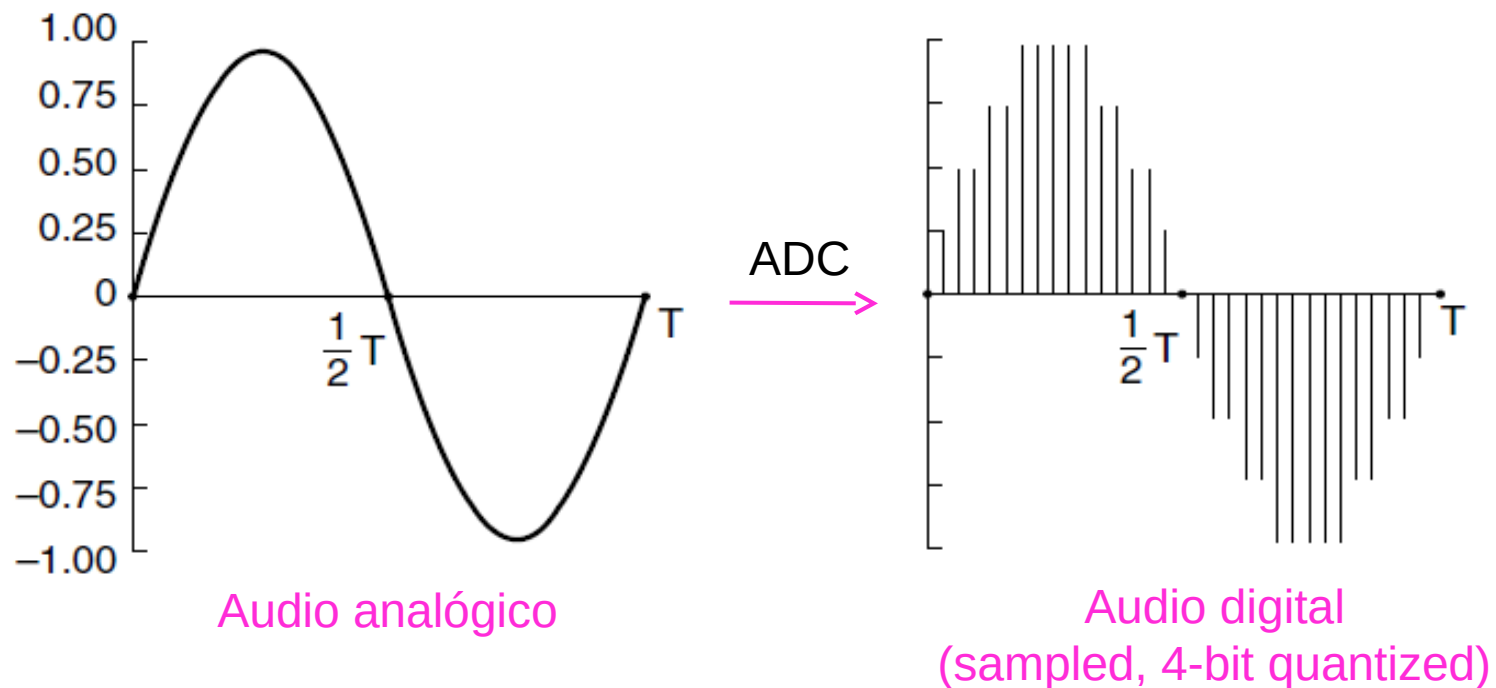


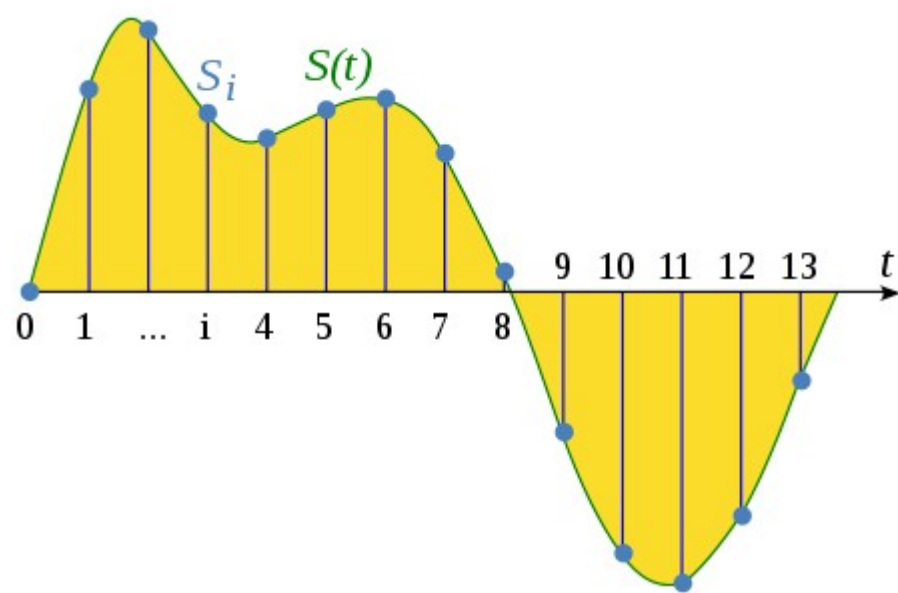
Text Caption

Compresión – Audio digital

Los ADC (Analog-to-Digital Converter) producen audio digital desde un microfono – Teorema del muestreo : 2 x ancho de banda

- Telefono: 8000 8-bit samples/segundo (64 Kbps); el audio en la computadora es usualmente mejor (e.g., 16 bit)

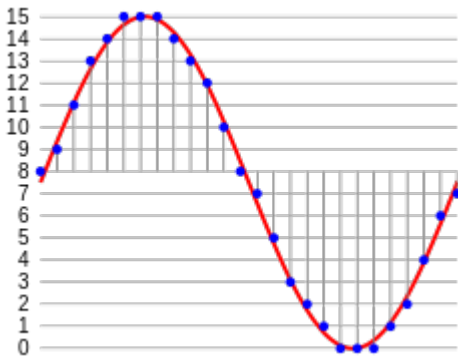






Compresión – Audio digital

El rango de frecuencia del oído humano es de 20 Hz a 20 000 Hz. Algunos animales, en particular los perros, pueden escuchar frecuencias más altas. El oído escucha el ruido en forma logarítmica, por lo que la proporción de dos sonidos con potencia A y B se expresa de manera convencional en dB (decibelees) como la cantidad $10 \log_{10} (A/B)$



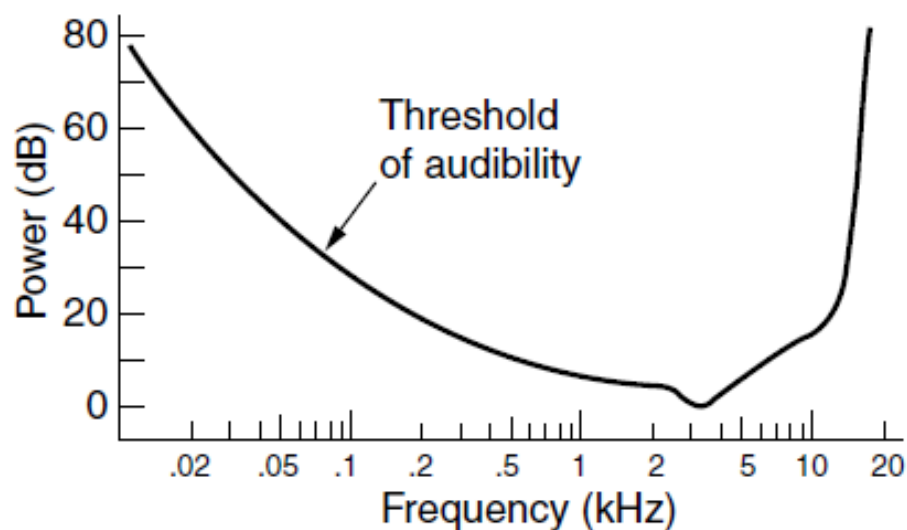
Ejemplo audio en calidad CD:
44000 muestras/seg * 16 bit
muestra => 704000 bits/seg * 2
canales = >1.4 Mbits / seg



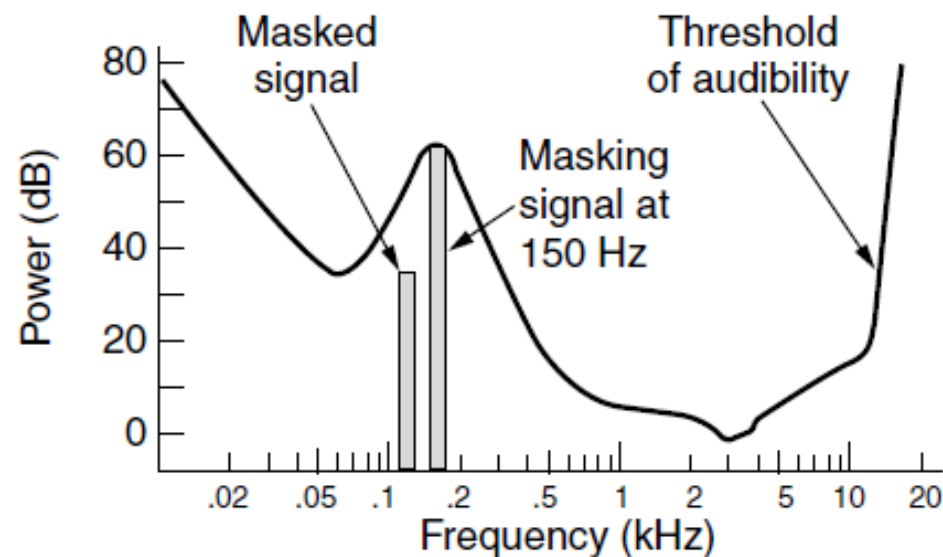
Compresión – Audio digital

El audio digital es usualmente comprimido con modelos psicoacústicos

- Los encoders (como AAC o MP3) explotan la percepción humana



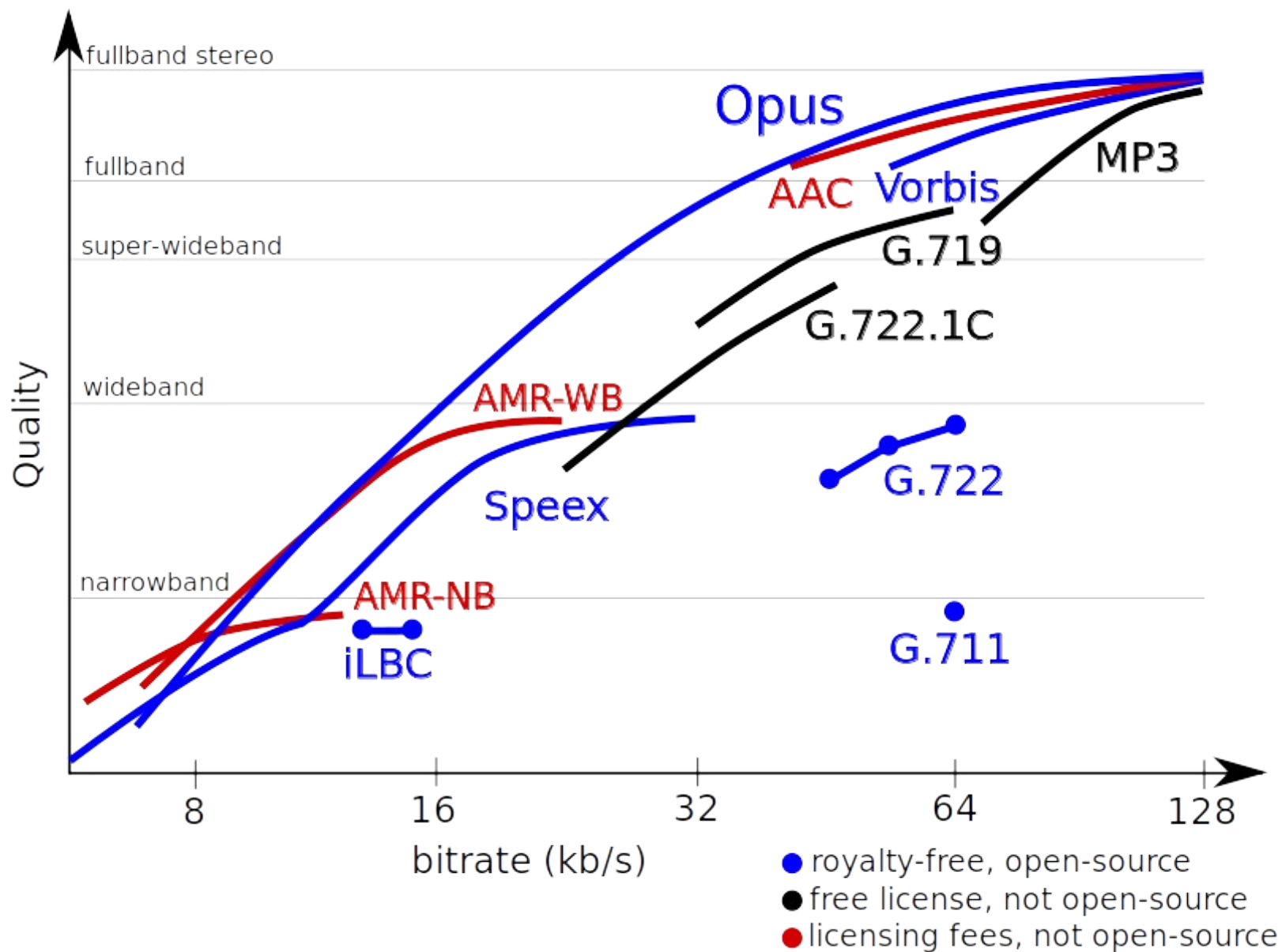
La sensibilidad del oído
varía con la frecuencia



Un tono fuerte puede
enmascarar un tono suave



Compresión – Audio digital





Compresión – video digital

Requerimientos de ancho de banda del video sin comprimir (RAW), segun su resolución, norma y profundidad de colores

525 NTSC uncompressed (YCbCr 4:2:2 chroma subsampling)

8-bit @ 720x486 @ 29.97 fps = 20 MB/s, or 70 GB/h.

10-bit @ 720x486 @ 29.97 fps = 27 MB/s, or 94 GB/h.

625 PAL uncompressed

8-bit @ 720x576 @ 25 fps = 20 MB/s, or 70 GB/h.

10-bit @ 720x576 @ 25 fps = 26 MB/s, or 93 GB/h.

720p HDTV uncompressed

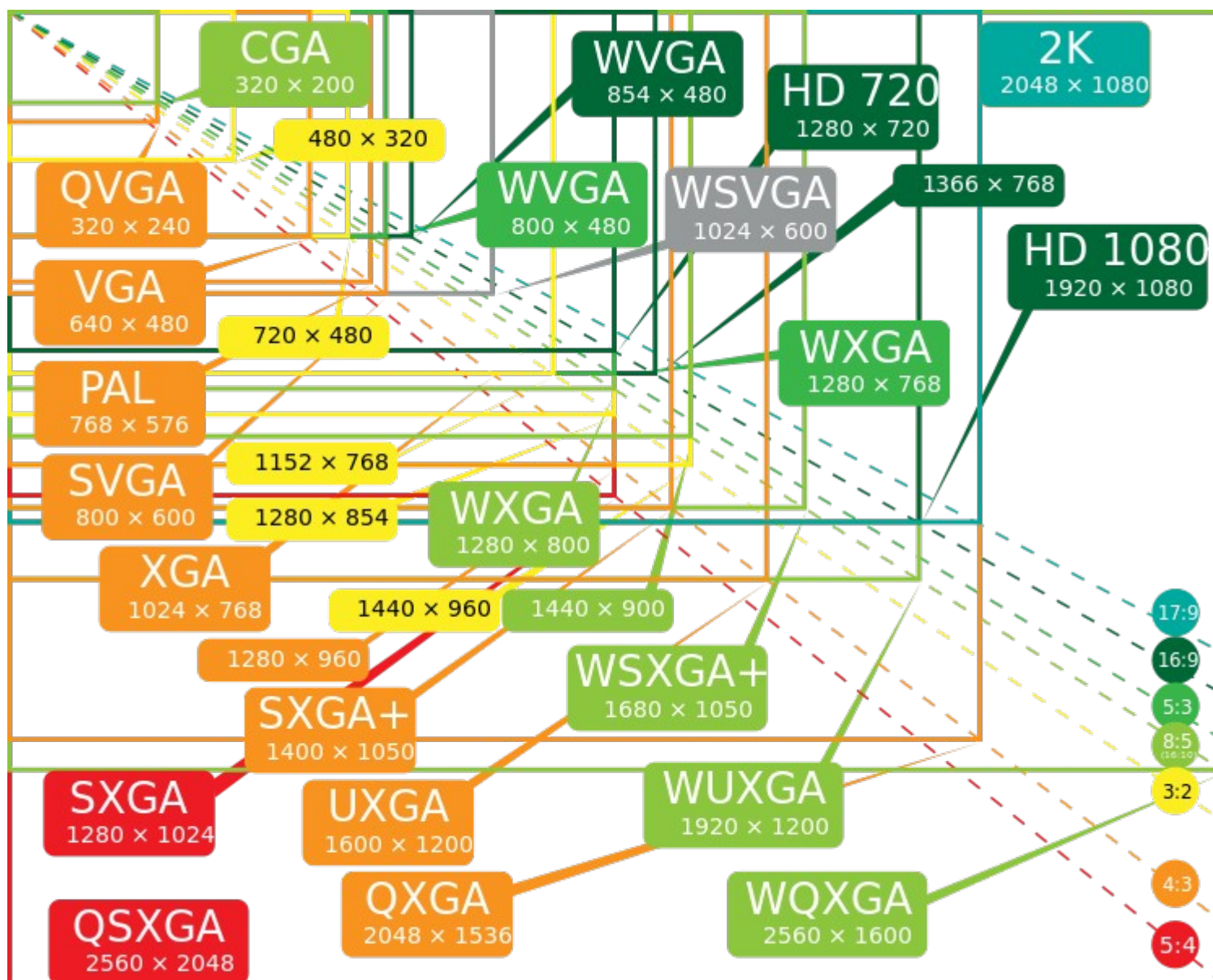
8-bit @ 1280x720 @ 59.94 fps = 105 MB/s, or 370 GB/h.

10-bit @ 1280x720 @ 59.94 fps = 140 MB/s, or 494 GB/h

8-bit @ 1920x1080 @ 29.97 fps = 119 MB/s, or 417 GB/h.

10-bit @ 1920x1080 @ 29.97 fps = 158 MB/s, or 556 GB/h.

Compresión – video digital





Compresión – Video digital

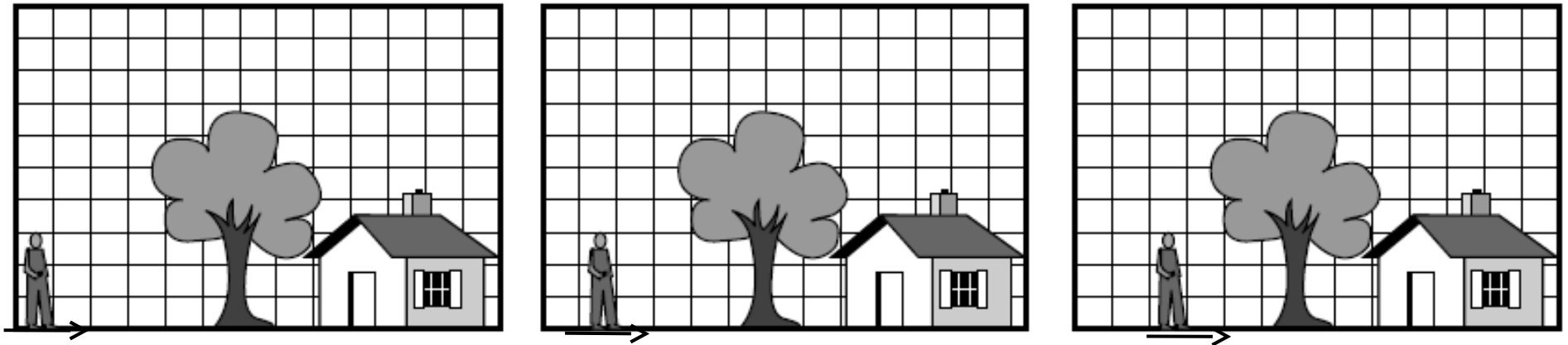
El objetivo es aprovechar la redundancia espacial y temporal

La redundancia espacial se resuelve aplicando JPEG a cada cuadro (DCT)

Compresión – video digital

MPEG comprime sobre una secuencia de cuadros, usando una técnica llamada motion tracking para remover redundancia temporal

- I (Intra-coded) cuadros son autocontenidos
- P (Predictive) cuadros usan predicción de block motion
- B (Bidirectional) cuadros pueden ser la predicción base para un cuadro futuro



Tres cuadros consecutivos con componentes estacionarios y en movimiento

Standard ECMA-404

2nd Edition / December 2017

The JSON Data Interchange Syntax

Standard



COPYRIGHT PROTECTED DOCUMENT

Contents

Page

1	Scope	1
2	Conformance	1
3	Normative references	1
4	JSON Text.....	2
5	JSON Values	2
6	Objects.....	3
7	Arrays	3
8	Numbers	3
9	String	4



Introduction

JSON* is a text syntax that facilitates structured data interchange between all programming languages. JSON is a syntax of braces, brackets, colons, and commas that is useful in many contexts, profiles, and applications. JSON stands for JavaScript Object Notation and was inspired by the object literals of JavaScript aka ECMAScript as defined in the ECMAScript Language Specification, Third Edition [1]. However, it does not attempt to impose ECMAScript's internal data representations on other programming languages. Instead, it shares a small subset of ECMAScript's syntax with all other programming languages. The JSON syntax is not a specification of a complete data interchange. Meaningful data interchange requires agreement between a producer and consumer on the semantics attached to a particular use of the JSON syntax. What JSON does provide is the syntactic framework to which such semantics can be attached

JSON syntax describes a sequence of Unicode code points. JSON also depends on Unicode in the hex numbers used in the `\u` escapement notation.

JSON is agnostic about the semantics of numbers. In any programming language, there can be a variety of number types of various capacities and complements, fixed or floating, binary or decimal. That can make interchange between different programming languages difficult. JSON instead offers only the representation of numbers that humans use: a sequence of digits. All programming languages know how to make sense of digit sequences even if they disagree on internal representations. That is enough to allow interchange.

Programming languages vary widely on whether they support objects, and if so, what characteristics and constraints the objects offer. The models of object systems can be wildly divergent and are continuing to evolve. JSON instead provides a simple notation for expressing collections of name/value pairs. Most programming languages will have some feature for representing such collections, which can go by names like `record`, `struct`, `dict`, `map`, `hash`, or `object`.

JSON also provides support for ordered lists of values. All programming languages will have some feature for representing such lists, which can go by names like `array`, `vector`, or `list`. Because objects and arrays can nest, trees and other complex data structures can be represented. By accepting JSON's simple convention, complex data structures can be easily interchanged between incompatible programming languages.

JSON does not support cyclic graphs, at least not directly. JSON is not indicated for applications requiring binary data.

It is expected that other standards will refer to this one, strictly adhering to the JSON syntax, while imposing semantics interpretation and restrictions on various encoding details. Such standards may require specific behaviours. JSON itself specifies no behaviour.

Because it is so simple, it is not expected that the JSON grammar will ever change. This gives JSON, as a foundational notation, tremendous stability.

JSON was first presented to the world at the JSON.org website in 2001. A definition of the JSON syntax was subsequently published as IETF RFC 4627 in July 2006. ECMA-262, Fifth Edition (2009) included a normative specification of the JSON grammar. This specification, ECMA-404, replaces those earlier definitions of the JSON syntax. Concurrently, the IETF published RFC 7158/7159 and in 2017 RFC 8259 as updates to RFC 4627. The JSON syntax specified by this specification and by RFC 8259 are intended to be identical.

* Pronounced /'dʒeɪ·sən/, as in "Jason and The Argonauts".

This Ecma Standard was developed by Technical Committee 39 and was adopted by the General Assembly of December 2017.

"COPYRIGHT NOTICE

© 2017 Ecma International

This document may be copied, published and distributed to others, and certain derivative works of it may be prepared, copied, published, and distributed, in whole or in part, provided that the above copyright notice and this Copyright License and Disclaimer are included on all such copies and derivative works. The only derivative works that are permissible under this Copyright License and Disclaimer are:

- (i) works which incorporate all or portion of this document for the purpose of providing commentary or explanation (such as an annotated version of the document),*
- (ii) works which incorporate all or portion of this document for the purpose of incorporating features that provide accessibility,*
- (iii) translations of this document into languages other than English and into different formats and*
- (iv) works by making use of this specification in standard conformant products by implementing (e.g. by copy and paste wholly or partly) the functionality therein.*

However, the content of this document itself may not be modified in any way, including by removing the copyright notice or references to Ecma International, except as required to translate it into languages other than English or into a different format.

The official version of an Ecma International document is the English language version on the Ecma International website. In the event of discrepancies between a translated version and the official version, the official version shall govern.

The limited permissions granted above are perpetual and will not be revoked by Ecma International or its successors or assigns.

This document and the information contained herein is provided on an "AS IS" basis and ECMA INTERNATIONAL DISCLAIMS ALL WARRANTIES, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY WARRANTY THAT THE USE OF THE INFORMATION HEREIN WILL NOT INFRINGE ANY OWNERSHIP RIGHTS OR ANY IMPLIED WARRANTIES OF MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE



The JSON Data Interchange Syntax

1 Scope

JSON is a lightweight, text-based, language-independent syntax for defining data interchange formats. It was derived from the ECMAScript programming language, but is programming language independent. JSON defines a small set of structuring rules for the portable representation of structured data.

The goal of this specification is only to define the syntax of valid JSON texts. Its intent is not to provide any semantics or interpretation of text conforming to that syntax. It also intentionally does not define how a valid JSON text might be internalized into the data structures of a programming language. There are many possible semantics that could be applied to the JSON syntax and many ways that a JSON text can be processed or mapped by a programming language. Meaningful interchange of information using JSON requires agreement among the involved parties on the specific semantics to be applied. Defining specific semantic interpretations of JSON is potentially a topic for other specifications. Similarly, language mappings of JSON can also be independently specified. For example, ECMA-262 defines mappings between valid JSON texts and ECMAScript's runtime data structures.

2 Conformance

A conforming JSON text is a sequence of Unicode code points that strictly conforms to the JSON grammar defined by this specification.

A conforming processor of JSON texts should not accept any inputs that are not conforming JSON texts. A conforming processor may impose semantic restrictions that limit the set of conforming JSON texts that it will process.

3 Normative References

The following referenced documents are indispensable for the application of this document. For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments) applies.

ISO/IEC 10646, *Information Technology – Universal Coded Character Set (UCS)*

The Unicode Consortium. The Unicode Standard <http://www.unicode.org/versions/latest>.

Bray, T., Ed. "The JavaScript Object Notation (JSON) Data Interchange Format", RFC 8259.

This specification and [RFC 8259] both provide specifications of the JSON grammar but do so using different formalisms. The intent is that both specifications define the same syntactic language. If a difference is found between them, Ecma International and the IETF will work together to update both documents. If an error is found with either document, the other should be examined to see if it has a similar error, and fixed if possible. If either document is changed in the future, Ecma International and the IETF will work together to ensure that the two documents stay aligned through the change. RFC 8259, also defines various semantic restrictions on the use of the JSON syntax. Those restrictions are not normative for this specification.

4 JSON Text

A JSON text is a sequence of tokens formed from Unicode code points that conforms to the JSON value grammar. The set of tokens includes six structural tokens, strings, numbers, and three literal name tokens.

The six structural tokens:

[U+005B left square bracket
{ U+007B left curly bracket
] U+005D right square bracket
} U+007D right curly bracket
: U+003A colon
, U+002C comma

These are the three literal name tokens:

true U+0074 U+0072 U+0075 U+0065
false U+0066 U+0061 U+006C U+0073 U+0065
null U+006E U+0075 U+006C U+006C

Insignificant whitespace is allowed before or after any token. Whitespace is any sequence of one or more of the following code points: character tabulation (U+0009), line feed (U+000A), carriage return (U+000D), and space (U+0020). Whitespace is not allowed within any token, except that space is allowed in strings.

5 JSON Values

A JSON value can be an *object*, *array*, *number*, *string*, *true*, *false*, or *null*.

value

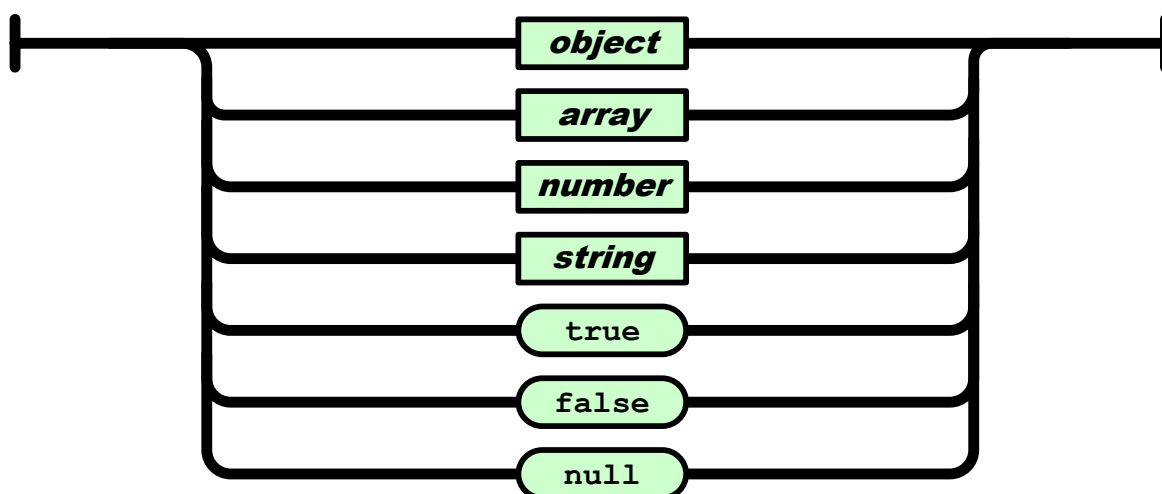


Figure 1 — value

6 Objects

An object structure is represented as a pair of curly bracket tokens surrounding zero or more name/value pairs. A name is a *string*. A single colon token follows each name, separating the name from the *value*. A single comma token separates a *value* from a following name. The JSON syntax does not impose any restrictions on the *strings* used as names, does not require that name *strings* be unique, and does not assign any significance to the ordering of name/value pairs. These are all semantic considerations that may be defined by JSON processors or in specifications defining specific uses of JSON for data interchange.

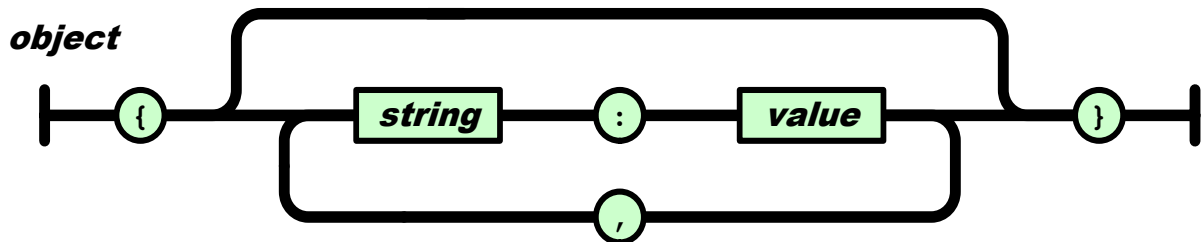


Figure 2 — object

7 Arrays

An array structure is a pair of square bracket tokens surrounding zero or more *values*. The *values* are separated by commas. The JSON syntax does not define any specific meaning to the ordering of the *values*. However, the JSON array structure is often used in situations where there is some semantics to the ordering.

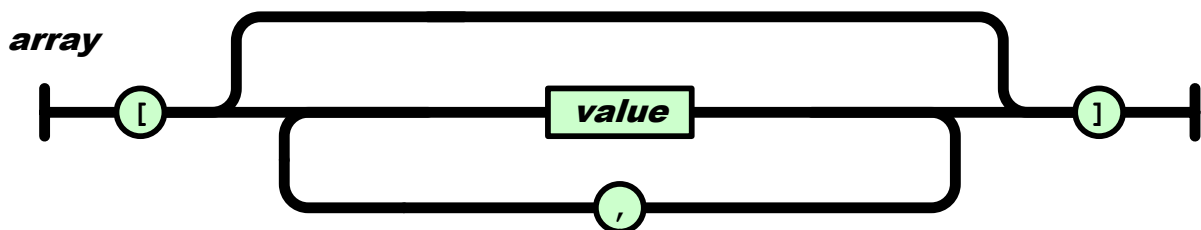


Figure 3 — array

8 Numbers

A number is a sequence of decimal digits with no superfluous leading zero. It may have a preceding minus sign (U+002D). It may have a fractional part prefixed by a decimal point (U+002E). It may have an exponent, prefixed by *e* (U+0065) or *E* (U+0045) and optionally + (U+002B) or - (U+002D). The digits are the code points U+0030 through U+0039.

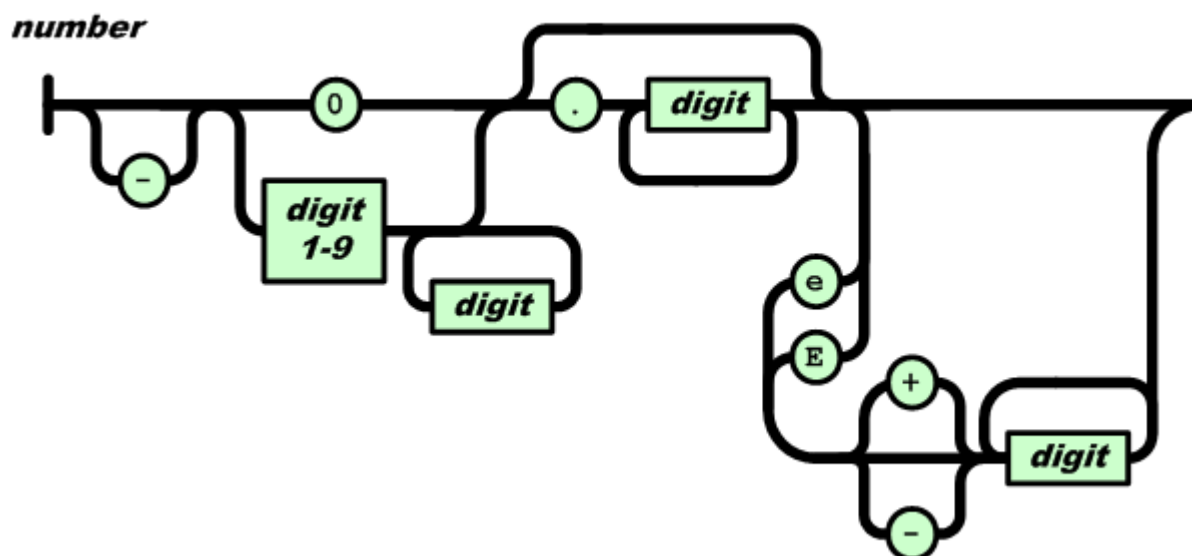


Figure 4 — number

Numeric values that cannot be represented as sequences of digits (such as *Infinity* and *NaN*) are not permitted.

9 String

A string is a sequence of Unicode code points wrapped with quotation marks (U+0022). All code points may be placed within the quotation marks except for the code points that must be escaped: quotation mark (U+0022), reverse solidus (U+005C), and the control characters U+0000 to U+001F. There are two-character escape sequence representations of some characters.

- \ " represents the quotation mark character (U+0022).
- \\ represents the reverse solidus character (U+005C).
- \/ represents the solidus character (U+002F).
- \b represents the backspace character (U+0008).
- \f represents the form feed character (U+000C).
- \n represents the line feed character (U+000A).
- \r represents the carriage return character (U+000D).
- \t represents the character tabulation character (U+0009).

So, for example, a string containing only a single reverse solidus character may be represented as "\\ ".

Any code point may be represented as a hexadecimal escape sequence. The meaning of such a hexadecimal number is determined by ISO/IEC 10646. If the code point is in the Basic Multilingual Plane (U+0000 through U+FFFF), then it may be represented as a six-character sequence: a reverse solidus, followed by the lowercase letter `u`, followed by four hexadecimal digits that encode the code point. Hexadecimal digits can be

digits (U+0030 through U+0039) or the hexadecimal letters **A** through **F** in uppercase (U+0041 through U+0046) or lowercase (U+0061 through U+0066). So, for example, a string containing only a single reverse solidus character may be represented as `"\u005C"`.

The following four cases all produce the same result:

```
"\u002F"
```

```
"\u002f"
```

```
"\"/"
```

```
"/"
```

To escape a code point that is not in the Basic Multilingual Plane, the character may be represented as a twelve-character sequence, encoding the UTF-16 surrogate pair corresponding to the code point. So for example, a string containing only the G clef character (U+1D11E) may be represented as `"\uD834\uDD1E"`. However, whether a processor of JSON texts interprets such a surrogate pair as a single code point or as an explicit surrogate pair is a semantic decision that is determined by the specific processor.

Note that the JSON grammar permits code points for which Unicode does not currently provide character assignments.

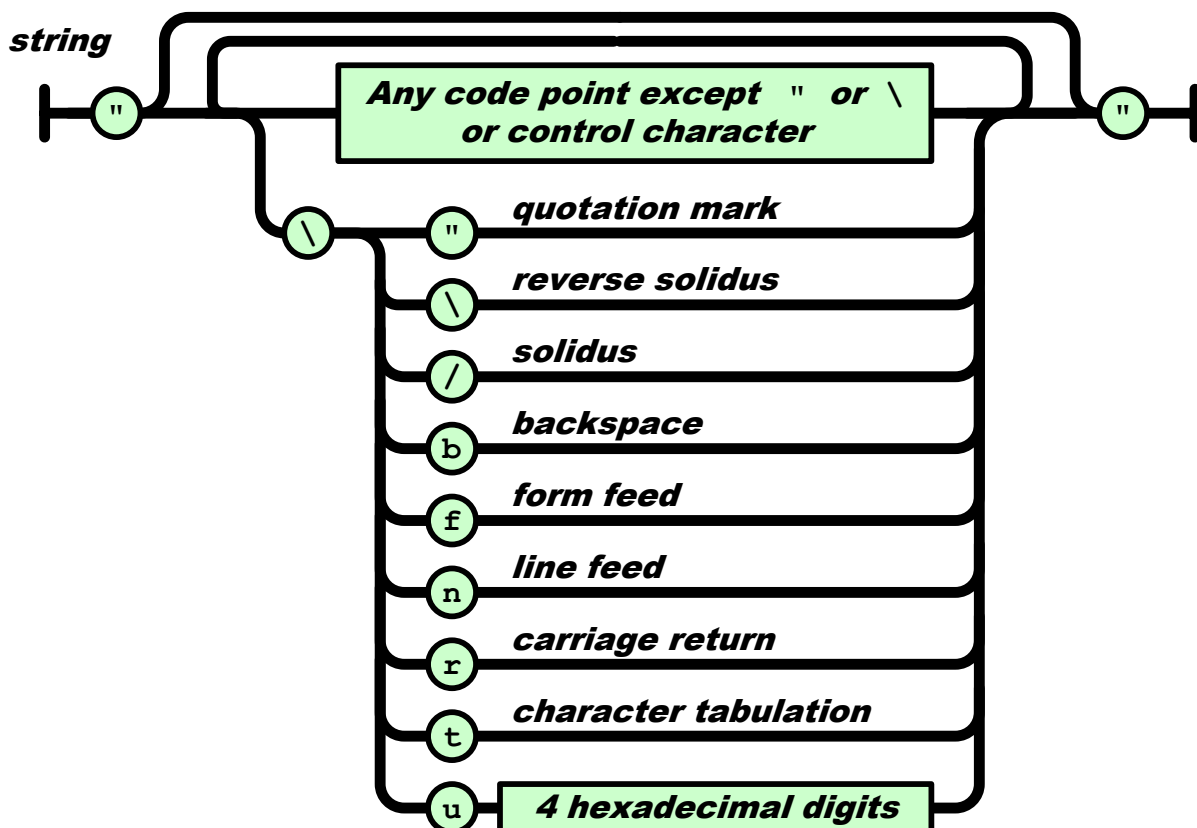


Figure 5 — string

Bibliography

- [1] ECMA-262, *ECMAScript® Language Specification*
- [2] IETF RFC 8259 *The JavaScript Object Notation (JSON) Data Interchange Format*

%PDF-1.x		Cabecera Header	
<pre>1 0 obj << /Type /Catalog /Pages 2 0 R >> endobj</pre>		Root Object El objeto 1 es un catálogo (/Catalog) que apunta a la página en el objeto 2.	CUERPO BODY
<pre>2 0 obj << /Type /Pages /Kids [3 0 R] /Count 1 >> endobj</pre>		El objeto 2 es un objeto de páginas (/Pages) que tiene una página como hijo.	
<pre>3 0 obj << /Type /Page /Parent 2 0 R /MediaBox [0 0 612 792] /Contents 4 0 R >> endobj</pre>		El objeto 3 es una página (/Page) que tiene un tamaño definido (MediaBox) y contenido en el objeto 4.	
<pre>4 0 obj << /Length 50 >> stream % Contenido del objeto (datos binarios) BT /F1 12 Tf 72 720 Td (¡Hola, mundo!) Tj ET endstream endobj</pre>		El objeto 4 es un objeto de contenido (/Contents) que tiene un flujo de datos binarios que representan el contenido de la página. En este caso, se muestra el texto "¡Hola, mundo!".	
<pre>xref 0 5 0000000000 65535 f 0000000017 00000 n 0000000096 00000 n 0000000174 00000 n 0000000327 00000 n</pre>		Se especifica la posición y el estado de cada objeto en el archivo PDF. La primera entrada, "0000000000 65535 f", indica que el objeto 0 está libre (f = free). Las siguientes entradas indican las posiciones y estados de los objetos restantes. ("n" significa "in use").	
<pre>trailer << /Size 5 /Root 1 0 R >> startxref 1234 %%EOF</pre>		Tamaño de la TRC (/Size) y objeto raíz (/Root). "startxref" especifica la posición del inicio de la sección xref. "1234" es la posición del inicio de la sección xref. "%%EOF" marca el final del archivo PDF.	
		TABLA de REFERENCIA CRUZADA	
		COLA	

%PDF-1.x	HEADER
1 0 obj << /Type /Catalog .. >> 20 0 obj << /page .. >>	BODY
ref 0 5 00000000000 65535 f 00000000017 00000 n	XREF
trailer << /Size 5 /Root 1 0 R >> startxref 1234 %%EOF	TRAILER

HEADER	BODY	XREF	TRAILER
--------	------	------	---------

CUANDO LO EDITAMOS PASA LO SIGUIENTE:

HEADER	BODY	XREF	TRAILER		BODY	XREF	TRAILER
--------	------	------	---------	--	------	------	---------

GnuPG básico

[Introducción](#)

[Ver claves públicas disponibles](#)

[Borrar claves de los anillos](#)

[Exportar claves](#)

[Encriptar mensajes](#)

[Firmar mensajes](#)

[Trabajar con claves en servidores](#)

[Clave de revocación](#)

[Programas que soportan GnuPG](#)

[Creación de claves](#)

[Ver claves privadas disponibles](#)

[Ver huella de la clave](#)

[Importar claves](#)

[Desencriptar mensajes](#)

[Verificar mensajes firmados](#)

[GnuPG subshell](#)

[Anillo de confianza](#)

[Historial del manual](#)

[Introducción](#)

GnuPG es una aplicación que sirve para encriptar mensajes, documentos ... GnuPG usa un sistema de claves públicas lo que quiere decir que cada usuario tiene una clave privada y una clave pública.

La clave privada es la que se usa para desencriptar aquello que nos envían encriptado con nuestra clave pública, La clave privada es una clave que solo ha de conocer el propietario ya que si alguien más la conociese podría desencriptar lo que nos mandan encriptado.

La clave pública es la que se da a la gente para que nos manden cosas encriptadas y usaran para encriptar aquello que nos quieran pasar.

El programa se puede obtener de la siguiente url <http://www.gnupg.org>

[Creación de claves](#)

Lo primero que hay que hacer una vez que se tiene GnuPG instalado es crear nuestra clave pública y privada. Para hacerlo hay usar el comando `gpg --gen-key`.

```
evolution:~# gpg --gen-key
gpg (GnuPG) 1.4.1; Copyright (C) 2005 Free Software Foundation, Inc.
This program comes with ABSOLUTELY NO WARRANTY.
This is free software, and you are welcome to redistribute it
under certain conditions. See the file COPYING for details.

gpg: directory `/root/.gnupg' created
gpg: creado un nuevo fichero de configuración `/root/.gnupg/gpg.conf'
gpg: AVISO: las opciones en `/root/.gnupg/gpg.conf' no están aún activas en esta ejecución
gpg: anillo `/root/.gnupg/secring.gpg' creado
gpg: anillo `/root/.gnupg/pubring.gpg' creado
Por favor seleccione tipo de clave deseado:
(1) DSA and ElGamal (default)
(2) DSA (sólo firmar)
(5) RSA (sólo firmar)
Su elección: 1
```

Al ser la primera vez que se ejecuta nos crea un directorio en el que guardara el fichero de configuración así como los archivos `secring.gpg` y `pubring.gpg`. En el primero se almacenaran las claves privadas y en el segundo las claves públicas.

La primera pregunta que hace es que tipo de clave queremos. Lo normal suele ser seleccionar la primera opción (DSA and ElGamal) que nos permite encriptar y firmar.


```
DSA keypair will have 1024 bits.  
ELG-E keys may be between 1024 and 4096 bits long.  
What keysize do you want? (2048) 2048  
El tamaño requerido es de 2048 bits
```

La siguiente pregunta es el tamaño de las claves que se puede elegir entre 1024 y 4096 bits. Por defecto se recomienda 2048, a mayor tamaño más segura es la clave. También a mayor tamaño más tiempo lleva encriptar y desencriptar.

```
Por favor, especifique el período de validez de la clave.  
0 = la clave nunca caduca  
<n> = la clave caduca en n días  
<n>w = la clave caduca en n semanas  
<n>m = la clave caduca en n meses  
<n>y = la clave caduca en n años  
¿Validez de la clave (0)? 0  
Key does not expire at all  
Is this correct? (y/N) y
```

La siguiente pregunta es cuanto tiempo de validez queremos que tenga la clave. La periodicidad se puede poner que no caduque nunca, que dure ciertos semanas, meses o años. En el caso de poner que caduque al cabo de cierto tiempo habrá que volver a generar las claves y volver a mandar la nueva clave pública a aquellos que usaban la que ha caducado. Por defecto viene la opción 0 que es que no caduque nunca.

```
You need a user ID to identify your key; the software constructs the user ID  
from the Real Name, Comment and Email Address in this form:  
"Heinrich Heine (Der Dichter) <heinrichh@duesseldorf.de>"  
  
Nombre y apellidos: Nombre Apellido  
Dirección de correo electrónico: prueba@prueba.com  
Comentario: Prueba de GnuPG  
Ha seleccionado este ID de usuario:  
"Nombre Apellido (Prueba de GnuPG) <prueba@prueba.com>"  
  
¿Cambia (N)ombre, (C)omentario, (D)irección o (V)ale/(S)alir? V
```

Ahora pregunta nuestro nombre y apellidos, dirección de correo y un comentario para la llave. Una vez introducidos todos los datos nos muestra cual es nuestro ID de usuario que lo crea a partir de los datos que le hemos introducido antes. Luego pregunta si queremos cambiar algún dato o si están bien los datos. Si estan correctos respondemos "V" y sigue adelante el proceso.


```

evolution:~# gpg --list-keys Nombre2
pub 1024D/3960CFFB 2005-08-14
uid Nombre2 Apellido2 (Prueba 2 de GnuPG) <prueba2@prueba2.com>
sub 2048g/0C083FDC 2005-08-14

evolution:~# gpg --list-keys Apellido2
pub 1024D/3960CFFB 2005-08-14
uid Nombre2 Apellido2 (Prueba 2 de GnuPG) <prueba2@prueba2.com>
sub 2048g/0C083FDC 2005-08-14

evolution:~# gpg --list-keys prueba2@prueba2.com
pub 1024D/3960CFFB 2005-08-14
uid Nombre2 Apellido2 (Prueba 2 de GnuPG) <prueba2@prueba2.com>
sub 2048g/0C083FDC 2005-08-14

evolution:~# gpg --list-keys 0x3960CFFB
pub 1024D/3960CFFB 2005-08-14
uid Nombre2 Apellido2 (Prueba 2 de GnuPG) <prueba2@prueba2.com>
sub 2048g/0C083FDC 2005-08-14

evolution:~# gpg --list-keys "Prueba 2 de GnuPG"
pub 1024D/3960CFFB 2005-08-14
uid Nombre2 Apellido2 (Prueba 2 de GnuPG) <prueba2@prueba2.com>
sub 2048g/0C083FDC 2005-08-14

evolution:~# gpg --list-keys "Nombre"
pub 1024D/712106AB 2005-08-14
uid Nombre Apellido (Prueba de GnuPG) <prueba@prueba.com>
sub 2048g/882790EC 2005-08-14

pub 1024D/3960CFFB 2005-08-14
uid Nombre2 Apellido2 (Prueba 2 de GnuPG) <prueba2@prueba2.com>
sub 2048g/0C083FDC 2005-08-14

evolution:~# █

```

[Ver claves privadas disponibles](#)

Para ver las claves privadas que tenemos disponibles hay que hacerlo con el comando `gpg --list-secret-keys`. Esto lo que haces listar las claves que hay disponibles dentro del fichero `secring.gpg`.

```

evolution:~# gpg --list-secret-keys
/root/.gnupg/secring.gpg
-----
sec 1024D/712106AB 2005-08-14
uid Nombre Apellido (Prueba de GnuPG) <prueba@prueba.com>
ssb 2048g/882790EC 2005-08-14

evolution:~# █

```

[Borrar claves de los anillos](#)

Se llama anillos a los archivos en los que se guardan las claves públicas y las privadas. Generalmente donde se guardan las claves públicas es el archivo `pubring.gpg` y en el que se guardan las claves secretas `secring.gpg`. Si se quiere borrar alguna clave primero hay que borrar la clave privada y después la pública. Si se intenta borrar primero la clave pública y esta tiene asociada una clave privada da un mensaje de error.

```

evolution:~# gpg --list-keys
/root/.gnupg/pubring.gpg
-----
pub   1024D/712106AB 2005-08-14
uid           Nombre Apellido (Prueba de GnuPG) <prueba@prueba.com>
sub   2048g/882790EC 2005-08-14

pub   1024D/3960CFFB 2005-08-14
uid           Nombre2 Apellido2 (Prueba 2 de GnuPG) <prueba2@prueba2.com>
sub   2048g/0C083FDC 2005-08-14

pub   1024D/001B9A17 2005-08-24
uid           Nombre4 Apellido4 (Prueba4) <prueba4@prueba4.com>
sub   2048g/DA02610C 2005-08-24

evolution:~# gpg --list-secret-keys
/root/.gnupg/secring.gpg
-----
sec   1024D/712106AB 2005-08-14
uid           Nombre Apellido (Prueba de GnuPG) <prueba@prueba.com>
ssb   2048g/882790EC 2005-08-14

sec   1024D/001B9A17 2005-08-24
uid           Nombre4 Apellido4 (Prueba4) <prueba4@prueba4.com>
ssb   2048g/DA02610C 2005-08-24

evolution:~# gpg --delete-keys Prueba4
gpg (GnuPG) 1.4.1; Copyright (C) 2005 Free Software Foundation, Inc.
This program comes with ABSOLUTELY NO WARRANTY.
This is free software, and you are welcome to redistribute it
under certain conditions. See the file COPYING for details.

gpg: ¡hay una clave secreta para esta clave pública! "Prueba4"!
gpg: use antes la opción "--delete-secret-key" para borrarla.
evolution:~# █

```

Para borrar claves privadas se hace con el comando `gpg --delete-secret-key ClaveID`

Para borrar claves públicas se hace con el comando `gpg --delete-key ClaveID`

```

evolution:~# gpg --delete-secret-keys Prueba4
gpg (GnuPG) 1.4.1; Copyright (C) 2005 Free Software Foundation, Inc.
This program comes with ABSOLUTELY NO WARRANTY.
This is free software, and you are welcome to redistribute it
under certain conditions. See the file COPYING for details.

sec 1024D/93A63FAC 2005-08-24 Nombre4 Apellido4 (Prueba4) <prueba4@prueba4.com>

Delete this key from the keyring? (y/N) y
This is a secret key! - really delete? (y/N) y
evolution:~#
evolution:~# gpg --delete-keys Prueba4
gpg (GnuPG) 1.4.1; Copyright (C) 2005 Free Software Foundation, Inc.
This program comes with ABSOLUTELY NO WARRANTY.
This is free software, and you are welcome to redistribute it
under certain conditions. See the file COPYING for details.

pub 1024D/93A63FAC 2005-08-24 Nombre4 Apellido4 (Prueba4) <prueba4@prueba4.com>

Delete this key from the keyring? (y/N) y
evolution:~# gpg --list-keys
gpg: comprobando base de datos de confianza
gpg: 3 marginal(s) needed, 1 complete(s) needed, PGP trust model
gpg: depth: 0 valid: 1 signed: 0 trust: 0-, 0q, 0n, 0m, 0f, 1u
/root/.gnupg/pubring.gpg
-----
pub 1024D/712106AB 2005-08-14
uid Nombre Apellido (Prueba de GnuPG) <prueba@prueba.com>
sub 2048g/882790EC 2005-08-14

pub 1024D/3960CFFB 2005-08-14
uid Nombre2 Apellido2 (Prueba 2 de GnuPG) <prueba2@prueba2.com>
sub 2048g/0C083FDC 2005-08-14

evolution:~#
evolution:~# gpg --list-secret-keys
/root/.gnupg/secring.gpg
-----
sec 1024D/712106AB 2005-08-14
uid Nombre Apellido (Prueba de GnuPG) <prueba@prueba.com>
ssb 2048g/882790EC 2005-08-14

evolution:~# █

```

Ver huella de la clave

Las claves están identificadas por lo que se llama huella. La huella es una serie de números que se usa para verificar si una clave pertenece realmente al propietario. Si se recibe una clave podemos ver cual es su huella y luego pedirle a su propietario que nos diga su huella. Si ambas coinciden la clave es correcta y no ha sido manipulada. Si no fuese igual es que ha sido modificada. La huella es como el md5 que verifica que un archivo no ha sido manipulado.

```

evolution:~/gnupg# gpg --fingerprint prueba@prueba.com
pub 1024D/712106AB 2005-08-14
    Key fingerprint = BCB8 45C8 A948 501E A360 851F EBEB 96C8 7121 06AB
uid Nombre Apellido (Prueba de GnuPG) <prueba@prueba.com>
sub 2048g/882790EC 2005-08-14

evolution:~/gnupg# █

```

Exportar claves

Las claves se pueden exportar a ficheros para que las podamos distribuir entre la gente que queremos que nos encripte o firme cosas o bien porque vamos a formatear el equipo y necesitamos

salvarlas.

Para exportar la clave publica se hace poniendo gpg --armor --output fichoeDeSalida --export ClaveID

```
evolution:~# gpg --armor --output prueba-public-key.asc --export prueba@prueba.com
evolution:~# more prueba-public-key.asc
-----BEGIN PGP PUBLIC KEY BLOCK-----
Version: GnuPG v1.4.1 (GNU/Linux)

mQGiBEL+60QRBADKr0jTjRQRuNS3sjukdL0bByjhsLEtxT3LIa/FYF18NydLaG3o
CGYMD8LtHSwsfnPFBu4KWFtTylgFX72TF00k8kdoXIbXwTacjDlIiv4xBy0Z7y80
R/ZDCmgYTB7dAj8IvUygCYTdkum2+fcIFqeK0RRsQ1ePHlNdNo42SGAN4wCg5kIf
ojlph3IQwC+hY06fD/AHSIED/AjI23uk9X/vt9kBIiBi+HKCBY7WonChYHf/xmJv
ykdyjs0JVIloIlWZyxxJ9Si0stPSZ2azG03SvA1sfK3c0LQfU+jxYJfAnkt4Y23a
5ogE0Bdwno3fQFJQmkZzXekL5KZXkNceoAqcglcFutMw8CDdUp8WUHZZLheu0cU
0XhPA/49N0bBgW4q0aRI2HOV0r+zcfoypaYGsE0+jciD0+NFA3ymAG4gYn9W10kP
bmbJnFH9Sjh40o33uhM7fFXPGWzPBRQq4/Jj5k97mo6A3CgBVJH3IlivvgzVx6dv
7cstTGE9bq4fvihluR3lV1SmEErU1nn/nJQGHnDVjUhuQQTn7Q1Tm9tYnJlIEFw
ZWxsaWRvIChQcnVlYmEgZGUgR251UEcpIDxcwcnVlYmFAcHJlZWJhLmNvbT6IXgQT
EQIAHgUCQv7rRAIbAwYLCQgHAwIDFQIDAxYCAQIeAQIXgAAKCRDr65bIcSEGq18E
AJ9EtZCnILQKP40+1XR6Ea06eBGEhgCfdKkB4pjcdW0z0utQavbrRpnnpByIXgQT
EQIAHgUCQv7rRAIbAwYLCQgHAwIDFQIDAxYCAQIeAQIXgAAKCRDr65bIcSEGq18E
AKCnodsFv0Ro1rhLWwpmF9X0lHKA9wCg2T+0K4pG0F+LTdFGZqhYR/t9rX+5Ag0E
Qv7rVBAIAJX0yGk0hmPmtEpJgv082D1EBad64ycydd0MZd+Z9J5MTKxL1kV8ecJE
PFld2Cvbl+ZaBnkj5mKi8a2/Qj+VhQI6Z8HXrEwmuu0GucBQ8kL2GmFAkV/kNVug
afZK0pIdgmjnbnc42Kh2YE02NZrfqe4aRSmYV0Ye2isn9g22G0bFnGBdkjnU193t
xn5KsW+Y9qx0zir4ksUwIafXZI3DptSeVw8398Lde8+zDZbd20D/IldXVyl87oqd
Asrd5v5qMMZEPpo8l+relkovkv95e5NtlPRnhILPwe0U3e8eFvWG+XdhQqlVUURG
wcsuvHpL7tzGzs0KIHIwI0yo2oT0pFsAAwYH/R9eu/u+9RVCSruh7EG5c rf7IGF
9cbp30YIFQzwm8qQ+5KZ9l7KoC7rQJLxTIzRzbaSW7cn5nARciKj+tiQcEbQ1DtK
LAiyAWSWY5DND8m4LcxPcGuTlKs0R1hZP16uinClyKP76/+MYDARZnBESHr+UuCu
mavV6A9Tqr/vKqGF4S3w0mhHFsVu0w7jHDlTE4KZBlItHesR5bacKnsWS6u7GAjm
QkjVWL15GuZIMVrI8RRrnHNeSuSnPouMSHpsL3hrp0416/t+dpAxWjuQkSFRaz30
MCJcH0cT4W9rJPRw6Pmhtzic8XQpwjy0wdTmKt1JKPqER3LzsJHnD9Re4TiISQY
EQIACQUcQv7rVAIbDAKCRDr65bIcSEGqyLPAJsHekZV+klIgResCyWEHDbvgVkj
oQCg4DQdEHNDahi9z7AW5jdXc0TvR1I=
=8Tnt
-----END PGP PUBLIC KEY BLOCK-----
evolution:~#
```

Para exportar la clave privada se haría poniendo gpg --armor --output fichoeDeSalida --export-secret-key ClaveID


```

evolution:~# gpg --armor --output prueba-secret-key.asc --export-secret-key prueba@prueba.com
evolution:~# more prueba-secret-key.asc
-----BEGIN PGP PRIVATE KEY BLOCK-----
Version: GnuPG v1.4.1 (GNU/Linux)

lQHhBEL+60QRBADKroJtjRQruNS3sjuKdL0bByjhsLEtxT31Ia/FYF18NydLaG3o
CGYmD8LthSwsfnFBu4KWFTtylgFX72TF00k8kdoXIbXwTacjDlIiv4xBy0Z7y80
R/ZDCmgYTB7dAj8IvUygCYTdkum2+fcIFqeK0RRsQ1ePHlNdNo42SGAN4wCg5kIf
ojlph3IQwC+hY06FD/AhSIED/AjI23uk9X/vt9k8Iibi+HKCBY7WonChYHf/xmJv
ykdYjs0JVILoILWZyyxJ9Si0stPSZ2azG03SvAlsFk3c0LQfU+jxYJfAnkt4Y23a
5ogE08dwno3fQFJQmkZzXeKL5KZXkNceAoAqcglcFutMMw8CDdUp8WUHZZLheu0cU
0XhPA/49N0bBgW4qQaRI2HOV0r+zcfoypaY6sE0+jciD0+NFA3ymAG4gYn9W1QkP
bmbJnFH9Sjh40o33uhM7fFXPGWzPBRQq4/Jj5k97mo6A3CgBVJH3ILivvgzVx6dv
7cstTGEEm9bq4fvihL uR3l V1SmEErU1nn/nJQGHnDVjUhuQQtn/4DAwL71Cho06br
nGCxs2uo4P0BBY21gluyN8TjCkNmWyYM80XHKF35VwB5T9k4suyR0ix6n10vvW6J
uap4nbQ1Tm9tYnJlIEFwZWxsawRvIchQcnVlYmEgZGUgR251UEcpIDxwcnVlYmFA
cHJlZWJhLmNvbT6IXgQTEQIAHgUCQv7rRAIbAwYLCQgHAwIDFQIDAxYCAQIEAQIX
gAAKCRDr65bIcSEgQl8EAKcnodsFv0Ro1rhLWwpmF9X0lHKA9wCg2T+0K4pG0F+L
TdFGZqhYR/t9rX+dAmIEQv7rVBAIAJX0yGk0hmPmtEpJgv082D1EBad64ycydd0M
Zd+Z9JsmTKxL1kV8ecJEPFLd2Cvbl+ZaBnKj5mKi8a2/Qj+VhQI6Z8HXrEwmuu0G
ucBQ8kL2GmFAkV/kNVugafZK0pIdgmjnbnc42Kh2YEO2NZrfqe4aRSmYV0Ye2isn
9g22G0bFnGBdkjnU193tXn5KsW+Y9qx0zir4ksUwIafXZI3DptSeVw8398Lde8+z
DZbd20D/ILDXVylB7oqdAsrd5v5qWMZEPpo8l+relkovkv95e5NtlPRnhILPweOU
3e8eFwG+XdhQqlVuURGwcsuvHpL7tzGzs0KIHiwI0yo2oT0pFsAAwYH/R9eu/u+
9RVCSruhG7EG5c rf7IGF9cbp30YIFQzwm8qQ+5KZ9l7KoC7rQJLxTIzRzbaSN7cn
5nARciKj+tiQcEbQ1DtKLAiyAWSWY5DND8m4LcxPcGuTlKs0R1hZP16uinClyKP7
6/+MYDARZnBEShr+UuCumavV6A9Tqr/vKqGF4S3w0mhHFsVu0w7jHDLTE4KZBLIt
HesR5bacKnsWS6u7GAjmQkjVWL15GuZIMVrI8RRrnHNeSuSnPouMShpsL3hrp04l
6/t+dpAxWjuQkSFRaz30MCJcH0cT4W9rJPRw6Pmhtzic8XQpwjy0wdTmKt1JKPqE
R3LzsJHnD9Re4Tj+AwMC+9QoaDum65xgsPq0M0B1PUDtPyM8hldtCVzl+iGV+8yv
R6MCKI7xGVjNs1DzDfnJmjSkXZfbb/jaBF3eW8kPno7jRFJQQS4X8RNMmMt8Jmi
iEKEGBECAAkFAkL+6lQCGwwACgkQ6+uMyHEhBqsizwCdG2qH6eePoGffcjDCI+yC
3jq/fecAn0HoA+IwTveoawxuf9/higf3PbAR
=8ldT
-----END PGP PRIVATE KEY BLOCK-----
evolution:~# █

```

Si quisieramos salvar todas las claves que tenemos valdría con copiar los archivos pubring.gpg y secring.gpg y luego cuando vayamos al nuevo equipo ponerlas en el directorio de GnuPG.

[Importar claves](#)

Si se quiere importar claves nuevas porque por ejemplo hemos formateado el equipo y queremos volver a tener nuestras claves las importamos con el comando `gpg --import ClaveID`. En el apartado anterior se han salvado las claves publica y privada pues ahora vamos a importarlasy. Primero importamos la publica y luego la privada.

```

evolution:~/claves# gpg --import prueba-public-key.asc
gpg: directory `/root/.gnupg' created
gpg: creado un nuevo fichero de configuración `/root/.gnupg/gpg.conf'
gpg: AVISO: las opciones en `/root/.gnupg/gpg.conf' no están aún activas en esta ejecución
gpg: anillo `/root/.gnupg/secring.gpg' creado
gpg: anillo `/root/.gnupg/pubring.gpg' creado
gpg: /root/.gnupg/trustdb.gpg: se ha creado base de datos de confianza
gpg: key 712106AB: public key "Nombre Apellido (Prueba de GnuPG) <prueba@prueba.com>" imported
gpg: Cantidad total procesada: 1
gpg:      importadas: 1
evolution:~/claves# gpg --import prueba-secret-key.asc
gpg: key 712106AB: secret key imported
gpg: key 712106AB: "Nombre Apellido (Prueba de GnuPG) <prueba@prueba.com>" 1 new signature
gpg: Cantidad total procesada: 1
gpg:      nuevas firmas: 1
gpg:      claves secretas leídas: 1
gpg:      claves secretas importadas: 1
evolution:~/claves# gpg --list-keys
/root/.gnupg/pubring.gpg
-----
pub    1024D/712106AB 2005-08-14
uid          Nombre Apellido (Prueba de GnuPG) <prueba@prueba.com>
sub    2048g/882790EC 2005-08-14
evolution:~/claves#

```

Ahora si queremos importar la clave de una amigo pues se haría igual.

```

evolution:~/claves# gpg --import prueba2-public-key.asc
gpg: key 3960CFFB: public key "Nombre2 Apellido2 (Prueba 2 de GnuPG) <prueba2@prueba2.com>" imported
gpg: Cantidad total procesada: 1
gpg:      importadas: 1
evolution:~/claves# gpg --list-keys
/root/.gnupg/pubring.gpg
-----
pub    1024D/712106AB 2005-08-14
uid          Nombre Apellido (Prueba de GnuPG) <prueba@prueba.com>
sub    2048g/882790EC 2005-08-14

pub    1024D/3960CFFB 2005-08-14
uid          Nombre2 Apellido2 (Prueba 2 de GnuPG) <prueba2@prueba2.com>
sub    2048g/0C083FDC 2005-08-14
evolution:~/claves#

```

Encriptar mensajes

Si se quiere encriptar mensajes se puede hacer poniendo `gpg --armor --recipient ClaveID --encrypt mensaje`. Si por ejemplo queremos encriptar el archivo `a.txt` habría que poner `gpg --armor --recipient prueba@prueba.com --encrypt a.txt`

```

evolution: ~/.gnupg# gpg --armor --recipient prueba@prueba.com --encrypt a.txt
evolution: ~/.gnupg# more a.txt.asc
-----BEGIN PGP MESSAGE-----
Version: GnuPG v1.4.1 (GNU/Linux)

hQIOA2/sL+2IJ5DsEaf/QmX0GGFaxQ7vKGuoGUqltnxD0gIZ8c0dbrXpZh2HN+NU
IGPF/fW0he/JKENJDwXUg3VinXhkJc2fZsE007iEiXPM1XeUfXAEALVrLlb80fU
vJyshR7K70nCKk4mKTZgMtp9NxxwFbzRbCI3N0dWLY9V9pBPSPiqhS9PFQ00L4DFd
/G/Yq6f3HMa+jW0o5aMexUuhHRCCyuMG9eN9La0LK8UsHSDsxEXgBHw7yM27Yrt0
WzUowL54tpceR/I66fyu+U4i50F+2ybXfwiPs70iWokJf5hecRTfUt01cvGxH5CJ
/P/u8g9hKLYUjyc4hGku6RFXfLYvNVZ30nYoNPrQFEQf+Pfqe4zk3wSjmRi2scFhJ
kdseCbGeWAin9H8J5EsXMmudLujBXsUEQ0kLHITmWpHHodkChcmTs+2I6SpdofK4
kJJzluGyWld4eB7V7da36EcSLrBNmtU4yAj2pIeFujZ4QdxZ4wL8cbCLKkKwSnCG
GJuxRXKLksaLbEA5KSHFmg1jJWt3ld0xZ80dCtftpuk8wZNLcxzXLaT0sDTLEQGB
gpIcFT8Pmb40loU/nPIDJ0UbzEnY1P6AufLr0gkrt3kUexPSjGTu6qVmaB1Jtcia
Gf3umrdzBaPtUYi1PeFAMDIA4SrW3zbCbv0afJ9o6ILBVh6CnHNnV0BLKX8kMqwBP
LNJFASWLA45E5iYmLfsreUpxbp8TWxPsGMGpg5q3N6cQ00+WcKZiVFRfcTkZDcy
rBGiEPcVVrdRR+4V78Rd9KakfPiaw5rm
=WB73
-----END PGP MESSAGE-----
evolution: ~/.gnupg#

```

También se puede encriptar a un fichero en concreto con la opción --output nombreFichero

```

evolution: ~/.gnupg# gpg --armor --output salida.txt --recipient prueba@prueba.com --encrypt a.txt
evolution: ~/.gnupg# more salida.txt
-----BEGIN PGP MESSAGE-----
Version: GnuPG v1.4.1 (GNU/Linux)

hQIOA2/sL+2IJ5DsEaf/Ytc78XgTLIbmVsomHcihRxMKF57i8lRP28gvWKY8Goet
06lma5die0dL55rUGT2z7aRshAK44eC6/kjHA00pvwDRhzPJ2jHuqyf13gI6CRUp
gSQ5qXZR5yMRP8r+hEgoSinogG8eHfa+3roXF6owneEARLrLc7MPHbgQvg/AIC8t
gBxt33CEBYLZh7haAvGJT1/oYhWRFzgPHl0eAiKjflWzXj5tTsukV18AFuXo+1ms
P+z3RYnFAY0HFvUr10kXbIY0E6bmzSTNU6e9cZc7ksPLYfwGxFZNhB5tTdXVklUd
yWdTRD4cZAddQ+IY4/RNeVGKYEkwkRp0Jy+wtAJbgf/Xe2ipy0FLjSka4k653Rn
dW0zmX98jaBRl0dTq/0vhSL5QlJDwyzJcYvL4tM1mB2kkLQ0Izz/VbdmGkoRzMXp
xmvFD6JRuHCJv+6bQA+n0snLM1qSxdhGFKyvTQhe/IwIR067WinmWS2xt0nL9WjJ
cHD9yxOKAC9k51FXlfof04kRfFDQvg8KYcjDhmCu9PIA019CD75foi/hvk9xUE20N
i5y/BZNruTceqXpE6HKe/C8y4uE/KzKcnQ6GX8TcrSKvpp84/qkKL3rXWW1VXXV0
yvmoG7RzbkRtM4e1LuM08B9cWGGa13/hAHf3UnaJR1pGyMiJyU7b4M/mISWoX3JZ
AdJFATzjQSSx/UnIJayJIXefJJcP9w2wN3Peak/H+vyWbCkZAZ88sxlePXoYPGcJ
8CLHPRIKw6FgKtb6X6JF6F/V56ABzEw
=Eib/
-----END PGP MESSAGE-----
evolution: ~/.gnupg#

```

Si en las opciones no se le pasa el parámetro --armor lo que se encripta lo deja en un archivo de tipo binario. Al poner la opción --armor transforma lo que se encripta en texto ASCII con el mensaje encriptado.

[Desencriptar mensajes](#)

Para desencriptar el mensaje que hemos encriptado antes hay que poner gpg --decrypt archivo. Para el caso anterior sería gpg --decrypt a.txt.asc.

```

evolution: ~/.gnupg# gpg --decrypt a.txt.asc

You need a passphrase to unlock the secret key for
user: "Nombre Apellido (Prueba de GnuPG) <prueba@prueba.com>"
2048-bit ELG-E key, ID 882790EC, created 2005-08-14 (main key ID 712106AB)

gpg: encrypted with 2048-bit ELG-E key, ID 882790EC, created 2005-08-14
      "Nombre Apellido (Prueba de GnuPG) <prueba@prueba.com>"
Hola
evolution: ~/.gnupg# more a.txt
Hola
evolution: ~/.gnupg#

```

Cuando desencriptamos algo se pide la password de nuestra clave para poder desencriptarlo. Para nuestro caso tenemos el archivo a.txt.asc encriptado al desencriptarlo nos deja el archivo a.txt y nos

muestra su contenido.

Firmar mensajes

Firmar mensajes sirve para que cuando a alguien le llegue un mensaje que hemos firmado la persona que lo ha recibido verifique con GnuPG que la firma es buena y que entonces hemos sido nosotros quien le ha enviado el mensaje. Por ejemplo vamos a firmar el archivo a.txt para ello se pondría `gpg --clearsign a.txt`. Esto nos creara el arhivo a.txt.asc con el contenido que se ve en la imagen.

```

evolution:~/gnupg# more a.txt
Hola
evolution:~/gnupg# gpg --clearsign a.txt

You need a passphrase to unlock the secret key for
user: "Nombre Apellido (Prueba de GnuPG) <prueba@prueba.com>"
1024-bit DSA key, ID 712106AB, created 2005-08-14

evolution:~/gnupg# more a.txt.asc
-----BEGIN PGP SIGNED MESSAGE-----
Hash: SHA1

Hola
-----BEGIN PGP SIGNATURE-----
Version: GnuPG v1.4.1 (GNU/Linux)

iD8DBQFC/yR+6+uWyHEhBqsRAkJWAJ4g7yRiEr9tH4S/JuECtpM83GQkHgCfYAlX
eY4V4P32zQ3NAukTCYR+pB8=
=FOM+
-----END PGP SIGNATURE-----
evolution:~/gnupg# █

```

Para firmar algo se pide la contraseña para poder firmarlo. Como se ve en la imagen lo que se ha hecho en el fichero firmado es añadir unas líneas que contienen la firma.

A la hora de firmar si se firma con el parámetro `--sign` en lugar de `--clearsign` nos generara un fichero de salida en binario con extensión `.gpg`. Para validar la firma y ver el contenido hay desencriptarlo con la opción `--decrypt`.

```

evolution:~/gnupg# more prueba.txt
Esto es una pequeña prueba.
evolution:~/gnupg# gpg --sign prueba.txt

You need a passphrase to unlock the secret key for
user: "Nombre Apellido (Prueba de GnuPG) <prueba@prueba.com>"
1024-bit DSA key, ID 712106AB, created 2005-08-14

evolution:~/gnupg# ls -la prueba*
-rw-r--r--  1 root root  28 2005-08-24 16:35 prueba.txt
-rw-r--r--  1 root root 120 2005-08-25 05:02 prueba.txt.gpg
evolution:~/gnupg# more prueba.txt.gpg
fEIA$ou0ZL«xëfqS0%êT8ôiw-.ÉWH-V(ÍKT(H-,Më!*ž:ìY@`úó02Î~ÿuÔŽ)%é{0`üè,q\p»ú3ÄMÿfŠëF>f0ó±^Li\ó
z
evolution:~/gnupg# gpg --decrypt prueba.txt.gpg
Esto es una pequeña prueba.
gpg: Signature made jue 25 ago 2005 05:02:23 CEST using DSA key ID 712106AB
gpg: Good signature from "Nombre Apellido (Prueba de GnuPG) <prueba@prueba.com>"
evolution:~/gnupg#

```

La firma también se puede hacer que se muestre en un fichero aparte con la opción -b. Esta opción se suele usar para firmar archivos binarios.

```

evolution: ~/.gnupg# ./a.out
Hola mundo
evolution: ~/.gnupg# file a.c
a.c: ASCII text
evolution: ~/.gnupg# file a.out
a.out: ELF 32-bit LSB executable, Intel 80386, version 1 (SYSV), for GNU/Linux 2.2.0, dynamically linked (uses shared libs), not stripped
evolution: ~/.gnupg# gpg -b a.out

You need a passphrase to unlock the secret key for
user: "Nombre Apellido (Prueba de GnuPG) <prueba@prueba.com>"
1024-bit DSA key, ID 712106AB, created 2005-08-14

evolution: ~/.gnupg# more a.out.sig
7hëëEq!«»>0Ç!0ÜÄ^LoÜQ^<>?tTn70+δ`^L
U¹Rm9ã
evolution: ~/.gnupg# gpg --verify a.out.sig
gpg: Signature made jue 25 ago 2005 13:01:44 CEST using DSA key ID 712106AB
gpg: Good signature from "Nombre Apellido (Prueba de GnuPG) <prueba@prueba.com>"
evolution: ~/.gnupg#

```

Verificar mensajes firmados

Para verificar mensajes firmados se hace poniendo `gpg --verify mensaje`. Para el caso anterior seria poner `gpg --verify a.txt.asc`

```

evolution: ~/.gnupg# gpg --verify a.txt.asc
gpg: Signature made dom 14 ago 2005 13:01:18 CEST using DSA key ID 712106AB
gpg: Good signature from "Nombre Apellido (Prueba de GnuPG) <prueba@prueba.com>"
evolution: ~/.gnupg#

```

Si la firma no fuese correcta podríamos ver un mensaje como el siguiente:

```

evolution: ~/.gnupg# gpg --verify a.txt.asc
gpg: CRC error; 74D39D - 14E33E
evolution: ~/.gnupg#

```

Trabajar con claves en servidores

Podemos buscar claves públicas de gente a la que queremos enviar mensajes cifrados en servidores de claves. Para hacer una búsqueda se hace poniendo los parámetros `--keyserver NombreDelServidor --search-keys ClaveID`. Si encuentra claves que coinciden con esa ID nos las muestras, luego tenemos tres opciones mostrar los siguientes si es que se han encontrado muchos, poner el número del registro que nos interesa (en ese caso nos importa la clave al anillo de claves públicas) o salir.

Si queremos importar una clave en concreto se hace con los parámetros `--keyserver NombreDelServidor --recv-keys ClaveID`.

Si queremos subir una clave a un servidor para que esté disponible para la gente se hace con los parámetros `--keyserver NombreDelServidor --send-keys ClaveID`

GnuPG subshell

GnuPG viene con una especie de shell que nos da multitud de opciones para trabajar con la clave, Nos permite firmar la clave, cambiar la contraseña, cambiar la fecha de expiración de la llave ...

Para acceder a esta shell hay que poner el parametro `--edit-key ClaveID`.

Clave de revocación

La clave de revocación es una clave que lo que hace es que cuando la importemos a nuestro anillo de claves invalide esa clave. Para generarla se hace con la opción `--gen-revoke`. Esta clave se puede crear nada más generar las llaves o bien cuando se halla comprometido. Hay gente que lo crea nada más crear las claves porque si por ejemplo ha olvidado la contraseña no podrá generar la clave de revocación ya que al final del proceso de generación se pide la contraseña. Esta clave ha de guardarse en un lugar seguro ya que si alguien la obtuviese podría revocar nuestras claves y dejarnos las claves inutilizadas.

```
evolution: ~/.gnupg# gpg --output Nombre5Revoke.asc --gen-revoke Nombre5

sec 1024D/CE99AF10 2005-08-25 Nombre5 Apellido5 (Prueba5) <prueba5@prueba5.com>

Create a revocation certificate for this key? (y/N) y
Por favor elija una razón para la revocación:
  0 = No se dio ninguna razón
  1 = La clave ha sido comprometida
  2 = La clave ha sido reemplazada.
  3 = La clave ya no está en uso
  Q = Cancelar
(Probablemente quería seleccionar 1 aquí)
Su decisión: 1
Introduzca una descripción opcional; acábela con una línea vacía:
> La clave ya no es segura
>
Razón para la revocación: La clave ha sido comprometida
La clave ya no es segura
Is this okay? (y/N) y

You need a passphrase to unlock the secret key for
user: "Nombre5 Apellido5 (Prueba5) <prueba5@prueba5.com>"
1024-bit DSA key, ID CE99AF10, created 2005-08-25

se fuerza salida con armadura ASCII.
Certificado de revocación creado.

Por favor consérvelo en un medio que pueda esconder; si alguien consigue
acceso a este certificado puede usarlo para inutilizar su clave.
Es inteligente imprimir este certificado y guardarlo en otro lugar, por
si acaso su medio resulta imposible de leer. Pero precaución: iel sistema
de impresión de su máquina podría almacenar los datos y hacerlos accesibles
a otras personas!
evolution: ~/.gnupg#
```

Si queremos revocar la clave hay que importar el fichero que tiene la clave de revocación y ya está. Una vez revocada la clave ya no podemos cifrar mensajes aunque si se pueden descifrar, aunque al descifrar se avisa de que la clave ha sido revocada..


```

evolution: ~/.gnupg# gpg --import Nombre5Revoke.asc
gpg: key CE99AF10: "Nombre5 Apellido5 (Prueba5) <prueba5@prueba5.com>" revocation certificate
imported
gpg: Cantidad total procesada: 1
gpg:      nuevas revocaciones de claves: 1
gpg: 3 marginal(s) needed, 1 complete(s) needed, PGP trust model
gpg: depth: 0  valid:   2  signed:   0  trust: 0-, 0q, 0n, 0m, 0f, 2u
evolution: ~/.gnupg# gpg --decrypt nombre5.txt.asc

You need a passphrase to unlock the secret key for
user: "Nombre5 Apellido5 (Prueba5) <prueba5@prueba5.com>"
2048-bit ELG-E key, ID 1B182FD4, created 2005-08-25 (main key ID CE99AF10)

gpg: NOTA: la clave ha sido revocada
gpg: razón para la revocación: La clave ha sido comprometida
gpg: comentario a la revocación: La clave ya no es segura
gpg: encrypted with 2048-bit ELG-E key, ID 1B182FD4, created 2005-08-25
      "Nombre5 Apellido5 (Prueba5) <prueba5@prueba5.com>"
Archivo a cifrar por Nombre5.
evolution: ~/.gnupg# gpg --armor -r Nombre5 --encrypt nombre5.txt
gpg: Nombre5: omitido: clave pública inutilizable
gpg: nombre5.txt: encryption failed: clave pública inutilizable
evolution: ~/.gnupg#

```

Al invalidar la clave tampoco se pueden firmar mensajes.

```

evolution: ~/.gnupg# gpg --default-key Nombre5 --sign nombre5.txt
gpg: no default secret key: clave secreta inutilizable
gpg: signing failed: clave secreta inutilizable
evolution: ~/.gnupg#

```

Anillo de confianza

Crear un anillo de confianza consiste en tener claves de gente firmada por otra gente que la han firmado y que con su firma aseguran que esa clave es realmente de quien dice ser y no ha sido alterada.

Si por ejemplo tenemos las persona A y B. Las personas A y B son amigas y se intercambian entre ellas las claves públicas, verifican sus fingerprints para ver que las claves son las correctas y quedan para ver las claves que se han pasado son correctas. Entonces una vez verificado que todo es correcto cada uno firma la clave de su amigo. Ahora si por ejemplo yo obtengo la clave de B y veo que está firmada por A (que es una persona que conozco y en la que confió) entonces me fío de que esa clave es la clave correcta de B y la puedo usar. Si por un casual quedaría firmar la clave de B con mi firma para abalar que su clave es buena sería bueno que me pusiese en contacto con él y verificásemos la clave.

```

evolution:~/gnupg# gpg --sign-key Nombre2

pub 1024D/3960CFFB created: 2005-08-14 expires: nunca usage: CS
trust: desconocido validity: desconocido
sub 2048g/0C083FDC created: 2005-08-14 expires: nunca usage: E
[ unknown] (1). Nombre2 Apellido2 (Prueba 2 de GnuPG) <prueba2@prueba2.com>

pub 1024D/3960CFFB created: 2005-08-14 expires: nunca usage: CS
trust: desconocido validity: desconocido
Huella de clave primaria: 03ED F15D 8552 614A FB74 FE80 3616 E327 3960 CFFB

Nombre2 Apellido2 (Prueba 2 de GnuPG) <prueba2@prueba2.com>

Are you sure that you want to sign this key with your
key "Nombre Apellido (Prueba de GnuPG) <prueba@prueba.com>" (712106AB)

Really sign? (y/N) y

You need a passphrase to unlock the secret key for
user: "Nombre Apellido (Prueba de GnuPG) <prueba@prueba.com>"
1024-bit DSA key, ID 712106AB, created 2005-08-14

evolution:~/gnupg# gpg --list-sigs
gpg: comprobando base de datos de confianza
gpg: 3 marginal(s) needed, 1 complete(s) needed, PGP trust model
gpg: depth: 0 valid: 1 signed: 1 trust: 0-, 0q, 0n, 0m, 0f, 1u
gpg: depth: 1 valid: 1 signed: 0 trust: 1-, 0q, 0n, 0m, 0f, 0u
/root/.gnupg/pubring.gpg
-----
pub 1024D/712106AB 2005-08-14
uid Nombre Apellido (Prueba de GnuPG) <prueba@prueba.com>
sig 3 712106AB 2005-08-14 Nombre Apellido (Prueba de GnuPG) <prueba@prueba.com>
sig 3 712106AB 2005-08-14 Nombre Apellido (Prueba de GnuPG) <prueba@prueba.com>
sub 2048g/882790EC 2005-08-14
sig 712106AB 2005-08-14 Nombre Apellido (Prueba de GnuPG) <prueba@prueba.com>

pub 1024D/3960CFFB 2005-08-14
uid Nombre2 Apellido2 (Prueba 2 de GnuPG) <prueba2@prueba2.com>
sig 3 3960CFFB 2005-08-14 Nombre2 Apellido2 (Prueba 2 de GnuPG) <prueba2@prueba2.com>
sig 712106AB 2005-08-25 Nombre Apellido (Prueba de GnuPG) <prueba@prueba.com>
sub 2048g/0C083FDC 2005-08-14
sig 3960CFFB 2005-08-14 Nombre2 Apellido2 (Prueba 2 de GnuPG) <prueba2@prueba2.com>

pub 1024D/CE99AF10 2005-08-25 [revoked: 2005-08-25]
rev CE99AF10 2005-08-25 Nombre5 Apellido5 (Prueba5) <prueba5@prueba5.com>
uid Nombre5 Apellido5 (Prueba5) <prueba5@prueba5.com>
sig 3 CE99AF10 2005-08-25 Nombre5 Apellido5 (Prueba5) <prueba5@prueba5.com>

evolution:~/gnupg# █

```

Programas que soportan GnuPG

Para utilizar GnuPG de forma gráfica y no tener que estar escribiendo los comandos podemos encontrar para KDE el programa KGPG y para Gnome el programa SeaHorse.

Hay algunos clientes de correo que también soportan GnuPG como Evolution o Mozilla Thunderbird al que hay que ponerle el plugin Enigmail para que tenga soporte de GnuPG.

Referencia del Manual:

<http://www.lostscene.com/manuales/gnupg.php>

Firma Digital en la Argentina

A cargo del Sec. de Gestión de la Información - Lic. Carlos Rico



Mundo Digital



El mercado



educación



Vida social



Gobierno

Gobierno Electrónico

Gobierno electrónico:

La Carta Iberoamericana de Gobierno Electrónico "el **Gobierno Electrónico** es el uso de las **tecnologías de la información y de la comunicación**, en los órganos de la **Administración Pública**, para **mejorar** la información y los servicios ofrecidos a los **ciudadanos**, orientar la **eficacia** y la **eficiencia** de la gestión pública e incrementar sustantivamente la **transparencia** del sector público y de la activa **participación** de los **ciudadanos**."



Gobierno Electrónico

Gobierno electrónico:



Las Firmas



Mundo Digital



Firma digitalizada



Firma electrónica



Mundo Natural



Firma Holográfica



Certificación

Que se requiere?:

- **Atribuir** el documento a su autor (una persona) en forma fehaciente

(autenticar al autor).
- **Verificar** la no alteración del contenido del documento luego de que fue firmado

(integridad del contenido).
- **Garantizar** el NO REPUDIO.

Objetivo:

Poder enviar un documento firmado a través de **medios electrónicos** de manera que ese documento cuente, por lo menos, con las mismas **características técnicas de seguridad y legales** que tiene un documento firmado **hológraficamente**.



Definición:

- La firma digital es una solución tecnológica que permite **autenticar el origen y verificar la integridad** del contenido de un mensaje de manera tal que ambas características sean **demostrables ante terceros**.
- Es el resultado de aplicarle a un documento digital un **procedimiento matemático** que requiere **información de exclusivo conocimiento** del firmante, encontrándose ésta bajo su **absoluto control**.
- Debe ser susceptible de **verificación por terceras partes**, de manera tal que dicha verificación permita simultáneamente **identificar al firmante y detectar cualquier alteración** del documento digital posterior a su firma.

Propiedades:

- **Autenticidad:** Poder atribuir el documento únicamente a su autor de forma fidedigna, de manera de **poder identificarlo**.
- **Integridad:** Estar vinculada a los datos del documento digital, poniendo en evidencia su **alteración** luego de que fue firmado.
- **Exclusividad:** Garantizar que la firma se encuentre bajo el **absoluto y exclusivo control** del firmante.
- **No repudio:** Garantizar que el emisor **no** pueda **negar o repudiar** su autoría o existencia; ser susceptible de **verificación ante terceros**.
- **Validez:** Haber sido producida con un **certificado emitido** por un **Certificador Licenciado**.

Qué se puede Firmar:

- Datos enviados a través de un formulario web.
- Una imagen, fotos o música.
- Un base de datos.
- Un disco rígido, un CD o un DVD.
- Una página o un sitio de Internet.
- Una transacción electrónica o un e-mail.
- Una hoja de cálculo o un documento de texto.
- El código fuente de un programa o un software.
- Uno o varios archivos en general.

Qué no es una firma digital:

- Una firma digitalizada (una firma manuscrita escaneada).
- Una contraseña o password.
- Un sistema biométrico.
- Un sistema de autenticación: este requisito sólo no alcanza.
- Una firma electrónica.
- Un documento encriptado (solo se garantiza la confidencialidad).

Para tener en cuenta:

La Seguridad de la Información requiere la **CONFIABILIDAD** del sistema e implica:

- CONFIDENCIALIDAD
- INTEGRIDAD
- DISPONIBILIDAD
- AUTENTICIDAD



Documento no firmado ni cifrado
Sin protección ni seguridad.



Documento firmado pero no cifrado
Autenticidad – Integridad - No Repudio.



Documento cifrado pero no firmado
Confidencialidad.



Documento firmado y cifrado
Todas las características anteriores.

Firma electronica:

- Se entiende por Firma electrónica al conjunto de datos electrónicos integrados, ligados o asociados de manera lógica a otros datos electrónicos, utilizado por el signatario como su medio de identificación, que carezca de algunos de los requisitos legales para ser considerada una firma digital.
- Importante:

**UN DOCUMENTO FIRMADO DIGITALMENTE QUE NO TENGA
UN CERTIFICADO EMITIDO POR UN CERTIFICADOR LICENCIADO.
ES UNA FIRMA ELECTRONICA PERO NO ES UNA FIRMA DIGITAL.**

Sistema de Firma Digital:

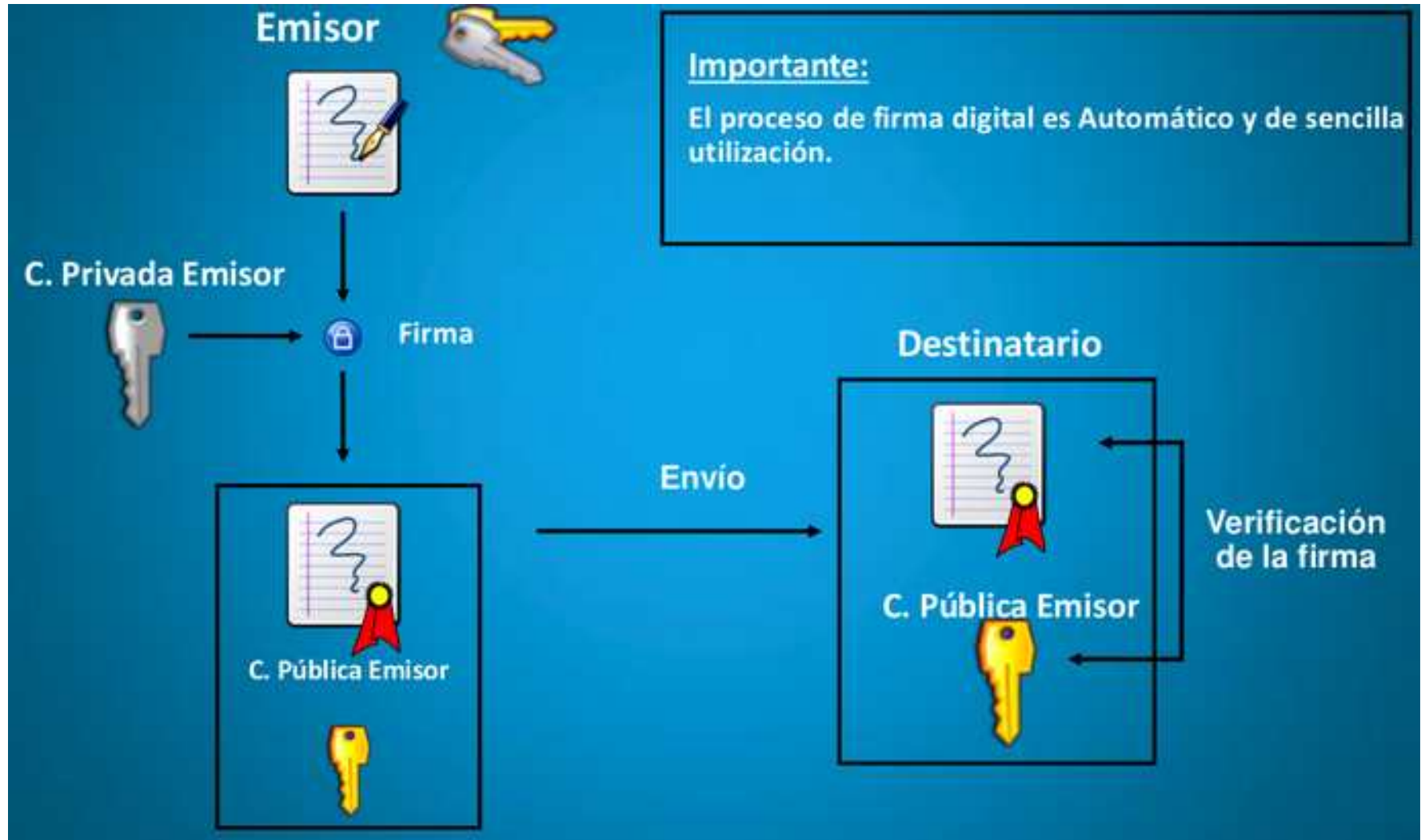
- **Quien firma (el suscriptor).**
- **Quien(es) necesita(n) verificar la firma.**
- **Quien testimonia que una firma digital pertenece a una cierta persona**
- **Quien controla el sistema.**

Ley N° 25.506 de Firma Digital:

- **Establece una Infraestructura de Firma Digital Nacional.**
- **Autoridad de Aplicación: Ministerio de Modernización.**
- **Certificadores Licenciados: Sistema de acreditación obligatorio.**
- **Firma digital: presunción de autoría e integridad, salvo prueba en contrario.**
- **Principio de equivalencia funcional.**
- **Firma electrónica: se invierte la carga probatoria.**
- **Despapelización del Estado.**
- **Neutralidad Tecnológica.**
- **Reconocimiento de certificados extranjeros.**
- **Protección del suscriptor del certificado.**

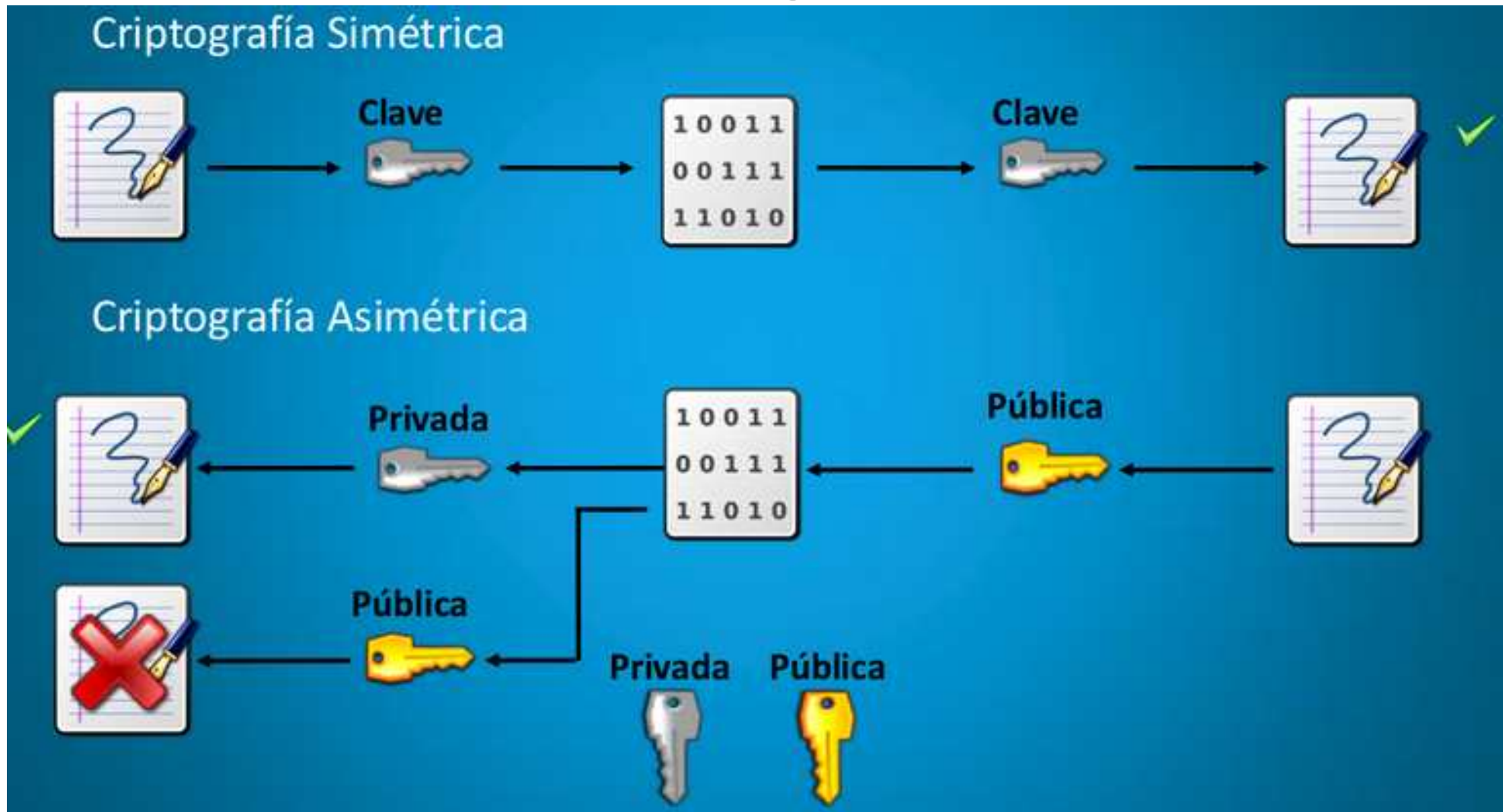
Firma Digital

Procedimiento: Clave Privada y Clave Publica.



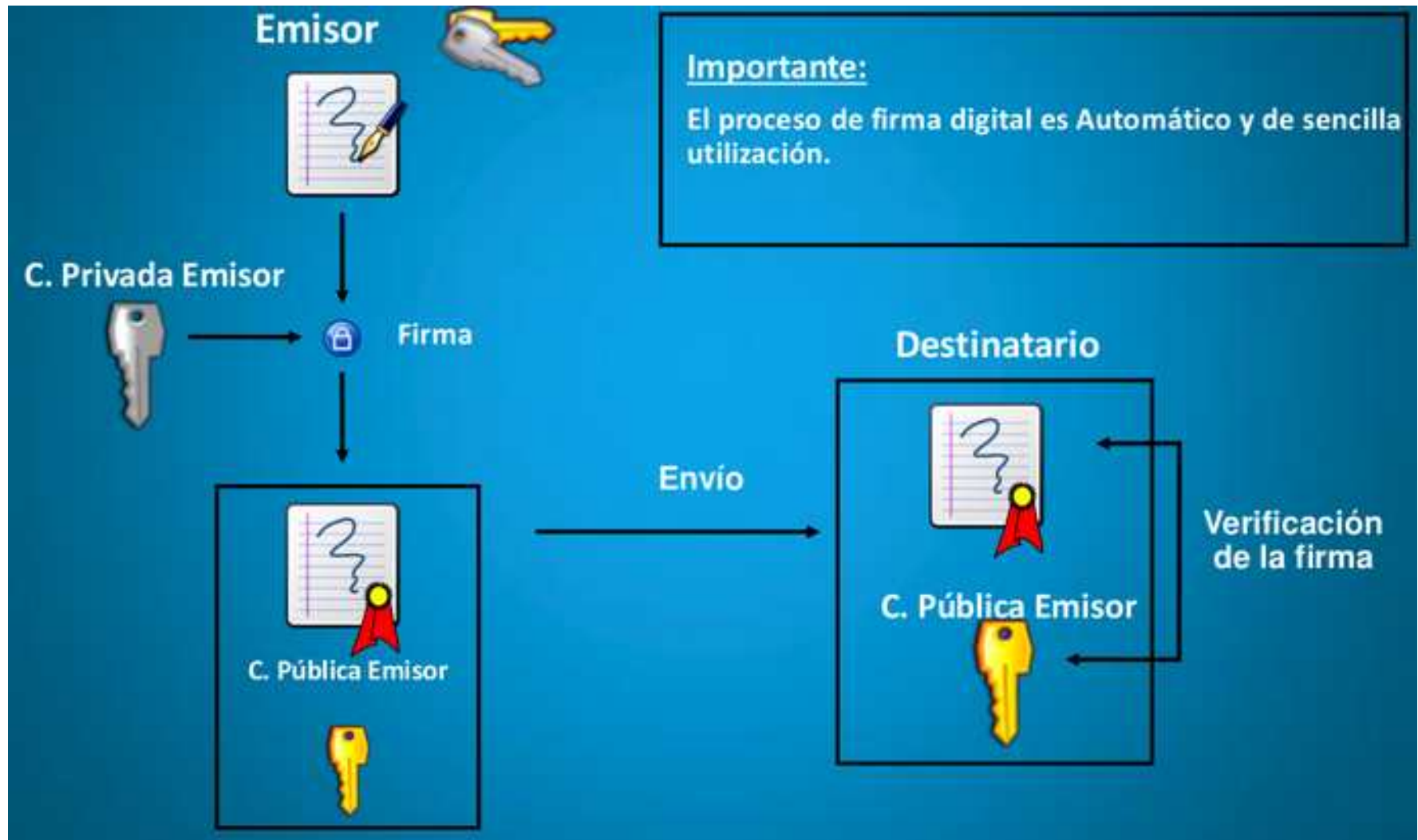
Firma Digital

Como funciona: Utiliza criptografía asimétrica.



Firma Digital

Procedimiento: Clave Privada y Clave Publica.



Firma Digital

Como funciona: Utiliza claves asimétricas Privada y Publica



Firma Digital

Como funciona: Utiliza Certificado de Clave Publica



Certificado de Clave Pública:

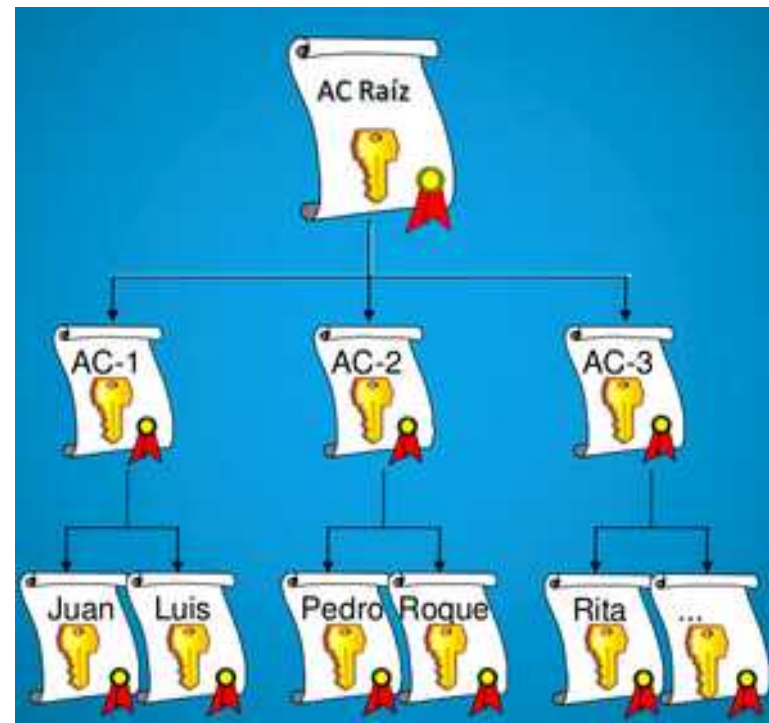
Los certificados de clave pública son documentos digitales firmados digitalmente por una **Autoridad Certificante** que vinculan la clave pública de una persona a sus datos de identidad.



AC RAIZ → Modernización.

AC N → Certificadores
Licenciados.

Personas Físicas



Firma Digital

Para Firmar: Donde almaceno mi clave Privada?



Computadoras
Teléfonos
tablets



No autorizado en la Argentina



Token
Dispositivo
criptográfico



**Por medio de un driver desde
un programa que permita
firmar.**

Protegido Con contraseña



Nube
Criptográfica



<https://firmar.gob.ar/firmador/#/>

<https://firmar.gob.ar/RA/>

**Accedo al URL de la PFDR
Con usuario y contraseña
Con doble seguridad dada por
Autorización enviada a mi
dispositivo movil.**

Para Verificar: Como verifico una Firma?



Con
Programas



**Requiere tener instalados los
certificados.**



Nube
Criptográfica



**Los Certificados los mantiene
La Plataforma.**

IMPORTANTE: La leyenda en un documento que aclare la Firma digital no implica su Firmado. Solo es verificable x comprobación del algoritmo con el certificado.

Autoridad de Registro: Nexa entre la AC y el subscriptor

Tiene a su cargo las funciones de validación de identidad y de otros datos de los solicitantes y suscriptores de certificados, registrando las presentaciones y trámites que les sean formulados por éstos.



Autoridad de Registro: Universidad Nacional de Mar del Plata

Universidad Nacional de Mar del Plata

Dirección: San Martín 2563, Piso 13 (7600) Mar del Plata , Buenos Aires

ginfo@mdp.edu.ar

0223-492-2684

Dirección: Habilitada a operar en modalidad móvil (7600) Mar del Plata , Buenos Aires

ginfo@mdp.edu.ar

0223-492-2684

<i>Oficial de Registro</i>	CALI Federico luis	federico@mdp.edu.ar
<i>Oficial de Registro</i>	KLINK Leandro esteban	leklink@mdp.edu.ar
<i>Responsable de Autoridad de Registro</i>	RICO Carlos alberto	ginfo@mdp.edu.ar
<i>Responsable de Soporte de Firma Digital</i>	ANDUEZA Federico ramon	anduezafr@mdp.edu.ar
<i>Responsable de Soporte de Firma Digital</i>	CALÍ Federico luis	federico@mdp.edu.ar
<i>Responsable de Soporte de Firma Digital</i>	IBARRA Luciano raúl	luciano@mdp.edu.ar

Seguridad Informatica para todos

PGP

GNUPG

Encriptar mensajes

Desencriptar mensajes

Firmar mensajes

Verificar mensajes firmados

Algunos Comandos

Anillo de confianza



Seguridad Informatica para todos

Lic. Carlos Alberto Rico

Ing. Felipe Evans

G.I.D.I.

Grupo de Ingeniería en Desarrollos Informaticos

Facultad de Ingeniería

U.N.M.d.P.

2009



Seguridad Informatica para todos

Criptología

Pretty Good Privacy

Privacidad bastante buena:

Zimmermann 1991.

Standard de Internet.

pgp -h



Seguridad Informatica para todos

GNU-PG

GnuPG es una aplicación que sirve para Encriptar y/o Firmar mensajes, documentos

GnuPG usa un sistema de claves públicas.

Cada Usuario tiene una clave privada y una clave pública.

El programa se puede obtener de la siguiente url <http://www.gnupg.org>



Seguridad Informatica para todos

GNU-PG

¿Para que ?

Encriptar mensajes

Desencriptar mensajes

Firmar mensajes

Verificar mensajes firmados

Anillo de confianza



Seguridad Informatica para todos

GNU-PG

Encriptar mensajes

Encripta quien envia.

Utiliza Clave Pública del destinatario.

```
gpg --armor --recipient ClaveID --encrypt mensaje
```



Seguridad Informatica para todos

GNU-PG

Desencriptar mensajes

Desencriptar quien recibe.

Utiliza su propia Clave Privada.

```
gpg --decrypt archivo
```



Seguridad Informatica para todos

GNU-PG

Firmar mensajes

Firma quien Envía

Utiliza su propia Clave Privada.

`gpg --clearsign a.txt` y genera: `a.txt.asc`

O

`gpg --sign a.txt` y genera: `a.txt.gpg` (encriptado)

Si Aplica `--sign` para desencriptar Usar `--decryp`



Seguridad Informatica para todos

GNU-PG

Verificar mensajes firmados

Verifica quien recibe

Utiliza la Clave Pública del que envió.

```
gpg --verify a.txt.asc
```



Seguridad Informatica para todos

GNU-PG

Anillo de confianza

Base del Escribano Virtual.

- 1) Escribano y usuario1 Intercambian sus Claves Publicas y las verifican con sus huellas.
- 2) Escribano Firma con su Clave Privada la Publica de Usuario1.
- 3) Usuario2 Recibe la Clave Publica de Usuario1 Firmada por Escribano.

Se basa en la confianza del Usuario2 sobre el Escribano.



Seguridad Informatica para todos

GNU-PG

Algunos Comandos

Crear Claves: `gpg --gen-key.`

Ver Publicas: `gpg --list-keys`

Ver Privadas: `gpg --list-secret-keys`

Borrar Priv.: `gpg --delete-secret-key ClaveID.` Archivo: `secreting.gpg`

Borrar Públ.: `gpg --delete-key ClaveID.` Archivo: `pubring.gpg`

Ver Huella: `pgp --fingerprint ClaveID` (Como el MD5)

Exportar Publica: `gpg --armor --output Archivo.asc --export ClaveID`

Exportar Privada: `gpg --armor --output Archivo.asc --export-secret-key ClaveID`



Seguridad Informatica para todos

GNU-PG

Algunos comandos

Importar Ambas: `gpg --import ClaveID`

Encriptar un mensaje : `gpg --armor --recipient ClaveID --encrypt mensaje`

Desencriptar mensaje : `gpg --decrypt archivo.asc`

Firmar mensaje: `gpg --clearsign archivo --> archivo.asc`

`gpg --sing archivo --> archivo.gpg`

`gpg --decrypt archivo.gpg` (para desencriptar la firma)

Verificar Firmas: `gpg --verify mensaje`



Seguridad Informatica para todos

GNU-PG

Algunos comandos

Buscar En servidores: `gpg --keyserver NombreServer --search-keys ClaveID.`

Importar desde Serv : `gpg --keyserver NombreServer --recv-keys ClaveID.`

exportar a un serv. : `gpg --keyserver NombreServer --send-keys ClaveID`

Acceder a la Shell: `gpg --edit-key ClaveID`

Revocar una Clave: `gpg --output archivo.asc --gen-revoke.`



Seguridad Informatica para todos

Firma Digital en la Argentina.

Autoridad Certificante de la Oficina Nacional de Tecnologías Información.

Autoridad Certificante

Oficina Nacional de Tecnologías de la Información (ONTI)

Subsecretaría de la Gestión Pública

Jefatura de Gabinete de Ministros

Abril, 2002

Sitio : <http://ca.pki.gov.ar/>



Seguridad Informatica para todos



Gracias por su Atención.

Summer Course - Parallel XML Processing

Peter Tröger

Blekinge Institute of Technology

Summer 2008

Examples partially adopted from www.w3schools.com - visit their site !

Parallel XML Processing ???

Three ways of
doing anything faster [Pfister]

- Work harder
- Work smarter
- Get help

ONLINE BUSINESS TOOLKIT

W3C outlines XML speed boosting plan

Tags: [XML](#), [W3C](#), [RRSHB](#), [MTOM](#)

Martin LaMonica CNET News.com

Published: 27 Jan 2005 09:05 GMT

Email Trackback Clip Link Print Post a comment

The **W3C**, the standards body in charge of developing XML, said on Tuesday that it has issued three recommendations designed to make handling XML-formatted data more efficient. The

specifications providers, including software infrastructure services applications, a variety of methods which can be s

The XML-binary agreed-on way as text, such as representing binary protocol called MTOM, which will members also do or RRSB, a method overall performance

High Speed Parsing Very Large XML Files

sponsored by Rogue Wave Software

Posted: 10 Jun 2008
Published: 01 Jun 2008
Format: PDF
Length: 6 Page(s)
Type: White Paper

ABSTRACT:

As the number of XML files in enterprise organizations significantly increases, architects, application developers, and data integration specialists must deal simultaneously with the growing number of XML formatted messages on the network as well as their rapidly expanding file sizes.

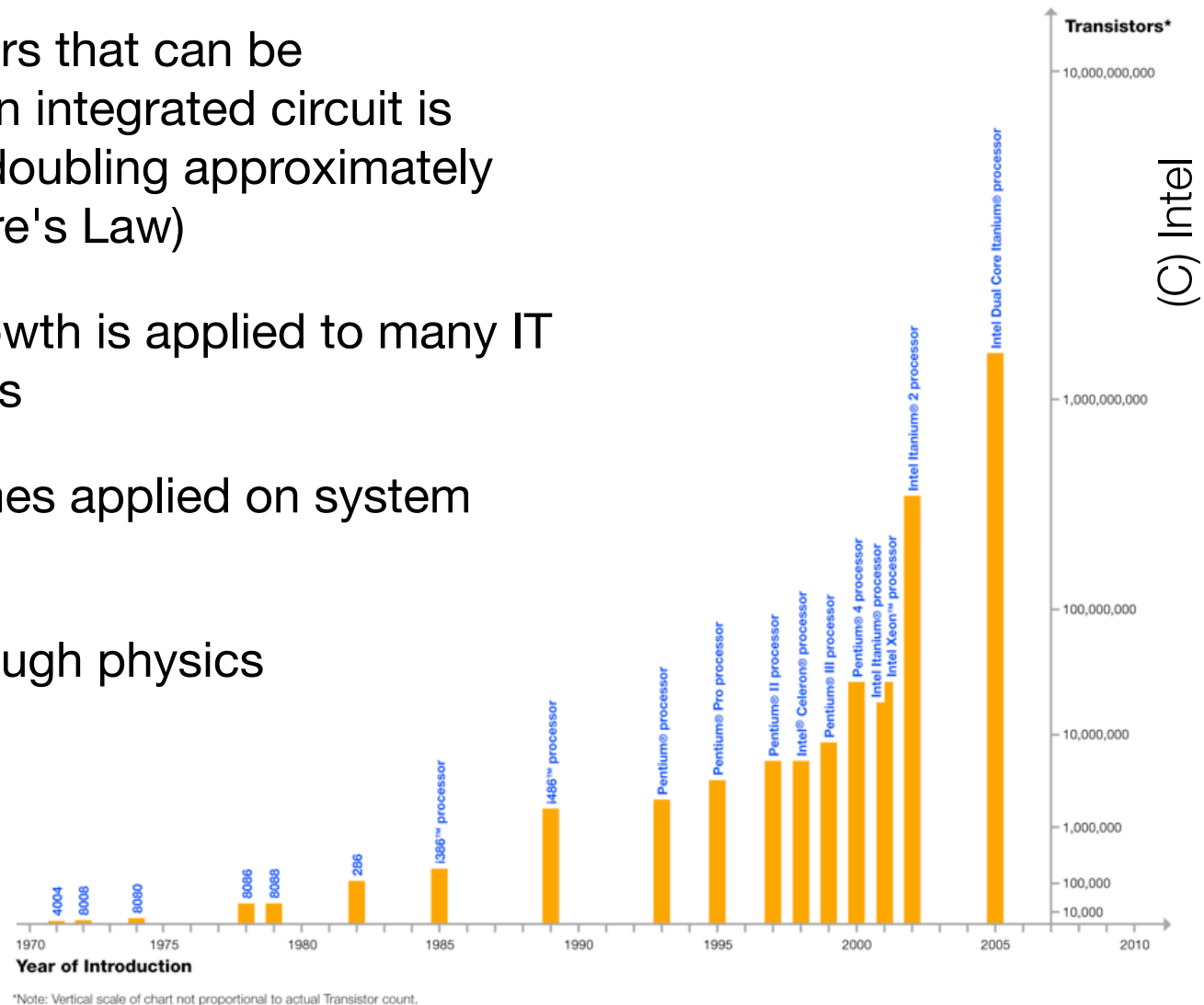
While XML has become the 'de facto' standard for transferring data from one step to the next in a business process or between applications, there are more and more occasions when those files are easily exceeding 50-100MB in size, causing performance slow downs and memory overloads. This paper describes the challenges associated with parsing very large XML files in today's system environments and offers a solution with HydraSDO for XML.

Rogue Wave introduces a new solution for parsing very large XML files: HydraSDO for XML. It is specifically



Work Harder

- „...the number of transistors that can be inexpensively placed on an integrated circuit is increasing exponentially, doubling approximately every two years. ...“ (Moore's Law)
 - Rule of exponential growth is applied to many IT hardware developments
 - Density rule is sometimes applied on system performance
- Meanwhile limitations through physics
 - Cooling, tunneling



Getting Help by Parallelism

- Where ?
 - Inside the processor (instruction-level parallelism, multicore)
 - Through multiple processors in one machine (multiprocessing)
 - Through multiple machines (distributed computing)
- How ?
 - Data Parallelism - Same operation on disjoint data sets
 - Control / Task Parallelism - Perform different operations at the same time
 - Program must be constructed of independent portions, or contain control structures to be carried out in parallel
 - Flow Parallelism - Overlapping work on data stream (pipelines, assembly line model)

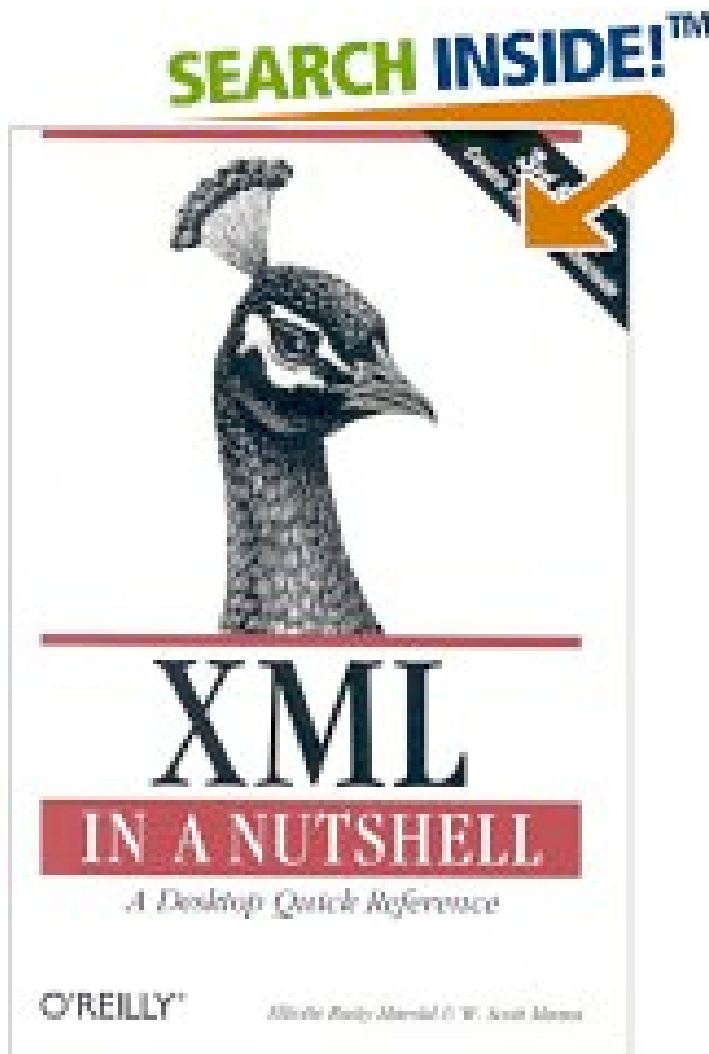
Three ways of XML processing speedup

- Faster hardware
(XSLT processors, ...)
- More condensed XML formats
(XOP, SOAP over TCP, ...)
- **Parallel processing
(multicore, SMP)**

Course Agenda

- *Investigate (and improve) state-of-the-art in parallelized XML processing*
- **Today / Tomorrow:** Crash course in XML technology and vocabulary
 - First assignment: Review of one XML parallelization paper
- 3.7.2008: Mutual presentation and discussion of reviewed paper (30 min)
 - Second assignment: Choosing one paper and performing in-depth analysis, including experiment
- 28.7. / 29.7.2008: Mutual presentation and discussion of experiment results
 - Third assignment: Written report about possible improvements of paper, or about parallel XML processing in the participants own research field
- This is a summer course - relax ...

Course Book



(C) Amazon

- You might also consult
 - Annotated XML specification (<http://www.xml.com/axml/testaxml.htm>)
 - All original specifications (<http://www.w3.org/TR>)
 - Endless amount of online tutorials
 - High quality from IBM DeveloperWorks
 - <http://www.w3schools.com>
- Papers about parallel XML processing are provided on the course home page

Outline

- **XML Basics**
 - Markup concepts, schema definition, DTD
 - Namespaces
- Document-centric XML
 - XSL Transformations, XPath
 - XML Rendering (CSS, XSL-FO)
- Data-centric XML
 - XML Schema, XQuery
 - Programming models

XML - The Origin

- **Standard Generalized Markup Language (SGML)**
 - Invented by IBM in the 70's, ISO standard 8879 in 1986
- Most successful SGML application - **HyperText Markup Language**
- Too complicated for complete implementation - „**Sounds Great Maybe Later**“
- Februar 1998: „Lite“ version of SGML for structural markup - **XML 1.0**
 - Development started 1996
 - Second edition 2000, third edition 2004, fourth edition 2006
- XML 1.1 published on 04 February 2004, second edition 2006
 - Minor changes only, not widely implemented
 - Support for Unicode 4.0 (e.g. Amharic) in markup

SGML in 1985

```
Document: Bungler OED           At: "<entry>"

<entry>
  <hwsec>
    <hwgp>
      <hwlem>bungler</hwlem>
      <pron>b<I>ʊ</I>ˈŋɡlə</pron>. </hwgp>
      <vfl>Also <vd>b</vd> <vf>bongler</vf>,
        </vfl>
      <etym>f. as prec. + <xra><xlem>-ER</xlem>
    </hwsec>
    <sen>One who bungles; a clumsy unskilful
      <quot>
        <qdat>1533 </qdat>
        <auth>MORE </auth>
        <wk>Answ. Poyson. Bk. </wk>Wks. (1557)
        <qtxt>He is even but a very bungler.
      </quot>
    </sen>
  </hwgp>
</entry>
```

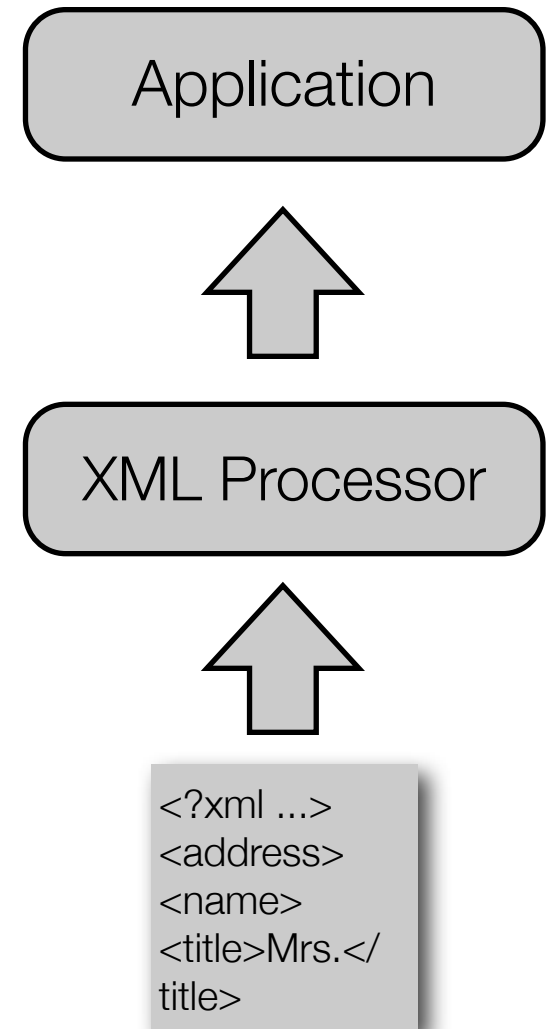
(C) Wikipedia

XML

- *eXtensible Markup Language*
- Design goals
 - Straightforwardly usable over the Internet
 - Support for a variety of applications
 - Compatible with SGML
- W3C recommendation(s) - lexical grammar and parser requirements
- Meta-markup language for text documents
 - W3C XML specification describes grammar for XML documents
 - Compliant documents are **well-formed**
- Restricted set of tags leads to one **XML application** (e.g. vector graphics)

XML is Everywhere

- Document format of choice for most newly developed software
- Stock trading, games, mobile phones, physics, journalism, comics, insurance business, multimedia, travel business, telecommunication, software, ...
- **XML technologies**
 - XML Schema, DOM, SAX, XHTML, XSLT, ...
- **XML applications - special XML-based formats**
 - SVG, MathML, RDF, SOAP, ...
 - Accordingly used by specialized software
- Narrative documents vs. data-centric documents



XML as Document Format

- „Narrative documents“ - reflects SGML background
 - Used for large-scale technical documentation
- Structuring of human-readable documents
 - Chapters, sections, headings, lists, paragraphs, ...
 - Removal of all markup roughly keeps the original text semantic
 - Non-textual information is typically meta-data
- No information about rendering of structured content
 - Separate XML technologies to describe layout (CSS)
 - Description of transformation to renderable format (XSLT, DSSSL)

XML as Data Format

- Original intention of XML was a document format
- Major issue of data format portability lead to XML as data format
 - Easy generation and parsing (compared to binary formats)
 - Support for complex nested data structures
 - Independent from programming language and operating system
- Interoperability ,glue‘ with XML-based network protocols
 - REST, XML-RPC, SOAP, BEEP, XMMP
- Has other implications than XML documents
 - Demand for strong typing (XML schema) and programming models

Common Misunderstandings

- HTML != XML, even though both came from SGML
 - HTML parsers accept sloppy markup, XML parsers not
 - HTML is for humans, XML for machines
 - Tags and elements are pre-defined in HTML; XML gives only syntax rules
 - XML tags describe semantic, HTML tags describe layout
- XML is not a programming language
 - Just a document, no compiler to executable code
- XML is not a network protocol
 - Protocol can send around XML documents (SOAP over HTTP)

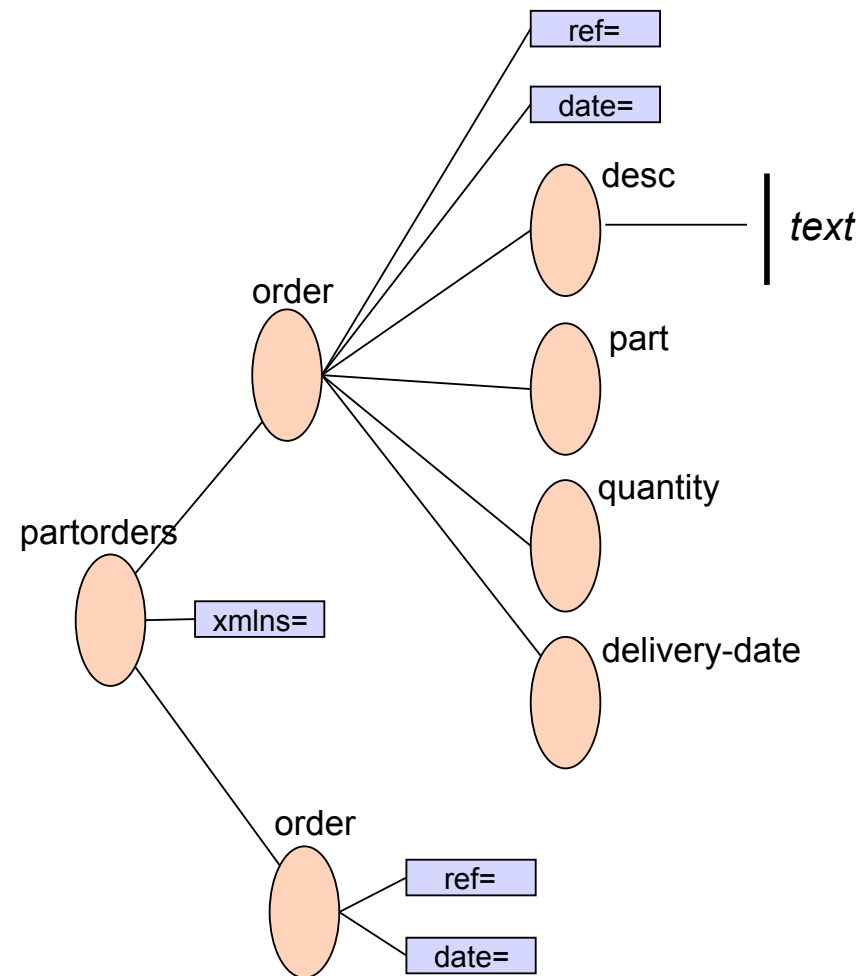
XML Markup Concepts

```
<address>
  <name>
    <title>Mrs.</title>
    <first-name>
      Mary
    </first-name>
    <last-name>
      McGoon
    </last-name>
  </name>
  <street>
    1401 Main Street
  </street>
  <city state="NC">Anytown</city>
  <postal-code>
    34829
  </postal-code>
</address>
```

- *Tag*
 - Text between angle brackets
 - Starting / ending tag
- *Element*
 - Start / end tag and *element content* in between
 - Child elements
 - Empty elements
- *Attribute*
 - Name-value pair inside the starting tag

XML Tree Structure

```
<partorders xmlns="...">
  <order date="..."
        ref="...">
    <desc> ..text..
  </desc>
  <part />
  <quantity />
  <delivery-date />
</order>
<order ref="..." .../>
</partorders>
```



(C) Ian Graham

Basic rules

- All XML documents must contain a single **root element**
- All elements (but the root element) have one parent element
- Elements can be nested, but not overlapped
- End tags are required
- Elements are case-sensitive
- Attributes must have (consistent) quoted values
- Attributes must be unique inside the element
- Recommended (but not mandatory) XML declaration on start
 - `<?xml version="1.0" encoding="ISO-8859-1" standalone="no"?>`

Basic rules

- White space
 - White space characters: tab, line-feed, carriage-return, single space
 - White space in element content is **significant**
 - White space in attributes is normalized (condensed to single one)
 - White space between elements is ignored
- EOL is normalized by XML processor
- Document must comply to defined character encoding (default UTF-8)

```
<?xml version="1.0" encoding="UTF-8"?>  
  <俄語>Данные</俄語>
```

XML Entities

- Macro definition for the document, as part of the DTD
- Predefined entity references for special characters
 - *<* for the less-than sign
 - *>* for the greater-than sign
 - *"* for a double-quote
 - *'* for a single quote (or apostrophe)
 - *&* for an ampersand
- Only detected and replaced in element content and attribute values

CDATA Section

```
<script>
  <![CDATA[
    function matchwo(a,b)
    {
      if (a < b && a < 0) then
        {
          return 1;
        }
      else
        {
          return 0;
        }
    }
  ]]>
</script>
```

- Special section for unparsed character data
- Intended for human editors or too many special characters
 - XML example in DOCTYPE document
 - JavaScript code in XHTML web page

Processing Instruction

```
<?xml-stylesheet
  type="text/css" media="screen"
  title="Regular fonts"
  href="regular.css"?>

<?mso-application
progid="InfoPath.Document"?>

<list>
  <?odbc2xml q1:select * from cartoon?>
  <item>
    <?odbc2xml q1.Name?> says
    <?odbc2xml q1.CatchPhrase?>
  </item>
</list>
```

- Provide information to specific applications reading the document
- PI's are markup, but no elements
 - Can occur everywhere, like comments
 - Typical use for style sheet declaration
- XML declaration is not a processing instruction

Schema

- **XML application** demands description of acceptable markup
 - **Schema definition**
- Comparison of document instances with schema definition
 - Compliant documents are **valid**
 - Validity of a XML document always depends on given schema definition
 - All documents must be **well-formed**, and might be **valid**
- Many different XML schema languages
 - **Document Type Definition (DTD)** language, as part of XML 1.0
 - **XML Schema** language, separate W3C specification
 - Schematron, Hook, RELAX NG, Examplotron,

XML Application Example

```
<?xml version="1.0" standalone="no"?>
<!DOCTYPE svg PUBLIC "-//W3C//DTD SVG 1.1//EN"
"http://www.w3.org/Graphics/SVG/1.1/DTD/svg11.dtd">

<svg width="100%" height="100%" version="1.1
  xmlns="http://www.w3.org/2000/svg">

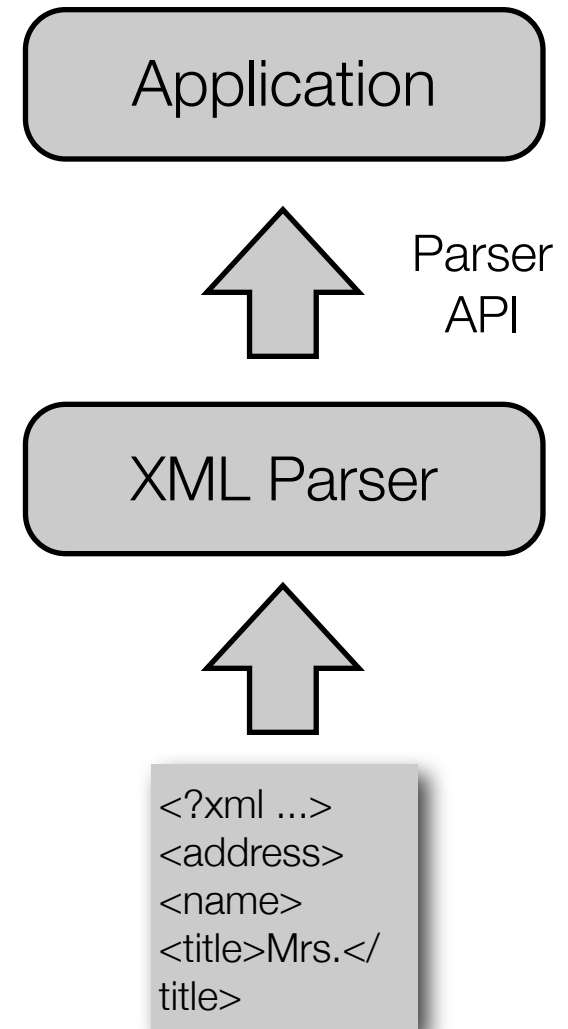
  <defs>
    <linearGradient id="orange_red" x1="0%" y1="0%" x2="100%" y2="0%">
      <stop offset="0%" style="stop-color:rgb(255,255,0);stop-opacity:1"/>
      <stop offset="100%" style="stop-color:rgb(255,0,0);stop-opacity:1"/>
    </linearGradient>
  </defs>

  <ellipse cx="200" cy="190" rx="85" ry="55" style="fill:url(#orange_red)"/>
</svg>
```



XML Parser

- Used by application trying to **understand** the XML document
 - Splits document in elements, attributes, and other parts
 - Hands over parts to the application
 - Always check if document is well-formed
- Possibility for **validating parser**
 - Documents contains URL to schema definition
 - On **validity error**, application decides over continuation
 - Validation is optional, and usually demands a Internet connection



XML Documents

- **Well-formed XML document**

- Follows the rules of the XML syntax
- Does not have an according DTD or schema definition

- **Valid XML document**

- Follows the rules defined by XML syntax and its DTD / Schema definition

- **Invalid document**

- Doesn't follow the XML syntax or its DTD / schema definition
- Rejected by parser

Document Type Definition (DTD)

- Applications have restrictions on XML document format - **vocabulary**
 - Allowed elements and their attributes
 - Content model - elements and data allowed inside element content
 - Required attributes, default values, list of valid values for attributes
- Application using the parser decides what to do with invalid document parts
- *Document type declaration* - specification of the DTD for a document
- *Document type definition* - the DTD file containing the specification
- Everything that is not allowed is forbidden
- Based for mutual understanding of a class of XML documents

Internal / External DTD Example

```
<?xml version="1.0"?>
<!DOCTYPE note [
  <!ELEMENT note (to,from,heading,body)>
  <!ELEMENT to      (#PCDATA)>
  <!ELEMENT from    (#PCDATA)>
  <!ELEMENT heading (#PCDATA)>
  <!ELEMENT body    (#PCDATA)>
]>
<note>
  <to>Tove</to>
  <from>Jani</from>
  <heading>Reminder</heading>
  <body>Don't forget me this weekend</
body>
</note>
```

```
<!DOCTYPE note SYSTEM "note.dtd">
```

- Typically external DTD, referred my multiple documents
- Parsed Character Data (PCDATA)
 - Pure text without tags and elements
- Document type declaration before root element
- Example: *www.bth.se*
- No exhaustive typing - intended for narrative documents

DTD Syntax Example

```
<!-- address.dtd -->
<!ELEMENT addressbook (address+)>
<!ELEMENT privbook (address*)>
<!ELEMENT address (name, street, city, state, postal-code)>
<!ELEMENT name (title?, first-name, (middle-initial | middle-name)?, last-name)>
<!ELEMENT title (#PCDATA)>
<!ELEMENT first-name (#PCDATA)>
<!ELEMENT last-name (#PCDATA)>
<!ELEMENT street (#PCDATA)>
<!ELEMENT city (#PCDATA)>
<!ELEMENT state (#PCDATA)>
<!ELEMENT postal-code (#PCDATA)>
```

- `<addressbook>` contains one or more addresses, `<privbook>` zero or more
- `<address>` element contains other elements in specified order (comma)
- `<name>` element contains optional `<title>` element (question mark), followed by first name and potentially a middle initial or middle name, and the last name

DTD Examples

- DocBook-XML DTD
 - 11.000 lines
 - Definition of XML format for text processing
- Scalable Vector Graphic (SVG) DTD
 - 1.000 lines
 - Definition of XML format for vector graphics
- Most exhaustive source of DTD definitions is the W3C itself
 - XHTML, HTML, MathML, SVG, PICS, RDF, SPARQL, SMIL, XML Schema

DTD Example: Dublin Core

- Original XML hype around the automated ,understanding‘ of documents
 - Dublin Core provides schema for annotation of documents
 - 15 standardized information items: title, creator, subject, ...

```
<?xml version="1.0"?>
<!DOCTYPE rdf:RDF PUBLIC "-//DUBLIN CORE//DCMES DTD 2002/07/31//EN"
                    "dcmes-xml-dtd.dtd">
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
        xmlns:dc="http://purl.org/dc/elements/1.1/">
  <rdf:Description
    rdf:about="http://www.ilrt.bristol.ac.uk/people/cmdjb/">
    <dc:title>Dave Beckett's Home Page</dc:title>
    <dc:creator>Dave Beckett</dc:creator>
    <dc:publisher>ILRT, University of Bristol</dc:publisher>
    <dc:date>2002-07-31</dc:date>
  </rdf:Description>
</rdf:RDF>
```

Namespaces

- Documents might combine markup from different XML applications
 - XHTML document with SVG pictures and MathML formula
- Two major goals for **XML namespaces** (W3C spec.)
 - Distinguish elements with same names from different vocabularies
 - Summarize all elements and attributes of one XML application
- Every element and attribute gets a **prefix**
 - Every prefix is mapped to an URI („xml“ is a reserved prefix)
 - Elements and attributes without prefix have a default URI
 - Elements and attributes with the same URI are in the same namespace
- Separate W3C specification, created one year after XML 1.0

XML Namespaces

```
<html xmlns="http://www.w3.org/HTML/1998/html4"
      xmlns:xdc="http://www.xml.com/books">
  <head><title>Book Review</title></head>
  <body>
    <xdc:bookreview>
      <xdc:title>XML: A Primer</xdc:title>
      <table>
        <tr align="center">
          <td>Author</td><td>Price</td>
          <td>Pages</td><td>Date</td></tr>
        <tr align="left">
          <td><xdc:author>Simon St. Laurent</xdc:author></td>
          <td><xdc:price>31.98</xdc:price></td>
          <td><xdc:pages>352</xdc:pages></td>
          <td><xdc:date>1998/01</xdc:date></td>
        </tr>
      </table>
    </xdc:bookreview>
  </body>
</html>
```

XML Namespaces Rules

- Namespace binding is valid for element itself and it's element content
- Default namespace
 - Applies when element / attribute has no namespace prefix
- URL's as namespace were chosen to re-use concept of domain names
 - No need for valid content, but typical source of confusion
 - This is ok: `<html xmlns:pt="mailto:peter@troeger.eu">`
- Standardized syntax: `[prefix]:[local part]`
 - Complete construct forms a **qualified name (QName)**
- In practice, people tend to overlook the shortcut nature of the prefix

Namespace Facts

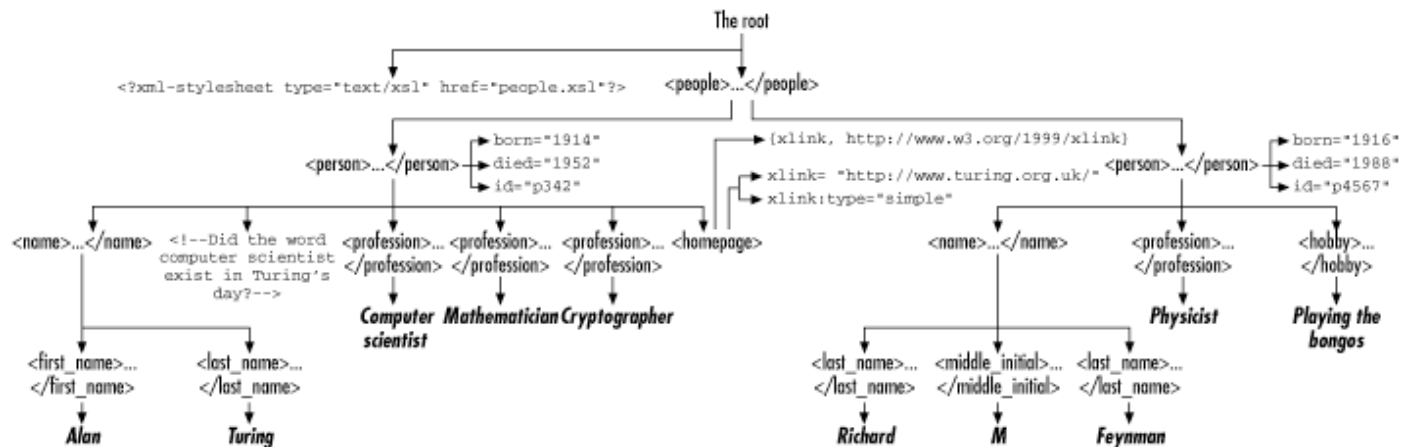
- Separation of XML specification and XML Namespaces specification
 - Non-namespace parser can still handle the documents
- Aware parser performs additional checks
 - Every prefix must be mapped to a URI
 - Configurable feature for most parsers
- DTDs and namespaces are independent
 - DTD definitions must exactly reflect chosen prefixes
 - Some tricks to make prefixes in DTDs still changeable (check course book)

Outline

- XML Basics
 - Markup concepts, schema definition, DTD
 - Namespaces
- **Document-centric XML**
 - XSL Transformations, XPath
 - XML Rendering (CSS, XSL-FO)
- Data-centric XML
 - XML Schema, XQuery
 - Programming models

XPath

- Non-XML language, used for identifying parts of XML documents
- Originally developed for XSLT
- Basis for other XML standards - XSLT, XForm, DOM, XML Schema, ...
- 7 types of nodes in a XML document
 - root nodes, element nodes, attribute nodes, text nodes, comment nodes, processing instruction nodes, name space nodes



(C) Harold et. al

XPath Location Path

- **Location path** for a set of nodes in a document
 - Build from sequential **location steps**
 - Every location step is processed relative to a **context node**
- Example: „/“ location step selects the root node of a document
- Example: Single element name as location step selects all child nodes
- Location steps can also address attributes („@attr“) or other node types
- Different wildcards for matching multiple nodes
- Location steps in a location path are separated by „/“
 - Either absolute or relative location path
- „//“ for ancestor, „..“ for parent element, „.“ for context node

XPath Examples

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<bookstore>
  <book category="COOKING">
    <title lang="en">Everyday Italian</title>
    <author>Giada De Laurentiis</author>
    <year>2005</year><price>30.00</price>
  </book>
  <book category="CHILDREN">
    <title lang="en">Harry Potter</title>
    <author>J K. Rowling</author>
    <year>2005</year><price>29.99</price>
  </book>
  <book category="WEB">
    <title lang="en">XQuery</title>
    <author>James McGovern</author>
    <year>2003</year><price>49.99</price>
  </book>
</bookstore>
```

- Select all nodes:
`"/bookstore/book"`
- Select the first book node:
`„/bookstore/book[0]"`
- Select the prices:
`„/bookstore/book/price/text()"`
- Select expensive books:
`"/bookstore/book[price>35]/price"`

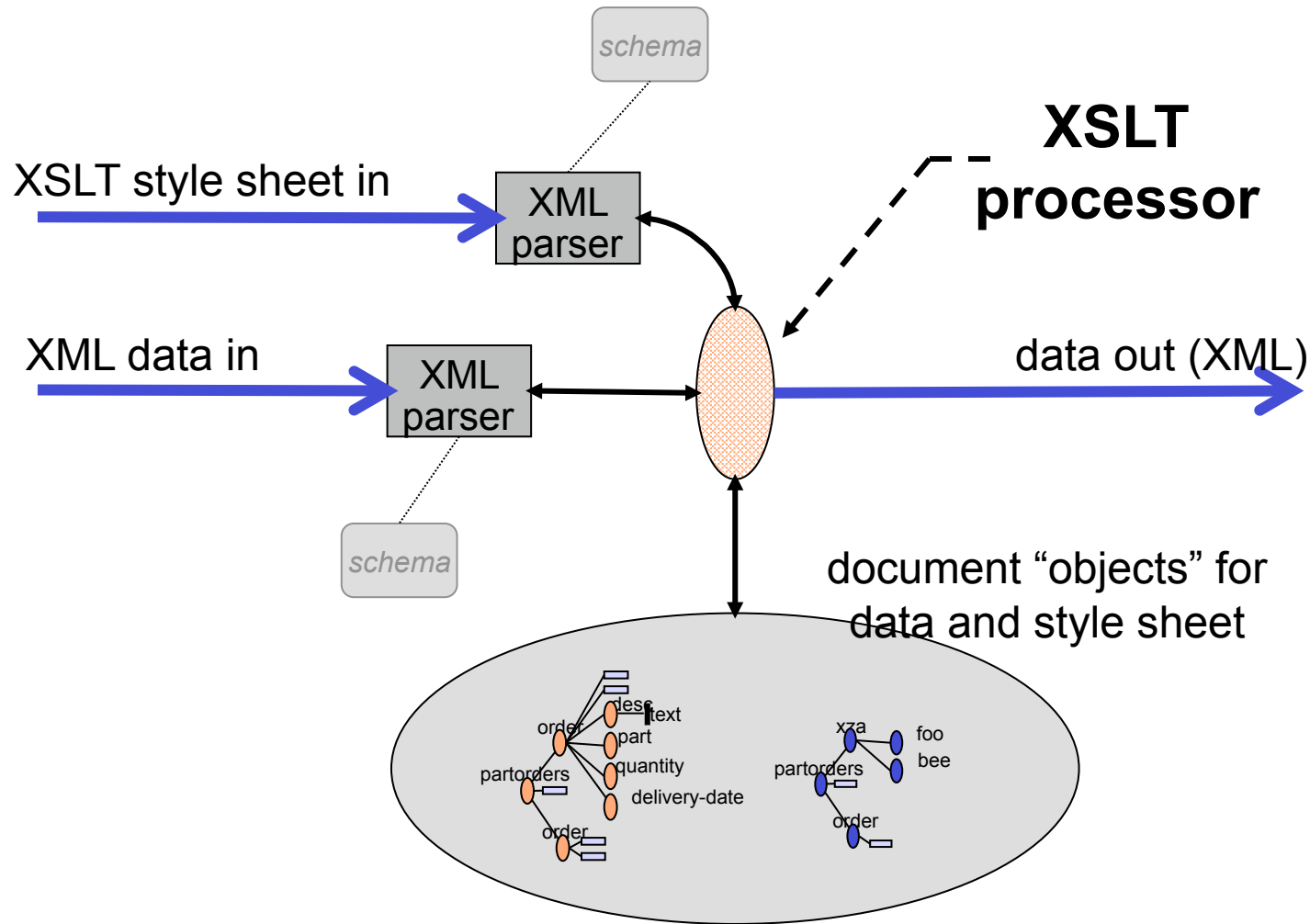
More XPath Features

- Number functions
- Boolean functions
- String functions
- Functions for handling of node sets
- See Wikipedia article for good explanation ...

XSL Transformations (XSLT)

- eXtensible Stylesheet Language (XSL)
 - HTML tags are predefined, so rendering is clear - not with XML !
 - XML-based formulation of so-called *style sheets*
 - Contains *XSL Transformations (XSLT)* and *XSL Formatting Objects (XSL-FO)*
- XSLT is a XML application
 - Define rules to transform *source tree* to *result tree*
 - An XSLT document is a *style sheet*, containing a *transformation template*
 - Contains *template rules*
 - Navigation in the source tree by *XPath*
- XSLT processor in browser (IE6), application server (Cocoon) or library (Xalan)

XSLT Concept



(C) Ian Graham

XSLT Concepts

```
<?xml version="1.0"?>
<xsl:stylesheet version="1.0" xmlns:xsl=...>
</xsl:stylesheet>
```

- Style sheet is a set of `<xsl:template>` elements
 - Rules tell the XSLT processor what to do with markup of the source tree
 - Empty style sheet just copies the content
 - `match` attribute - determines pattern for matching input
- Style sheet must output well-formed markup
 - Otherwise, style sheet itself is not well-formed

```
<?xml version="1.0"?>
<xsl:stylesheet version="1.0"
    xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
  <xsl:template match="person">
    <p>Eine Person</p>
  </xsl:template>
</xsl:stylesheet>
```

XSLT Concepts

- `<xsl:value-of>`
 - Extract value of XML element and add it to the output stream
 - Element value: Text without tags and with resolved entity references
- `<xsl:for-each>`
 - Select every element of a specified node set
 - Support for filtering and sorting
- Support for different kinds of conditions (if, choose, ...)
- `<xsl:apply-templates>`
 - Define processing order for source tree (based on XPath)

```

<?xml version="1.0" encoding="ISO-8859-1"?>
<xsl:stylesheet version="1.0"
    xmlns:xsl="http://www.w3.org/1999/XSL/Transform" >
  <xsl:template match="/">
    <html><body>
      <h2>My CD Collection</h2>
      <table border="1">
        <tr bgcolor="#9acd32">
          <th align="left">Title</th>
          <th align="left">Artist</th>
        </tr>
        <xsl:for-each select="catalog/cd">
          <tr>
            <td><xsl:value-of select="title"/></td>
            <td><xsl:value-of select="artist"/></td>
          </tr>
        </xsl:for-each>
      </table>
    </body></html>
  </xsl:template>
</xsl:stylesheet>

```

```

<?xml version="1.0" encoding="ISO-8859-1"?>
<?xml-stylesheet type="text/xsl" href="cat.xsl"?>
<catalog>
  <cd>
    <title>Empire Burlesque</title>
    <artist>Bob Dylan</artist>
    <country>USA</country>
    <company>Columbia</company>
    <price>10.90</price>
    <year>1985</year>
  </cd>

```

Integrated Template Rules

- 7 types of nodes in a XML document
 - root nodes, element nodes, attribute nodes, text nodes, comment nodes, processing instruction nodes, name space nodes
- Default XSLT rule for every node type
 - Text and attribute nodes are copied
 - For element and root nodes, template rules are applied to all child nodes
 - Comment and processing instruction nodes are ignored
 - Name space nodes are ignored, but all necessary name space declaration are written to the output

XSLT Output Configuration

- `<xsl:output>`
 - *method*=“xml” | “html” | “text” | qname
 - Default method is *html* if root node has an element child, the local name of first such element is “html” (any case), and the namespace is null
 - Otherwise, default method is *xml*
 - *version* = <version of the output method>
 - XML: default is 1.0, HTML: default is 4.0
 - *encoding*=<name of output encoding>
 - *media-type*=<MIME type> (text output: default is text/plain)
 - ...

More concepts in XSLT

- XSLT is a turing-complete (programming) language
- Numbering and sorting of output elements
- Conditional processing
- Iterations
- Extension elements and functions
- Import of other style sheets
- Utilization of complex XPath expressions

Cascading Stylesheets (CSS)

- Non-XML language for describing presentation of XML documents
- CSS is defined in different levels
 - Level 1 - only used for HTML formatting
 - Level 2 - XML and HTML can be formatted in a similar manner
- Trivial syntax
 - Every line denotes an element and the styles to be applied
 - Over 100 style properties, most with useful default values
- Included in HTML / XML page as processing instruction
- Some simple selectors: * (all non-covered elements), attributes, ID's, ...
- Exhaustive output formatting by XSL Formatting Objects (XSL-FO)

CSS Example

```
<?xml-stylesheet href="style.css" type="text/css"?>
<card>
  <name>John Doe</name>
  <title>CEO, Widget Inc.</title>
  <email>john.doe@widget.com</email>
  <phone>(202) 555-1414</phone>
  <logo url="widget.gif"/>
</card>
```

```
card { background-color: #cccccc; border: none;}
name { display: block; font-size: 20pt; margin-left: 0;}
title { display: block; margin-left: 20pt;}
email { display: block; margin-left: 20pt;}
phone { display: block; margin-left: 20pt;}
```

Outline

- XML Basics
 - Markup concepts, schema definition, DTD
 - Namespaces
- Document-centric XML
 - XSL Transformations, XPath
 - XML Rendering (CSS, XSL-FO)
- **Data-centric XML**
 - XML Schema, XQuery
 - Programming models

XML Schema

- Alternative to DTD definition - description of document structure and data
 - XML schema definition („XSD“) has an XML syntax
 - Can be processed as any other XML
 - Example: XSLT for transformation of XML schema to web form
 - Improved data type support
 - Integers, floats, datetime, decimal, string, URL, sets, ...
 - Simple and complex data types, inheritance
 - More expressive power
 - String length, regular expressions, support for XML namespaces
- XML described by XML schema definition is called **instance document**

XML Schema Example - Namespace Support

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
           targetNamespace="http://www.troeger.eu">
  ...
</xs:schema>
```

```
<bookstore
  type="science fiction"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://www.troeger.eu bookstore.xsd" >
  <book>
    <author>...</author>
    <title>...</title>
    <publisher>...</publisher>
    <isbn>...</isbn>
    <price>...</price>
  </book>
  <book>
    ...
  </book>
</bookstore>
```

XML Schema Example - Documentation

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:annotation>
    <xs:documentation xml:lang="en">...</xs:documentation>
  </xs:annotation>
  <xs:element name="bookstore">
    <xs:complexType>
      ...
    </xs:complexType>
  </xs:element>
</xs:schema>
```

- Child elements of `<xs:schema>` with the type `<xs:element>` determine possible root nodes
- Possibility for in-build documentation by `<xs:annotation>` element

```
<bookstore type="science fiction">
  <book>
    <author>...</author>
    <title>...</title>
    <publisher>...</publisher>
    <isbn>...</isbn>
    <price>...</price>
  </book>
  <book>
    ...
  </book>
</bookstore>
```

XML Schema Example - Matryoshka Design

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="bookstore">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="book" maxOccurs="unbounded">
          ...
        </xs:element>
      </xs:sequence>
      <xs:attribute name="type" type="xs:string"/>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

- One `<bookstore>` element contains multiple books and an type attribute

```
<bookstore type="science fiction">
  <book>
    <author>...</author>
    <title>...</title>
    <publisher>...</publisher>
    <isbn>...</isbn>
    <price>...</price>
  </book>
  <book>
    ...
  </book>
</bookstore>
```

XML Schema Example - Matryoshka Design

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="bookstore">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="book" maxOccurs="unbounded">
          <xs:complexType>
            <xs:sequence>
              <xs:element name="author" type="xs:string"/>
              <xs:element name="title" type="xs:string"/>
              <xs:element name="publisher" type="xs:string"/>
              <xs:element name="isbn" type="xs:string"/>
              <xs:element name="price" type="xs:string"/>
            </xs:sequence>
          </xs:complexType>
        </xs:element>
      </xs:sequence>
      <xs:attribute name="type"
        type="xs:string"/>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

```
<bookstore type="science fiction">
  <book>
    <author>...</author>
    <title>...</title>
    <publisher>...</publisher>
    <isbn>...</isbn>
    <price>...</price>
  </book>
  <book>... </book>
</bookstore>
```


XML Schema - Element Definition

- **Simple element**, contains no more elements or attributes
 - `<xs:element name="xxx" type="yyy"/>`
 - **Content can have type:** anyURI, base64Binary, boolean, byte, integer, language, dateTime, duration, string, ...
 - May have default value or fixed value
- **Complex element** can have attributes and child elements
 - Complex type definition directly bound to element definition
 - `<xs:element name="xxx"><xs:complexType>...`
 - Complex type definition referred by type attribute in element definition
 - `<xs:element name="xxx" type="ctype"/>...`
`<xs:complexType name="ctype">`

XML Schema - Attributes

- Attribute is part of a complex type
 - `<xs:attribute name="xxx" type="yyy"/>`
 - May have default value or fixed value
 - By default optional, can be mandatory
 - `<xs:attribute name="lang" type="xs:string" use="required"/>`

XML Schema - Restrictions

- Define acceptable values for element content or attributes

```
<xs:element name="age">
  <xs:simpleType>
    <xs:restriction base="xs:integer">
      <xs:minInclusive value="0"/>
      <xs:maxInclusive value="120"/>
    </xs:restriction>
  </xs:simpleType>
</xs:element>
```

```
<xs:element name="car" type="carType"/>
<xs:simpleType name="carType">
  <xs:restriction base="xs:string">
    <xs:enumeration value="Golf"/>
    <xs:enumeration value="BMW"/>
  </xs:restriction>
</xs:simpleType>
```

```
<xs:element name="letter">
  <xs:simpleType>
    <xs:restriction base="xs:string">
      <xs:pattern value="[a-z]"/>
    </xs:restriction>
  </xs:simpleType>
</xs:element>
```

```
<xs:element name="password">
  <xs:simpleType>
    <xs:restriction base="xs:string">
      <xs:minLength value="5"/>
      <xs:maxLength value="8"/>
    </xs:restriction>
  </xs:simpleType>
</xs:element>
```

XML Schema - Complex Type Indicators

- Order indicators: `all`, `choice`, `sequence`
 - `<all>`: Child elements can occur in any order and only once
 - `<choice>`: Either one or another child element can occur
 - `<sequence>`: Child elements must occur in specific order
- Occurrence indicators: `maxOccurs`, `minOccurs`
 - Default value 1 for all other indicators

```
<xs:element name="person">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="full_name" type="xs:string"/>
      <xs:element name="child_name" type="xs:string" maxOccurs="10"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>
```

XML Schema - Extensibility

- `<any>` element specifies content not further declared by this schema
 - Allows placeholder semantic
 - Additional markup is then declared by another schema file

```
<xs:element name="person">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="firstname" type="xs:string"/>
      <xs:element name="lastname" type="xs:string"/>
      <xs:any minOccurs="0"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>
```

```
<xs:element name="children">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="childname" type="xs:string"
        maxOccurs="unbounded"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>
```

XML Schema - Extensibility

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<persons xmlns="http://www.troeger.eu"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:SchemaLocation="http://www.troeger.eu family.xsd
    http://www.bth.se children.xsd">

  <person>
    <firstname>Hege</firstname>
    <lastname>Refsnes</lastname>
    <children>
      <childname>Cecilie</childname>
    </children>
  </person>
  <person>
    <firstname>Stale</firstname>
    <lastname>Refsnes</lastname>
  </person>
</persons>
```

XML Schema - There is a lot more ...

- Complex types with mixed content
- Data type details
- Element substitution
- Schema validation
 - Input is document, document schema, and schema of XML schema
 - Resulting documents are called „schema-valid“
 - XSD validator tools online, use tools for editing
- XSD-based XML applications: WSDL, XML Digital Signatures, OWL, ...

Other Schema Languages

Everybody who actually touches the technology has known the truth for years, and it's time to stop sweeping it under the rug. W3C XML Schemas (XSD) suck. They are hard to read, hard to write, hard to understand, have interoperability problems, and are unable to describe lots of things you want to do all the time in XML.

...

It's a pity; when XSD came out people thought that since it came from the W3C, same as XML, it must be the way to go, and it got baked into a bunch of other technology before anyone really had a chance to think it over. So now lots of people say "Well, yeah, it sucks, but we're stuck with it." Wrong! The time has come to declare it a worthy but failed experiment, tear down the shaky towers with XSD in their foundation, and start using RELAX NG for all significant XML work.

Tim Bray, 2006/11/28

Relax NG

```
<addressBook>
  <card>
    <name>John Smith</name>
    <email>js@example.com</email>
  </card>
  <card>
    <name>Fred Bloggs</name>
    <email>fb@example.net</email>
  </card>
</addressBook>
```

```
<element name="addressBook"
  xmlns="http://relaxng.org/ns/structure/1.0">
  <zeroOrMoreoptional
```

XQuery

- Designed as SQL counterpart - a query language for XML documents
 - Relies on XPath, similar concept of node types
 - Supported by all major database engines
 - W3C recommendation since 2007
- Select nodes from a XML file
 - **Functions** for data extraction
 - **Path expressions** for navigating through the document
 - **Predicates** for limiting the returned result set
- All XQuery constructs are expressions

XQuery Expressions

- Path expressions

- `doc("books.xml")/bookstore/book[price<30]`

- FLOWR („For, Let, Where, Order by, Return“) expressions

- **for** `$x in doc("books.xml")/bookstore/book`
where `$x/price>30`
order by `$x/title`
return `$x/title`

```
<ul>
{
  for $x in doc("books.xml")/bookstore/book/
  title
  order by $x
  return <li>{data($x)}</li>
}
</ul>
```

XQuery - Complex Example

```
declare variable $firstName as
xs:string external;
<videos featuring="{ $firstName}">
{
  let $doc := .
  for $v in $doc//video,
      $a in $doc//actors/actor
  where ends-with($a, $firstName)
      and $v/actorRef = $a/@id
  order by $v/year
  return
    <video year="{ $v/year}">
      { $v/title }
    </video>
}
</videos>
```

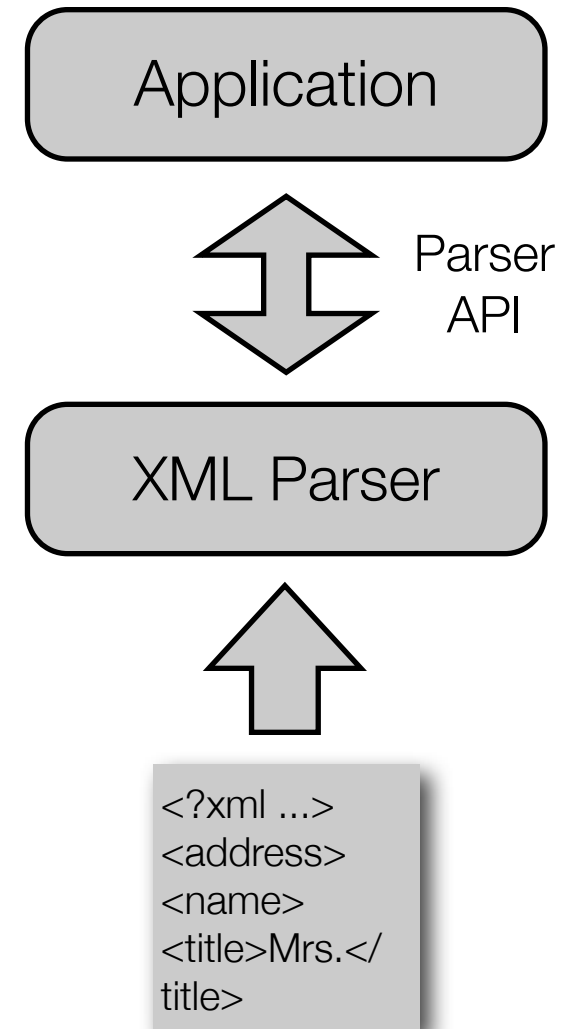
See http://www.stylusstudio.com/xquery_primer.html

XQuery Features

- Conditionals - Formulate if-then-else expressions
- Comparisons - General comparisons (=, !=, <, <=, ...) and value comparisons
- Adding new elements and new attributes to output
- Over 100 build-in functions
 - String values, numeric values, date / time handling, QName manipulation, sequence manipulation
 - Support for own function definitions (again in XQuery vocabulary)
- Originally intended as query language for XML data sets, meanwhile also possible replacement for XSLT
 - XQuery is better in data handling, while XSLT is better for narrative docs

Programming Models

- XML parser is used by application trying to **understand** the XML document
 - Splits document in elements, attributes, and other parts
 - Hands over parts to the application
 - Checks if document is well-formed / valid
- Application demands API to the parser
 - Reflects the programming model for XML processing on the application side
 - Must be standardized in order to get portable applications / exchangeable parsers



XML Processing Models

- XML parser moves from document start to document end
- **Event-oriented XML processing**
 - Parser notifies the application of processing events
 - Reading of start tag, content, end tag, processing instructions, ...
 - Advantage of efficiency and speed
 - Application can process incoming data while it is read by parser
 - Application needs to implement a state machine
 - API's in practice: *Simple API for XML (SAX)* and *Expat*

XML Processing Models

- **Tree-based XML processing**

- XML documents are naturally described as tree structures (e.g. XPath)
- After complete parsing, application can access tree representation of the XML document
 - Application don't have to deal with incomplete content
 - Easy changes to the in-memory tree structure
 - Full-fledged navigation capabilities (e.g. XPath)
 - Possible high memory footprint
- API's in practice: Document Object Model (DOM), XOM

XML Processing Models

- **Pull-based XML processing**

- Similarities to event-based processing
- Application demands content from the parser
 - Easier application code
 - No need for complicated parse state consideration
 - Still speed and efficiency advantage as with event-based approach
- API's in practice: Streaming API for XML (StAX), .NET XMLReader class

- Combined approaches

- Generation of complete trees out of parsing events

Document Object Model (DOM)

- Set of W3C recommendations
 - Representation of hierarchical documents in memory
 - Original specification is independent from programming language (IDL)
 - Supports reading and manipulation of XML / HTML / CSS
- Set of abstract interfaces, realized by implementations
 - Interfaces are separated in modules, applications can support only parts
 - Example: XML parser don't need to implement HTML-specific parts
- Different versions defined as **levels**
 - Other XML specifications extended DOM for their specific needs (e.g. SVG)

DOM Kernel Concepts

- Starting point is the generic *Node* interface
 - Support for localisation of parent nodes, child nodes, and ancestor nodes
 - Traverse document tree without knowledge of node types
 - Different node attributes: *nodeType*, *nodeValue*, *namespaceURI*, ...
- Specific interfaces for different node types, all derived from *Node*
 - *Comment*, *Document*, *Element*, *Entity*, *ProcessingInstruction*, *Text*, ...
 - *Node* object is never created directly by the parser or the application
- Several interfaces to ease up work with DOM
 - *NodeList* - ordered access to content of a node
 - *NamedNodeMap* - Accessing unordered set of elements (e.g. attributes)

```

public class ElementPrinting {
    public static void main(String args[]) {
        try
        {
            DOMParser parser = new DOMParser();
            parser.parse(args[0]);
            DocumentImpl document = (DocumentImpl)parser.getDocument();
            Node root = document.getLastChild();
            NodeIteratorImpl iterator =
                (NodeIteratorImpl)document.createNodeIterator(root,
                    NodeFilter.SHOW_ELEMENT, (NodeFilter)allelements, true);
            printElements(iterator);
        }
        catch (Exception e) {
            e.printStackTrace();
        }
    }

    public static void printElements(NodeIteratorImpl iter) {
        Node n;
        while ((n = iter.nextNode()) != null) {
            System.out.println(n.getNodeName());
        }
    }
}

class AllElements implements NodeFilter {
    public short acceptNode (Node n) {
        if (n.getNodeType() == Node.ELEMENT_NODE)
            return FILTER_ACCEPT;
        return FILTER_SKIP;
    }
}

```

```
public class XmlCreateAndPrint {  
    public XmlCreateAndPrint() {  
        try {  
            DocumentBuilderFactory dbfac=DocumentBuilderFactory.newInstance();  
            DocumentBuilder docBuilder = dbfac.newDocumentBuilder();  
            Document doc = docBuilder.newDocument();  
  
            //create the root element and add it to the document  
            Element root = doc.createElement("root");  
            doc.appendChild(root);  
  
            //create child element, add an attribute, and add to root  
            Element child = doc.createElement("child");  
            child.setAttribute("name", "value");  
            root.appendChild(child);  
  
            //add a text element to the child  
            Text text = doc.createTextNode("Hello World");  
            child.appendChild(text);  
  
            //Output the in-memory XML tree by transformation to tree  
            TransformerFactory transfac = TransformerFactory.newInstance();  
            Transformer trans = transfac.newTransformer();  
            StringWriter sw = new StringWriter();  
            StreamResult result = new StreamResult(sw);  
            DOMSource source = new DOMSource(doc);  
            trans.transform(source, result);  
            String xmlString = sw.toString();  
            System.out.println("Here's the xml:\n\n" + xmlString);  
  
        } catch (Exception e) {System.out.println(e);}  
    }  
}
```


DOM Example (Python)

```
import sys
from xml.dom import minidom, Node

def showNode(node):
    if node.nodeType == Node.ELEMENT_NODE:
        print 'Element name: %s' % node.nodeName
        for (name, value) in node.attributes.items():
            print '    Attr -- Name: %s  Value: %s' % (name, value)
        if node.attributes.get('ID') is not None:
            print '    ID: %s' % node.attributes.get('ID').value

def main():
    doc = minidom.parse(sys.argv[1])
    node = doc.documentElement
    showNode(node)
    for child in node.childNodes:
        showNode(child)

if __name__ == '__main__':
    main()
```

Simple API for XML (SAX)

- Originally developed as Java interface, but ported to other OO languages (C++, Python, Perl, Eiffel)
 - `org.xml.sax` package
- Since 2004 SAX2 available (namespace support)
- `XMLReader` interface abstracts access to the parser
 - Properties and features to configure all aspects of the parser
- `ContentHandler` interface is implemented by the application
 - `XMLReader` is configured with an instance of the implementation
 - Functions are called by the parser when the event occurs
- Support for filter implementations

ContentHandler interface

```
package org.xml.sax;

public interface ContentHandler
{
    public void setDocumentLocator (Locator locator);
    public void startDocument () throws SAXException;
    public void endDocument() throws SAXException;
    public void startPrefixMapping (String prefix, String uri)
        throws SAXException;
    public void endPrefixMapping (String prefix) throws SAXException;
    public void startElement (String uri, String localName,
        String qName, Attributes atts) throws SAXException;
    public void endElement (String uri, String localName, String qName)
        throws SAXException;
    public void characters (char ch[], int start, int length)
        throws SAXException;
    public void ignorableWhitespace (char ch[], int start, int length)
        throws SAXException;
    public void processingInstruction (String target, String data)
        throws SAXException;
    public void skippedEntity (String name) throws SAXException;
}
```

Registering the Content Handler

```
XMLReader parser =  
org.xml.sax.helpers.XMLReaderFactory.createXMLReader();  
  
// Create a new instance and register it with the parser  
ContentHandler contentHandler = new MyHandlerImplementation();  
parser.setContentHandler(contentHandler);
```

Python Example

```
from xml.sax import make_parser
from xml.sax.handler import ContentHandler

class DocHandler(ContentHandler):
    def startDocument(self):
        self.Storage = ""
    def endDocument(self):
        # Print approximate number of words
        print len(string.split(self.Storage))
    def characters(self, content):
        self.Storage = self.Storage + content

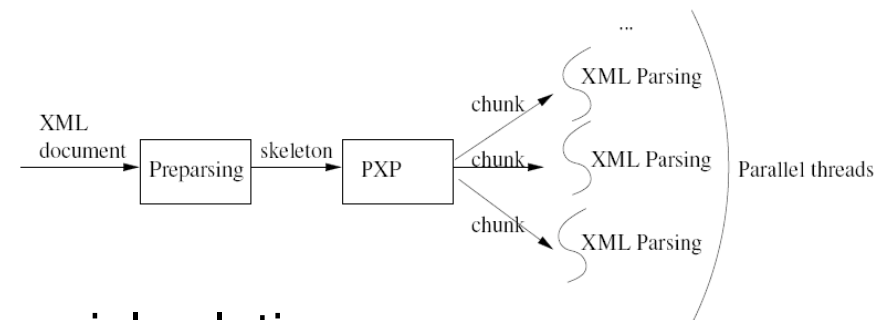
parser = make_parser()
parser.setContentHandler(DocHandler())
parser.parse(open("foo.xml"))
```

There is more ...

- XML information set
- Unicode in XML
 - `xml:lang` definition according to ISO-639
 - Sub-codes according to ISO-3166
- XPointer, XLinks, XInclude
- All the XML applications
- Fancy parser interfaces and schema languages
- ...

Parallelization ?

- Different technologies provide different parallelization possibilities
 - Parallelized XML parsing
 - 6 papers - most obvious (and most interesting ?) approach
 - PXP project (University of Indiana)
 - Parallelized XPath evaluation - 2 papers
 - No standalone solution
 - Parallelized XSLT processor
 - 1 paper, but unknown number of commercial solutions
 - Parallelized XQuery evaluation - 5 papers, pure database topic
 - Parallelized style sheet - based rendering ???



Other Unsolved Questions

- Degree of multi-threading in common open source XML processors
 - Apache Xerces, Gnome libxml2, Expat, Sun Java XML support (XML parser, DOM, SAX, XML Schema)
 - Apache Xindice (XML database)
 - Apache Xalan, Sablotron (XSLT processor, XPath implementation)
 - Apache Batik (SVG toolkit)
- Getting performance measurements
 - Sun's XMLTest, XMLBench project



Carlos Alberto Rico
Ing. Felipe Evans
Hernan Hinojal
G.I.D.I.



Grupo de Ingeniería en Desarrollos Informaticos
Facultad de Ingeniería
U.N.M.d.P.

2018



SEGURIDAD INFORMATICA

¿Que es Seguridad?



Característica o cualidad que implica:

“ Un sistema este libre de peligro, daño o riesgo”.

“La seguridad es una utopía, es decir una característica inalcanzable. Una tendencia”

Fiabilidad: Probabilidad que un sistema se comporte como tal. Que actúe de acuerdo a lo esperado.



SEGURIDAD INFORMATICA

¿Que queremos Proteger?

Fundamentalmente:



Hardware , el Software, Datos

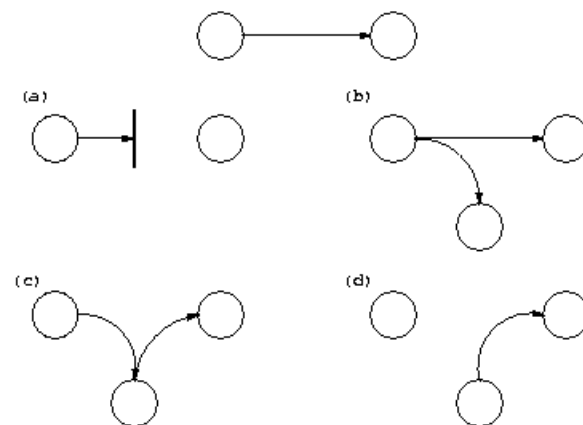
LA INFORMACIÓN



SEGURIDAD INFORMATICA

¿Que queremos Proteger??

Los tipos de Ataques:



a) Interrupción: Un objeto del sistema se pierda, inutilizable o indisponible.

b) Interceptación: Acceso no autorizada a un objeto del sistema.

c) Modificación: Interceptar y modificar.

d) Fabricación: Objeto original fabricado.



SEGURIDAD INFORMATICA

¿De que nos queremos Proteger?



Personas.

Amenazas Lógicas.

Catastrofes.



SEGURIDAD INFORMATICA

¿De que nos queremos Proteger?



Personal. (accidentes por error o desconocimiento)

Ex-empleados.

Curiosos. (Atacantes no destructivos).

Crakers. (scaners, exploit).

Terroristas .(ataques a DB, WEBS).

Intrusos renumerados. (Pagos x otras empresas).



SEGURIDAD INFORMATICA

¿De que nos queremos Proteger?



Software incorrecto. (Bugs,etc).

Herramientas de seguridad. (Nessus, SATAN,etc).

Puertas traseras.

Bombas Lógicas (activadas ante un evento)

Canales cubiertos.

Virus.(el Huésped).

Gusanos. (propagación en red “portador de virus”)

Caballos de Troya.

Programas conejo o bacteria.

Técnicas Salami

Amenazas lógicas.



SEGURIDAD INFORMATICA

¿De que nos queremos Proteger?



Naturales. (Entorno geográfico).

Artificiales. (Infraestructura.)



SEGURIDAD INFORMÁTICA

¿Como podemos Protegernos

Con que mecanismos de seguridad:



Prevención. Detección. Recuperación. Análisis forense.

“Mejor prevenir que curar!!!!”

ELEMENTOS BÁSICOS DE LA CRIPTOGRAFÍA

EMISOR

MENSAJE

HOLA MUNDO

METODO
DE
CIFRADO



LLAVE

MENSAJE
CIFRADO

XSBRWIEIXA

EL MENSAJE CIFRADO NO
PUEDE SER ENTENDIDO POR
CUALQUIER INTERCEPTOR NO
AUTORIZADO



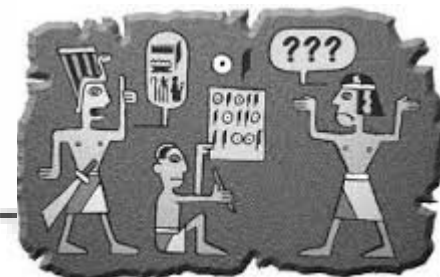
RECEPTOR

HOLA MUNDO

MENSAJE
DECIFRADO



METODO
DE
DESCIFRADO





SEGURIDAD INFORMATICA

